



Departamento de
Computação - **UFSCar**



Algoritmos e Estruturas de Dados II

**Algoritmos de ordenação, estruturas de dados avançadas
e algoritmos em grafos**

Alexandre L. M. Levada
Departamento de Computação - UFSCar

8 de julho de 2026

Prefácio

A Ciência da Computação está presente em praticamente todos os aspectos da sociedade moderna. Dos mecanismos de busca na Internet aos sistemas de navegação por satélite, das redes sociais aos modelos de inteligência artificial, praticamente toda tecnologia computacional é construída sobre algoritmos e estruturas de dados eficientes. Por trás de cada aplicação existe um conjunto de decisões cuidadosamente planejadas para organizar informações e resolver problemas da melhor maneira possível.

Esta apostila foi elaborada com o objetivo de apresentar os principais conceitos de Algoritmos e Estruturas de Dados de forma clara, intuitiva e fundamentada. Mais do que descrever técnicas de programação, buscamos desenvolver no leitor o pensamento algorítmico: a capacidade de abstrair problemas, modelá-los matematicamente, analisar diferentes estratégias de solução e compreender seus custos computacionais.

Ao longo dos capítulos, o conteúdo é apresentado de maneira gradual, combinando definições formais, demonstrações matemáticas, exemplos ilustrativos, algoritmos clássicos e análises de complexidade. Sempre que possível, procuramos explicar não apenas *como* um algoritmo funciona, mas principalmente *por que* ele funciona e quais são as ideias fundamentais que motivam sua construção.

Esperamos que este material sirva tanto como apoio às disciplinas de graduação quanto como uma referência para consultas futuras. Mais importante, desejamos que ele desperte a curiosidade do leitor para explorar áreas mais avançadas da Computação, lembrando que dominar algoritmos significa adquirir uma forma estruturada de pensar, capaz de transcender linguagens de programação, tecnologias e ferramentas específicas.

Se esta apostila conseguir estimular o prazer pela resolução de problemas e mostrar que elegância, criatividade e rigor matemático podem caminhar juntos no desenvolvimento de algoritmos, então ela terá cumprido plenamente seu propósito.

Bons estudos!

Sumário

1	Uma Breve Revisão à Complexidade de Algoritmos	5
1.1	O Modelo RAM (<i>Random Access Machine</i>)	6
1.2	Notação Big-O	8
1.3	Classes de Complexidade de Algoritmos	9
1.4	Analisando Algoritmos	10
1.5	Considerações Finais	17
2	Algoritmos de Ordenação	18
2.1	BubbleSort	19
2.2	InsertionSort	24
2.3	Selection Sort	29
2.4	Shellsort	33
2.5	Heapsort	40
2.6	Mergesort	49
2.7	Quicksort	55
2.8	Limitante Inferior para Ordenação Baseada em Comparações	66
2.9	Countingsort	68
2.10	Radixsort	72
2.11	Bucketsort	75
2.12	Comparação entre algoritmos de ordenação	79
2.13	Considerações Finais	80
3	Árvores de Busca Digitais (Tries)	82
3.1	Busca em árvores binárias digitais	83
3.2	Inserção em árvores binárias digitais	86
3.3	Remoção em árvores binárias digitais	87
3.4	Análise da Complexidade	89
3.5	Considerações Finais	90
4	Skip Lists	92
4.1	Análise da Complexidade	94
4.2	Inserção em Skip Lists	98
4.3	Remoção em Skip Lists	101
4.4	Considerações Finais	103
5	Árvores Rubro-Negras	105
5.1	Altura Máxima de Árvores Rubro-Negras	106
5.2	Inserção em Árvores Rubro-Negras	108
5.2.1	Ajustes após inserção	114

5.3	Remoção em Árvores Rubro-Negras	117
5.4	Árvores AVL versus Árvores Rubro-Negras	124
5.5	Considerações Finais	125
6	Hash Tables	127
6.1	Tabelas de Acesso Direto	127
6.2	Tabelas de Espalhamento	129
6.3	Tratamento de Colisões por Encadeamento Lógico	131
6.4	Análise da Complexidade da Busca	133
6.5	Funções Hash	136
6.5.1	Método da Divisão	137
6.5.2	Método da Multiplicação	138
6.6	Hashing Universal	139
6.6.1	Família Universal de Funções Hash	140
6.6.2	Construção de uma Família Universal	140
6.6.3	Como o Hashing Universal é Utilizado na Prática?	141
6.7	Endereçamento Aberto (<i>Open Addressing</i>)	142
6.8	Linear Probing (Sondagem Linear)	143
6.9	Quadratic Probing (Sondagem Quadrática)	144
6.10	Double Hashing (Espalhamento Duplo)	145
6.11	Análise da Complexidade no Endereçamento Aberto	147
6.12	Calculando Probabilidades de Colisões	153
6.12.1	O paradoxo do aniversário	154
6.12.2	Probabilidade de colisões em Tabelas Hash	154
6.13	Número esperado de colisões	156
6.14	Perfect Hashing (Espalhamento Perfeito)	157
6.15	Considerações Finais	164
7	Grafos: Fundamentos Básicos	166
7.1	Definições Básicas	167
7.2	Representações Computacionais de Grafos	170
7.3	Matriz de Adjacências	170
7.4	Matriz de Incidência	171
7.5	Lista de Adjacências	171
7.6	Quando utilizar cada representação?	172
7.7	Tipos de Grafos	174
7.7.1	Grafos direcionados	174
7.7.2	Grafos ponderados	174
7.7.3	Multigrafos	175
7.7.4	Caminhos, Ciclos e Conectividade	175

7.7.5	Conectividade	175
7.7.6	Subgrafos	175
7.8	Considerações Finais	176
8	Busca em Grafos: BFS e DFS	177
8.1	Árvores e suas Propriedades	178
8.2	Relação entre busca em grafos e árvores	180
8.3	Busca em Largura (Breadth-First Search – BFS)	182
8.3.1	Exemplo de execução da Busca em Largura	183
8.3.2	Complexidade da Busca em Largura	185
8.4	Corretude da Busca em Largura	188
8.5	Busca em Profundidade (Depth-First Search – DFS)	190
8.5.1	Variáveis utilizadas	190
8.5.2	Implementação recursiva	191
8.5.3	Complexidade da Busca em Profundidade	193
8.5.4	Propriedades da DFS-tree e Vértices de Corte	194
8.5.5	Detecção de Ciclos com Busca em Profundidade (DFS)	198
8.6	Ordenação Topológica	199
8.6.1	Quando a ordenação topológica existe?	200
8.6.2	A ideia da solução usando DFS	200
8.6.3	Por que o algoritmo funciona?	204
8.7	Considerações Finais	206
9	Caminhos mínimos em grafos ponderados	209
9.1	Algoritmo de Dijkstra	210
9.1.1	Caminhos mínimos	210
9.1.2	A operação de relaxamento	210
9.1.3	Dijkstra como uma generalização da BFS	213
9.1.4	Análise da Complexidade	215
9.1.5	Prova de Corretude do Algoritmo de Dijkstra	219
9.2	Limitações do Algoritmo de Dijkstra: Arestas com Pesos Negativos	221
9.3	O Algoritmo de Bellman–Ford	223
9.4	Dijkstra com Múltiplas Fontes (Multi-Source Dijkstra)	226
9.5	A heurística A*	231
9.6	Considerações Finais	239

1 Uma Breve Revisão à Complexidade de Algoritmos

Na Ciência da Computação, mais especificamente no estudo de algoritmos, uma das primeiras perguntas que surgem é: *como medir a eficiência de um algoritmo?* Em outras palavras, dados dois algoritmos A_1 e A_2 , como determinar qual deles é mais eficiente? Em geral, estamos interessados em duas quantidades principais:

- **Tempo de execução** (*complexidade de tempo*);
- **Quantidade de memória utilizada** (*complexidade de espaço*).

A quantidade de memória exigida por um algoritmo depende essencialmente das variáveis e estruturas de dados alocadas durante sua execução, o que torna essa medida relativamente simples de estimar. Já o tempo de execução requer uma análise mais formal e aprofundada do algoritmo.

Uma dúvida natural de muitos programadores é: por que não simplesmente cronometrar o tempo gasto pela execução de uma implementação do algoritmo?

A resposta é que o tempo medido dependeria de diversos fatores externos ao algoritmo em si, tais como:

- a linguagem de programação utilizada (por exemplo, C, Python ou Java);
- o hardware da máquina onde o programa é executado (processador, memória, etc.).

O objetivo da análise de algoritmos é justamente realizar uma avaliação que seja *independente desses fatores externos*, ou seja, uma análise que dependa apenas da estrutura do algoritmo em questão.

O estudo da complexidade computacional é relativamente antigo. Ele foi formulado de forma sistemática no início da década de 1960 e, desde então, tornou-se a abordagem padrão para avaliar o custo de algoritmos.

As operações elementares executadas por uma CPU são chamadas de *instruções*, e o tempo necessário para executá-las é conhecido como sua *latência*. As instruções são armazenadas na memória e executadas sequencialmente pelo processador, que mantém um estado interno armazenado em vários registradores.

Um desses registradores é o *ponteiro de instrução* (*instruction pointer*), que indica o endereço da próxima instrução a ser lida e executada. Cada instrução altera o estado do processador de alguma forma (incluindo a atualização do ponteiro de instrução), podendo também modificar dados na memória principal. Além disso, cada instrução pode exigir um número diferente de ciclos de CPU para ser concluída antes que a próxima instrução possa ser iniciada.

Para estimar o tempo real de execução de um programa, seria necessário somar as latências de todas as instruções executadas e dividir esse valor pela frequência de clock do processador, isto é, pelo número de ciclos que a CPU executa por segundo.

Entretanto, esse processo é extremamente complexo e, na prática, praticamente inviável para a análise de algoritmos. Por esse motivo, utiliza-se uma abordagem abstrata, na qual contamos o número de operações elementares executadas pelo algoritmo em função do tamanho da entrada.

1.1 O Modelo RAM (*Random Access Machine*)

O modelo RAM (*Random Access Machine*) é uma abstração de uma CPU e constitui um dos modelos teóricos de computação mais simples e amplamente utilizados na análise de algoritmos. Nesse modelo, assume-se a existência de um banco potencialmente ilimitado de células de memória, cada uma das quais pode armazenar um número ou caractere arbitrário.

As células de memória são numeradas e podem ser acessadas diretamente por meio de seu endereço. Uma característica fundamental do modelo é que o tempo necessário para acessar qualquer célula da memória é sempre o mesmo, independentemente de sua posição. Em outras palavras, todas as operações básicas, incluindo leitura e escrita na memória, operações aritméticas e operações booleanas, são consideradas como tendo custo constante, correspondendo a uma única unidade de tempo.

Dessa forma, o modelo RAM pode ser entendido como um modelo de computação com memória infinita e custo de acesso uniforme (*uniform memory access*). Essa abstração simplifica significativamente a análise teórica de algoritmos e, por essa razão, é amplamente adotada no estudo da complexidade computacional.

O tempo de execução de um algoritmo no modelo RAM é medido contabilizando-se o número total de etapas necessárias para processar uma determinada entrada de dados. Nesse contexto, cada operação elementar executada pelo algoritmo corresponde a uma unidade de tempo.

O modelo RAM opera segundo alguns princípios básicos:

- Operações aritméticas ou lógicas elementares, tais como $+$, $-$, $*$, $/$, comparações e testes condicionais (por exemplo, `if`), são consideradas operações simples que ocorrem em um único intervalo de tempo.
- Estruturas de controle mais complexas, como laços de repetição (*loops*) e chamadas de sub-rotinas, são compostas por múltiplas operações elementares e, portanto, correspondem a vários intervalos de tempo.
- Todo acesso à memória (leitura ou escrita) ocorre exatamente em um intervalo de tempo.

Na prática, esses princípios são equivalentes a assumir que a CPU dispõe de um conjunto básico de instruções, como:

LOAD, STORE, READ, WRITE, ADD, SUBTRACT, MULTIPLY, DIVIDE, TEST,
JUMP, HALT

e que todas essas instruções são executadas em tempo constante, isto é, em um único intervalo de tempo fixo e previamente determinado.

Esse modelo captura as funcionalidades essenciais dos computadores modernos, embora não represente fielmente todos os seus detalhes de implementação. Por exemplo, no modelo RAM uma operação de adição e uma operação de multiplicação possuem o mesmo custo unitário. Entretanto, em computadores reais, a multiplicação geralmente requer mais ciclos de processamento do que uma soma.

Apesar dessas simplificações, o modelo RAM se mostra extremamente eficaz para a análise de algoritmos, especialmente no contexto da *análise assintótica*, na qual o interesse principal é compreender como o custo de execução cresce em função do tamanho da entrada.

Como forma de simplificar ainda mais os cálculos envolvidos na análise, frequentemente adota-se a estratégia de contabilizar apenas o número de operações de atribuição realizadas por um algoritmo A , o que corresponde, em essência, ao número de acessos à memória executados durante sua execução.

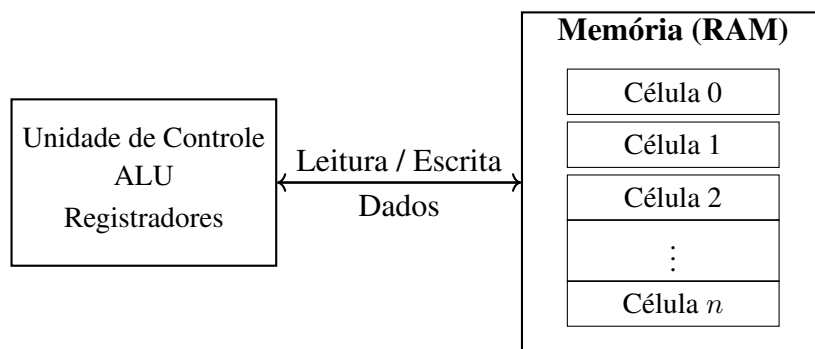


Figura 1: Ilustração conceitual do modelo RAM. A CPU executa instruções e acessa diretamente qualquer célula da memória, assumindo custo uniforme para todas as operações.

Para simplificar a análise de algoritmos no modelo RAM, costuma-se adotar algumas hipóteses básicas:

1. Cada comando de atribuição executa em tempo constante, ou seja, possui complexidade $O(1)$.
2. Uma função $T(n)$ deve retornar o número de atribuições executadas quando a entrada do problema resolvido pelo algoritmo possui tamanho n .

Vejamos um exemplo simples. Considere a função a seguir, que calcula a soma dos n primeiros números inteiros.

```
sum_of_n(n) {  
    soma = 0  
    for i = 1 to n  
        soma = soma + i  
    return soma  
}
```

Note que a instrução de inicialização `soma = 0` é executada apenas uma vez. Já o laço `for` executa n vezes, realizando uma operação de atribuição a cada iteração. Dessa forma, o número total de atribuições executadas pelo algoritmo é

$$T(n) = n + 1.$$

Observe que, quando n cresce muito, apenas a parte dominante da função se torna relevante. Nesse caso, o termo dominante é n , enquanto a constante 1 torna-se desprezível. Por essa razão, dizemos que o algoritmo possui crescimento linear, sendo descrito assintoticamente por

$$T(n) = O(n).$$

1.2 Notação Big-O

Em vez de nos preocuparmos em contar exatamente o número de instruções executadas por um programa, é geralmente mais conveniente analisar apenas a *ordem de grandeza* da função de custo $T(n)$. Em outras palavras, estamos interessados em compreender o comportamento da função $T(n)$ quando o tamanho da entrada n cresce arbitrariamente.

Suponha que exista uma função $f(n)$, definida para todos os inteiros $n \geq 0$, e constantes positivas c e m tais que

$$T(n) \leq c f(n)$$

para todo $n \geq m$ (isto é, para n suficientemente grande). Nesse caso, dizemos que $T(n)$ é *Big-O de* $f(n)$, ou simplesmente

$$T(n) = O(f(n)).$$

Intuitivamente, a notação Big-O fornece um **limite superior assintótico** para o crescimento da função de custo de um algoritmo. Considere o algoritmo a seguir, que calcula a soma dos elementos de uma matriz quadrada de ordem n .

```
sum_of_matrix(M, n) {
    soma = 0
    for i = 1 to n {
        soma_linha = 0
        for j = 1 to n
            soma_linha += M[i,j]
        soma += soma_linha
    }
    return soma
}
```

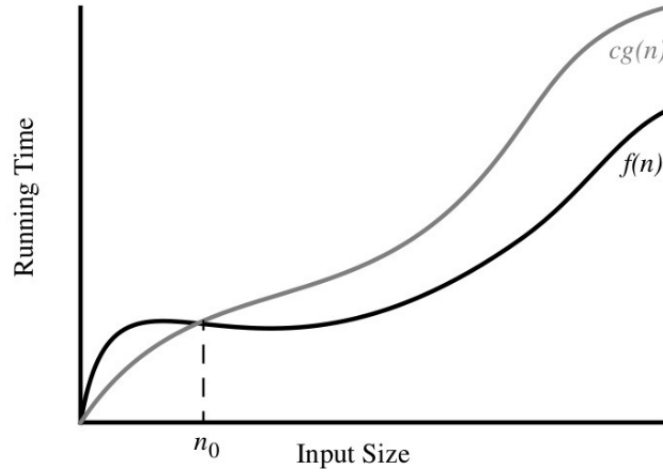


Figura 2: A função $g(n)$ multiplicada por uma constante c fornece um limite superior para a função $f(n)$ para todo n maior que um limite n_0 .

Vamos estimar o número de instruções executadas por esse algoritmo. Observe que o laço mais interno (controlado por j) executa n iterações. Assim, para cada valor de i no laço externo, realizamos aproximadamente $n + 2$ atribuições.

Como existem n possíveis valores para i , obtemos

$$T(n) = n(n + 2) + 1$$

ou, equivalentemente,

$$T(n) = n^2 + 2n + 1.$$

Agora, considere $c = 2$ e $f(n) = n^2$. Temos que

$$n^2 + 2n + 1 \leq 2n^2$$

para todo $n > 2$. Portanto,

$$T(n) = O(n^2).$$

Note que poderíamos ter escolhido outras funções maiores, como $f(n) = n^3$, que também satisfariam a definição. Entretanto, o objetivo da notação Big-O é fornecer o **limite superior mais justo possível** para o crescimento da função de custo.

1.3 Classes de Complexidade de Algoritmos

Os algoritmos costumam ser classificados de acordo com a ordem de crescimento assintótico da função $T(n)$ que descreve sua complexidade de tempo. Essa classificação permite comparar algoritmos independentemente de constantes multiplicativas e termos de menor ordem. A seguir, apresen-

tamos as principais classes de complexidade, organizadas conforme a função $f(n)$ associada ao seu crescimento assintótico.

Tabela 1: Principais classes de complexidade assintótica

Função $f(n)$	Classe de Complexidade
1	Constante
$\log n$	Logarítmica
n	Linear
$n \log n$	Log-linear
n^2	Quadrática
n^3	Cúbica
n^k	Polinomial
2^n	Exponencial
$n!$	Fatorial

Observações

- Algoritmos de complexidade constante $O(1)$ possuem tempo de execução independente do tamanho da entrada.
- Algoritmos logarítmicos $O(\log n)$ são típicos de estratégias que dividem o problema sucessivamente pela metade, como na busca binária.
- Algoritmos lineares $O(n)$ percorrem a entrada uma única vez.
- Algoritmos $O(n \log n)$ aparecem frequentemente em métodos eficientes de ordenação baseados em divisão e conquista.
- Complexidades polinomiais $O(n^k)$ são consideradas tratáveis do ponto de vista computacional.
- Complexidades exponenciais $O(2^n)$ e fatoriais $O(n!)$ crescem muito rapidamente, tornando-se inviáveis mesmo para valores moderados de n .

1.4 Analisando Algoritmos

Considere inicialmente um exemplo com duas estruturas de repetição em série.

```
Func_A(n) {  
    c = 0  
    for i = 1 to n  
        c += 1  
    for j = 1 to n  
        c += 2  
    return c  
}
```

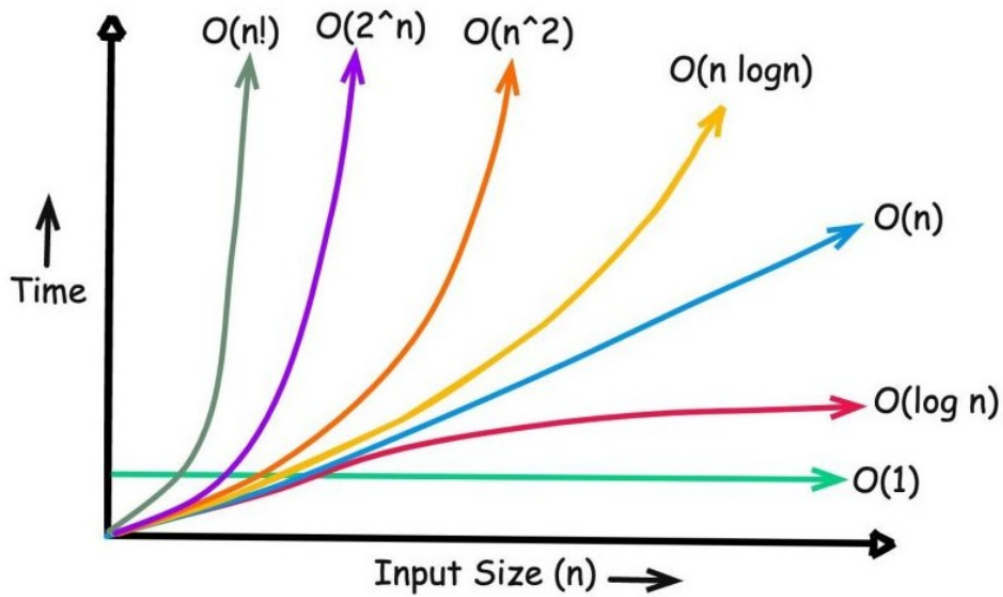


Figura 3: O gráfico mostra a taxa de crescimento de diversos tipos de funções usadas na complexidade de algoritmos.

Temos uma atribuição inicial (1 operação) e dois loops com n iterações cada. Cada laço contribui com n atribuições. Logo,

$$T(n) = 2n + 1,$$

o que implica

$$T(n) = O(n).$$

Agora considere um algoritmo com duas estruturas de repetição aninhadas.

```
Func_B(n) {
  c = 0
  for i = 1 to n {
    for j = 1 to n
      c = c + 1
  }
  return c
}
```

O loop interno executa n operações. Como o loop externo executa n vezes, temos $n \cdot n = n^2$ incrementos, além da inicialização. Assim,

$$T(n) = n^2 + 1,$$

portanto,

$$T(n) = O(n^2).$$

Se no loop interno o limite fosse 10 em vez de n , teríamos

$$T(n) = 10n + 1,$$

o que implica

$$T(n) = O(n).$$

Considere agora um exemplo em que o loop interno depende da variável do loop externo.

```
Func_C(n) {
    c = 0
    for i = 1 to n {
        for j = 1 to i
            c = c + 1
        }
    }
    return c
}
```

Observe que:

- Quando $i = 1$, o loop interno executa uma vez;
- Quando $i = 2$, executa duas vezes;
- ...
- Quando $i = n$, executa n vezes.

Logo, o número total de incrementos é:

$$1 + 2 + 3 + \cdots + n = \sum_{k=1}^n k.$$

Para resolver esse somatório, note que:

$$(k+1)^2 = k^2 + 2k + 1,$$

o que implica

$$(k+1)^2 - k^2 = 2k + 1.$$

Somando ambos os lados de $k = 1$ até n :

$$\sum_{k=1}^n [(k+1)^2 - k^2] = \sum_{k=1}^n (2k+1).$$

O lado esquerdo é uma soma telescópica:

$$\sum_{k=1}^n [(k+1)^2 - k^2] = (n+1)^2 - 1.$$

Logo,

$$(n+1)^2 - 1 = \sum_{k=1}^n (2k+1).$$

Aplicando a propriedade associativa:

$$(n+1)^2 - 1 = 2 \sum_{k=1}^n k + \sum_{k=1}^n 1.$$

Como

$$\sum_{k=1}^n 1 = n,$$

temos:

$$(n+1)^2 - 1 = 2 \sum_{k=1}^n k + n.$$

Expandindo:

$$n^2 + 2n + 1 - 1 = 2 \sum_{k=1}^n k + n,$$

$$n^2 + 2n = 2 \sum_{k=1}^n k + n.$$

Isolando o somatório:

$$2 \sum_{k=1}^n k = n^2 + n,$$

$$\sum_{k=1}^n k = \frac{n(n+1)}{2}.$$

Portanto,

$$T(n) = \frac{n(n+1)}{2} + 1 = \frac{1}{2}(n^2 + n) + 1.$$

Assim, o termo dominante é n^2 , e concluímos que

$$T(n) = O(n^2).$$

O próximo exemplo apresenta uma função em que a variável controladora é dividida por 2 a cada iteração.

```
Func_D(n) {  
    c = 0  
    i = n  
    while i > 1 {  
        c = c + 1  
        i = floor(i/2)    // divisao inteira  
    }  
    return c  
}
```

Essa função calcula quantas vezes um número pode ser dividido por 2 até se tornar menor ou igual a 1. Por exemplo, para $n = 16$:

$$16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$$

A variável c termina valendo 4, pois

$$2^4 = 16.$$

Para $n = 25$:

$$25 \rightarrow 12 \rightarrow 6 \rightarrow 3 \rightarrow 1$$

Temos $c = 4$, pois

$$2^4 < 25 < 2^5.$$

Para $n = 40$:

$$40 \rightarrow 20 \rightarrow 10 \rightarrow 5 \rightarrow 2 \rightarrow 1$$

Temos $c = 5$, pois

$$2^5 < 40 < 2^6.$$

De forma geral, o número de iterações do laço é:

$$\lfloor \log_2 n \rfloor.$$

Como dentro do `while` existem duas instruções, o número total de operações é:

$$T(n) = 2 + 2\lfloor \log_2 n \rfloor,$$

onde $\lfloor x \rfloor$ representa a função piso, isto é, o maior inteiro menor ou igual a x .

Logo,

$$T(n) = O(\log_2 n).$$

Considere agora o seguinte algoritmo:

```
Func_E(n) {  
    c = 0  
    for i = 1 to n  
        c = c + Func_D(n)  
    return c  
}
```

Como a função `Func_D(n)` possui complexidade $O(\log_2 n)$ e o laço externo executa n vezes, o número total de operações é da ordem de:

$$T(n) = n \log_2 n.$$

Portanto, a complexidade assintótica é:

$$T(n) = O(n \log_2 n).$$

A seguir apresentamos duas funções para verificar se um número inteiro n é primo. Vamos analisar a complexidade de cada uma e justificar qual delas é mais eficiente.

```
prime_A(n) {  
    if n == 1  
        return False  
    for i = 2 to n-1 {  
        resto = n % i  
        if resto == 0  
            return False  
    }  
    return True  
}
```

Primeiramente, analisemos o algoritmo `prime_A`. Se $n \in \{0, 1, 2, 3\}$, o algoritmo executa apenas algumas instruções condicionais, resultando em complexidade $T(n) = O(1)$, que corresponde ao melhor caso. Caso contrário, o laço executa de $i = 2$ até $i = n - 1$. Logo, o número de iterações é:

$$(n - 1) - 2 + 1 = n - 2.$$

Em cada iteração é realizada uma operação de resto da divisão. Assim, no pior caso:

$$T(n) = n - 2,$$

o que implica:

$$T(n) = O(n).$$

Agora analisemos o algoritmo `prime_B`. Novamente, se $n \in \{0, 1, 2, 3\}$, temos:

$$T(n) = O(1).$$

Caso contrário, o laço `while` executa enquanto $i \leq \sqrt{n}$.

```
prime_B(n) {
    if n == 1
        return False
    if n == 2
        return True
    if n % 2 == 0
        return False
    i = 3
    while i <= sqrt(n) {
        resto = n % i
        if resto == 0
            return False
        else
            i += 2
    }
    return True
}
```

Como i inicia em 3 e é incrementado de 2 em 2, o número de iterações é aproximadamente:

$$\frac{\sqrt{n} - 3}{2}.$$

Em cada iteração são realizadas duas operações principais: o cálculo do resto e o incremento de i . Assim, o número total de operações é proporcional a:

$$T(n) \approx 2 \cdot \frac{\sqrt{n} - 3}{2}.$$

Assintoticamente, temos:

$$T(n) = O(\sqrt{n}) = O(n^{1/2}).$$

Comparando as duas complexidades:

$$O(n) \quad \text{vs} \quad O(n^{1/2}),$$

observa-se que

$$\lim_{n \rightarrow \infty} \frac{\sqrt{n}}{n} = 0.$$

Logo, $n^{1/2}$ cresce muito mais lentamente que n . Portanto, o algoritmo `prime_B` é assintoticamente mais eficiente que `prime_A`. Em termos práticos, isso significa que, para valores suficientemente grandes de n , nenhum computador fará com que `prime_A` seja mais rápido que `prime_B` (embora, para valores pequenos de n , diferenças de constante possam eventualmente inverter essa relação).

1.5 Considerações Finais

Neste capítulo apresentamos uma breve revisão dos principais conceitos relacionados à análise de complexidade de algoritmos. Discutimos inicialmente a motivação para esse tipo de análise, destacando a necessidade de avaliar a eficiência de algoritmos de forma independente de fatores externos, tais como linguagem de programação, arquitetura de hardware ou detalhes específicos de implementação.

Para tornar essa análise viável, introduzimos o modelo de computação *RAM* (*Random Access Machine*), que fornece uma abstração simplificada do funcionamento de um computador. Nesse modelo, assumimos que operações elementares possuem custo constante e que qualquer acesso à memória ocorre em tempo uniforme. Embora esse modelo não capture todos os detalhes de computadores reais, ele permite realizar análises matemáticas tratáveis e, na prática, fornece boas estimativas para o comportamento de algoritmos.

A partir dessas hipóteses, mostramos como estimar o custo de um algoritmo por meio de uma função $T(n)$, que expressa o número de operações executadas em função do tamanho da entrada. Em seguida, discutimos a importância da análise assintótica e introduzimos a notação Big-O, que permite descrever o limite superior do crescimento de $T(n)$ de maneira concisa e independente de constantes ou termos de menor ordem.

Essas ferramentas conceituais constituem a base para o estudo de algoritmos eficientes e serão amplamente utilizadas ao longo desta disciplina. Nos próximos capítulos, aplicaremos esses princípios para analisar algoritmos mais sofisticados, incluindo métodos eficientes de ordenação, estruturas de dados avançadas e algoritmos fundamentais em grafos.

2 Algoritmos de Ordenação

A ordenação de dados é um dos problemas mais clássicos e fundamentais da Ciência da Computação. De maneira geral, o problema consiste em reorganizar os elementos de uma coleção de dados de acordo com uma determinada relação de ordem, como por exemplo ordem crescente ou decrescente. Embora a definição do problema seja simples, o estudo de algoritmos de ordenação revelou, ao longo das últimas décadas, uma rica variedade de estratégias algorítmicas e resultados teóricos importantes.

A importância prática da ordenação é difícil de ser superestimada. Em muitas aplicações computacionais, dados precisam ser organizados antes que outras operações possam ser realizadas de forma eficiente. Por exemplo, estruturas de dados de busca, sistemas de banco de dados, motores de busca e aplicações de análise de dados frequentemente dependem de conjuntos de dados ordenados para acelerar consultas, agregações e operações de filtragem. Em diversos casos, o custo de ordenar os dados representa uma parcela significativa do tempo total de processamento de um sistema.

Do ponto de vista teórico, os algoritmos de ordenação também desempenham um papel central no estudo da análise de algoritmos. Muitos conceitos fundamentais, como análise de complexidade, técnicas de divisão e conquista, estratégias iterativas e recursivas, além de limites inferiores para certos problemas computacionais, podem ser ilustrados de maneira clara por meio do estudo desses algoritmos. Por essa razão, algoritmos de ordenação são frequentemente utilizados como exemplos introdutórios em cursos de algoritmos e estruturas de dados.

Neste capítulo estudaremos alguns dos principais algoritmos de ordenação utilizados na prática. Inicialmente serão apresentados métodos mais simples, que ajudam a compreender os princípios básicos do problema. Em seguida, analisaremos algoritmos mais sofisticados e eficientes, capazes de ordenar grandes volumes de dados com melhor desempenho assintótico. Para cada algoritmo, discutiremos seu funcionamento, apresentaremos exemplos ilustrativos e analisaremos sua complexidade de tempo e espaço.

A ordenação de dados é um dos problemas mais importantes da Ciência da Computação, aparecendo em uma grande variedade de aplicações práticas e servindo como base para diversas outras operações sobre conjuntos de dados.

Problema: ordenação de dados

Entrada: uma sequência arbitrária de chaves

$$a_1, a_2, a_3, \dots, a_n$$

(elementos armazenados em uma lista ou vetor).

Saída: uma permutação da sequência de entrada tal que

$$a'_1 \leq a'_2 \leq a'_3 \leq \dots \leq a'_n$$

ou seja, os elementos organizados em ordem crescente.

Uma primitiva básica utilizada em muitos dos algoritmos de ordenação baseados em trocas que estudaremos é a função `swap(a, b)`, responsável por trocar as posições de duas chaves.

```
swap(a, b) {  
    temp = a  
    a = b  
    b = temp  
}
```

Como podemos observar, essa operação envolve apenas um número constante de atribuições. Portanto, sua complexidade é $O(1)$, o que é um aspecto fundamental na análise da complexidade de algoritmos de ordenação, já que muitas dessas técnicas baseiam-se na repetição dessa operação para reorganizar os elementos da sequência.

2.1 BubbleSort

O algoritmo *Bubble Sort* é uma das abordagens mais simples para a ordenação de dados. A ideia básica consiste em percorrer o vetor diversas vezes e, a cada passagem, fazer com que o maior elemento ainda não ordenado “flutue” para o final da lista (isto é, para a posição mais à direita possível).

Esse padrão de movimentação lembra o comportamento de bolhas em um líquido, que tendem a subir até atingir seu próprio nível. É justamente dessa analogia que surge o nome do algoritmo, também conhecido como *método da bolha*.

No melhor caso, quando os elementos já se encontram ordenados, o algoritmo necessita de apenas n operações relevantes, onde n representa o número de elementos do vetor. No entanto, no pior caso, são realizadas aproximadamente n^2 operações. Portanto, diz-se que a complexidade do método é de **ordem quadrática**. Por essa razão, o Bubble Sort não é recomendado para aplicações que exigem alto desempenho ou que manipulam grandes volumes de dados. A seguir apresentamos um pseudocódigo para o algoritmo.

```
BubbleSort(L, n) {  
    // Percorre cada elemento do array L  
    for i = n-1 downto 1 {  
        // Flutua o maior elemento para a posição mais à direita  
        for j = 1 to i {  
            if L[j] > L[j+1]  
                swap(L[j], L[j+1])  
        }  
    }  
}
```

Exemplo de execução

Considere o seguinte vetor inicial:

$[5, 2, 13, 7, -3, 4, 15, 10, 1, 6]$

A seguir mostramos algumas das principais etapas do processo de ordenação.

1ª passagem (o maior elemento vai para a última posição)

$[5, 2, 13, 7, -3, 4, 15, 10, 1, 6]$

$[2, 5, 13, 7, -3, 4, 15, 10, 1, 6]$

$[2, 5, 7, 13, -3, 4, 15, 10, 1, 6]$

$[2, 5, 7, -3, 13, 4, 15, 10, 1, 6]$

$[2, 5, 7, -3, 4, 13, 15, 10, 1, 6]$

$[2, 5, 7, -3, 4, 13, 10, 15, 1, 6]$

$[2, 5, 7, -3, 4, 13, 10, 1, 15, 6]$

$[2, 5, 7, -3, 4, 13, 10, 1, 6, 15]$

Após a primeira passagem, o maior elemento do vetor (15) já se encontra na última posição.

2ª passagem (segundo maior elemento)

$[2, 5, 7, -3, 4, 13, 10, 1, 6, 15]$

$[2, 5, -3, 7, 4, 13, 10, 1, 6, 15]$

$[2, 5, -3, 4, 7, 13, 10, 1, 6, 15]$

$[2, 5, -3, 4, 7, 10, 13, 1, 6, 15]$

$[2, 5, -3, 4, 7, 10, 1, 13, 6, 15]$

$[2, 5, -3, 4, 7, 10, 1, 6, 13, 15]$

Passagens seguintes

O mesmo processo continua sendo aplicado, fazendo com que os maiores elementos restantes avancem gradualmente para o final do vetor. Após algumas iterações adicionais, obtemos:

$[-3, 1, 2, 4, 5, 6, 7, 10, 13, 15]$

É importante observar que a garantia de que o vetor está completamente ordenado só ocorre após a execução de todas as passagens previstas pelo algoritmo.

Análise da Complexidade

Para analisar o desempenho do algoritmo Bubble Sort, consideraremos três cenários distintos: **pior caso**, **caso médio** e **melhor caso**. Essa abordagem é comum na análise de algoritmos, pois o comportamento de um método pode variar significativamente dependendo da configuração inicial dos dados de entrada.

Pior caso

O pior caso ocorre quando o vetor de entrada está ordenado em **ordem decrescente**. Nessa situação, todos os elementos precisam ser reposicionados para que o vetor fique em ordem crescente.

Observando o pseudo-código do algoritmo, percebe-se que o laço interno executa de 1 até i . Na primeira passagem temos $i = n - 1$, na segunda $i = n - 2$, e assim sucessivamente até $i = 1$. Dessa forma, o número total de comparações realizadas é dado por

$$T(n) = (n - 1) + (n - 2) + \cdots + 1.$$

Sabemos que o somatório

$$1 + 2 + \cdots + n = \frac{n(n + 1)}{2}.$$

Logo,

$$T(n) = \frac{n(n - 1)}{2} = \frac{n^2 - n}{2}.$$

Portanto, a complexidade assintótica do algoritmo no pior caso é

$$T(n) = O(n^2).$$

Melhor caso

É possível realizar uma pequena modificação no algoritmo Bubble Sort para melhorar seu desempenho quando o vetor já está ordenado. A ideia consiste em contar quantas trocas (*swaps*) são realizadas em cada passagem. Caso nenhuma troca seja efetuada, isso significa que a lista já se encontra ordenada e o algoritmo pode terminar antecipadamente. O pseudo-código modificado é apresentado a seguir.

```

BubbleSort_M(L, n) {
    for i = n-1 downto 1 {
        s = 0
        for j = 1 to i {
            if L[j] > L[j+1] {
                swap(L[j], L[j+1])
                s = s + 1
            }
        }
        if s == 0
            break
    }
}

```

Com essa modificação, se a lista de entrada já estiver ordenada, o algoritmo não realizará nenhuma troca durante a primeira passagem. Assim, o laço externo é interrompido imediatamente após percorrer o vetor uma única vez. Nesse cenário, o número de operações realizadas é proporcional ao tamanho da entrada, resultando em uma complexidade

$$T(n) = O(n).$$

Entretanto, é importante observar que a versão original do Bubble Sort apresentada anteriormente não realiza essa verificação de trocas. Dessa forma, mesmo quando a lista já está ordenada, o algoritmo continua executando todas as passagens previstas, o que resulta em complexidade

$$T(n) = O(n^2)$$

mesmo no melhor caso.

Caso médio

No caso médio, assumimos a hipótese de que todas as possíveis configurações da entrada são equiprováveis. Dessa forma, o custo esperado do algoritmo pode ser obtido calculando-se a média do número de operações realizadas em todos os possíveis casos.

Recorde que, no pior caso, o número de comparações em cada iteração do laço externo é: na primeira iteração $n - 1$, na segunda $n - 2$, e assim sucessivamente até chegar a 1 comparação na última iteração. Para obter o caso médio, podemos imaginar que o número de trocas efetivamente realizadas em cada passo seja um valor intermediário entre esses extremos. Assim, podemos calcular uma média do somatório anterior para diferentes valores de k , variando de 1 até $n - 1$. Formalmente, podemos escrever

$$T(n) = \frac{1}{n-1} \sum_{k=1}^{n-1} \sum_{i=1}^k i.$$

Sabemos que o somatório

$$1 + 2 + 3 + \cdots + k = \frac{k(k+1)}{2}.$$

Substituindo na expressão anterior, obtemos

$$T(n) = \frac{1}{n-1} \sum_{k=1}^{n-1} \frac{k(k+1)}{2}$$

ou, de forma equivalente,

$$T(n) = \frac{1}{2(n-1)} \sum_{k=1}^{n-1} (k^2 + k).$$

Separando os termos, temos

$$T(n) = \frac{1}{2(n-1)} \left(\sum_{k=1}^{n-1} k^2 + \sum_{k=1}^{n-1} k \right). \quad (*)$$

Vamos chamar o primeiro somatório de A e o segundo de B . O valor de B é facilmente calculado, pois sabemos que

$$\sum_{k=1}^{n-1} k = \frac{n(n-1)}{2}.$$

Resta então calcular o valor de

$$A = \sum_{k=1}^{n-1} k^2.$$

Para isso, utilizamos o conceito de soma telescópica. Observe que

$$(k+1)^3 = k^3 + 3k^2 + 3k + 1,$$

de modo que

$$(k+1)^3 - k^3 = 3k^2 + 3k + 1.$$

Aplicando o somatório em ambos os lados para $k = 1, \dots, n-1$, obtemos

$$\sum_{k=1}^{n-1} [(k+1)^3 - k^3] = 3 \sum_{k=1}^{n-1} k^2 + 3 \sum_{k=1}^{n-1} k + \sum_{k=1}^{n-1} 1.$$

O lado esquerdo é uma soma telescópica, cujo valor é

$$n^3 - 1.$$

Assim,

$$n^3 - 1 = 3 \sum_{k=1}^{n-1} k^2 + 3 \frac{n(n-1)}{2} + (n-1).$$

Rearranjando os termos, obtemos

$$\sum_{k=1}^{n-1} k^2 = \frac{n^3 - 1}{3} - \frac{n(n-1)}{2} - \frac{n-1}{3}. \quad (**)$$

Substituindo (**) em (*), temos

$$T(n) = \frac{1}{2(n-1)} \left[\frac{n^3 - 1}{3} - \frac{n(n-1)}{2} - \frac{n-1}{3} + \frac{n(n-1)}{2} \right].$$

Observando que $n^3 - 1 = (n-1)(n^2 + n + 1)$, podemos cancelar o fator $(n-1)$, obtendo

$$T(n) = \frac{1}{6}(n^2 + n).$$

Portanto, o custo médio do algoritmo cresce proporcionalmente a n^2 , o que implica que

$$T(n) = O(n^2).$$

2.2 InsertionSort

O *Insertion Sort*, ou *ordenação por inserção*, é um algoritmo de ordenação que, dado um vetor inicial, constrói o vetor ordenado final inserindo um elemento por vez na posição correta. Assim como outros algoritmos de ordenação de complexidade quadrática, ele é bastante eficiente para problemas com entradas pequenas, sendo frequentemente o mais eficiente entre os algoritmos dessa classe.

Uma forma intuitiva de compreender o funcionamento do Insertion Sort é compará-lo com a maneira como algumas pessoas organizam um baralho durante um jogo de cartas. Imagine que você está segurando algumas cartas na mão e que elas já estão ordenadas. Ao receber uma nova carta, você precisa inseri-la na posição correta entre as cartas que já possui, de modo que a ordenação seja mantida.

A cada nova carta recebida, você começa comparando essa carta com as demais cartas da sua mão até encontrar a posição correta para inseri-la. Dependendo do valor da nova carta, ela pode ser maior ou menor do que algumas das cartas já presentes. Assim, você desloca as cartas necessárias até abrir espaço para inserir a nova carta no local adequado.

Depois da inserção, sua mão volta a estar totalmente ordenada. Em seguida, você recebe outra carta e repete o mesmo procedimento. Esse processo se repete até que todas as cartas tenham sido

inseridas. Essa é exatamente a ideia por trás do algoritmo de ordenação por inserção.

No contexto de um vetor, percorremos suas posições começando pelo índice 2. Cada nova posição corresponde a um elemento que precisa ser inserido na sublista já ordenada à esquerda dessa posição.

O pseudo-código do algoritmo é apresentado a seguir.

```
InsertionSort(L, n) {  
    // Percorre cada elemento de L  
    for i = 2 to n {  
        k = i  
        // Insere o pivô na posição correta  
        while k > 1 and L[k] < L[k-1] {  
            swap(L[k], L[k-1])  
            k = k - 1  
        }  
    }  
}
```

Exemplo de execução

Considere o seguinte vetor de entrada:

$[5, 2, 13, 7, -3, 4, 15, 10, 1, 6]$

A seguir apresentamos algumas etapas do processo de ordenação.

1ª passagem

$[5, 2, 13, 7, -3, 4, 15, 10, 1, 6] \rightarrow [2, 5, 13, 7, -3, 4, 15, 10, 1, 6]$

2ª passagem

$[2, 5, 13, 7, -3, 4, 15, 10, 1, 6] \rightarrow [2, 5, 13, 7, -3, 4, 15, 10, 1, 6]$

3ª passagem

$[2, 5, 13, 7, -3, 4, 15, 10, 1, 6] \rightarrow [2, 5, 7, 13, -3, 4, 15, 10, 1, 6]$

4ª passagem

$[2, 5, 7, 13, -3, 4, 15, 10, 1, 6] \rightarrow [-3, 2, 5, 7, 13, 4, 15, 10, 1, 6]$

5ª passagem

$[-3, 2, 5, 7, 13, 4, 15, 10, 1, 6] \rightarrow [-3, 2, 4, 5, 7, 13, 15, 10, 1, 6]$

6ª passagem

$$[-3, 2, 4, 5, 7, 13, 15, 10, 1, 6] \rightarrow [-3, 2, 4, 5, 7, 13, 15, 10, 1, 6]$$

7ª passagem

$$[-3, 2, 4, 5, 7, 13, 15, 10, 1, 6] \rightarrow [-3, 2, 4, 5, 7, 10, 13, 15, 1, 6]$$

8ª passagem

$$[-3, 2, 4, 5, 7, 10, 13, 15, 1, 6] \rightarrow [-3, 1, 2, 4, 5, 7, 10, 13, 15, 6]$$

9ª passagem

$$[-3, 1, 2, 4, 5, 7, 10, 13, 15, 6] \rightarrow [-3, 1, 2, 4, 5, 6, 7, 10, 13, 15]$$

Fim: a garantia de que o vetor está completamente ordenado só é obtida após a execução de todas as passagens do algoritmo.

Análise da complexidade

Para analisar o desempenho do algoritmo Insertion Sort, consideraremos três cenários distintos: **melhor caso, pior caso e caso médio**.

Melhor caso

O melhor caso ocorre quando o vetor já se encontra ordenado. Nesse cenário, cada elemento considerado como pivô já está em sua posição correta e, portanto, nenhuma troca é necessária.

Assim, o laço interno não é executado e, dentro do laço externo, apenas uma verificação é realizada para cada iteração. Dessa forma, o número total de operações é dado por

$$T(n) = \sum_{i=2}^n 1 = n - 1.$$

Portanto, a complexidade do algoritmo no melhor caso é

$$T(n) = O(n).$$

Pior caso

O pior caso ocorre quando o vetor está ordenado em ordem decrescente. Nesse caso, cada novo pivô precisa ser deslocado até o início do vetor.

Assim, o primeiro pivô realiza uma troca, o segundo realiza duas trocas, o terceiro realiza três trocas, e assim sucessivamente. Logo, temos

$$T(n) = \sum_{i=2}^n \left(1 + \sum_{j=1}^{i-1} 2 \right).$$

Expandindo os somatórios, obtemos

$$T(n) = \sum_{i=2}^n 1 + \sum_{i=2}^n \sum_{j=1}^{i-1} 2.$$

O primeiro somatório resulta em

$$\sum_{i=2}^n 1 = n - 1.$$

Já o segundo pode ser escrito como

$$\sum_{i=2}^n 2(i - 1).$$

Assim,

$$T(n) = (n - 1) + 2 \sum_{i=2}^n (i - 1).$$

Sabemos que

$$\sum_{i=2}^n (i - 1) = \sum_{k=1}^{n-1} k = \frac{n(n - 1)}{2}.$$

Substituindo na expressão anterior, obtemos

$$T(n) = (n - 1) + 2 \left(\frac{n(n - 1)}{2} \right).$$

Simplificando,

$$T(n) = n(n - 1) + (n - 1) = n^2 - 1.$$

Portanto, a complexidade assintótica no pior caso é

$$T(n) = O(n^2).$$

Caso médio

Para o caso médio, podemos utilizar uma estratégia semelhante à adotada na análise do Bubble Sort. Assumimos que todas as configurações possíveis da entrada são equiprováveis e calculamos o custo médio do algoritmo.

Nesse caso, podemos escrever

$$T(n) = \frac{1}{n-1} \sum_{k=1}^{n-1} \left[\sum_{i=2}^{k+1} 1 + \sum_{i=2}^{k+1} \sum_{j=1}^{i-1} 2 \right].$$

Chamando a soma entre colchetes de S , temos

$$S = \sum_{i=2}^{k+1} 1 + \sum_{i=2}^{k+1} \sum_{j=1}^{i-1} 2.$$

O primeiro termo resulta em

$$\sum_{i=2}^{k+1} 1 = k.$$

Já o segundo pode ser escrito como

$$\sum_{i=2}^{k+1} 2(i-1).$$

Portanto,

$$S = k + 2 \sum_{i=2}^{k+1} (i-1).$$

Sabemos que

$$\sum_{i=2}^{k+1} (i-1) = \sum_{i=1}^k i = \frac{k(k+1)}{2}.$$

Assim,

$$S = k + 2 \left(\frac{k(k+1)}{2} \right).$$

Simplificando,

$$S = k^2 + 2k.$$

Substituindo esse resultado em $T(n)$, temos

$$T(n) = \frac{1}{n-1} \sum_{k=1}^{n-1} (k^2 + 2k).$$

Separando os somatórios,

$$T(n) = \frac{1}{n-1} \left(\sum_{k=1}^{n-1} k^2 + 2 \sum_{k=1}^{n-1} k \right).$$

Sabemos que

$$\sum_{k=1}^{n-1} k = \frac{n(n-1)}{2}$$

e

$$\sum_{k=1}^{n-1} k^2 = \frac{(n-1)n(2n-1)}{6}.$$

Substituindo esses valores, obtemos uma expressão quadrática em n . Assim, o comportamento assintótico do algoritmo no caso médio é

$$T(n) = O(n^2).$$

Portanto, tanto no pior caso quanto no caso médio o Insertion Sort possui complexidade quadrática, embora seu desempenho seja linear quando a entrada já se encontra ordenada.

2.3 Selection Sort

A *ordenação por seleção* (*Selection Sort*) é um método de ordenação baseado na ideia de selecionar repetidamente o menor elemento da parte ainda não ordenada do vetor e movê-lo para a primeira posição disponível à esquerda. O funcionamento do algoritmo é simples: inicialmente, procura-se o menor valor em todo o vetor e ele é colocado na primeira posição. Em seguida, busca-se o segundo menor valor entre os elementos restantes e ele é colocado na segunda posição. Esse procedimento continua sucessivamente até que todos os elementos estejam ordenados.

Em cada iteração, o algoritmo compara um elemento com todos os demais elementos à sua direita, com o objetivo de encontrar o menor valor. Independentemente da configuração inicial do vetor, o número de comparações realizadas pelo algoritmo é aproximadamente proporcional ao quadrado do tamanho da entrada. Assim, a complexidade do algoritmo é sempre de ordem quadrática.

Entre as principais vantagens do Selection Sort podemos destacar sua simplicidade de implementação, o fato de não utilizar vetor auxiliar (e portanto consumir pouca memória adicional) e o bom desempenho para vetores pequenos. Por outro lado, sua principal desvantagem é a baixa eficiência quando aplicado a vetores grandes, devido ao seu custo quadrático. O pseudo-código do algoritmo é apresentado a seguir.

```
SelectionSort(L, n) {  
    // Percorre todos os elementos de L  
    for i = 1 to n {  
        menor = i  
        // Encontra o menor elemento  
        for k = i+1 to n {  
            if L[k] < L[menor]
```

```

        menor = k
    }
    // Troca a posição do elemento i com o menor
    swap(L[menor], L[i])
}
}

```

Exemplo de execução

O exemplo a seguir ilustra os passos necessários para ordenar o seguinte vetor de entrada:

$[5, 2, 13, 7, -3, 4, 15, 10, 1, 6]$

1ª passagem

$[5, 2, 13, 7, -3, 4, 15, 10, 1, 6] \rightarrow [-3, 2, 13, 7, 5, 4, 15, 10, 1, 6]$

2ª passagem

$[-3, 2, 13, 7, 5, 4, 15, 10, 1, 6] \rightarrow [-3, 1, 13, 7, 5, 4, 15, 10, 2, 6]$

3ª passagem

$[-3, 1, 13, 7, 5, 4, 15, 10, 2, 6] \rightarrow [-3, 1, 2, 7, 5, 4, 15, 10, 13, 6]$

4ª passagem

$[-3, 1, 2, 7, 5, 4, 15, 10, 13, 6] \rightarrow [-3, 1, 2, 4, 5, 7, 15, 10, 13, 6]$

5ª passagem

$[-3, 1, 2, 4, 5, 7, 15, 10, 13, 6] \rightarrow [-3, 1, 2, 4, 5, 7, 15, 10, 13, 6]$

6ª passagem

$[-3, 1, 2, 4, 5, 7, 15, 10, 13, 6] \rightarrow [-3, 1, 2, 4, 5, 6, 15, 10, 13, 7]$

7ª passagem

$[-3, 1, 2, 4, 5, 6, 15, 10, 13, 7] \rightarrow [-3, 1, 2, 4, 5, 6, 7, 10, 13, 15]$

8ª passagem

$[-3, 1, 2, 4, 5, 6, 7, 10, 13, 15] \rightarrow [-3, 1, 2, 4, 5, 6, 7, 10, 13, 15]$

9ª passagem

$$[-3, 1, 2, 4, 5, 6, 7, 10, 13, 15] \rightarrow [-3, 1, 2, 4, 5, 6, 7, 10, 13, 15]$$

Fim: a garantia de que o vetor está completamente ordenado só é obtida após a execução de todas as passagens do algoritmo.

Análise da complexidade

Para analisar o desempenho do algoritmo Selection Sort, consideraremos três cenários distintos: **pior caso, melhor caso e caso médio.**

Pior caso

No pior caso, o segundo laço percorre os elementos do vetor desde $i + 1$ até n . Na primeira iteração temos $i = 1$, na segunda $i = 2$, e assim sucessivamente até $i = n - 1$. Além disso, a atribuição realizada dentro do comando `for` ocorre a cada iteração.

Assim, o número total de operações pode ser expresso por

$$T(n) = \sum_{i=1}^n \left(2 + \sum_{j=i+1}^n 1 \right).$$

Reorganizando os termos, obtemos

$$T(n) = 2 \sum_{i=1}^n 1 + \sum_{i=1}^n \sum_{j=i+1}^n 1.$$

Observe que

$$\sum_{i=1}^n 1 = n.$$

Além disso,

$$\sum_{j=i+1}^n 1 = n - (i + 1) + 1 = n - i.$$

Portanto,

$$T(n) = 2n + \sum_{i=1}^n (n - i).$$

Separando os termos do somatório,

$$T(n) = 2n + \sum_{i=1}^n n - \sum_{i=1}^n i.$$

Sabemos que

$$\sum_{i=1}^n n = n^2$$

e

$$\sum_{i=1}^n i = \frac{n(n+1)}{2}.$$

Substituindo esses resultados, obtemos

$$T(n) = 2n + n^2 - \frac{n(n+1)}{2}.$$

Simplificando,

$$T(n) = \frac{n^2}{2} + \frac{3n}{2}.$$

Assim, a complexidade assintótica é

$$T(n) = O(n^2).$$

Melhor caso

Mesmo quando o vetor já está ordenado, o laço interno do algoritmo Selection Sort precisa percorrer todos os elementos restantes do vetor para verificar se existe algum valor menor.

Uma característica importante do comando `for` é que ele possui uma instrução de atribuição implícita. Por exemplo,

```
for i = 1 to n
  print(i)
```

é equivalente a

```
i = 1
while i <= n {
  print(i)
  i = i + 1
}
```

Assim, mesmo no melhor caso, todas as comparações ainda precisam ser realizadas. Portanto, o número total de operações permanece proporcional a n^2 . Logo, a complexidade no melhor caso também é

$$T(n) = O(n^2).$$

Caso médio

Como tanto o melhor caso quanto o pior caso possuem complexidade quadrática, o comportamento médio do algoritmo também será da mesma ordem. Portanto, para entradas aleatórias, o custo médio do Selection Sort também cresce quadraticamente com o tamanho da entrada, isto é,

$$T(n) = O(n^2).$$

2.4 Shellsort

O algoritmo *Shellsort* foi proposto originalmente por :contentReference[oaicite:0]index=0 como uma otimização do método *Insertionsort*. Uma das principais limitações do algoritmo *Insertionsort* é a quantidade de comparações necessárias para encontrar a posição correta de um elemento no vetor. Por exemplo, quando o menor elemento está localizado na última posição do vetor (pior caso), são necessárias $n - 1$ comparações e trocas para movê-lo até o início da lista. Esse comportamento torna o algoritmo pouco eficiente quando o número de elementos n é grande.

A ideia central do algoritmo *Shellsort* consiste em permitir trocas entre elementos que estão distantes entre si no vetor, introduzindo o conceito de *gap*. Considere um valor inteiro h , denominado *gap*. Os elementos separados por h posições são comparados e rearranjados de modo a formar subsequências ordenadas. Dizemos então que o vetor está ***h-ordenado***.

Durante a execução do algoritmo, o valor do *gap* h é gradualmente reduzido até atingir o valor 1. Quando isso ocorre, o vetor torna-se completamente ordenado. Em outras palavras, quando a sequência está 1-ordenada, o algoritmo termina.

É possível observar que o algoritmo *Insertionsort* pode ser interpretado como um caso particular do *Shellsort* quando $h = 1$. No entanto, no *Shellsort*, quando finalmente chegamos a esse valor de h , a lista já se encontra parcialmente ordenada, pois diversas trocas foram realizadas anteriormente utilizando valores maiores de h . Dessa forma, apenas um pequeno número de operações adicionais é necessário para concluir a ordenação.

Portanto, a lógica fundamental utilizada pelo *Shellsort* é a mesma do *Insertionsort*: inserir cada elemento na posição correta dentro de um conjunto ordenado. A diferença é que, ao permitir trocas entre elementos distantes, o algoritmo consegue reduzir significativamente o número total de operações necessárias para ordenar o vetor.

Exemplo de execução do Shellsort

Considere o vetor inicial

$$[5, 2, 13, 7, -3, 4, 15, 10, 1, 6].$$

Neste exemplo utilizaremos a sequência de *gaps* originalmente proposta por Shell, na qual o valor de h é sucessivamente dividido por 2:

$$h = \frac{n}{2}, \frac{n}{4}, \frac{n}{8}, \dots, 1.$$

Como $n = 10$, temos a sequência de gaps

$$h = 5, 2, 1.$$

1ª iteração: $h = 5$ Com $h = 5$, comparamos elementos separados por cinco posições. Isso gera as seguintes subsequências (de índices):

$$(0, 5), (1, 6), (2, 7), (3, 8), (4, 9).$$

Aplicando o princípio do Insertionsort em cada subsequência:

$$[5, 2, 13, 7, -3, 4, 15, 10, 1, 6]$$

$$[4, 2, 13, 7, -3, 5, 15, 10, 1, 6] \quad (5 > 4 \Rightarrow troca)$$

$$[4, 2, 10, 7, -3, 5, 15, 13, 1, 6] \quad (13 > 10 \Rightarrow troca)$$

$$[4, 2, 10, 1, -3, 5, 15, 13, 7, 6] \quad (7 > 1 \Rightarrow troca)$$

Após processar todas as subsequências, o vetor torna-se parcialmente ordenado.

2ª iteração: $h = 2$ Agora os elementos são comparados com distância 2. As subsequências formadas são:

$$(0, 2, 4, 6, 8) \quad (1, 3, 5, 7, 9).$$

Aplicando novamente o mesmo princípio de inserção dentro de cada subsequência:

$$[4, 2, 10, 1, -3, 5, 15, 13, 7, 6]$$

$$[-3, 2, 4, 1, 7, 5, 10, 13, 15, 6]$$

$$[-3, 1, 4, 2, 7, 5, 10, 6, 15, 13]$$

Nesse ponto o vetor já está bastante próximo de estar completamente ordenado.

3ª iteração: $h = 1$ Quando $h = 1$, o algoritmo torna-se equivalente ao *Insertionsort*. Entretanto, como o vetor já está parcialmente ordenado pelas etapas anteriores, poucas trocas são necessárias.

$[-3, 1, 4, 2, 7, 5, 10, 6, 15, 13]$

$[-3, 1, 2, 4, 5, 6, 7, 10, 13, 15]$

Esse exemplo ilustra como o Shellsort reduz gradualmente a desordem do vetor utilizando diferentes valores de *gap*. Quando finalmente chegamos a $h = 1$, o vetor já está quase ordenado, tornando a etapa final extremamente eficiente.

A seguir apresentamos o algoritmo *Shellsort* em sua forma padrão. A ideia central consiste em iniciar com um valor de *gap* relativamente grande e reduzi-lo progressivamente até atingir o valor 1. Para cada valor de *gap*, executa-se uma variação do algoritmo *Insertionsort*, considerando apenas elementos separados por *gap* posições. Dessa forma, cada subsequência formada por esse intervalo é ordenada localmente.

```
ShellSort(L, n) {
    // iniciamos com um gap grande e depois reduzimos
    gap = n//2
    while gap > 0 {
        // realiza um Insertionsort utilizando o gap atual
        for i = gap to n-1 {
            // elemento a ser inserido na subsequência
            j = i
            // encontra a posição correta considerando o gap
            while j >= gap and L[j-gap] > L[j] {
                swap(L[j], L[j-gap])
                j = j - gap
            }
        }
        // diminui o valor do gap
        gap = gap//2
    }
}
```

Observe que não é necessário iniciar o laço interno em $i = 0$, pois os primeiros elementos de cada subsequência já estão trivialmente ordenados. Assim, o algoritmo começa em $i = \text{gap}$, inserindo gradualmente os demais elementos em suas posições corretas dentro de cada subsequência. Quando o valor de *gap* finalmente se torna igual a 1, o procedimento equivale a executar o algoritmo *Insertionsort* sobre um vetor que já está parcialmente ordenado, o que torna essa última etapa bastante eficiente.

Análise da Complexidade

Como o algoritmo *Shellsort* depende essencialmente da sequência de *gaps* utilizada durante o processo de ordenação, não existe uma única complexidade associada ao algoritmo. Diferentes sequências de *gaps* podem levar a desempenhos bastante distintos. De fato, a análise rigorosa do comportamento do Shellsort no caso médio ainda permanece um problema em aberto para várias sequências de *gaps*. Para simplificar a exposição, consideraremos nesta análise a sequência original proposta por Shell, na qual o valor do *gap* é sucessivamente dividido por dois.

Melhor caso O melhor caso ocorre quando a lista de entrada já se encontra ordenada. Nessa situação, o laço `while` interno responsável pelas trocas não é executado nenhuma vez. Restam apenas os comandos estruturais do algoritmo:

```
gap = n//2
while gap > 0 {
    for i = gap to n
        j = i
        gap = gap//2
}
```

Observe que o valor de *gap* é reduzido sucessivamente por divisões por 2. Portanto, o número de iterações do laço externo `while` é aproximadamente

$$\log_2 n.$$

Como em cada iteração percorremos o vetor de tamanho n , o custo total pode ser estimado por

$$T(n) = n \log n + \log n.$$

Assim, a complexidade assintótica no melhor caso é

$$T(n) = O(n \log n).$$

Pior caso O pior caso ocorre quando a lista está em ordem decrescente. Nessa situação, o número máximo de trocas depende diretamente do valor do *gap*. Inicialmente temos

$$\frac{n}{2} \text{ sequências de tamanho } 2,$$

depois

$$\frac{n}{4} \text{ sequências de tamanho } 4,$$

depois

$\frac{n}{8}$ sequências de tamanho 8,

e assim sucessivamente. Podemos aproximar o número total de trocas por

$$T(n) = \left(\frac{n}{2}\right) 1 + \left(n - \frac{n}{4}\right) 3 + \left(n - \frac{n}{8}\right) 7 + \cdots + \left(n - \frac{n}{2^k}\right) (2^k - 1).$$

Aplicando a distributiva e reorganizando os termos, obtemos

$$T(n) = \frac{n}{2} (1 + 3 + 7 + 15 + \cdots + (2^k - 1)).$$

Denotando o somatório entre parênteses por S ,

$$S = \sum_{i=1}^k \frac{2^i - 1}{2^i}.$$

Esse somatório pode ser manipulado algebricamente e resulta em

$$S = 2(2^k - 1) - 2k + \left(1 - \frac{1}{2^k}\right).$$

A condição de parada do algoritmo ocorre quando $gap = 1$. Como a sequência de $gaps$ é obtida por divisões sucessivas por dois, temos

$$n = 2^k \quad \Rightarrow \quad k = \log_2 n.$$

Substituindo esse resultado na expressão anterior, chegamos a

$$S = 2(n - 1) - 2 \log_2 n + 1 - \frac{1}{n}.$$

Como $T(n) = nS$, obtemos

$$T(n) = 2n^2 - 2n - 2n \log_2 n + n - 1.$$

Assim, a complexidade assintótica no pior caso é

$$T(n) = O(n^2).$$

Caso médio No caso médio podemos adotar uma aproximação semelhante à utilizada no pior caso, assumindo que aproximadamente metade das trocas possíveis seja realizada. Dessa forma,

$$T(n) = \left(\frac{n}{2}\right) 1 + \left(n - \frac{n}{4}\right) 2 + \left(n - \frac{n}{8}\right) 4 + \cdots + \left(n - \frac{n}{2^k}\right) 2^{k-1}.$$

Após reorganizar os termos, obtemos

$$T(n) = \frac{n}{2} (1 + 3 + 7 + 15 + \dots + (2^k - 1)).$$

Sabemos que o somatório

$$A = 1 + 3 + 7 + 15 + \dots + (2^k - 1)$$

pode ser aproximado por

$$A = 2(n - 1) - \log_2 n.$$

Logo,

$$T(n) = \frac{n}{2} [2(n - 1) - \log_2 n] = n^2 - n - \frac{n}{2} \log_2 n.$$

Portanto, a complexidade assintótica do Shellsort no caso médio, usando a sequência de *gaps* original de Shell, também é

$$T(n) = O(n^2).$$

Sequências de *gap*

Após a proposta original do algoritmo *Shellsort*, diversos estudos propuseram novas formas de definir os valores de *gap* utilizados nas iterações do algoritmo. O consenso geral é que, a cada passo, o valor do *gap* h deve ser reduzido progressivamente, até que na última iteração ele atinja o menor valor possível, isto é, $h = 1$.

Diversas estratégias alternativas para a sequência de *gaps* mostraram que é possível melhorar significativamente o desempenho do algoritmo em termos de complexidade computacional. A seguir apresentamos algumas das sequências de *gaps* propostas posteriormente à descoberta do algoritmo.

1. Sequência de Shell

$$\frac{n}{2}, \frac{n}{4}, \frac{n}{8}, \dots, 1$$

Essa é a sequência original proposta por Donald Shell e consiste em divisões sucessivas do tamanho da lista por 2.

2. Sequência de Hibbard

$$1, 3, 7, \dots, 2^k - 1 \quad (k \geq 1)$$

A ideia consiste em gerar os termos da sequência até que o maior valor seja menor ou igual a $n/2$. Durante a execução do algoritmo, os valores de h são utilizados do maior para o menor.

Com essa sequência, pode-se mostrar que o pior caso possui complexidade

$$O(n^{3/2}).$$

3. Sequência de Knuth

$$1, 4, 13, 40, \dots$$

Os termos dessa sequência são definidos pela recorrência

$$h_k = \frac{3^k - 1}{2},$$

sendo $k \geq 1$ e escolhendo-se o maior valor que seja menor que $n/3$. Assim como nas outras sequências, os valores de h são utilizados do maior para o menor. O pior caso também apresenta complexidade

$$O(n^{3/2}).$$

4. Sequência de Pratt

$$1, 2, 3, 4, 6, 8, 9, 12, \dots$$

Essa sequência é formada por números da forma

$$2^p 3^q,$$

ou seja, inteiros cujos fatores primos são apenas 2 e 3. Novamente, os valores de *gap* são utilizados do maior para o menor. Nesse caso, pode-se mostrar que o pior caso possui complexidade

$$O(n(\log n)^2).$$

5. Incrementos de Sedgewick

$$1, 5, 19, 41, 109, \dots$$

Essa sequência pode ser definida pelas expressões

$$h_k = \begin{cases} 9(2^k - 2^{k/2}) + 1, & \text{se } k \text{ é par} \\ 8 \cdot 2^k - 6 \cdot 2^{(k+1)/2} + 1, & \text{se } k \text{ é ímpar} \end{cases}$$

Os valores são novamente gerados até que o maior termo seja menor ou igual a $n/2$, sendo utilizados do maior para o menor durante a execução do algoritmo. Para essa sequência, o pior caso é dado por

$$O(n^{4/3}).$$

Existem diversas outras sequências de *gaps* propostas na literatura; apresentamos aqui apenas algumas das mais conhecidas. Entretanto, com sequências mais sofisticadas, frequentemente não é possível obter expressões analíticas simples para o custo $T(n)$.

É interessante observar que diferentes sequências de *gaps* podem produzir complexidades de pior caso significativamente melhores que $O(n^2)$. Por exemplo, utilizando a sequência de Hibbard obtém-se complexidade $O(n^{3/2})$; com a sequência de Pratt obtém-se $O(n(\log n)^2)$; e com a sequência de Sedgewick obtém-se $O(n^{4/3})$.

Conforme mencionado anteriormente, determinar a complexidade do *Shellsort* no caso médio para sequências de *gaps* diferentes da sequência original de Shell permanece um problema em aberto. Resultados teóricos indicam que essa complexidade pode variar entre

$$O(n^{5/4}) \quad \text{e} \quad O(n(\log n)^2).$$

2.5 Heapsort

Veremos a seguir como empregar estruturas de dados do tipo *heap* para projetar um algoritmo de ordenação eficiente, conhecido como *Heapsort*. Ele pode ser entendido como uma otimização do Selectionsort, em que o processo de encontrar o mínimo do array é realizado de maneira mais eficiente utilizando um heap.

A estrutura de dados Heap

Um *Heap* é uma estrutura de dados especial baseada em **árvores binárias completas**. Uma árvore binária completa é aquela em que todos os níveis estão completamente preenchidos, exceto possivelmente o último, que é preenchido da esquerda para a direita.

Uma árvore binária completa é considerada uma estrutura de dados do tipo *heap* quando satisfaz uma das seguintes propriedades:

1. Todo nó i é menor ou igual aos seus dois filhos (*min-heap*); ou
2. Todo nó i é maior ou igual aos seus dois filhos (*max-heap*).

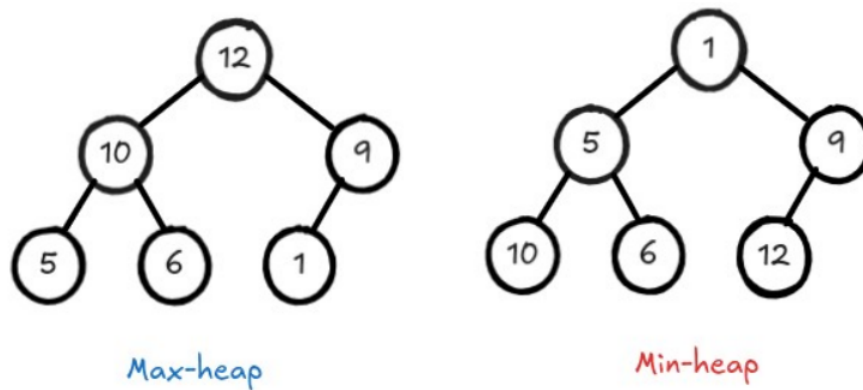


Figura 4: Exemplo de estruturas do tipo min-heap e max-heap.

No caso de um *min-heap*, o menor elemento da estrutura encontra-se na raiz da árvore. Já no caso de um *max-heap*, o maior elemento está localizado na raiz.

Em implementações práticas, um heap é normalmente armazenado em um vetor. Se $T[i]$ denota o i -ésimo elemento (ou nó) do heap com n elementos, então as relações entre pais e filhos podem ser obtidas diretamente por meio de operações aritméticas sobre os índices:

- Filho à esquerda: $T[2i + 1]$, se $2i + 1 \leq n$;
- Filho à direita: $T[2i + 2]$, se $2i + 2 \leq n$;
- Pai: $T[\lfloor \frac{i-1}{2} \rfloor]$, onde $\lfloor \cdot \rfloor$ denota a função piso.

Essa representação permite implementar árvores binárias completas de forma extremamente eficiente em memória, sem a necessidade de ponteiros.

Uma pergunta natural que surge nesse contexto é: *como transformar um vetor arbitrário em um min-heap*? Para isso, utilizamos um procedimento denominado *heapificação*, geralmente implementado por meio da função `heapify`, que reorganiza os elementos do vetor de forma a satisfazer a propriedade de heap.

Como heapificar um array/árvore?

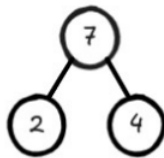
A partir de uma árvore binária completa, podemos modificá-la para se tornar um *max-heap* executando uma função chamada `heapify` em todos os elementos que não são folhas. Para isso, assume-se que a árvore está representada na forma de um vetor.

Como a função `heapify` utiliza recursão, seu funcionamento pode parecer um pouco complexo à primeira vista. Para compreender melhor sua lógica, podemos começar analisando um exemplo simples com apenas três elementos.

Considere uma árvore com três nós. Podemos ter dois cenários distintos. No primeiro caso, a raiz já é o maior elemento da árvore. Nesse cenário, não é necessário realizar nenhuma operação, pois a propriedade de *max-heap* já está satisfeita. Esse corresponde ao caso base da recursão, no qual atingimos a condição de parada.

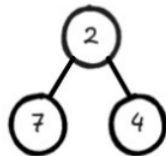
No segundo cenário, um dos filhos possui valor maior que o valor da raiz. Nesse caso, precisamos realizar uma troca entre a raiz e o maior de seus filhos para restaurar a propriedade de *max-heap*.

Cenário 1



pai já é maior que os 2 filhos

Cenário 2



substituir o pai pelo maior dos 2 filhos

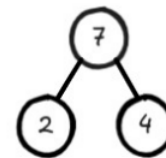
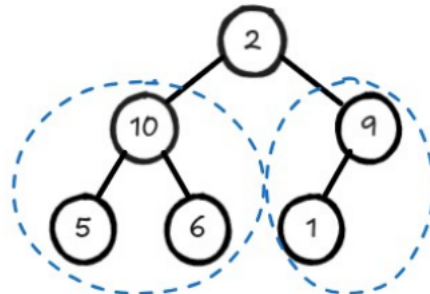


Figura 5: Para criar um max-heap, devemos substituir o pai pelo maior dos 2 filhos.

Agora considere um cenário um pouco mais geral, em que o heap possui mais de um nível. Suponha que a raiz viole a propriedade de *max-heap*, mas que as duas subárvores (à esquerda e à direita) já sejam *max-heaps*.



Ambas subárvores são max-heaps

Figura 6: Essa substituição é válida apenas se as subárvores a esquerda e a direita forem max-heaps.

Para restaurar a propriedade do heap em toda a árvore, precisamos empurrar o elemento da raiz para baixo até que ele atinja sua posição correta.

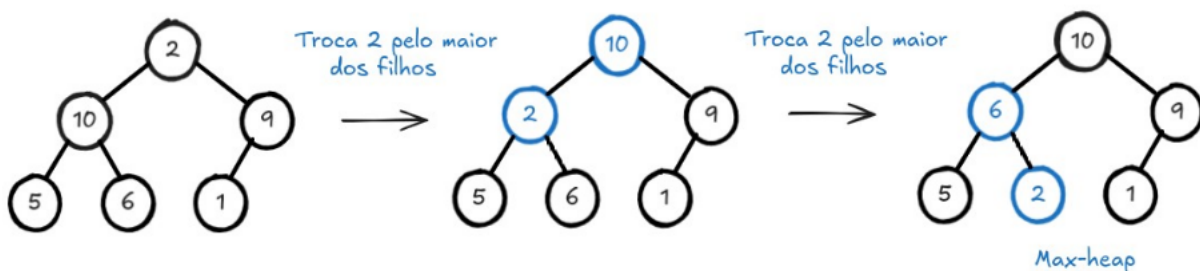


Figura 7: A função *heapify* deve continuar recursivamente até que a chave atinja sua posição correta no heap.

Em outras palavras, para manter a propriedade de *max-heap* em uma árvore onde ambas as subárvores já são *max-heaps*, precisamos executar o procedimento `heapify` repetidamente no elemento da raiz até que ele seja maior que seus filhos ou até que se torne um nó folha. A função a seguir ilustra esse procedimento.

```
// Função recursiva para heapificar um elemento da árvore
heapify(L, n, i) {
    // Encontra o maior entre o nó raiz i e os filhos
    largest = i
    l = 2 * i + 1
    r = 2 * i + 2
    // Se filho a esquerda é maior que pai
    if l < n and L[i] < L[l]
        largest = l
    // Se filho a direita é maior que o maior
    if r < n and L[largest] < L[r]
        largest = r
    // Se o nó raiz i não é o maior, troca e continua heapify
    if largest != i {
        swap(L[i], L[largest])
        heapify(L, n, largest)
    }
}
```

Essa função funciona tanto para o caso base quanto para árvores de qualquer tamanho, pois o parâmetro i define qual elemento deve ser processado. Dessa forma, é possível mover o elemento da raiz para sua posição correta, preservando a propriedade de *max-heap*, desde que as subárvores já satisfaçam essa propriedade.

Entretanto, se as subárvores à esquerda ou à direita da raiz não forem *max-heaps*, uma única chamada da função `heapify` não será suficiente para transformar um vetor arbitrário em um heap.

Surge então a questão: como construir um *max-heap* a partir de um array arbitrário? O índice do último elemento que não é folha em uma árvore binária completa é dado por

$$\left\lfloor \frac{n}{2} \right\rfloor - 1,$$

pois a cada nível de uma árvore binária o número de nós dobra. Por exemplo, em uma árvore binária completa com $n = 15$ nós, o índice do último nó que não é folha é

$$\frac{15}{2} - 1 = 7 - 1 = 6.$$

Já em uma árvore binária completa com $n = 8$ nós, o índice do último nó que não é folha é

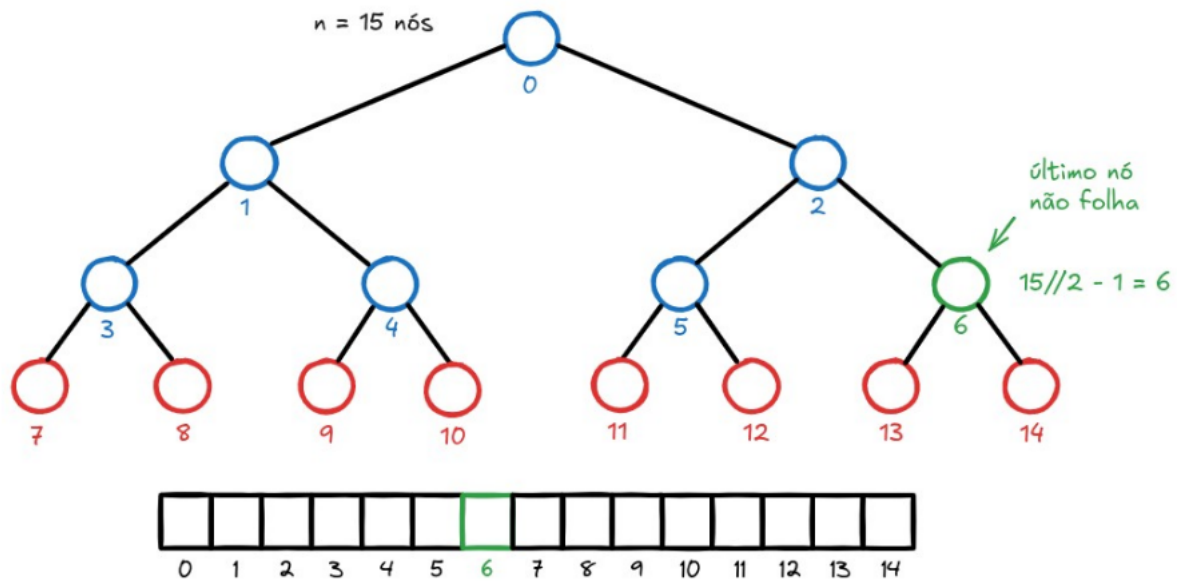


Figura 8: Repare que o elemento de índice 6 representa o último nó não folha da árvore.

$$\frac{8}{2} - 1 = 4 - 1 = 3.$$

Esses exemplos ilustram como podemos identificar os nós internos da árvore, que são justamente aqueles sobre os quais o procedimento `heapify` deve ser aplicado para construir um heap a partir de um vetor arbitrário.

Dessa forma, a função responsável por construir um *max-heap* a partir de um array arbitrário pode ser descrita pelo seguinte pseudo-código:

```
// Constrói max heap
build_max_heap(L, n) {
    for i = n//2 - 1 downto 0
        heapify(L, n, i)
    return L
}
```

O algoritmo *Heapsort* pode ser entendido como uma otimização do algoritmo *Insertion Sort*. A ideia central consiste em utilizar a estrutura de dados heap para encontrar rapidamente o maior elemento do conjunto e posicioná-lo na posição correta do vetor. De forma resumida, o funcionamento do algoritmo pode ser descrito pelos seguintes passos:

1. Inicialmente, o vetor de entrada é transformado em um *max-heap*. Como a árvore representada na forma vetorial passa a satisfazer a propriedade de *max-heap*, o maior elemento estará localizado na raiz da árvore (índice 0).
2. **Swap**: o elemento da raiz é trocado com o último elemento do vetor.

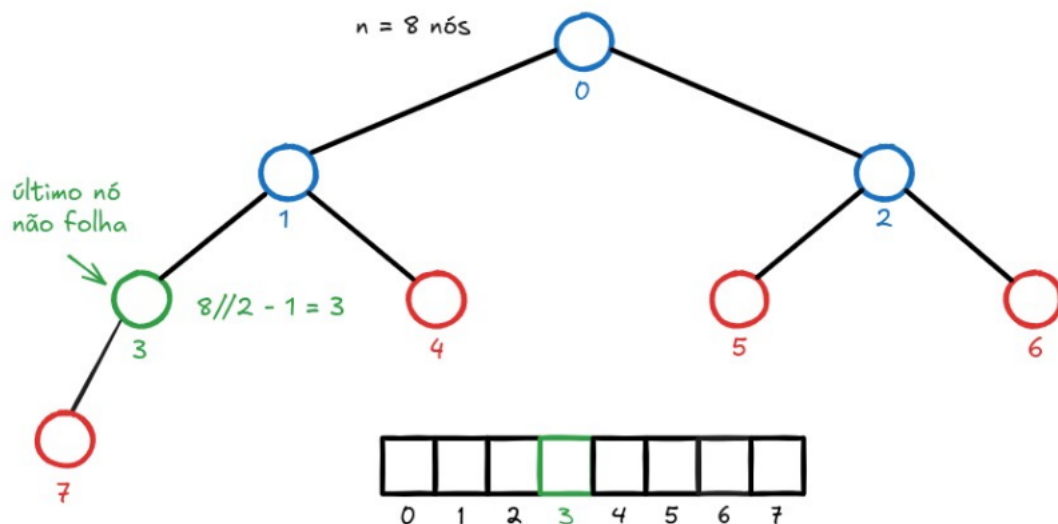


Figura 9: Repare que o elemento de índice 3 representa o último nó não folha da árvore.

3. **Remove:** o tamanho efetivo do heap é reduzido em uma unidade, removendo da estrutura o elemento que acabou de ser colocado na posição final do vetor.
4. **Heapify:** aplica-se a função `heapify` para restaurar a propriedade de *max-heap*, trazendo novamente o maior elemento para a raiz da árvore.
5. Repete-se o processo dos passos 2 a 4 até que todos os elementos tenham sido processados e o vetor esteja completamente ordenado.

A função a seguir apresenta o pseudo-código do algoritmo *Heapsort*, utilizando as funções auxiliares descritas anteriormente.

```
heapsort(L, n) {
    // Constrói max heap
    L = build_max_heap(L, n)
    for i = n-1 downto 1 {
        // Troca
        swap(L[i], L[0])
        // Heapifica o elemento raiz
        heapify(L, i, 0)
    }
}
```

As figuras a seguir ilustram o funcionamento do algoritmo em um exemplo didático.

Considerando o max-heap criado anteriormente, iremos aplicar o algoritmo Heapsort no vetor inicial. As figuras a seguir ilustram o passo a passo do método.

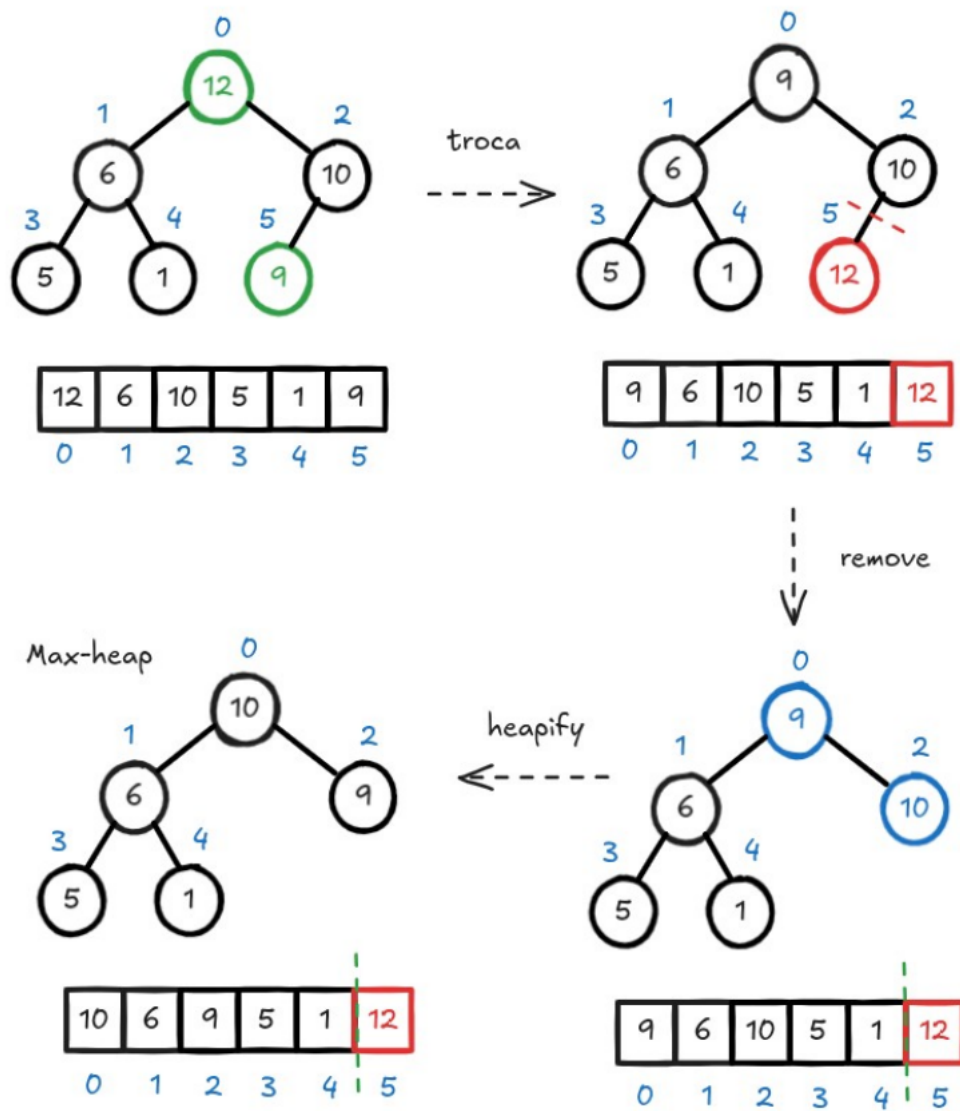
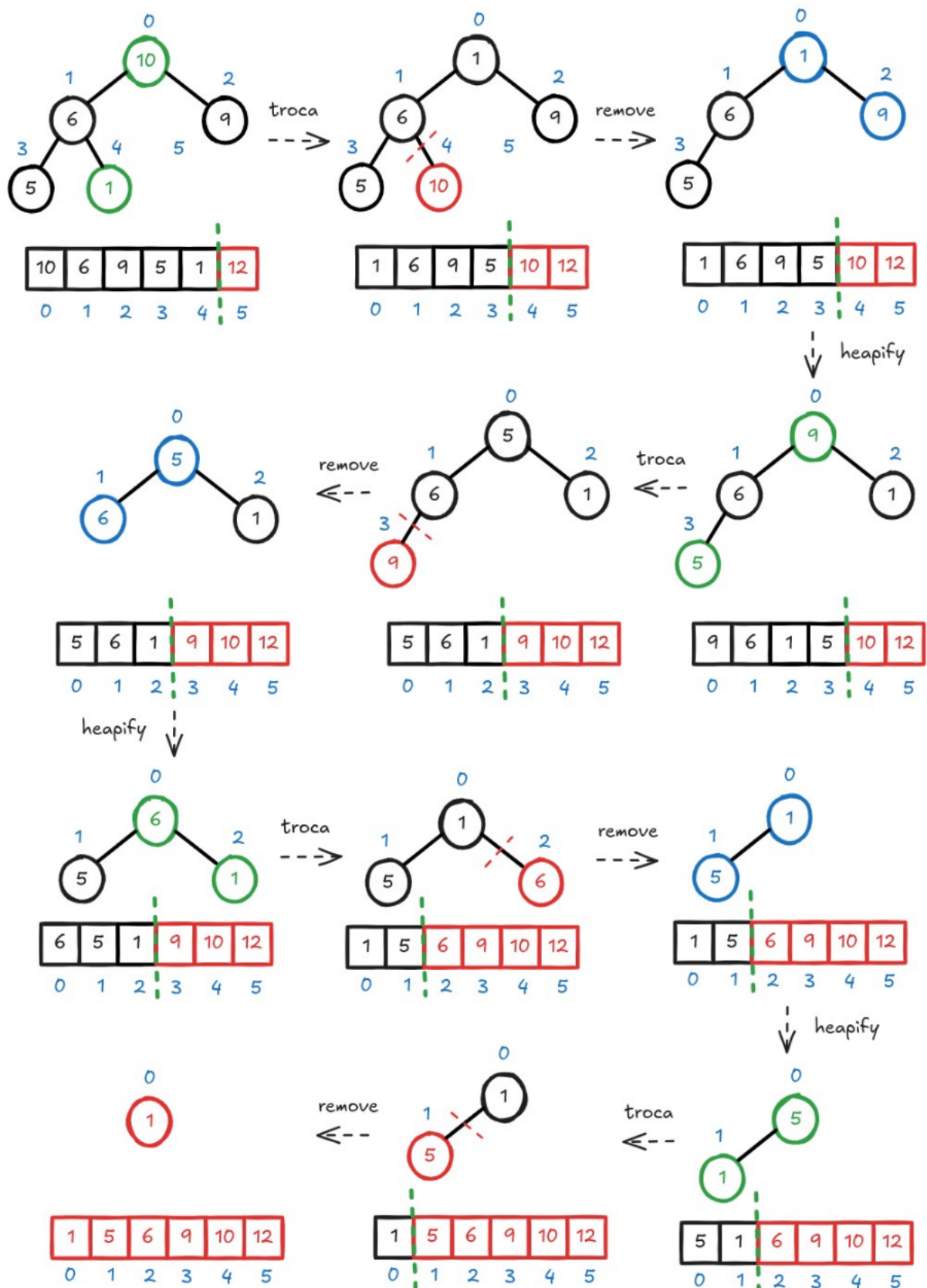


Figura 10: O primeiro passo do heapsort consiste em construir um max-heap.



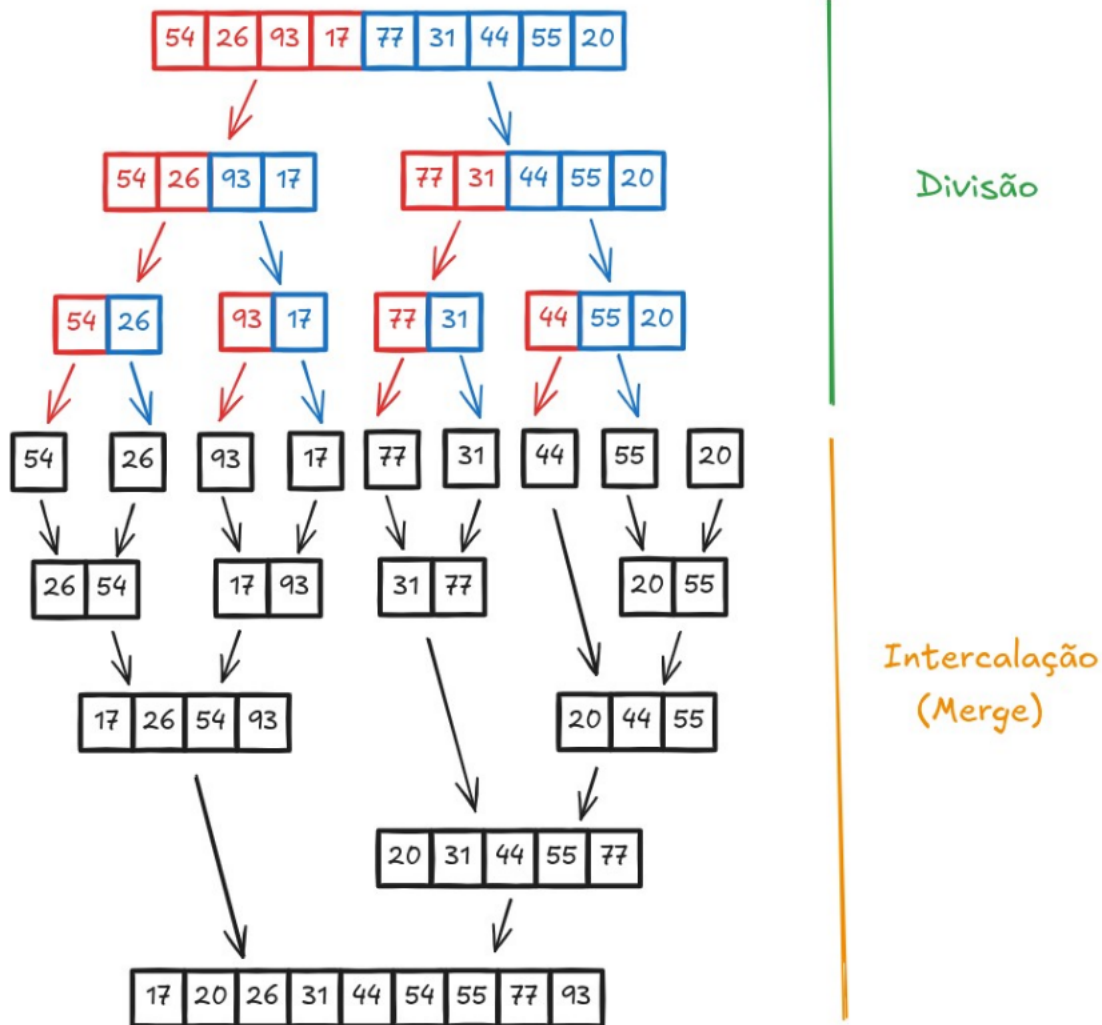


Figura 12: Ilustração gráfica do algoritmo mergesort: primeiro divide-se recursivamente o array em sublistas unitárias e depois é realizada a intercalação das listas duas a duas.

2.6 Mergesort

O algoritmo *Mergesort* utiliza a abordagem conhecida como *Dividir para Conquistar*. A ideia básica consiste em dividir um problema grande em vários subproblemas menores, resolver cada um desses subproblemas recursivamente e, em seguida, combinar as soluções obtidas para construir a solução do problema original. Trata-se de um algoritmo recursivo que divide uma lista continuamente pela metade. Se a lista estiver vazia ou possuir apenas um elemento, então ela já está ordenada por definição, caracterizando o *caso base* da recursão.

Caso a lista possua mais de um elemento, ela é dividida em duas metades aproximadamente iguais. Em seguida, o algoritmo *Merge Sort* é invocado recursivamente para ordenar cada uma dessas metades. Após as duas sublistas estarem ordenadas, realiza-se a operação fundamental do algoritmo, denominada *intercalação (merge)*. Intercalar consiste no processo de combinar duas listas menores já ordenadas para formar uma nova lista única e também ordenada. Dessa forma, o algoritmo possui duas fases principais: a fase de divisão e a fase de intercalação.

Exemplo de execução

Considere o vetor de entrada

$[5, 2, 13, 7, -3, 4, 15, 10, 1, 6]$.

O algoritmo Merge Sort possui duas fases principais: **divisão** (*divide*) e **intercalação** (*merge*). A seguir mostramos cada uma dessas fases.

Passo 1: Divisão recursiva Inicialmente o vetor é dividido recursivamente até que todas as sublistas possuam apenas um elemento.

Nível 0

$[5, 2, 13, 7, -3, 4, 15, 10, 1, 6]$

Nível 1

$[5, 2, 13, 7, -3]$ $[4, 15, 10, 1, 6]$

Nível 2

$[5, 2]$ $[13, 7, -3]$ $[4, 15]$ $[10, 1, 6]$

Nível 3

$[5]$ $[2]$ $[13]$ $[7, -3]$ $[4]$ $[15]$ $[10]$ $[1, 6]$

Nível 4 (caso base)

$[5]$ $[2]$ $[13]$ $[7]$ $[-3]$ $[4]$ $[15]$ $[10]$ $[1]$ $[6]$

Neste ponto, todas as sublistas possuem apenas um elemento e estão ordenadas por definição.

Passo 2: Intercalação (Merge) Agora começa o processo de reconstrução da lista por meio da intercalação de sublistas ordenadas.

Nível 3

$[7]$ $[-3] \rightarrow [-3, 7]$

$[1]$ $[6] \rightarrow [1, 6]$

Nível 2

$[5]$ $[2] \rightarrow [2, 5]$

$[13]$ $[-3, 7] \rightarrow [-3, 7, 13]$

$[4]$ $[15] \rightarrow [4, 15]$

$$[10] [1, 6] \rightarrow [1, 6, 10]$$

Nível 1

$$[2, 5] [-3, 7, 13] \rightarrow [-3, 2, 5, 7, 13]$$

$$[4, 15] [1, 6, 10] \rightarrow [1, 4, 6, 10, 15]$$

Nível 0 (resultado final)

$$[-3, 2, 5, 7, 13] [1, 4, 6, 10, 15] \rightarrow [-3, 1, 2, 4, 5, 6, 7, 10, 13, 15]$$

Ao final do processo de intercalação, obtemos o vetor completamente ordenado. A seguir apresentamos o algoritmo merge, responsável por intercalar duas sublistas ordenadas. O custo computacional dessa operação é $O(n)$.

```
merge(A, low, q, high) {
    // q é o ponto médio
    nL = q - low + 1 // número de elementos da sublista esquerda
    nR = high - q    // número de elementos da sublista direita
    int L[0...nL-1]  // sublista a esquerda
    int R[0...nR-1]  // sublista a direita
    // Copia elementos para L
    for i = 0 to nL-1
        L[i] = A[low + i]
    // Copia elementos para R
    for j = 0 to nR-1
        R[j] = A[(q+1) + j]
    // Inicializa variáveis
    i = 0
    j = 0
    k = 0
    // Loop principal
    while i < nL and j < nR {
        if L[i] <= R[j] {
            A[k] = L[i]
            i = i + 1
        }
        else {
            A[k] = R[j]
            j = j + 1
        }
    }
```

```

    k = k + 1
}
// se sobrar elementos em L
while i < nL {
    A[k] = L[i]
    i = i + 1
    k = k + 1
}
# se sobrar elementos em R
while j < nR {
    A[k] = R[j]
    j = j + 1
    k = k + 1
}
}

```

A figura a seguir ilustra o funcionamento da função merge.

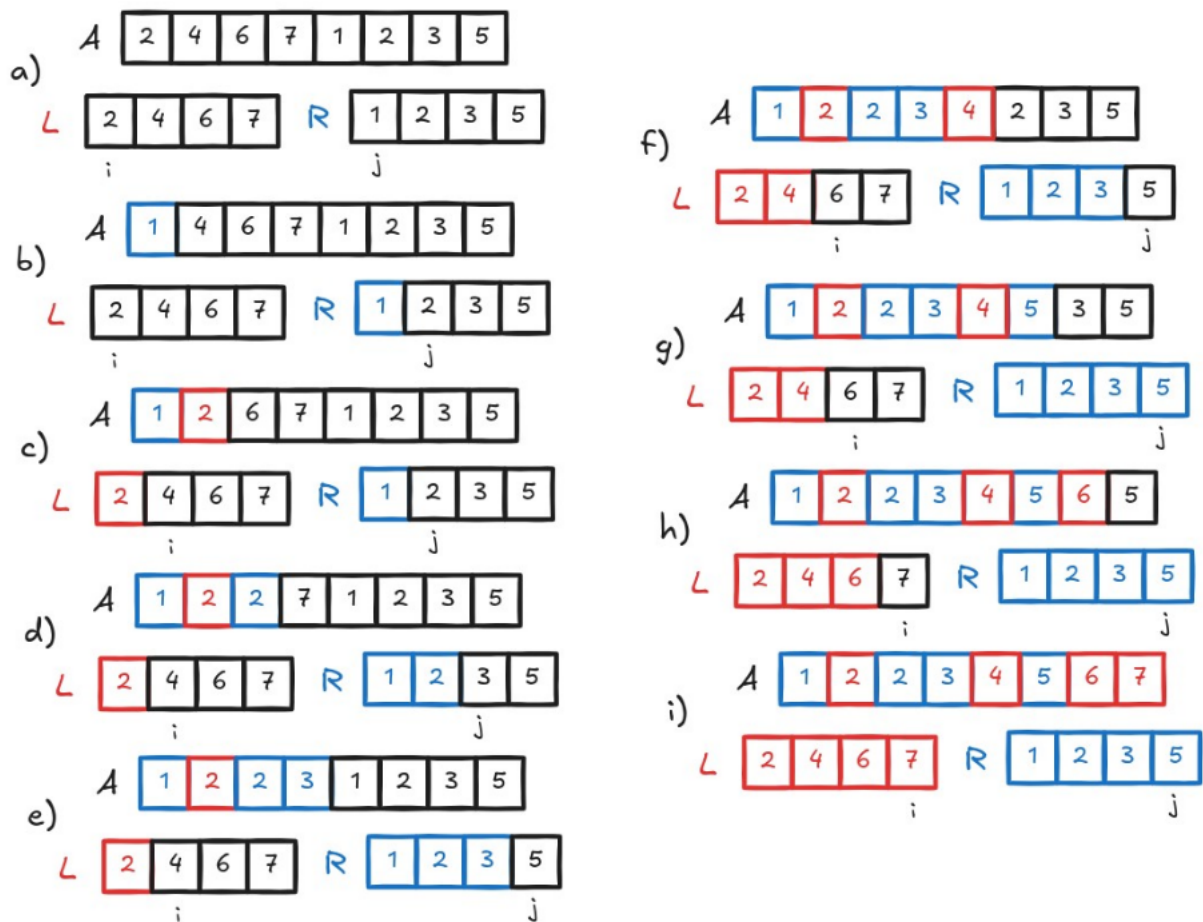


Figura 13: Ilustração gráfica do processo de intercalação de dois arrays (merge).

A seguir é apresentada a função mergesort, responsável por realizar a ordenação de uma lista

utilizando a estratégia de *dividir para conquistar*.

```
mergesort(A, low, high) {  
    // condição de parada: lista tem zero ou um elemento  
    if low >= high  
        return  
    // calcula o meio  
    q = floor((low + high)/2)  
    // Aplica mergesort recursivamente nas metades  
    mergesort(A, low, q)  
    mergesort(A, q + 1, high)  
    // Após a recursão faz o merge  
    merge(A, low, q, high)  
}
```

A função `mergesort` apresentada acima começa verificando o *caso base* da recursão. Se o tamanho da lista for menor ou igual a um elemento, então a lista já está ordenada por definição e nenhum processamento adicional é necessário. Caso contrário, isto é, se o tamanho da lista for maior do que um, dividimos o vetor em duas partes aproximadamente iguais. Em seguida, aplicamos recursivamente o algoritmo `mergesort` à metade esquerda e à metade direita do vetor.

É importante observar que a lista pode não possuir um número par de elementos. Isso, contudo, não representa um problema, pois a diferença de tamanho entre as duas sublistas será de no máximo um elemento. Quando a função `mergesort` retorna das chamadas recursivas, após processar a sublista esquerda (LE) e a sublista direita (LD), ambas já se encontram ordenadas. Nesse momento, a etapa final do algoritmo consiste em intercalar essas duas listas ordenadas menores para formar uma única lista maior também ordenada.

A operação de intercalação é realizada pela função `merge`, que insere os elementos de volta na lista original *A* um a um, selecionando repetidamente o menor elemento entre as duas listas ordenadas.

Análise da complexidade

Uma característica importante do algoritmo *Merge Sort* é que ele apresenta essencialmente o mesmo comportamento no **melhor caso**, **caso médio** e **pior caso**. Isso ocorre porque o algoritmo sempre divide o vetor recursivamente e sempre executa o mesmo processo de intercalação, independentemente da ordem inicial dos elementos. Os três passos característicos dos algoritmos baseados na estratégia de *dividir para conquistar*, aplicados ao Merge Sort, são os seguintes:

1. **Dividir:** calcular o ponto médio do subvetor, o que pode ser feito em tempo constante $O(1)$;
2. **Conquistar:** resolver recursivamente dois subproblemas, cada um de tamanho aproximadamente $n/2$, contribuindo com $T(n/2) + T(n/2)$ para o tempo total;

3. **Combinar:** intercalar os dois subvetores ordenados em um único vetor ordenado, operação que exige tempo $O(n)$.

Assim, o tempo de execução do algoritmo pode ser descrito pela seguinte relação de recorrência:

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n).$$

Essa recorrência é semelhante àquela encontrada na análise do algoritmo Quicksort. Expandindo a recorrência, temos

$$T(n) = 2T\left(\frac{n}{2}\right) + n$$

$$T(n) = 2 \left[2T\left(\frac{n}{4}\right) + \frac{n}{2} \right] + n$$

$$T(n) = 4T\left(\frac{n}{4}\right) + 2n$$

Prosseguindo com a expansão,

$$T(n) = 8T\left(\frac{n}{8}\right) + 3n$$

e, de maneira geral, após k expansões obtemos

$$T(n) = 2^k T\left(\frac{n}{2^k}\right) + kn.$$

Para alcançar o caso base $T(1)$, devemos ter

$$\frac{n}{2^k} = 1,$$

o que implica

$$n = 2^k \quad \Rightarrow \quad k = \log_2 n.$$

Substituindo esse valor na expressão anterior, obtemos

$$T(n) = 2^{\log_2 n} T(1) + n \log_2 n.$$

Como $2^{\log_2 n} = n$, segue que

$$T(n) = nT(1) + n \log_2 n.$$

Como $T(1) = O(1)$, concluímos que a complexidade assintótica do Merge Sort em qualquer caso é

$$T(n) = O(n \log_2 n).$$

Uma das principais limitações do algoritmo Merge Sort é que ele executa todo o processo de divisão e intercalação mesmo quando a lista já está ordenada. Por esse motivo, a complexidade no melhor caso é idêntica à complexidade no pior caso, isto é,

$$O(n \log n).$$

Para valores muito grandes de n e listas compostas por números gerados aleatoriamente, pode-se mostrar que o número médio de comparações realizadas pelo algoritmo Merge Sort é aproximadamente αn menor do que no pior caso, onde

$$\alpha = -1 + \sum_{k=0}^{\infty} \frac{1}{2^k + 1}$$

e numericamente

$$\alpha \approx 0.2645.$$

2.7 Quicksort

O algoritmo *Quicksort* é baseado na estratégia de *Dividir para Conquistar*, pois divide o problema de ordenar um vetor em subproblemas menores, que podem ser resolvidos de forma mais simples e eficiente. A ideia central do algoritmo consiste em escolher um elemento do vetor, denominado *pivô*, e reorganizar os demais elementos de forma que todos os valores menores que o pivô fiquem à sua esquerda, enquanto os valores maiores fiquem à sua direita. Após esse particionamento, o algoritmo aplica o mesmo procedimento recursivamente às duas partes do vetor. De forma geral, o funcionamento do Quicksort pode ser descrito em três etapas principais:

1. Escolha do pivô

Seleciona-se um elemento do vetor para servir como pivô. Existem diferentes estratégias possíveis para essa escolha:

- o primeiro elemento da lista;
- o último elemento da lista;
- a mediana entre o primeiro, o elemento central e o último elemento da lista (estratégia geralmente mais eficiente).

2. Particionamento

Nesta etapa, o vetor é reorganizado de modo que todos os elementos menores que o pivô sejam posicionados antes dele (à esquerda), enquanto todos os elementos maiores sejam colocados após ele (à direita).



Figura 14: Ilustração do array após o processo de particionamento com um pivô c .

Ao final desse processo, o pivô estará exatamente em sua posição final no vetor ordenado. Existem diversas estratégias para implementar essa etapa de particionamento.

3. Ordenação recursiva

Após o particionamento, o algoritmo aplica recursivamente o Quicksort aos dois subvetores resultantes (à esquerda e à direita do pivô). O caso base da recursão ocorre quando o subvetor possui tamanho igual a 1 (ou zero elementos), situação em que ele já está ordenado por definição.

Exemplo de execução do Quicksort

Considere o vetor inicial

$[5, 2, 13, 7, -3, 4, 15, 10, 1, 6]$

Neste exemplo utilizaremos como pivô sempre o **último elemento** do subvetor considerado.

Passo 1: Primeiro particionamento Escolhemos como pivô o último elemento:

$\text{pivô} = 6$

Reorganizando o vetor de modo que todos os elementos menores que o pivô fiquem à esquerda e os maiores à direita, obtemos

$[5, 2, -3, 4, 1 \mid 6 \mid 13, 7, 15, 10]$

O pivô já se encontra em sua posição final no vetor ordenado. Agora aplicamos o Quicksort recursivamente às duas partes.

Esquerda: $[5, 2, -3, 4, 1]$ Direita: $[13, 7, 15, 10]$

Passo 2: Particionamento da metade esquerda Subvetor:

$[5, 2, -3, 4, 1]$

Escolhemos

$\text{pivô} = 1$

Após o particionamento:

$[-3 \mid 1 \mid 5, 2, 4]$

Subproblemas gerados:

$[-3]$ $[5, 2, 4]$

Observação: o subvetor $[-3]$ possui apenas um elemento, portanto já está ordenado.

Passo 3: Particionamento da metade direita Subvetor:

$[13, 7, 15, 10]$

Escolhemos

$\text{pivô} = 10$

Após o particionamento:

$[7 \mid 10 \mid 15, 13]$

Subproblemas gerados:

$[7]$ $[15, 13]$

Novamente, o subvetor $[7]$ já está ordenado por possuir apenas um elemento.

Passo 4: Novos particionamentos Restam dois subvetores com mais de um elemento.

Subvetor:

$[5, 2, 4]$

Escolhendo

$\text{pivô} = 4$

Temos

[2 | 4 | 5]

Subvetor:

[15, 13]

Escolhendo

pivô = 13

Temos

[13 | 15]

Passo final Substituindo os subvetores ordenados no vetor original, obtemos

[-3, 1, 2, 4, 5, 6, 7, 10, 13, 15]

Todos os subvetores restantes possuem apenas um elemento, logo o processo recursivo termina e o vetor está completamente ordenado.

A seguir apresentamos o algoritmo `Quicksort`, juntamente com uma função auxiliar responsável pelo particionamento da lista.

```
Quicksort(L, low, high) {  
    if low < high {  
        q = partition(L, low, high)  
        Quicksort(L, low, q - 1)  
        Quicksort(L, q + 1, high)  
    }  
}
```

Para que o algoritmo `Quicksort` seja eficiente, é fundamental que a operação de particionamento seja realizada em tempo linear, isto é, com custo computacional $O(n)$.

A seguir apresentamos uma função eficiente para realizar o particionamento da lista, utilizando como pivô o último elemento do vetor (particionamento de Lomuto). Uma pequena observação técnica que vale a pena mencionar é que existem duas formas eficientes de realizar o particionamento. O método de Hoare, que utiliza dois ponteiros que caminham em direções opostas, e o método de Lomuto, em que um único ponteiro de separação é empregado e o pivô é reposicionado no final (nosso caso).

```
partition(L, low, high) {  
    x = L[high]          // pivô
```

```

i = low - 1      // maior índice do lado esquerdo
for j = low to high - 1 {    // percorre os elementos, exceto o pivô
    if L[j] <= x {           // elemento pertence ao lado esquerdo?
        i = i + 1           // atualiza índice da região esquerda
        swap(L[i], L[j])    // move elemento para o lado esquerdo
    }
}
swap(L[i+1], L[high])        // coloca o pivô na posição correta
return i + 1                 // retorna o índice final do pivô
}

```

Durante o processo de particionamento, o vetor é reorganizado de modo que todos os elementos menores ou iguais ao pivô fiquem posicionados à sua esquerda, enquanto os elementos maiores permanecem à direita. Ao final do algoritmo, o pivô é colocado entre essas duas regiões, ocupando sua posição definitiva no vetor ordenado. O índice retornado pela função indica exatamente essa posição final do pivô, permitindo que o algoritmo Quicksort seja aplicado recursivamente aos dois subvetores gerados. A figura a seguir ilustra o funcionamento do algoritmo para particionar uma lista em duas sublistas.

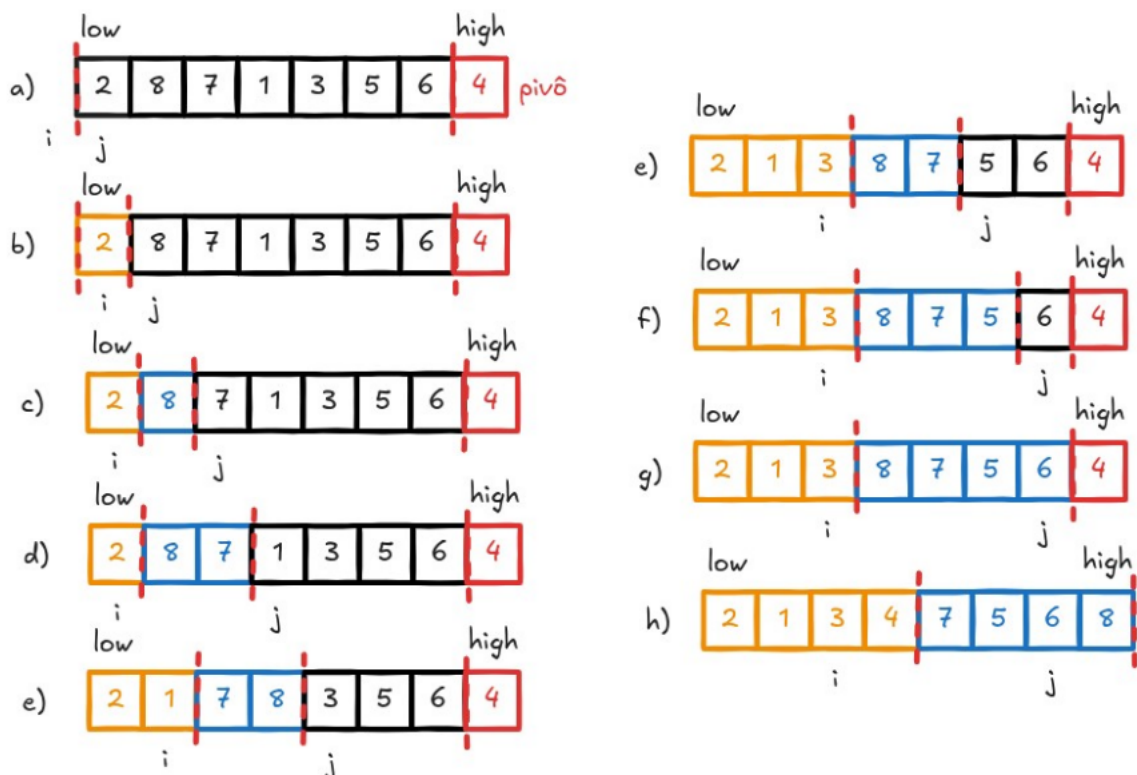


Figura 15: Ilustração do processo de particionamento de um array com o método de Lomuto.

Análise da complexidade

Para analisar a complexidade do algoritmo *Quicksort*, consideramos três cenários distintos: **pior caso**, **melhor caso** e **caso médio**. A eficiência do algoritmo depende diretamente da qualidade do particionamento produzido pela escolha do pivô.

Pior caso O pior caso ocorre quando o pivô escolhido é sempre o menor ou o maior elemento do vetor. Nessa situação, o particionamento gera sublistas extremamente desbalanceadas.

Na primeira chamada recursiva temos uma lista de tamanho n . Após o particionamento, obtemos uma sublista com $n - 1$ elementos, o pivô em sua posição final e uma sublista vazia. Dessa forma, podemos escrever a seguinte relação de recorrência:

$$T(n) = T(n - 1) + T(0) + n = T(n - 1) + n$$

Isso ocorre porque estamos decompondo um problema de tamanho n em um subproblema de tamanho $n - 1$ e outro de tamanho 0. Entretanto, para realizar o particionamento da lista são necessárias n operações, já que a função `partition` percorre todos os elementos do vetor.

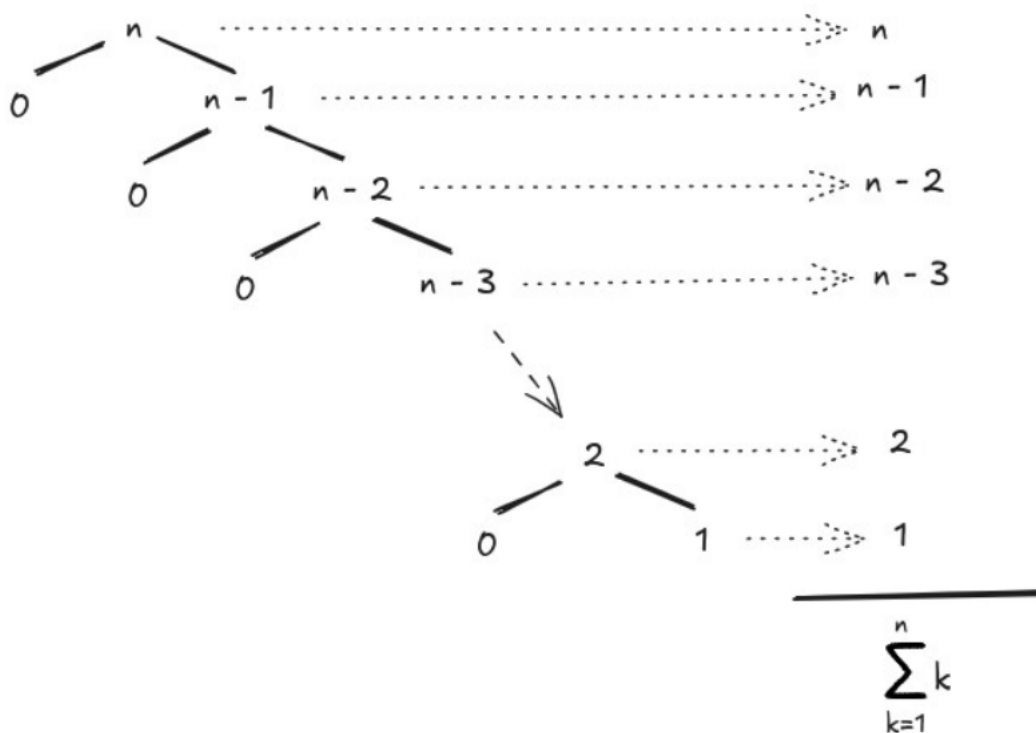


Figura 16: Árvore de recursão para o pior caso do algoritmo Quicksort.

Somando os custos em cada nível da recursão, obtemos o seguinte somatório:

$$T(n) = 1 + 2 + 3 + \cdots + (n - 1) + n$$

ou, de forma equivalente,

$$T(n) = \sum_{k=1}^n k$$

cujo resultado é

$$T(n) = \frac{n(n+1)}{2} = \frac{n^2 + n}{2}.$$

Portanto, a complexidade assintótica no pior caso é

$$T(n) = O(n^2).$$

Melhor caso O melhor caso ocorre quando o pivô divide o vetor em duas partes exatamente do mesmo tamanho. Nesse caso, após o particionamento obtemos duas sublistas com aproximadamente $n/2$ elementos cada.

A relação de recorrência pode então ser escrita como

$$T(n) = T\left(\frac{n}{2}\right) + T\left(\frac{n}{2}\right) + n = 2T\left(\frac{n}{2}\right) + n.$$

Nesse caso estamos decompondo um problema de tamanho n em dois subproblemas de tamanho $n/2$. O custo adicional n corresponde à operação de particionamento realizada pela função `partition`.

Como a cada nível da recursão o tamanho do problema é reduzido pela metade, a altura da árvore de recursão é

$$h = \log_2 n.$$

Além disso, o custo total em cada nível da árvore é proporcional a n . Portanto, somando os custos ao longo dos níveis da árvore de recursão, obtemos

$$T(n) = O(n \log_2 n).$$

Assim, no melhor caso o algoritmo Quicksort apresenta complexidade $O(n \log n)$.

Caso médio Para analisar o caso médio do algoritmo *Quicksort*, utilizamos uma estratégia baseada no cálculo da média sobre todas as possíveis partições do vetor. Supondo que cada elemento tenha a mesma probabilidade de ser escolhido como pivô, os tamanhos possíveis das partições são

$$(1, n-1), (2, n-2), (3, n-3), \dots, (n-1, 1).$$

Assim, a relação de recorrência para o tempo médio de execução pode ser escrita como

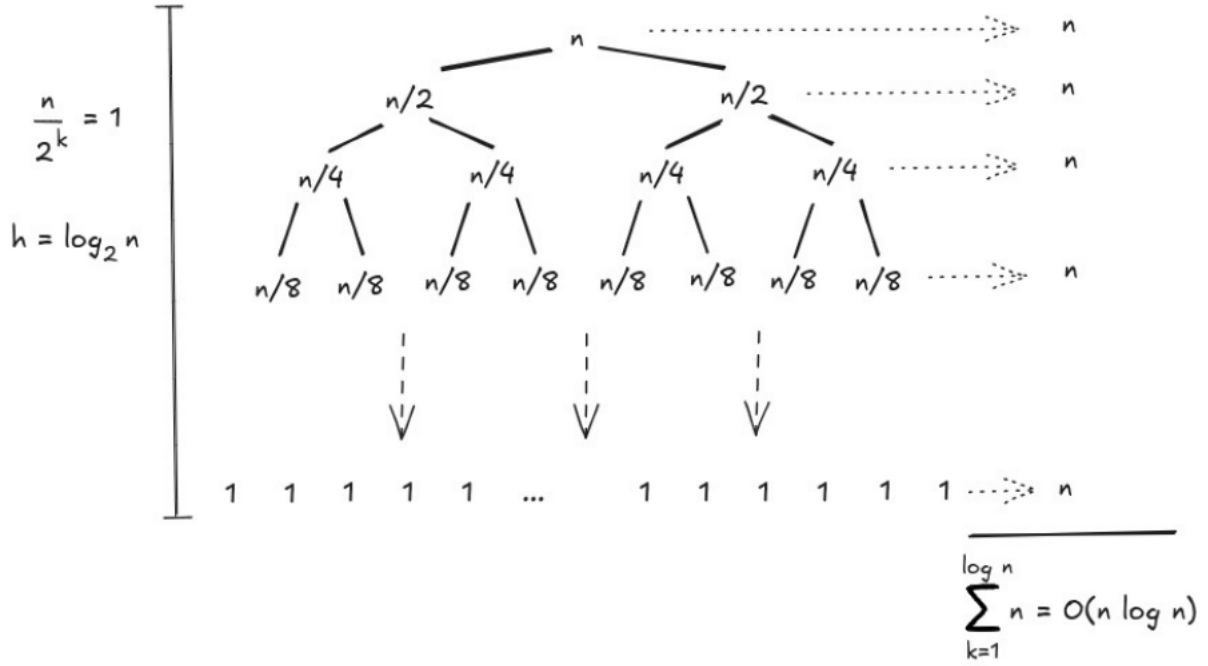


Figura 17: Árvore de recursão para o melhor caso do algoritmo Quicksort.

$$T(n) = \frac{1}{n} \sum_{i=1}^n [T(i-1) + T(n-i)] + O(n).$$

Separando os somatórios, obtemos

$$T(n) = \frac{1}{n} \sum_{i=1}^n T(i-1) + \frac{1}{n} \sum_{i=1}^n T(n-i) + cn.$$

Observe que os dois somatórios são idênticos, pois os termos aparecem apenas em ordem inversa. Dessa forma, podemos escrever

$$T(n) = \frac{2}{n} \sum_{i=0}^{n-1} T(i) + cn.$$

Multiplicando ambos os lados por n , temos

$$nT(n) = 2 \sum_{i=0}^{n-1} T(i) + cn^2 \quad (\text{I})$$

Agora reescrevemos a identidade anterior substituindo n por $n-1$:

$$(n-1)T(n-1) = 2 \sum_{i=0}^{n-2} T(i) + c(n-1)^2 \quad (\text{II})$$

Subtraindo (II) de (I) e eliminando constantes, obtemos

$$nT(n) - (n-1)T(n-1) = 2T(n-1) + 2cn.$$

Isolando o termo $T(n)$, temos

$$nT(n) = (n+1)T(n-1) + 2cn.$$

Dividindo ambos os lados por $n(n+1)$, obtemos

$$\frac{T(n)}{n+1} = \frac{T(n-1)}{n} + \frac{2c}{n+1}.$$

Utilizando a recorrência para expandir $T(n-1)$, temos

$$\frac{T(n)}{n+1} = \frac{T(n-2)}{n-1} + \frac{2c}{n} + \frac{2c}{n+1}.$$

Repetindo o processo para $T(n-2)$, obtemos

$$\frac{T(n)}{n+1} = \frac{T(n-3)}{n-2} + \frac{2c}{n-1} + \frac{2c}{n} + \frac{2c}{n+1}.$$

Continuando o processo até chegar em $T(1)$, obtemos

$$\frac{T(n)}{n+1} = \frac{T(1)}{2} + 2c \left(\frac{1}{3} + \frac{1}{4} + \frac{1}{5} + \cdots + \frac{1}{n+1} \right).$$

Como $T(1) = O(1)$, podemos escrever

$$\frac{T(n)}{n+1} = O(1) + 2c \sum_{i=3}^{n+1} \frac{1}{i}.$$

Para aproximar esse somatório, utilizamos o fato de que a série harmônica pode ser aproximada por uma integral:

$$\ln a = \int_1^a \frac{dx}{x}.$$

Assim,

$$\sum_{i=1}^n \frac{1}{i} = O(\log n).$$

Substituindo essa aproximação, obtemos

$$T(n) = (n+1) (O(1) + 2c O(\log n)).$$

Portanto,

$$T(n) = O(n \log n).$$

Uma estratégia empírica frequentemente utilizada para melhorar o desempenho do algoritmo Quicksort consiste em escolher como pivô a *mediana de três*, isto é, a mediana entre o primeiro elemento, o elemento central e o último elemento do vetor.

Escolha aleatória do pivô Uma das limitações do algoritmo *Quicksort* é que sua eficiência depende diretamente da qualidade das partições produzidas durante a execução. Quando o pivô escolhido é consistentemente o menor ou o maior elemento da lista, as partições tornam-se extremamente desbalanceadas, resultando em uma complexidade de pior caso $O(n^2)$.

Uma estratégia simples e bastante eficaz para reduzir a probabilidade desse comportamento é utilizar a **aleatorização na escolha do pivô**. Em vez de selecionar sempre o primeiro ou o último elemento da lista, escolhe-se um índice aleatório r no intervalo $[low, high]$ e utiliza-se o elemento $L[r]$ como pivô.

A principal vantagem dessa abordagem é que ela elimina a dependência do desempenho do algoritmo em relação à ordem inicial dos dados. Mesmo que o vetor esteja previamente ordenado ou organizado de maneira adversa, cada execução do algoritmo terá uma grande probabilidade de produzir partições relativamente equilibradas.

Do ponto de vista probabilístico, a escolha aleatória do pivô torna muito improvável que ocorram repetidamente partições extremamente desbalanceadas. Como consequência, o tempo de execução esperado do algoritmo passa a ser

$$O(n \log n),$$

mesmo quando os dados de entrada possuem alguma estrutura específica.

Outra vantagem importante é que essa estratégia é simples de implementar. Basta trocar o elemento escolhido aleatoriamente com o último elemento do vetor e então aplicar o procedimento de particionamento usual. Dessa forma, o algoritmo mantém a mesma lógica de particionamento, mas passa a apresentar um comportamento muito mais robusto em relação à distribuição dos dados de entrada.

Em resumo, a aleatorização do pivô não altera a complexidade de pior caso do Quicksort, mas reduz significativamente a probabilidade de que esse cenário ocorra na prática, tornando o algoritmo mais eficiente e estável em aplicações reais.

Partições desbalanceadas e comportamento prático Uma característica importante do algoritmo *Quicksort* é que, na prática, ele tende a operar próximo do caso médio na maior parte das execuções. Mesmo quando as partições produzidas pelo pivô são relativamente desbalanceadas, por exemplo do tipo 90% – 10%, o algoritmo ainda mantém complexidade assintótica $O(n \log n)$.

Considere o caso em que, a cada etapa de particionamento, o vetor é dividido em duas partes contendo aproximadamente $0.9n$ e $0.1n$ elementos. Nesse cenário, a árvore de recursão torna-se assimétrica.

A altura do ramo mais curto da árvore (lado esquerdo) é

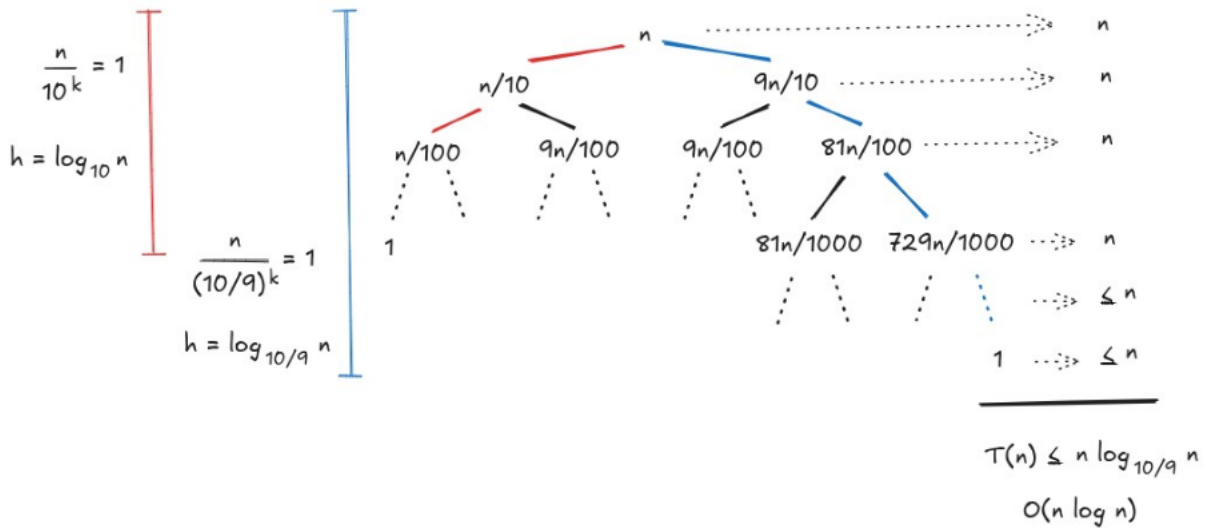


Figura 18: Árvore de recursão para o caso de uma partição 90%-10% no algoritmo Quicksort. Mesmo quando uma partição é bem desbalanceada, o comportamento do algoritmo Quicksort ainda é bem próximo do caso médio, ou seja, a complexidade é $O(n \log n)$.

$$h_L = \log_{10} n,$$

pois a cada nível o tamanho do subproblema é reduzido por um fator de 10. Já a altura do ramo mais longo (lado direito) é

$$h_R = \log_{10/9} n,$$

uma vez que o maior subproblema contém aproximadamente $0.9n$ elementos a cada nível da recursão.

Observa-se que, até a altura h_L , o custo por nível da árvore de recursão é aproximadamente n , pois ainda existem vários subproblemas sendo processados. Após esse ponto, o custo por nível passa a ser menor que n , já que o tamanho total dos subproblemas restantes diminui progressivamente. Somando os custos ao longo dos níveis da árvore de recursão, obtemos

$$T(n) \leq cn \log_{10/9} n.$$

Utilizando a mudança de base do logaritmo, temos

$$\log_{10/9} n = \frac{\log_2 n}{\log_2(10/9)}.$$

Como

$$\log_2(10/9) \approx 0.152,$$

segue que

$$\log_{10/9} n \approx \frac{\log_2 n}{0.152} \approx 6.578 \log_2 n.$$

Portanto,

$$T(n) = O(n \log_2 n).$$

Esse resultado mostra que mesmo com partições relativamente desbalanceadas, o algoritmo Quicksort continua apresentando complexidade $O(n \log n)$, o que explica seu excelente desempenho na prática.

2.8 Limitante Inferior para Ordenação Baseada em Comparações

Os algoritmos de ordenação estudados até o momento são todos baseados em comparações. Em outras palavras, eles obtêm informação sobre a sequência por meio de uma operação que recebe dois elementos e retorna qual deles é maior, permitindo decidir se eles devem ou não ser trocados de posição. Para obter um limitante inferior para o número de comparações necessárias para ordenar uma sequência, vamos comparar duas grandezas:

1. o número de permutações que uma sequência de tamanho n pode apresentar;
2. o número de sequências distintas que um algoritmo consegue distinguir após realizar k comparações.

Com relação ao primeiro ponto, é fácil notar que o número de possíveis permutações de uma sequência de tamanho n é

$$n! = n \times (n - 1) \times (n - 2) \times (n - 3) \times \cdots \times 1.$$

Com relação ao segundo ponto, note que inicialmente todas as sequências são indistinguíveis para o algoritmo. A cada comparação realizada, obtemos uma decisão binária (por exemplo, $a_i < a_j$ ou $a_i \geq a_j$). No melhor dos casos, cada comparação divide o espaço de busca em duas partes, dobrando o número de sequências que podem ser diferenciadas. Esse processo pode ser representado por uma árvore binária de decisões. Em uma árvore binária de altura k (isto é, com k comparações), o número máximo de folhas é

$$2^k.$$

Cada folha representa uma possível sequência identificada pelo algoritmo. Portanto, um algoritmo que realiza k comparações pode distinguir no máximo 2^k sequências diferentes. Nesse ponto podemos aplicar o princípio da casa dos pombos. Se temos mais pombos do que casas, então pelo menos uma casa deverá conter mais de um pombo. No contexto da ordenação:

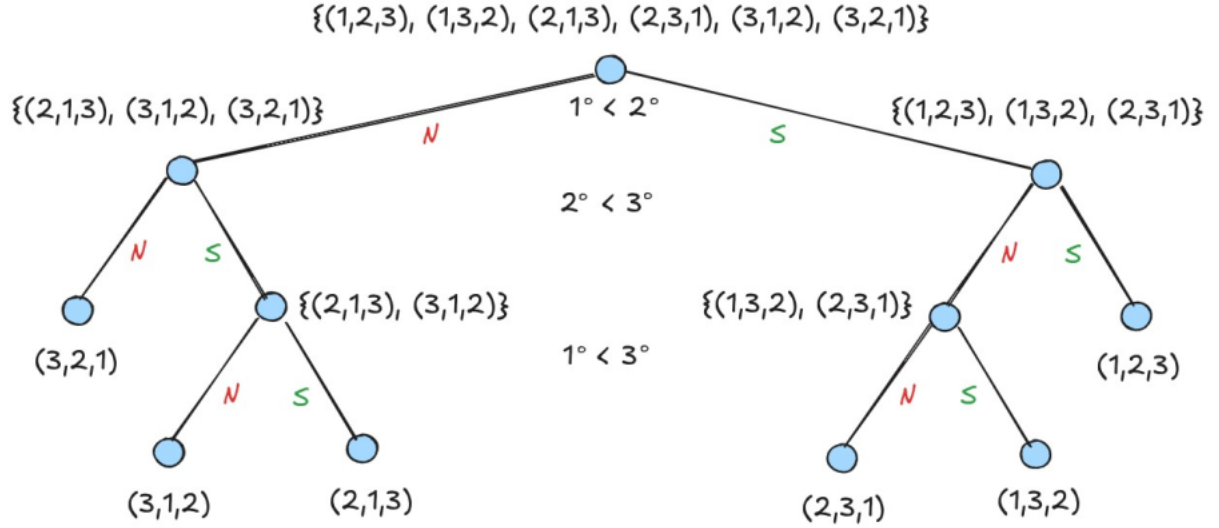


Figura 19: O número máximo de folhas em uma árvore binária de k níveis é 2^k .

- **Pombos:** $n!$ possíveis permutações da sequência;
- **Casas:** 2^k folhas da árvore de decisão.

Assim, se

$$n! > 2^k,$$

então não é possível distinguir todas as permutações possíveis. Portanto, para que o algoritmo seja capaz de ordenar qualquer sequência, é necessário que

$$2^k \geq n!.$$

Para estimar esse valor, vamos obter um limitante inferior para $n!$. Pela definição de fatorial, podemos escrever

$$n! = n(n-1)(n-2) \cdots \left(\frac{n}{2} + 1\right) \left(\frac{n}{2}\right) \left(\frac{n}{2} - 1\right) \cdots 3 \cdot 2 \cdot 1.$$

Observe que existem aproximadamente $n/2$ termos maiores ou iguais a $n/2$. Portanto,

$$n! \geq \left(\frac{n}{2}\right) \left(\frac{n}{2}\right) \left(\frac{n}{2}\right) \cdots \left(\frac{n}{2}\right),$$

onde aparecem $n/2$ fatores. Assim,

$$n! \geq \left(\frac{n}{2}\right)^{n/2}.$$

Substituindo essa desigualdade na expressão anterior, obtemos

$$2^k \geq n! \geq \left(\frac{n}{2}\right)^{n/2}.$$

Aplicando logaritmo na base 2, chegamos a

$$k \geq \log_2 \left(\left(\frac{n}{2} \right)^{n/2} \right) = \frac{n}{2} \log_2 \left(\frac{n}{2} \right).$$

Assim, concluímos que

$$k = \Omega(n \log n).$$

Portanto, qualquer algoritmo de ordenação baseado exclusivamente em comparações precisa realizar, no mínimo,

$$\Omega(n \log n)$$

comparações (e consequentemente trocas) no pior caso.

Ordenação não baseada em comparações

Até o momento estudamos algoritmos de ordenação cujo funcionamento se baseia exclusivamente em comparações entre elementos. Como vimos na seção anterior, qualquer algoritmo desse tipo possui um limitante inferior de complexidade $\Omega(n \log n)$ no pior caso. Isso significa que, independentemente da estratégia utilizada, não é possível ordenar uma sequência arbitrária utilizando apenas comparações com complexidade assintótica melhor que essa.

Entretanto, esse limitante não se aplica quando exploramos propriedades adicionais dos dados a serem ordenados. Em particular, quando os elementos pertencem a um domínio conhecido (por exemplo, números inteiros dentro de um intervalo limitado) ou possuem uma estrutura que pode ser explorada diretamente, é possível desenvolver algoritmos de ordenação que não dependem de comparações entre pares de elementos.

Esses algoritmos utilizam estratégias alternativas, como contagem de ocorrências, distribuição de elementos em classes ou processamento dígito a dígito. Como consequência, podem alcançar complexidades assintóticas inferiores a $O(n \log n)$, chegando em alguns casos a $O(n)$ sob certas condições.

Nesta subseção iremos estudar três algoritmos clássicos de ordenação não baseados em comparações: o *Countingsort*, que utiliza a contagem direta das ocorrências dos valores; o *Radixsort*, que ordena os elementos processando seus dígitos individualmente; e o *Bucketsort*, que distribui os elementos em diferentes classes (*buckets*) antes de ordená-los localmente.

2.9 Countingsort

O algoritmo *Countingsort* é um método de ordenação que não se baseia em comparações entre elementos. Em vez disso, ele explora uma propriedade importante dos dados: os elementos a serem

ordenados devem pertencer a um domínio discreto e limitado, tipicamente um conjunto de inteiros em um intervalo conhecido.

A ideia central do algoritmo consiste em contar quantas vezes cada valor possível aparece na sequência de entrada. Essas contagens são armazenadas em um vetor auxiliar, no qual cada posição representa um valor do domínio dos dados. A partir dessas informações é possível determinar diretamente a posição correta de cada elemento no vetor ordenado.

De forma mais específica, o algoritmo constrói inicialmente um vetor de contagem que registra o número de ocorrências de cada valor. Posteriormente, esse vetor é transformado em um vetor de somas acumuladas, permitindo determinar quantos elementos da lista possuem valor menor ou igual a um determinado valor. Essa informação é então utilizada para posicionar cada elemento em sua posição final no vetor ordenado.

Uma característica importante do *Countingsort* é que ele é um algoritmo estável, isto é, elementos com valores iguais preservam a mesma ordem relativa em que aparecem na sequência original. Essa propriedade é particularmente útil quando o algoritmo é utilizado como subrotina em métodos de ordenação mais complexos, como o *Radixsort*.

Quando o intervalo de valores possíveis não é muito grande em relação ao tamanho da entrada, o *Countingsort* apresenta complexidade temporal $O(n + k)$, onde n é o número de elementos da lista e k é o tamanho do intervalo de valores possíveis. Nessas condições, ele pode ser significativamente mais eficiente do que algoritmos de ordenação baseados em comparações.

Funcionamento do algoritmo Countingsort

O algoritmo *Countingsort* ordena os elementos de uma sequência contando o número de ocorrências de cada valor. Para compreender a intuição por trás do método, vamos descrever seus passos utilizando um exemplo ilustrativo.

Considere a seguinte lista de entrada:

$$L = [4, 2, 2, 8, 3, 3, 1]$$

O funcionamento do algoritmo pode ser descrito pelas etapas a seguir.

Passo 1. Encontrar o maior elemento da lista Inicialmente identificamos o maior elemento da lista, que determina o tamanho do vetor auxiliar de contagem.

$$\text{max} = 8$$

Passo 2. Criar um vetor auxiliar inicializado com zeros Criamos um vetor H de tamanho $(\text{max} + 1)$, inicialmente composto apenas por zeros.

$$H = [0, 0, 0, 0, 0, 0, 0, 0, 0]$$

Passo 3. Contar o número de ocorrências Percorremos a lista L e armazenamos em H o número de ocorrências de cada elemento. Cada posição do vetor corresponde a um valor possível na lista.

$$H = [0, 1, 2, 2, 1, 0, 0, 0, 1]$$

Observe que algumas posições podem permanecer com valor zero, caso nenhum elemento correspondente apareça na lista.

Passo 4. Calcular a soma cumulativa Em seguida calculamos a soma cumulativa dos elementos de H , obtendo um novo vetor C .

$$C = [0, 1, 3, 5, 6, 6, 6, 6, 7]$$

Esse vetor indica quantos elementos da lista possuem valor menor ou igual a cada índice.

Passo 5. Determinar a posição correta de cada elemento Para cada elemento $L[i]$, consultamos o vetor C e colocamos o elemento na posição

$$C[L[i]] - 1$$

em um novo vetor S , que terá o mesmo tamanho de L .

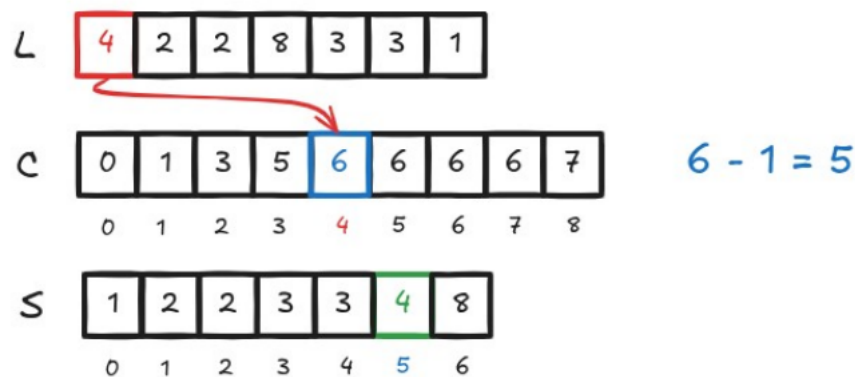


Figura 20: O elemento $L[i]$ deve ser colocado na posição $C[L[i]] - 1$.

Passo 6. Atualizar o vetor cumulativo Após posicionar um elemento em S , diminuimos em uma unidade o valor correspondente em C . Dessa forma, garantimos que elementos iguais sejam colocados em posições consecutivas.

A seguir apresentamos o algoritmo *Countingsort* em pseudo-código, o que permitirá analisar sua complexidade.

```
Countingsort(L, n) {
    m = max(L)          // maior elemento de L
    C = zeros(m+1)      // vetor de contagem (0 até m)
```

```

S = zeros(n)          // vetor ordenado
// conta número de ocorrências
for i = 0 to n-1
    C[L[i]] = C[L[i]] + 1
// calcula soma acumulada
for i = 1 to m
    C[i] = C[i] + C[i-1]
// realiza o mapeamento para posição correta
for i = 0 to n-1 {
    C[L[i]] = C[L[i]] - 1
    S[C[L[i]]] = L[i]
}
return S
}

```

Análise da complexidade

Inicialmente, observe que a operação de encontrar o maior elemento da lista L possui complexidade $O(n)$, pois é necessário percorrer todos os elementos da lista. Analisando agora as três estruturas de repetição do algoritmo (`for`), podemos escrever a função de custo $T(n)$ como a soma dos custos de cada uma das etapas. Assim,

$$T(n) = O(n) + \sum_{i=0}^{n-1} 1 + \sum_{i=1}^m 1 + 2 \sum_{i=0}^{n-1} 1$$

onde:

- o primeiro termo corresponde à busca pelo maior elemento da lista;
- o primeiro somatório corresponde à contagem do número de ocorrências;
- o segundo somatório corresponde ao cálculo da soma cumulativa;
- o último somatório corresponde à etapa de construção do vetor ordenado.

Simplificando os termos, obtemos

$$T(n) = O(n) + O(n) + O(m) + O(n) = O(n + m),$$

onde n representa o número de elementos da lista e m representa o maior valor presente na lista.

Em geral, o algoritmo apresenta excelente desempenho quando o maior elemento da lista não é muito grande, pois isso reduz o valor de m . Por exemplo, quando $m < n$, o termo dominante passa a ser n , e a complexidade do algoritmo torna-se simplesmente

$O(n)$.

Entretanto, o algoritmo também apresenta limitações. A principal delas ocorre quando o maior elemento da lista é muito grande. Nesse caso, o termo m domina a expressão da complexidade, resultando em um custo $O(m)$. Isso acontece porque o vetor de contagem precisa possuir tamanho proporcional ao maior valor da lista.

Assim, quando $m \gg n$, o vetor de contagem torna-se muito maior que a própria lista de entrada. Mesmo que n seja pequeno (por exemplo, uma lista com apenas 100 elementos), o algoritmo pode apresentar um tempo de execução elevado devido ao grande tamanho do vetor auxiliar.

Limitações do Countingsort Apesar de sua eficiência em diversos cenários, o algoritmo *Countingsort* apresenta algumas limitações importantes. A principal delas está relacionada ao intervalo de valores possíveis dos elementos da lista. Como o algoritmo utiliza um vetor auxiliar de tamanho proporcional ao maior valor presente na entrada, seu custo em tempo e espaço depende diretamente desse intervalo. Assim, quando os elementos da lista assumem valores muito grandes, o vetor de contagem torna-se excessivamente grande, mesmo que o número de elementos a serem ordenados seja relativamente pequeno. Nesses casos, o algoritmo pode se tornar inviável tanto em termos de tempo quanto de uso de memória.

Outra limitação é que o *Countingsort* só pode ser aplicado diretamente a dados que pertencem a um domínio discreto e relativamente pequeno, tipicamente inteiros não negativos. Quando os valores são muito grandes ou quando os elementos possuem múltiplos dígitos significativos, uma alternativa interessante consiste em explorar a representação posicional desses valores.

Essa ideia motiva o algoritmo *Radixsort*, que realiza a ordenação processando os elementos dígito a dígito. Em vez de depender do valor numérico completo de cada elemento, o *Radixsort* aplica repetidamente um algoritmo de ordenação estável (frequentemente o próprio *Countingsort*) sobre cada posição de dígito. Dessa forma, é possível ordenar números potencialmente grandes mantendo uma complexidade linear em relação ao tamanho da entrada.

2.10 Radixsort

O algoritmo *Radixsort* pode ser considerado uma generalização do *Countingsort*. Trata-se de um método de ordenação baseado no agrupamento de elementos de acordo com seus dígitos, iniciando pelo dígito menos significativo.

A ideia consiste em ordenar os elementos sucessivamente de acordo com cada posição de dígito. Primeiro realiza-se a ordenação considerando o dígito das unidades; em seguida, o dígito das dezenas; depois o das centenas, milhares, e assim por diante, até alcançar o número de dígitos do maior elemento presente na lista. Considere, por exemplo, a seguinte lista de inteiros:

[121, 432, 564, 23, 1, 45, 788]

Como o maior elemento é 788, temos que o número máximo de dígitos é

$$k = 3.$$

Para facilitar o processamento dos dígitos, podemos representar todos os números com três dígitos, adicionando zeros à esquerda quando necessário:

[121, 432, 564, 023, 001, 045, 788].

O processo de ordenação ocorre então da seguinte forma.

Ordenação pelo dígito da unidade

[121, 001, 432, 023, 564, 045, 788]

Ordenação pelo dígito da dezena

[001, 121, 023, 432, 045, 564, 788]

Ordenação pelo dígito da centena

[001, 023, 045, 121, 432, 564, 788]

Ao final dessas três etapas, a lista encontra-se completamente ordenada. Uma estratégia bastante eficiente consiste em utilizar uma versão modificada do *Countingsort* como subrotina do *Radixsort*. Como em cada passo lidamos apenas com um único dígito (de 0 a 9), o valor máximo considerado é pequeno, o que torna o *Countingsort* extremamente eficiente nesse contexto. A seguir apresentamos o pseudocódigo do algoritmo *Countingsort_R*, utilizado internamente no *Radixsort*.

```
Countingsort_R(L, n, d) {  
    // d indica qual dígito está sendo considerado  
    S = zeros(n)           // array auxiliar  
    m = (L[0]/d) % 10      // dígito d do primeiro elemento  
    for i = 1 to n-1 {  
        if (L[i]/d) % 10 > m  
            m = (L[i]/d) % 10  
    }  
    C = zeros(m+1)  
    for i = 0 to n-1  
        C[(L[i]/d) % 10] += 1      // número de ocorrências  
    for i = 1 to m  
        C[i] += C[i-1]           // soma acumulada
```

```

for i = 0 to n-1 {
    C[(L[i]/d) % 10] -= 1
    S[C[(L[i]/d) % 10]] = L[i]
}
for i = 0 to n-1
    L[i] = S[i]
}

```

A seguir apresentamos o pseudocódigo do algoritmo *Radixsort*. A ideia consiste em executar o procedimento `Countingsort_R` repetidamente, uma vez para cada dígito do maior elemento da lista.

```

Radixsort(L, n) {
    m = max(L)
    d = 1 // inicia pelo dígito da unidade
    while m/d > 0 {
        Countingsort_R(L, n, d)
        d = d * 10 // avança para o próximo dígito
    }
}

```

Análise da Complexidade

Para analisar a complexidade do algoritmo *Radixsort*, devemos primeiramente examinar a complexidade da versão modificada do *Countingsort*, denominada `Countingsort_R`. Observando o pseudocódigo do algoritmo, nota-se que ele possui cinco estruturas de repetição (`for`) executadas sequencialmente. Assim, podemos escrever a função de custo $T(n)$ associada ao `Countingsort_R` como

$$T(n) = \sum_{i=1}^{n-1} 1 + \sum_{i=0}^{n-1} 1 + \sum_{i=1}^m 1 + 2 \sum_{i=0}^{n-1} 1 + \sum_{i=0}^{n-1} 1.$$

Agrupando os termos, obtemos

$$T(n) = O(n) + O(n) + O(m) + O(n) + O(n) = O(n + m).$$

Entretanto, no caso do *Radixsort*, o valor de m corresponde ao maior dígito considerado em cada passo do algoritmo. Como os dígitos variam apenas entre 0 e 9, temos que

$$m \leq 9.$$

Assim, m é uma constante pequena, e o termo dominante passa a ser n . Portanto, a complexidade do algoritmo `Countingsort_R` torna-se

$$O(n).$$

Prosseguindo com a análise do *Radixsort*, podemos escrever sua função de custo como

$$T(n) = O(n) + \sum_{i=1}^k [O(n) + 1],$$

onde k representa o número de dígitos do maior elemento da lista L . Simplificando a expressão, obtemos

$$T(n) = O(n) + k O(n) + k.$$

Como, em geral, o número de dígitos k é muito menor que n e pode ser tratado como uma constante em muitos cenários práticos, concluímos que a complexidade do algoritmo *Radixsort* é

$$T(n) = O(n).$$

2.11 Bucketsort

O algoritmo *Bucketsort* é um método eficiente para ordenar um conjunto de n chaves distribuídas uniformemente em um determinado intervalo (hipótese fundamental do algoritmo). Assim como o *Countingsort* e o *Radixsort*, ele não é baseado em comparações, o que significa que não depende diretamente de trocas de posições entre elementos durante o processo principal de ordenação.

Seja $I = [0, K)$ um intervalo arbitrário. O primeiro passo do algoritmo consiste em dividir esse intervalo, de tamanho K , em n baldes (*buckets*), cada um com tamanho $\frac{K}{n}$.

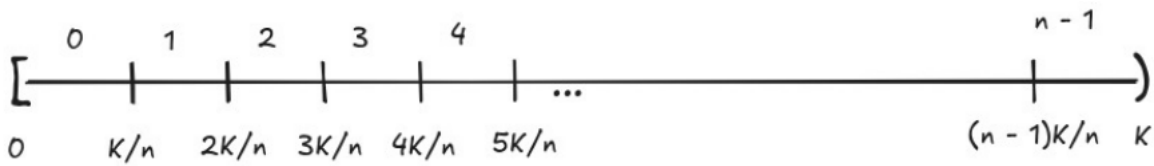


Figura 21: Ao dividir o intervalo I em n baldes, cada balde tem tamanho K/n .

A ideia central é simples: cada elemento da lista L a ser ordenada é colocado em seu respectivo balde de acordo com seu valor. Sob a hipótese de que os dados estão uniformemente distribuídos no intervalo, espera-se que cada balde receba apenas um pequeno número de elementos. Dessa forma, o custo computacional para ordenar os elementos dentro de cada balde torna-se constante, isto é, $O(1)$.

Após a distribuição dos elementos, percorremos os baldes na ordem em que aparecem e copiamos os elementos ordenados de cada balde para uma lista final. Esse processo produz a sequência ordenada. A seguir apresentamos o algoritmo *Bucketsort* em pseudocódigo, o que permite posteriormente analisar sua complexidade computacional.

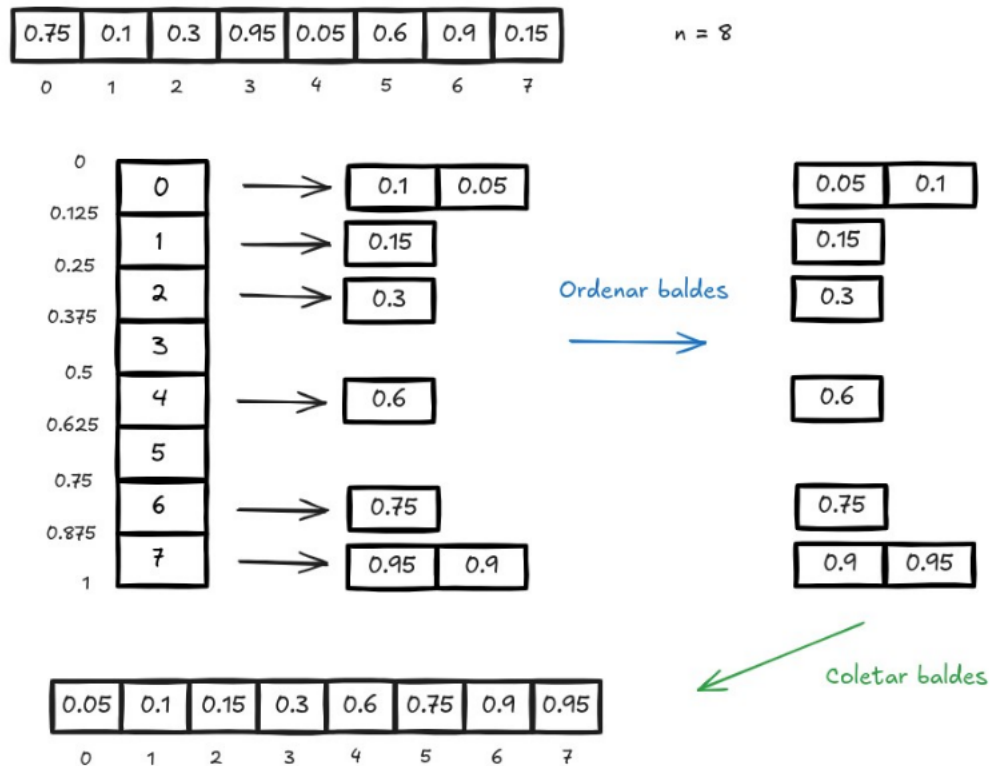


Figura 22: Ilustração gráfica do algoritmo Bucketsort.

```

Bucketsort(L, n) {
    // Inicializa baldes vazios
    for i = 0 to n-1
        B[i] = NIL
    // Distribui chaves nos baldes
    for i = 0 to n-1 {
        if L[i] == 1
            index = n - 1
        else {
            index = floor(n * L[i])
            insert(B[index], L[i])
        }
    }
    // ordena cada um dos baldes
    for i = 0 to n-1
        sort(B[i])
    // realiza a concatenação dos baldes ordenados
    S = concatenate(B[0], B[1], ..., B[n-1])
    return S
}

```

Uma observação importante refere-se ao caso em que os dados estão uniformemente distribuídos

em um intervalo arbitrário $[a, b)$. Podemos normalizar esses valores para o intervalo $[0, 1)$ utilizando a transformação

$$y_i = \frac{x_i - x_{\min}}{x_{\max} - x_{\min}}.$$

Após a ordenação, é possível retornar os dados ao intervalo original $[a, b)$ aplicando a transformação inversa

$$x_i = x_{\min} + (x_{\max} - x_{\min})y_i.$$

Essa normalização permite aplicar o algoritmo *Bucketsort* de forma conveniente mesmo quando os dados originais não estão naturalmente no intervalo $[0, 1)$.

Análise da complexidade

Observe que o algoritmo *Bucketsort* utiliza três primitivas básicas:

1. **insert**(p, x): insere o elemento x na lista apontada pela referência p . Essa operação possui complexidade $O(1)$.
2. **sort**(p): ordena a lista apontada pela referência p . Caso utilizemos um algoritmo de ordenação baseado em comparações, sua complexidade pode variar entre $O(n \log n)$ e $O(n^2)$, dependendo do algoritmo utilizado.
3. **concatenate**(B, n): retorna a lista obtida pela concatenação das listas $B[0], B[1], \dots, B[n-1]$. Essa operação possui complexidade $O(kn)$, onde k representa o tamanho médio dos baldes. Como k é constante, o custo torna-se $O(n)$.

O ponto crítico da análise está na primitiva `sort()`, sendo necessário realizar uma análise probabilística. Seja X_i o número de elementos na lista $B[i]$. Definimos ainda a seguinte variável binária:

$$X_{ij} = \begin{cases} 1, & \text{se o elemento } j \text{ de } L \text{ foi para a lista } B[i], \\ 0, & \text{caso contrário.} \end{cases}$$

Observe que

$$X_i = \sum_j X_{ij}.$$

Denotemos por Y_i o número de comparações necessárias para ordenar a lista $B[i]$. Note que

$$Y_i \leq X_i^2,$$

pois no pior caso o custo de ordenação é $O(n^2)$. Como desejamos analisar o caso médio, podemos escrever

$$E[Y_i] \leq E[X_i^2] = E \left[\left(\sum_j X_{ij} \right)^2 \right].$$

Expandindo o valor esperado, obtemos

$$E[X_i^2] = E \left[\sum_j X_{ij} \sum_k X_{ik} \right] = E \left[\sum_j \sum_k X_{ij} X_{ik} \right].$$

Podemos separar os termos da diagonal e fora da diagonal, obtendo

$$E[Y_i] \leq \sum_j E[X_{ij}] + \sum_j \sum_{k \neq j} E[X_{ij} X_{ik}].$$

Como X_{ij} é uma variável aleatória binária, temos

$$P(X_{ij} = 0) + P(X_{ij} = 1) = 1.$$

Pela definição de valor esperado,

$$E[X_{ij}] = \sum_{x \in \{0,1\}} x P(X_{ij} = x) = 0 \cdot P(X_{ij} = 0) + 1 \cdot P(X_{ij} = 1) = P(X_{ij} = 1).$$

Assumindo que a probabilidade de um elemento cair em qualquer um dos n baldes é uniforme, obtemos

$$E[X_{ij}] = P(X_{ij} = 1) = \frac{1}{n}.$$

Para calcular o valor esperado do produto, observe que, para $j \neq k$, as variáveis aleatórias são independentes. Assim,

$$E[X_{ij} X_{ik}] = E[X_{ij}] E[X_{ik}] = \frac{1}{n} \cdot \frac{1}{n} = \frac{1}{n^2}.$$

Substituindo esses valores na expressão anterior, temos

$$E[Y_i] \leq \sum_{j=1}^n \frac{1}{n} + \sum_{j=1}^n \sum_{k \neq j}^n \frac{1}{n^2}.$$

Como existem $n(n-1)$ termos no segundo somatório, segue que

$$E[Y_i] \leq 1 + \frac{n(n-1)}{n^2} = 1 + \frac{n^2 - n}{n^2} = 2 - \frac{1}{n}.$$

Esse é o número médio de comparações necessárias para ordenar o balde $B[i]$. Como existem n baldes, o número total esperado de comparações é

$$E[Y] = E \left[\sum_{i=1}^n Y_i \right] = \sum_{i=1}^n E[Y_i] = n \left(2 - \frac{1}{n} \right) = 2n - 1.$$

Portanto, o custo esperado é $O(n)$. Considerando todas as operações do algoritmo, temos

$$n \cdot O(1) + O(n) + O(n) + O(n),$$

o que resulta finalmente em

$$O(n).$$

Assim, sob a hipótese de distribuição uniforme dos dados, o algoritmo *Bucketsort* apresenta complexidade linear no caso médio. Porém, se a distribuição dos dados se afasta da uniforme (i.e., normal), podemos ter uma concentração de chaves em alguns baldes, fazendo a complexidade computacional aumentar.

2.12 Comparação entre algoritmos de ordenação

Em resumo, a Tabela 2 apresenta uma comparação entre as complexidades dos principais algoritmos de ordenação discutidos neste capítulo.

Algoritmo	Melhor caso	Caso médio	Pior caso
BubbleSort	$O(n)$	$O(n^2)$	$O(n^2)$
InsertionSort	$O(n)$	$O(n^2)$	$O(n^2)$
SelectionSort	$O(n^2)$	$O(n^2)$	$O(n^2)$
MergeSort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
QuickSort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$
Heapsort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Countingsort	$O(n + m)$	$O(n + m)$	$O(n + m)$
Radixsort	$O(nd)$	$O(nd)$	$O(nd)$
Bucketsort	$O(n)$	$O(n)$	$O(n \log n)$

Tabela 2: Comparação entre as complexidades de algoritmos de ordenação.

Para os interessados em aprofundar o estudo sobre algoritmos de ordenação, existem diversos materiais disponíveis na internet. Um conceito adicional interessante está relacionado à *estabilidade* dos algoritmos de ordenação.

Um algoritmo de ordenação é considerado **estável** quando mantém a ordem relativa dos registros que possuem chaves iguais. A estabilidade de um algoritmo de ordenação torna-se particularmente importante em problemas nos quais os elementos possuem múltiplos atributos e a ordenação é realizada em mais de uma etapa. Nesse tipo de situação, cada etapa pode ordenar os registros de acordo com um critério diferente, preservando o resultado das ordenações anteriores. Por exemplo, em bases de dados é comum ordenar registros primeiro por um campo secundário (como data) e depois por um campo principal (como nome). Se o algoritmo utilizado na segunda etapa for estável, os registros que

possuem o mesmo valor no campo principal manterão a ordem relativa definida pelo campo secundário. Esse comportamento também é relevante em aplicações como processamento de linguagem natural, sistemas de busca, análise de logs e manipulação de registros estruturados, onde diferentes critérios de ordenação podem ser aplicados sequencialmente e a preservação da ordem anterior garante consistência e previsibilidade no resultado final.

A Tabela 3 apresenta quais dos algoritmos discutidos são estáveis.

Algoritmo	Estável
Bubblesort	Sim
Selectionsort	Não
Insertionsort	Sim
Shellsort	Não
Heapsort	Não
Quicksort	Não
Mergesort	Sim
Countingsort	Sim
Bucketssort	Sim

Tabela 3: Estabilidade de alguns algoritmos de ordenação.

A seguir indicamos alguns links interessantes para visualização e estudo dos algoritmos de ordenação:

- **15 sorting algorithms in 6 minutes** (animações sonorizadas muito boas para visualização):
<https://www.youtube.com/watch?v=kPRA0W1kECg>
- **Visualizações passo a passo dos algoritmos:**
<https://visualgo.net/en/sorting>
- **Algoritmos de ordenação representados como danças em grupo:**
<https://www.youtube.com/user/AlgoRythmics>

2.13 Considerações Finais

Neste capítulo foram apresentados alguns dos principais algoritmos de ordenação utilizados em computação, juntamente com suas ideias fundamentais, implementações em pseudocódigo e análises de complexidade. A ordenação é um dos problemas clássicos da área de algoritmos e estruturas de dados, possuindo papel central em diversas aplicações, como processamento de dados, bancos de dados, recuperação de informação, aprendizado de máquina e computação científica.

Inicialmente foram discutidos algoritmos simples baseados em comparações, como *BubbleSort*, *InsertionSort* e *SelectionSort*. Apesar de apresentarem complexidade quadrática no pior caso, esses algoritmos são conceitualmente simples e possuem valor pedagógico importante, além de serem úteis em cenários com conjuntos de dados pequenos ou parcialmente ordenados.

Em seguida, foram apresentados algoritmos mais eficientes baseados em comparações, como *MergeSort*, *QuickSort* e *Heapsort*. Esses métodos alcançam complexidade $O(n \log n)$, que corresponde ao limite inferior para algoritmos de ordenação baseados exclusivamente em comparações. Cada um desses algoritmos apresenta características próprias, envolvendo diferentes estratégias como divisão e conquista, estruturas de dados auxiliares e organização em árvores.

Posteriormente, discutimos algoritmos de ordenação que não são baseados em comparações, como *Countingsort*, *Radixsort* e *Bucketsort*. Esses métodos exploram propriedades específicas dos dados, como o domínio dos valores ou sua distribuição estatística, permitindo atingir complexidade linear em determinadas condições. Entretanto, tais algoritmos também apresentam limitações e hipóteses sobre os dados de entrada que devem ser consideradas na prática.

Outro aspecto importante discutido foi a estabilidade dos algoritmos de ordenação, propriedade relevante em aplicações nas quais os registros possuem múltiplos atributos e podem ser ordenados sucessivamente por diferentes critérios. Além disso, foi apresentado o limite inferior teórico para algoritmos de ordenação baseados em comparações, mostrando que não é possível, no pior caso, ordenar n elementos utilizando menos do que $\Omega(n \log n)$ comparações.

De modo geral, não existe um algoritmo de ordenação universalmente melhor para todos os cenários. A escolha do método mais adequado depende de diversos fatores, como o tamanho do conjunto de dados, a distribuição dos valores, as restrições de memória, a necessidade de estabilidade e o custo das operações envolvidas. Por essa razão, compreender os diferentes paradigmas e características desses algoritmos é essencial para projetar soluções eficientes em problemas computacionais reais.

3 Árvores de Busca Digitais (Tries)

Estruturas de dados baseadas em árvores desempenham um papel central no desenvolvimento de algoritmos eficientes para busca, inserção e armazenamento de informações. Entre essas estruturas, as chamadas *árvores de busca digitais*, também conhecidas como *tries* (do inglês *retrieval*), destacam-se por explorar diretamente a representação dos dados em termos de seus dígitos ou caracteres. Diferentemente das árvores de busca tradicionais, como as árvores binárias de busca, que realizam comparações entre chaves completas, as *tries* organizam os elementos de acordo com seus prefixos.

A ideia fundamental por trás de uma *trie* consiste em decompor cada chave em uma sequência de símbolos, como caracteres de uma string ou dígitos de um número, e utilizar cada posição dessa sequência para guiar o caminho dentro da árvore. Dessa forma, cada nível da estrutura corresponde a uma posição específica da chave, e cada aresta representa um possível valor do símbolo naquela posição. Essa organização permite realizar operações de busca e inserção de forma extremamente eficiente, frequentemente com custo proporcional ao tamanho da chave, e não ao número total de elementos armazenados.

Estruturas do tipo *trie* são amplamente utilizadas em diversas aplicações envolvendo processamento de strings e manipulação de sequências simbólicas. Exemplos típicos incluem sistemas de *autocomplete*, corretores ortográficos, dicionários computacionais, roteamento de redes, compressão de dados e algoritmos de processamento de linguagem natural. Nessas aplicações, a capacidade de explorar prefixos comuns entre diferentes chaves permite reduzir o custo de busca e melhorar a organização dos dados.

Neste capítulo apresentaremos os conceitos fundamentais das árvores de busca digitais, discutindo sua estrutura, suas operações básicas e suas propriedades de desempenho. Também analisaremos diferentes variantes dessas estruturas e suas aplicações práticas, mostrando como elas podem ser utilizadas para resolver de maneira eficiente problemas que envolvem manipulação e busca em grandes conjuntos de strings ou sequências discretas.

Nas árvores AVL, a posição de um nó na estrutura depende essencialmente da comparação entre as chaves. Em contraste, nas árvores de busca digitais (*Tries*), a posição de um nó é determinada pela comparação entre os bits (ou símbolos) que compõem as chaves. Em vez de comparar diretamente valores inteiros ou strings completas, o algoritmo navega pela estrutura examinando progressivamente os dígitos que formam cada chave.

Esse tipo de estrutura de dados apresenta algumas vantagens e desvantagens importantes:

- **Vantagem:** a implementação é relativamente simples, pois não exige operações de balanceamento ou rotações, como ocorre em estruturas como árvores AVL ou árvores rubro-negras.
- **Desvantagem:** o desempenho depende diretamente do tamanho das chaves armazenadas. Quanto maior o número de dígitos ou caracteres que compõem cada chave, maior será o custo das operações de busca, inserção e remoção.

Suponha agora um conjunto finito de chaves Ω dado por

$$\Omega = \{A, B, C, D, \dots, W, X, Y, Z\}.$$

Considere que cada chave é codificada em binário de acordo com sua posição no conjunto. Como existem 26 símbolos, são necessários 5 bits para representá-los, pois

$$2^5 = 32.$$

Uma possível codificação binária para esses símbolos é apresentada a seguir:

$A : 00001$	$H : 01000$	$O : 01111$	$V : 10110$
$B : 00010$	$I : 01001$	$P : 10000$	$W : 10111$
$C : 00011$	$J : 01010$	$Q : 10001$	$X : 11000$
$D : 00100$	$K : 01011$	$R : 10010$	$Y : 11001$
$E : 00101$	$L : 01100$	$S : 10011$	$Z : 11010$
$F : 00110$	$M : 01101$	$T : 10100$	
$G : 00111$	$N : 01110$	$U : 10101$	

Essa representação binária permite construir uma árvore de busca digital na qual cada nível da árvore corresponde a um dos bits da chave. A navegação na árvore ocorre examinando-se sucessivamente os bits da chave, direcionando a busca para o filho esquerdo ou direito dependendo do valor do bit considerado.

3.1 Busca em árvores binárias digitais

A estratégia de busca em árvores binárias digitais consiste em comparar um bit da chave procurada com o bit correspondente associado ao nó corrente. Entretanto, como cada chave é composta por vários bits, surge a seguinte questão: qual bit deve ser comparado em cada etapa da busca? A resposta depende da profundidade em que o nó corrente se encontra na árvore.

De forma geral, cada nível da árvore corresponde a uma posição específica do código binário da chave. Assim, se estamos na raiz da árvore, comparamos o primeiro bit da chave (da esquerda para a direita). Se estamos no segundo nível, comparamos o segundo bit; no terceiro nível, comparamos o terceiro bit; e assim sucessivamente. Dessa forma, a profundidade máxima da árvore está diretamente relacionada com o número de bits necessários para codificar as chaves armazenadas. A figura a seguir ilustra um exemplo desse processo de busca.

Em resumo, suponha que desejamos buscar a chave G , cuja representação binária é

$$G = 00111.$$

Partindo da raiz da árvore, devemos examinar o primeiro bit da chave. Como esse bit é 0, seguimos

Buscar as seguintes chaves:

G: 00111 (ok)

R: 10010 (ok)

B: 00010 (x)

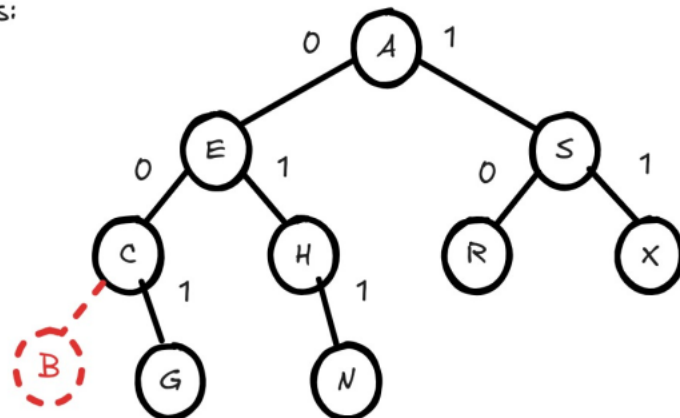


Figura 23: Ilustração gráfica do processo de busca em uma árvore de busca digital.

para o filho à esquerda. No segundo nível da árvore, analisamos o segundo bit, que também é 0, o que nos leva novamente ao filho à esquerda. No terceiro nível, examinamos o terceiro bit, que é 1, e portanto seguimos para o filho à direita. Esse processo continua até que a chave seja encontrada ou até que um nó nulo seja alcançado, indicando que a chave não está presente na estrutura.

Podemos definir o nó de uma árvore de busca digital como uma estrutura que armazena uma chave binária e referências para seus possíveis filhos, correspondentes aos valores 0 e 1 do bit analisado em cada nível da árvore.

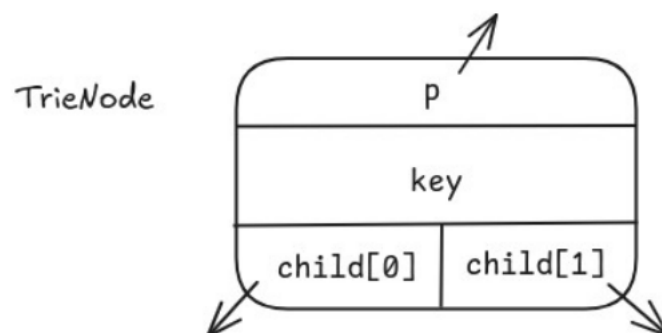


Figura 24: Ilustração gráfica de um nó de uma árvore de busca digital (Trie).

Para projetarmos um algoritmo de busca em árvores binárias digitais, considere inicialmente um cenário simples em que as chaves possuem apenas 3 bits. Nesse caso, não é possível ter caminhos com tamanho maior que 3, pois cada nível da árvore corresponde à análise de um dos bits da chave. Isso implica que qualquer chave presente na árvore deve estar a uma distância de, no máximo, 3 da raiz.

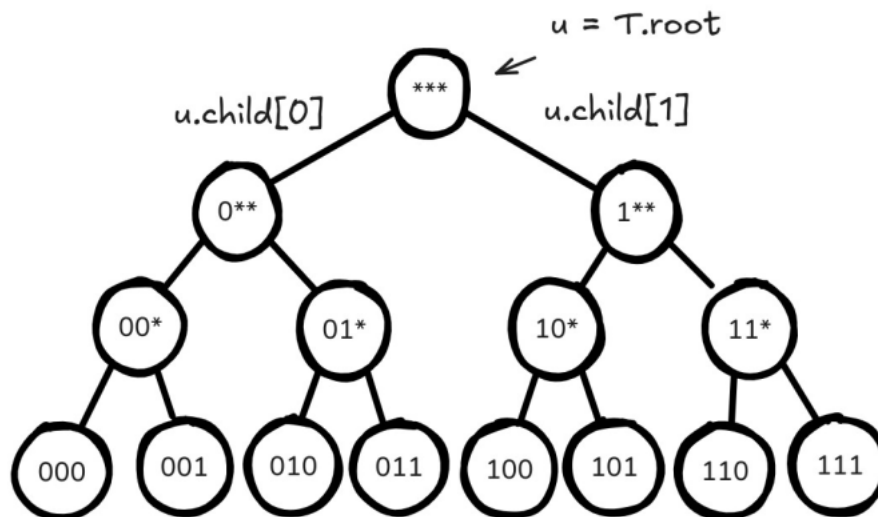


Figura 25: Ilustração gráfica da organização de uma árvore de busca digital.

Generalizando essa ideia, o algoritmo de busca em uma árvore digital cujas chaves possuem w bits pode ser descrito pelo pseudocódigo a seguir:

```

Trie_Search(T, x) {
    u = T.root
    for i = 0 to w-1 {
        c = (x.key >> (w-1)-i) & 1    // extrai o bit a ser comparado
        if u.key != x.key
            u = u.child[c]              // desce um nível
        else
            return True                  // chave encontrada
        if u == NIL
            return False                // chave não encontrada
    }
}

```

O comando utilizado para extrair o bit a ser analisado emprega operadores binários. Suponha, por exemplo, que $w = 3$ bits e que a chave a ser buscada seja 100. A ideia é que, ao iniciar na raiz, devemos analisar o primeiro bit (o mais significativo). Como nesse ponto temos $i = 0$, deslocamos os bits de $x.key$ duas posições para a direita, obtendo 001. Em seguida aplicamos o operador lógico $\&$ (*and*) com o valor 1, isolando assim o bit menos significativo, que neste caso é 1. Antes de apresentar o algoritmo de inserção em árvores binárias digitais, é importante destacar uma propriedade fundamental dessa estrutura.

Propriedade fundamental. Toda chave armazenada em uma árvore de busca digital encontra-se em algum ponto do caminho definido pela sequência de bits que compõem essa chave.

3.2 Inserção em árvores binárias digitais

Considere agora o processo de inserção em uma árvore de busca digital. Seja M a chave do nó a ser inserido, com representação binária

$$M = 01101.$$

Nesse caso, a chave deve ser inserida na primeira posição livre ao longo do caminho definido por essa sequência de bits. A figura a seguir ilustra uma sequência de inserções em uma árvore de busca digital.

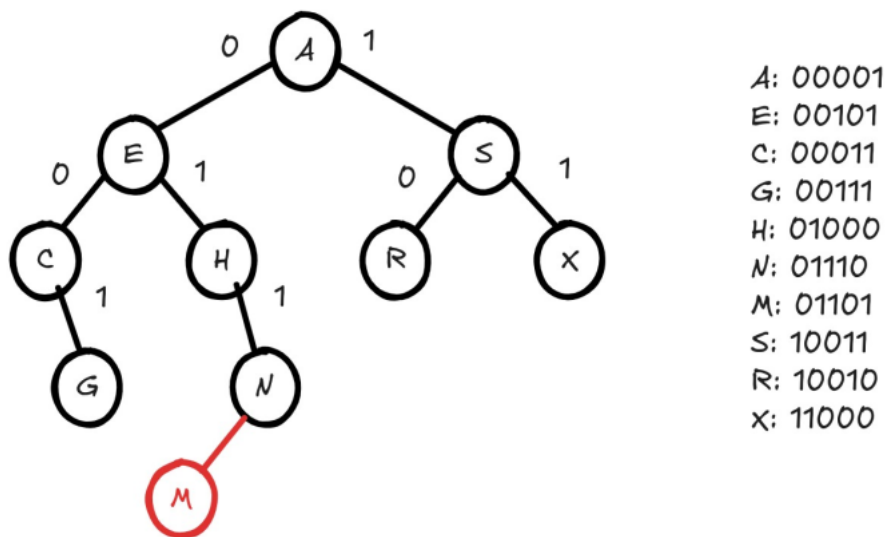


Figura 26: Ilustração gráfica da inserção de um nó em uma árvore de busca digital.

Observe, entretanto, que a árvore não mantém necessariamente as chaves em ordem no sentido tradicional de árvores binárias de busca. Por exemplo, pode ocorrer que $E > A$ e mesmo assim E esteja localizado na subárvore esquerda de A . Ainda assim, uma propriedade importante é preservada: todas as chaves localizadas na subárvore esquerda possuem prefixos binários diferentes das chaves da subárvore direita, de acordo com o bit analisado naquele nível.

Outro detalhe relevante é que todas as chaves presentes em uma subárvore no nível k compartilham os mesmos $k - 1$ bits iniciais. Isso ocorre porque cada nível da árvore corresponde à análise de um bit específico da chave. Uma consequência importante dessa propriedade é que, em árvores de busca digitais, não é necessário realizar operações explícitas de balanceamento. Após a inserção das chaves, a árvore tende a permanecer aproximadamente balanceada devido à forma como os bits das chaves definem os caminhos dentro da estrutura. Dessa forma, elimina-se a necessidade de operações de rotação, comuns em outras estruturas como árvores AVL ou árvores rubro-negras. Supondo que todas as chaves sejam distintas e possuam w bits, o número máximo de elementos que podem ser armazenados é

$$n = 2^w.$$

Isso ocorre porque a altura máxima da árvore é justamente w , e cada sequência possível de w bits corresponde a um caminho distinto na estrutura. Sob a hipótese de distribuição uniforme das chaves, a altura da árvore pode ser expressa como

$$h = \log_2 n = w.$$

O algoritmo de inserção em uma árvore digital de w bits pode ser descrito pelo seguinte pseudo-código:

```
Trie_Insert(T, x) {
    u = T.root
    for i = 0 to w-1 {
        c = (x.key >> (w-1)-i) & 1    // extrai bit a ser comparado
        if u.key == x.key
            return False                // chave já pertence à árvore
        if u.child[c] == NIL
            break                        // encontrou posição para inserir
        u = u.child[c]
    }
    u.child[c] = x
    x.p = u                            // u deve ser pai de x
}
```

3.3 Remoção em árvores binárias digitais

A remoção de nós em árvores binárias digitais pode ser um pouco mais delicada do que em outras estruturas de dados, pois em determinados casos a *propriedade fundamental* da estrutura pode ser violada. O caso mais simples ocorre quando o nó a ser removido é uma folha, isto é, quando ele não possui filhos. Nessa situação, basta remover o nó desligando a referência correspondente no nó pai.

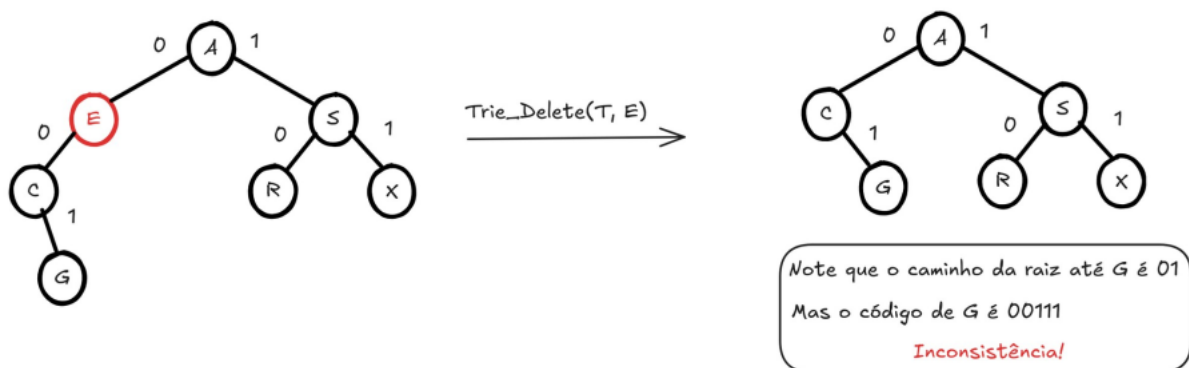


Figura 27: Exemplo de remoção do nó E em uma árvore de busca digital. Note que a simples remoção de E gera uma inconsistência na árvore.

Entretanto, quando o nó a ser removido possui um ou dois filhos, podem surgir inconsistências. Considere, por exemplo, uma árvore binária digital e suponha que desejamos remover a chave E . Após realizar simplesmente a remoção do nó correspondente, pode ocorrer que outra chave, como C , passe a ocupar uma posição que não respeita o caminho determinado pelos seus bits.

Suponha ainda que a chave G possua a codificação

$$G = 00111.$$

Entretanto, após a remoção de E , o caminho da raiz até G passa a ser 01. Note que isso gera uma inconsistência, pois 01 não é prefixo da representação binária de G . Dessa forma, a propriedade fundamental da árvore digital é violada.

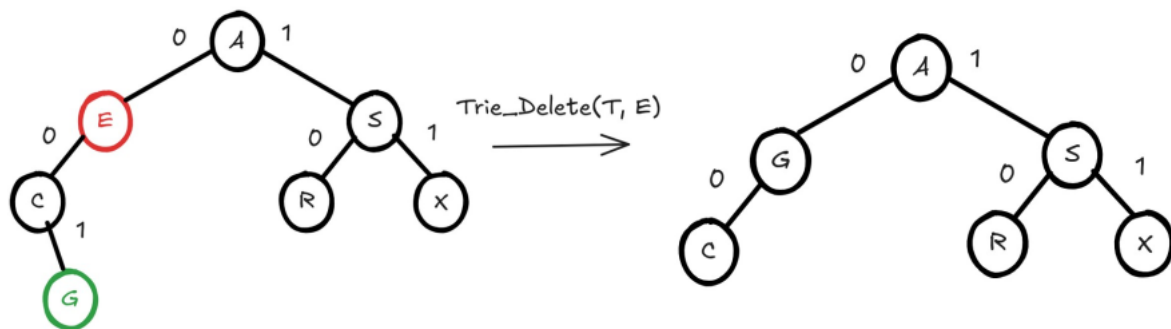


Figura 28: Ao substituir o nó E por um descendente que seja um nó folha, a propriedade chave das árvores de busca digitais é restaurada.

Para resolver esse problema, devemos substituir o nó removido por algum de seus descendentes que seja uma folha. Após realizar essa substituição, basta remover a folha correspondente. Esse procedimento funciona porque um nó folha pode ser posicionado em qualquer ponto do caminho que leva até ele sem violar a propriedade da estrutura. Por exemplo, as chaves

$$E = 00101 \quad \text{e} \quad G = 00111$$

compartilham o mesmo prefixo inicial, o que permite utilizar o nó correspondente a G para substituir E , removendo em seguida a folha original. O algoritmo de remoção em uma árvore digital de w bits pode ser descrito pelo seguinte pseudocódigo.

```
Trie_Delete(T, x) {
    u = T.root
    for i = 0 to w-1 {
        c = (x.key >> (w-1)-i) & 1    // extrai bit a ser comparado
        if u.key == x.key
            break                      // encontrou chave a ser removida
        if u.child[c] == NIL
            return False               // chave não pertence à árvore
        u = u.child[c]
```

```

    }
    if i == w-1 {                                // nó é folha
        u.p.child[c] = NIL                        // remove o nó
    }
    else {                                        // nó não é folha
        z = u
        while z.child[0] != NIL or z.child[1] != NIL {
            if z.child[0] != NIL {
                z = z.child[0]
                c = 0                            // viemos da esquerda
            }
            else {
                z = z.child[1]
                c = 1                            // viemos da direita
            }
        }
        u.key = z.key                            // substitui chave
        z.p.child[c] = NIL                       // remove folha
    }
}

```

3.4 Análise da Complexidade

Nesta seção analisamos a complexidade das principais operações em árvores de busca digitais (*tries*): busca, inserção e remoção. Diferentemente das árvores binárias de busca tradicionais, nas quais o custo das operações depende do número de elementos armazenados, nas *tries* o custo está diretamente relacionado ao tamanho das chaves, isto é, ao número de bits ou caracteres que as compõem.

Suponha que cada chave seja representada por uma sequência de w bits. Nesse caso, a profundidade máxima da árvore também será w , pois cada nível corresponde à análise de um dos bits da chave. Lembrando que o número de bits para representar uma chave n é igual a $w = \log_2 n$.

Busca A operação de busca consiste em percorrer a árvore a partir da raiz, examinando sucessivamente os bits da chave procurada. Em cada nível da árvore, apenas um bit é analisado para decidir qual filho deve ser visitado. Como existem no máximo w níveis, o número máximo de iterações do algoritmo de busca é w .

Assim, a complexidade da operação de busca é dada por

$$T_{busca}(w) = O(w).$$

Observe que esse custo é independente do número total de chaves armazenadas na estrutura.

Inserção A operação de inserção segue essencialmente o mesmo caminho que a operação de busca. Inicialmente percorremos a árvore examinando os bits da chave até encontrar a primeira posição livre ao longo do caminho definido por sua sequência binária. Em seguida, o novo nó é inserido nessa posição.

Como no pior caso o algoritmo precisa examinar todos os w bits da chave, a complexidade da operação de inserção também é

$$T_{inse}(w) = O(w).$$

Novamente, esse custo depende apenas do tamanho da chave e não da quantidade total de elementos armazenados.

Remoção A operação de remoção também envolve inicialmente a busca da chave a ser removida, o que requer no máximo $w = \log_2 n$ passos. Caso o nó seja uma folha, sua remoção ocorre em tempo constante.

Entretanto, quando o nó possui filhos, é necessário encontrar um descendente folha para substituí-lo antes da remoção. No pior caso, pode ser necessário percorrer um caminho adicional até alcançar essa folha, cujo comprimento também é limitado pela altura da árvore.

Dessa forma, a complexidade da operação de remoção é dada por

$$T_{remo}(w) = O(w).$$

Portanto, as três operações fundamentais em árvores de busca digitais apresentam complexidade linear em relação ao tamanho da chave:

$$T_{busca} = O(w) = O(\log_2 n)$$

$$T_{inse} = O(w) = O(\log_2 n)$$

$$T_{remo} = O(w) = O(\log_2 n)$$

Como em muitos cenários práticos o tamanho das chaves w é limitado (e frequentemente pequeno em comparação com o número de elementos armazenados), as árvores de busca digitais podem oferecer desempenho muito eficiente para operações de busca e atualização. Por outro lado, quando as chaves são muito longas, o custo das operações pode se tornar mais elevado, o que constitui uma das principais limitações dessa estrutura de dados.

3.5 Considerações Finais

Neste capítulo foram apresentadas as árvores de busca digitais, conhecidas como *tries*, uma estrutura de dados que organiza chaves com base em sua representação interna em termos de bits ou caracteres. Diferentemente das árvores binárias de busca tradicionais, nas quais a navegação depende

de comparações entre chaves completas, as *tries* exploram diretamente a estrutura dos dados, percorrendo a árvore de acordo com os dígitos que compõem cada chave.

Foram discutidas as operações fundamentais dessa estrutura (busca, inserção e remoção) mostrando como cada uma delas se baseia na análise sequencial dos bits das chaves. Essa característica faz com que a complexidade dessas operações dependa essencialmente do tamanho da chave, e não do número total de elementos armazenados na árvore. Como resultado, as operações possuem custo $O(w)$, onde w representa o número de bits ou caracteres utilizados para codificar as chaves.

Uma vantagem importante das árvores de busca digitais é a simplicidade de sua implementação. Como a posição de cada chave é determinada diretamente por sua sequência de bits, não há necessidade de manter propriedades explícitas de balanceamento nem realizar operações de rotação, como ocorre em estruturas como árvores AVL ou árvores rubro-negras. Além disso, a estrutura naturalmente compartilha prefixos comuns entre diferentes chaves, o que pode resultar em ganhos de eficiência em aplicações envolvendo grandes conjuntos de strings.

Por outro lado, essa estrutura também apresenta algumas limitações. Como o custo das operações depende diretamente do tamanho das chaves, o desempenho pode ser impactado quando as chaves são muito longas. Além disso, o número potencial de nós da estrutura pode crescer rapidamente quando o alfabeto de símbolos é grande ou quando as chaves não compartilham muitos prefixos em comum.

Apesar dessas limitações, as árvores de busca digitais são amplamente utilizadas em aplicações que envolvem manipulação e busca em sequências de caracteres, como dicionários computacionais, sistemas de *autocomplete*, compressão de dados, roteamento de redes e processamento de linguagem natural. Nessas situações, a capacidade de organizar e acessar dados de acordo com seus prefixos torna essa estrutura particularmente eficiente.

Em síntese, as *tries* constituem uma alternativa interessante às estruturas tradicionais de busca, especialmente quando lidamos com dados que possuem representação sequencial natural, como strings ou códigos binários. O entendimento de suas propriedades e limitações é fundamental para escolher a estrutura de dados mais adequada em diferentes problemas computacionais.

4 Skip Lists

Skip Lists, ou listas com saltos, são estruturas de dados probabilísticas que permitem realizar as operações de busca, inserção e remoção com complexidade esperada $O(\log n)$. Essas estruturas combinam algumas das vantagens de vetores ordenados, como a eficiência da busca binária, com a flexibilidade dinâmica das listas encadeadas. Uma *Skip List* é composta por uma hierarquia de listas encadeadas organizadas em diferentes níveis. À medida que o nível aumenta, a estrutura torna-se mais esparsa, ou seja, menos elementos aparecem nas listas superiores. Algumas características importantes dessa estrutura são:

- Uma *Skip List* é composta por múltiplas camadas de listas encadeadas ordenadas.
- Quanto maior o nível da lista, maior sua esparsidade, ou seja, menos elementos são armazenados nesse nível.
- A estrutura utiliza um parâmetro conhecido como *fator de dispersão* $d > 1$, que controla a probabilidade de que um elemento presente no nível i seja replicado no nível $i + 1$.

De forma resumida, uma *Skip List* é construída em camadas da seguinte maneira:

1. A camada inferior corresponde a uma lista encadeada ordenada tradicional contendo todos os elementos.
2. Cada camada superior funciona como uma espécie de *via expressa*. Um elemento presente na camada i aparece na camada $i + 1$ com probabilidade

$$q = \frac{1}{d}.$$

Por exemplo, se $d = 2$, cada elemento possui 50% de chance de aparecer no nível imediatamente superior. Podemos interpretar esse processo como o lançamento de uma moeda: se o resultado for *cara*, o elemento da camada i é replicado na camada $i + 1$; caso contrário, ele não é promovido.

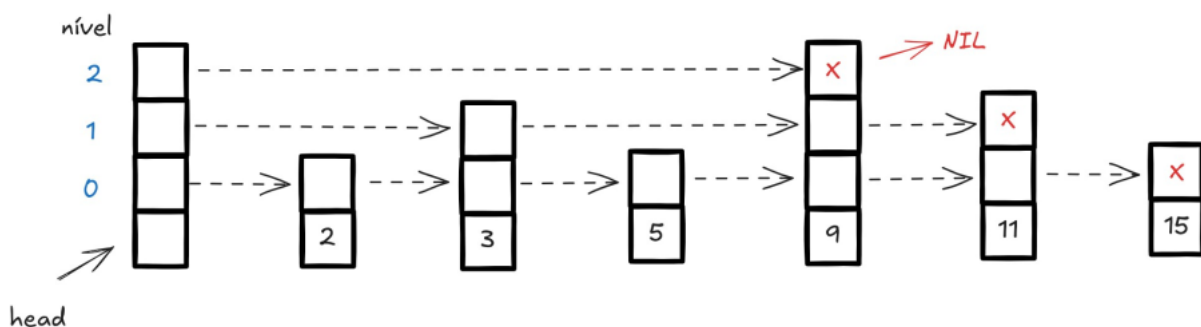


Figura 29: Ilustração gráfica de uma Skip List com três níveis: 0, 1 e 2.

Pode-se demonstrar que, em média, cada elemento aparece em

$$\frac{d}{d-1}$$

listas diferentes. O primeiro nó da estrutura é geralmente implementado como um vetor de referências (ponteiros), que funciona como as cabeças das diferentes listas encadeadas que compõem os níveis da *Skip List*.

Busca em Skip Lists

A busca em uma *Skip List* sempre deve ser iniciada no nível mais alto da estrutura, isto é, na lista mais esparsa. Caso a chave não seja encontrada nesse nível, o algoritmo desce progressivamente para os níveis inferiores até chegar à lista base. A figura a seguir ilustra o processo de busca pela chave 8.

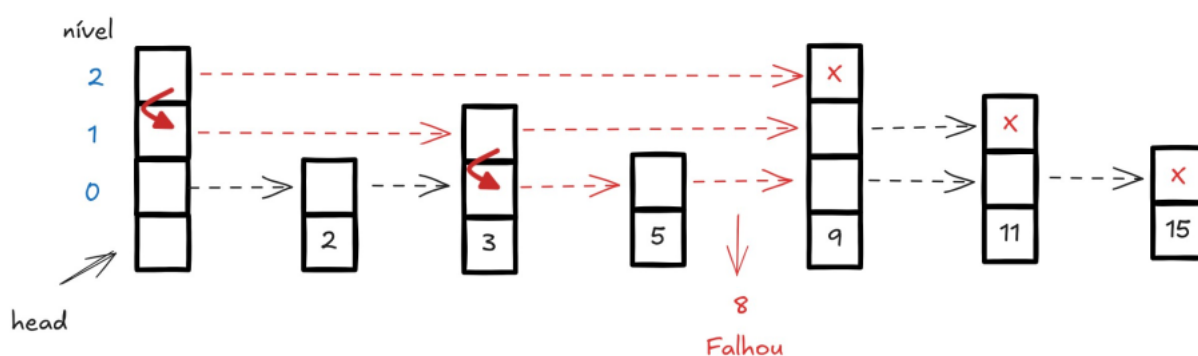


Figura 30: Ilustração gráfica da busca pela chave 8 em uma Skip List.

A lógica da busca pela chave 8 consiste em iniciar no nível 2 e percorrer a lista enquanto o valor da chave corrente for menor que 8. Esse processo continua até que encontremos um elemento com $x.key > 8$ ou até o final da lista.

Suponha que a chave não seja encontrada no nível 2. Nesse caso, o algoritmo desce para o nível 1. Nesse nível, o algoritmo passa pela chave 3 e chega à chave 9. Como novamente não encontrou a chave procurada, o algoritmo desce para o nível 0, reiniciando a busca a partir do último elemento visitado no nível superior (no caso, o elemento 3). Em seguida, passa pela chave 5 e chega à chave 9. Como esse é o nível mais baixo da estrutura e a chave 8 não foi encontrada, o algoritmo conclui que essa chave não pertence à *Skip List*.

Antes de apresentar o algoritmo de busca, é importante entender como as informações são organizadas em um nó da estrutura. Cada nó de uma *Skip List* possui, além do campo `key`, um vetor de referências (ponteiros) para outros nós da estrutura. Esse vetor, geralmente denominado `forward`, permite que um mesmo nó esteja ligado a diferentes níveis da lista, possibilitando a navegação rápida entre os elementos.

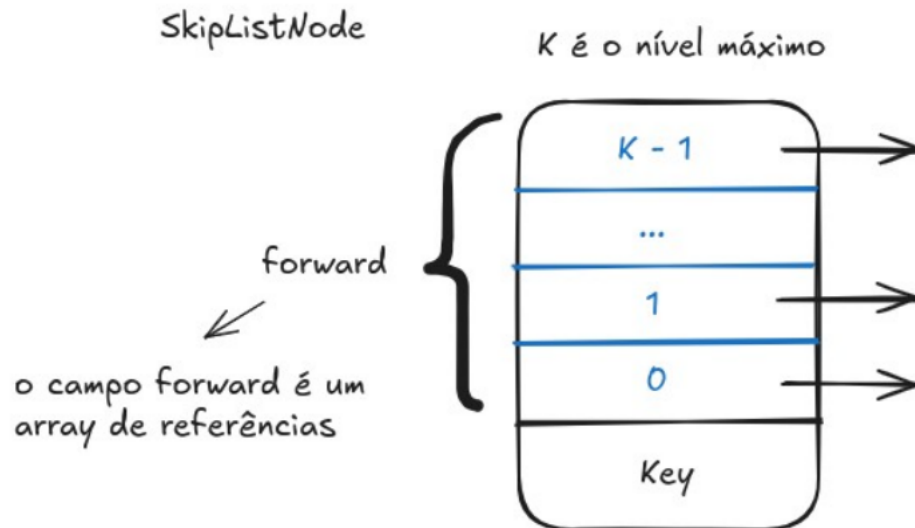


Figura 31: Ilustração gráfica da estrutura de um nó de uma Skip List.

A seguir apresentamos o pseudocódigo do algoritmo utilizado para buscar uma chave em uma *Skip List*.

```
SkipList_Search(L, key) {
    x = L.head
    for i = L.level downto 0 {
        while x.forward[i].key < key
            x = x.forward[i]
    }
    // neste ponto x aponta para o antecessor da chave buscada
    x = x.forward[0]
    if x.key == key
        return x.key
    else
        return False
}
```

4.1 Análise da Complexidade

O fator de dispersão d representa a probabilidade de um elemento do nível i ser replicado para o nível $i + 1$. Em outras palavras, podemos dizer que, ao passar para o próximo nível, o número de nós na lista encadeada diminui, em média, por um fator $q = 1/d$. Assim, o número esperado de elementos em cada nível pode ser representado da seguinte forma:

Nível	Número esperado de nós
0	n
1	$\frac{n}{d}$
2	$\frac{n}{d^2}$
3	$\frac{n}{d^3}$
$\vdots$	$\vdots$
k	$\frac{n}{d^k}$

No nível mais alto da estrutura espera-se que exista apenas um elemento. Logo,

$$\frac{n}{d^k} = 1$$

o que implica

$$n = d^k.$$

Portanto,

$$k = \log_d n.$$

Assim, uma *Skip List* contendo n elementos possui, no máximo, $1 + \log_d n$ níveis. Além disso, espera-se que para cada nó presente no nível $i + 1$ existam aproximadamente d nós no nível i . Dessa forma, durante uma busca, espera-se realizar em média $d/2$ deslocamentos horizontais em um nível antes de descer para o nível inferior. Portanto, considerando o produto entre o número esperado de níveis e o número esperado de deslocamentos por nível, o tempo médio de busca é dado por

$$\frac{d}{2}(1 + \log_d n) = O(\log_d n),$$

assumindo d como uma constante. Assim, as operações de busca, inserção e remoção em uma *Skip List* possuem complexidade média logarítmica.

Eficiência de Espaço

Podemos agora analisar o número médio de nós armazenados em uma *Skip List*. No primeiro nível existem n nós, no segundo $\frac{n}{d}$, no terceiro $\frac{n}{d^2}$, e assim sucessivamente. Logo, o número total esperado de nós é dado por:

$$S = n + \frac{n}{d} + \frac{n}{d^2} + \frac{n}{d^3} + \dots$$

Colocando n em evidência:

$$S = n \left(1 + \frac{1}{d} + \frac{1}{d^2} + \frac{1}{d^3} + \dots \right).$$

A expressão entre parênteses corresponde à soma de uma progressão geométrica infinita com termo inicial $a_0 = 1$ e razão $q = 1/d$. Logo,

$$S = n(1 + q + q^2 + q^3 + \cdots).$$

Seja a soma S' definida por

$$S' = qS = n(q + q^2 + q^3 + \cdots).$$

Calculando a diferença $S - S'$, obtemos

$$S - S' = (1 - q)S = n.$$

Assim,

$$S = \frac{n}{1 - q}.$$

Substituindo $q = \frac{1}{d}$, temos

$$S = \frac{n}{1 - \frac{1}{d}} = \frac{n}{\frac{d-1}{d}} = \frac{nd}{d-1}.$$

Portanto,

$$S = O(n).$$

Conclui-se que a eficiência de espaço de uma *Skip List* é linear em relação ao tamanho da entrada, sendo assintoticamente equivalente ao consumo de memória de uma lista encadeada tradicional.

Relação entre Tempo e Espaço

Podemos analisar o comportamento das *Skip Lists* considerando diferentes valores para o fator de dispersão d . Esse parâmetro controla simultaneamente a altura esperada da estrutura e a quantidade total de nós armazenados.

a) Suponha que $d = 2$.

Nesse caso, o número esperado de níveis é

$$\log_2 n.$$

A quantidade média de nós armazenados na estrutura é dada por

$$\frac{2n}{2-1} = 2n.$$

Logo, o consumo de memória é aproximadamente proporcional a $2n$, enquanto o tempo médio de busca é

$$O(\log_2 n).$$

b) Suponha agora que $d = 10$.

O número esperado de níveis passa a ser

$$\log_{10} n.$$

Podemos escrever essa expressão em função do logaritmo na base 2:

$$\log_{10} n = \frac{\log_2 n}{\log_2 10} \approx \frac{\log_2 n}{3.32} \approx 0.30 \log_2 n.$$

Entretanto, em cada nível espera-se percorrer aproximadamente $d/2$ elementos antes de descer para o próximo nível. Logo, o custo médio de busca é aproximadamente

$$\frac{d}{2} \log_d n = 5 \log_{10} n \approx 1.5 \log_2 n.$$

Em relação ao espaço utilizado, temos

$$\frac{10n}{10-1} = \frac{10}{9}n \approx 1.11n.$$

Portanto, podemos observar que quanto maior for o fator de dispersão d , menor será o consumo de memória da *Skip List*, porém maior será o custo médio de busca. Em outras palavras, existe um compromisso (*trade-off*) entre tempo de execução e eficiência de espaço.

Análise da Probabilidade

A construção de uma *Skip List* depende de decisões aleatórias. A cada novo nível da estrutura temos menos nós, mais precisamente $\frac{1}{d}$ do número de nós do nível anterior. Para obter esse comportamento, cada nó deve pertencer ao nível i com probabilidade

$$\left(\frac{1}{d}\right)^i.$$

Por razões didáticas, considere o caso em que $d = 2$. Nesse cenário, podemos gerar um número aleatório uniforme no intervalo $[0, 1]$ para decidir o nível de um novo elemento.



Figura 32: Intervalos para o cálculo da probabilidade de uma chave cair no nível k em uma Skip List com fator de dispersão $d = 2$.

O nível é definido de acordo com o subintervalo no qual o número sorteado cai:

- se cair no intervalo $[0, 1/2)$, o nó aparece até o nível 1;
- se cair no intervalo $[1/2, 3/4)$, o nó aparece até o nível 2;
- se cair no intervalo $[3/4, 7/8)$, o nó aparece até o nível 3;
- se cair no intervalo $[7/8, 15/16)$, o nó aparece até o nível 4;
- e assim sucessivamente.

Probabilidades	1	0.5	0.25	0.125	0.0625	...
Níveis	0	1	2	3	4	5 ...

Figura 33: Probabilidades de uma chave cair no nível k em uma Skip List com fator de dispersão $d = 2$.

Observe que a probabilidade de um valor cair em cada subintervalo é dada exatamente pela diferença entre o limite superior e o limite inferior do intervalo. Dessa forma, obtemos uma distribuição geométrica que garante que níveis superiores sejam progressivamente mais esparsos, característica essencial para o funcionamento eficiente das *Skip Lists*.

4.2 Inserção em Skip Lists

A operação de inserção em uma *Skip List* baseia-se em dois passos principais. Primeiro, é necessário sortear aleatoriamente o nível no qual o novo nó aparecerá. Em seguida, deve-se localizar a posição correta da chave na estrutura, processo bastante semelhante ao algoritmo de busca apresentado anteriormente. A função a seguir ilustra o algoritmo responsável por sortear um nível aleatório para um novo nó.

```
Random_Level(q) {
    // o parâmetro q = 1/d deve ser definido previamente
    level = 1
```

```

// geralmente define-se um nível máximo para evitar níveis muito
altos
while random() < q and level < maxlevel
    level = level + 1
return level
}

```

Note que, se $q = 0.5$, o processo equivale a lançar repetidamente uma moeda: enquanto o resultado for “cara”, o nó sobe para o próximo nível, até atingir o nível máximo permitido. A figura a seguir ilustra o processo de inserção da chave 8 em uma *Skip List*.

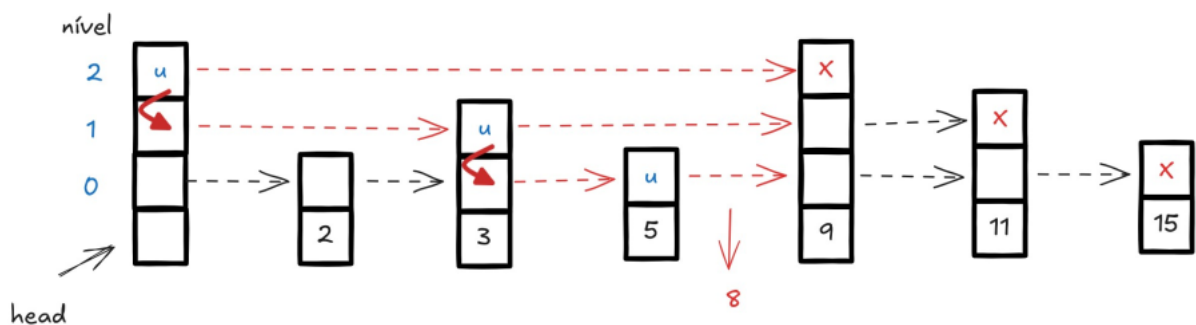


Figura 34: Antes de inserir a chave 8, devemos encontrar sua posição correta na estrutura.

Após encontrar a posição correta do novo elemento, devemos atualizar os ponteiros dos nós predecessores em cada nível da estrutura.

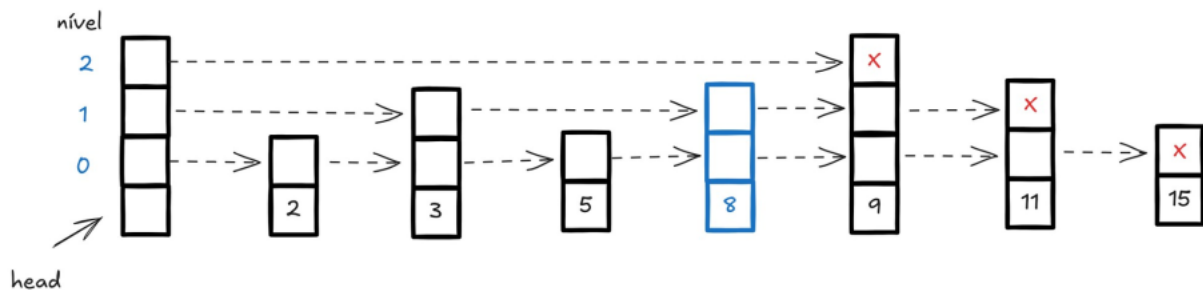


Figura 35: Após encontrar a posição da chave 8, devemos realizar o encadeamento lógico.

Para isso, utiliza-se uma variável auxiliar denominada *update*, que armazena referências para os nós imediatamente anteriores ao ponto de inserção em cada nível da estrutura. Assim, *update* deve ser um vetor de K referências, uma para cada nível da *Skip List*.

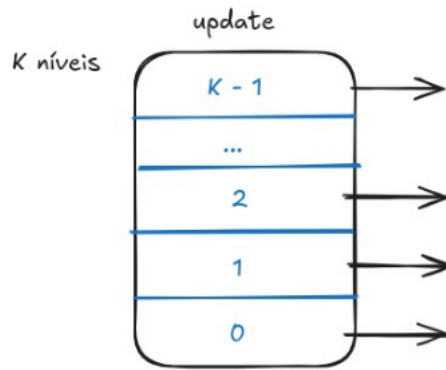


Figura 36: Para o encadeamento lógico, é preciso uma variável auxiliar que é definida como um array de referências (uma para cada nível da Skip List).

A seguir apresentamos o pseudocódigo para o algoritmo de inserção em uma *Skip List*. O funcionamento é o seguinte: se a chave *searchKey* ainda não existe na estrutura, ela é inserida em sua posição correta. Caso a chave já exista, seu valor é simplesmente atualizado para *newKey*.

```
SkipList_Insert(L, searchKey, newKey) {
    SkipListNode update[0...K-1] // array de referências para nós
    x = L.head
    // encontra a posição correta do nó
    for i = L.level downto 0 {
        while x.forward[i].key < searchKey
            x = x.forward[i]
        update[i] = x
    }
    // aqui, x.forward[0] aponta para o local de inserção
    x = x.forward[0]
    // se chave já existe, atualiza valor
    if x.key == searchKey
        x.key = newKey
    else {
        // se chave não existe, insere novo nó
        level = Random_Level(q)
        // se nível sorteado é maior que o nível atual da lista
        if level > L.level {
            for i = L.level+1 to level
                update[i] = L.head
            L.level = level
        }
        // Cria novo nó
        x = Make_Node(level, searchKey)
        // Realiza o encadeamento lógico
```

```

for i = 0 to level {
    x.forward[i] = update[i].forward[i]
    update[i].forward[i] = x
}
}
}

```

Como *Skip Lists* são estruturas de dados aleatorizadas, a inserção de um mesmo conjunto fixo de n chaves nunca produzirá exatamente a mesma estrutura. A distribuição dos nós pelos diferentes níveis depende dos sorteios realizados durante as inserções, de modo que a organização final da estrutura pode variar a cada execução do algoritmo.

4.3 Remoção em Skip Lists

A remoção de um nó em uma *Skip List* é semelhante ao processo de inserção, pois também exige a identificação dos nós predecessores em cada nível da estrutura. Entretanto, além de atualizar corretamente as referências (ponteiros), é necessário verificar se o nó removido era o último elemento presente em algum nível superior da estrutura.

Caso o nó removido seja o único elemento de um determinado nível k , após a sua remoção esse nível se tornará uma lista vazia. Nesse caso, o nível deve ser eliminado da estrutura, reduzindo o número total de níveis da *Skip List*. Esse cenário ocorre, por exemplo, ao remover a chave 9 da estrutura ilustrada na figura a seguir.

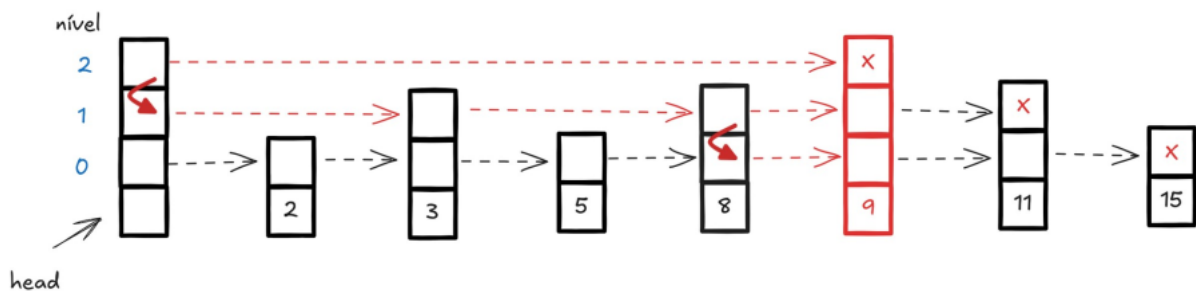


Figura 37: Para remover um nó da Skip List, é preciso identificar os antecessores em cada nível da estrutura com uma variável auxiliar.

Observe que, após a remoção desse elemento, o nível mais alto da *Skip List* passa a ser vazio. Portanto, esse nível deve ser descartado, fazendo com que a altura total da estrutura diminua em uma unidade. De forma semelhante ao algoritmo de inserção, utiliza-se um vetor auxiliar *update* para armazenar os nós predecessores do elemento a ser removido em cada nível da estrutura.

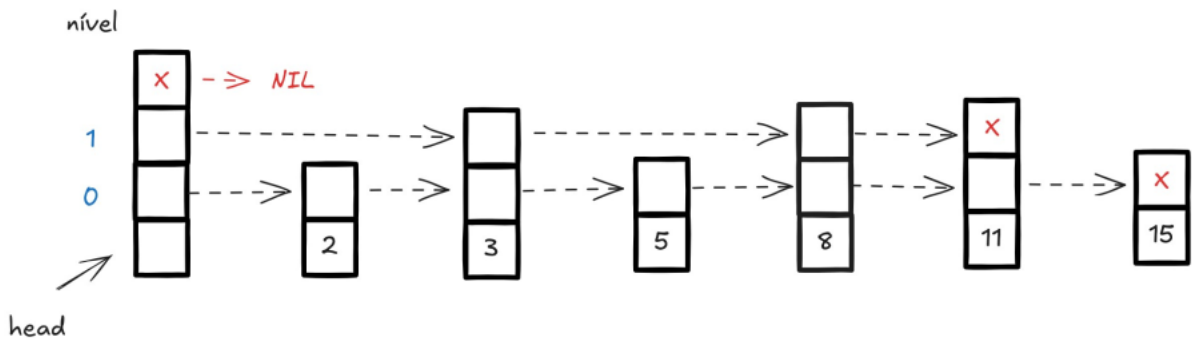


Figura 38: Se após a remoção de um nó, um nível da Skip List ficar vazio, ele pode ser removido.

A seguir apresentamos o pseudocódigo para o algoritmo de remoção em uma *Skip List*.

```
SkipList_Delete(L, searchKey) {
    SkipListNode update[0...K-1]
    x = L.head
    for i = L.level downto 0 {
        while x.forward[i].key < searchKey
            x = x.forward[i]
        update[i] = x
    }
    // neste ponto x.forward[0] aponta para o nó a ser removido
    x = x.forward[0]
    // Encontrou a chave a ser removida
    if x.key == searchKey {
        for i = 0 to L.level {
            // Se "passa por cima" de x, não altera o encadeamento
            if update[i].forward[i] != x
                break
            update[i].forward[i] = x.forward[i]
        }
        free(x) // desaloca o nó
        // remove níveis vazios da estrutura
        while L.level > 0 and L.header.forward[L.level] == NIL
            L.level = L.level - 1
    }
}
```

Análise da Complexidade

Tanto a inserção quanto a remoção de nós em *Skip Lists* começam com uma busca para localizar a posição correta onde as referências (ponteiros) devem ser ajustadas. Após essa etapa, o algoritmo

realiza apenas modificações nas referências dos nós predecessores, o que leva tempo constante para cada nível da estrutura.

Assim, o custo total dessas operações pode ser expresso como

$$O(\log_d n) + O(k),$$

onde k representa o número de níveis da *Skip List*. Como tipicamente $k \ll n$, o termo dominante é

$$O(\log_d n).$$

Portanto, as operações de busca, inserção e remoção em uma *Skip List* apresentam complexidade média logarítmica. Na prática, observa-se frequentemente que as operações de inserção e remoção em *Skip Lists* podem ser mais rápidas do que as operações equivalentes em árvores AVL. Isso ocorre principalmente no caso da remoção, pois nas árvores AVL é necessário localizar o sucessor do nó removido e realizar possíveis rotações para manter a propriedade de balanceamento.

Outra vantagem importante das *Skip Lists* é que as chaves são mantidas naturalmente em ordem. Dessa forma, operações como encontrar o menor ou o maior elemento da estrutura podem ser realizadas em tempo constante. Com base em diversos estudos empíricos, observou-se que *Skip Lists* podem superar árvores AVL em determinadas situações, às vezes de forma significativa. Por exemplo, *Skip Lists* são particularmente eficientes quando se trabalha com conjuntos de chaves já ordenadas (em ordem crescente ou decrescente). Mesmo quando as chaves aparecem em ordem aleatória, a estrutura apresenta bom desempenho em operações como contagem de elementos menores que um determinado valor e exclusão de elementos. Por outro lado, quando a operação predominante é a busca em conjuntos de dados não ordenados, árvores AVL podem apresentar desempenho mais previsível. Entretanto, quando o número de inserções e remoções é elevado, *Skip Lists* tendem a apresentar melhor desempenho prático.

Além disso, do ponto de vista de implementação, *Skip Lists* são consideravelmente mais simples do que árvores AVL. As árvores AVL exigem operações de rotação e manutenção rigorosa do fator de balanceamento, o que torna sua implementação mais complexa. Em contrapartida, *Skip Lists* utilizam apenas operações de ajuste de ponteiros e decisões aleatórias, o que simplifica bastante sua implementação.

Em resumo, pode-se afirmar que *Skip Lists* constituem uma alternativa elegante e eficiente às árvores balanceadas, oferecendo desempenho comparável em muitas situações e com menor complexidade de implementação.

4.4 Considerações Finais

As *Skip Lists* representam uma alternativa elegante e eficiente às estruturas clássicas de dados baseadas em árvores balanceadas. Ao utilizar aleatorização para controlar a altura da estrutura, elas conseguem oferecer complexidade média $O(\log n)$ para as operações fundamentais de busca, inserção

e remoção, mantendo ao mesmo tempo uma implementação consideravelmente mais simples do que estruturas como árvores AVL ou árvores rubro-negras.

Uma característica particularmente interessante das *Skip Lists* é que sua organização em múltiplos níveis permite explorar paralelismo e concorrência de forma mais natural do que em árvores balanceadas tradicionais. Por essa razão, essas estruturas são amplamente utilizadas em sistemas concorrentes e paralelos, especialmente quando múltiplas operações de leitura e escrita precisam ocorrer simultaneamente sobre uma mesma estrutura de dados.

Em sistemas distribuídos, por exemplo, *Skip Lists* são frequentemente utilizadas para implementar filas de prioridades concorrentes altamente escaláveis. A estrutura probabilística reduz a necessidade de mecanismos complexos de balanceamento, o que diminui a contenção entre threads e melhora o desempenho em ambientes com grande volume de operações simultâneas.

Outro contexto importante de aplicação é o desenvolvimento de sistemas operacionais. Em algumas situações, *Skip Lists* têm sido consideradas como alternativa a estruturas tradicionais como árvores AVL e árvores rubro-negras no kernel do Linux, principalmente quando se deseja simplificar a implementação de determinadas estruturas internas sem comprometer significativamente o desempenho assintótico.

Além disso, *Skip Lists* também aparecem em aplicações de grande escala relacionadas à recuperação de informação. Motores de busca (*search engines*), por exemplo, utilizam estruturas semelhantes para organizar e acessar rapidamente grandes volumes de dados. Um caso notável é o projeto de código aberto Apache Lucene, amplamente utilizado em sistemas de indexação e busca textual.

Em síntese, as *Skip Lists* demonstram que técnicas probabilísticas podem ser empregadas com sucesso no projeto de estruturas de dados eficientes. Elas combinam bom desempenho teórico, simplicidade de implementação e excelente adaptabilidade a ambientes concorrentes e distribuídos, o que explica sua presença em diversas aplicações modernas de software.

Note que todas as árvores mostradas na figura acima ferem alguma das 4 propriedades, ou seja, nenhuma delas é uma árvore rubro-negra válida. Exemplos de árvores rubro-negras válidas são mostradas a seguir.

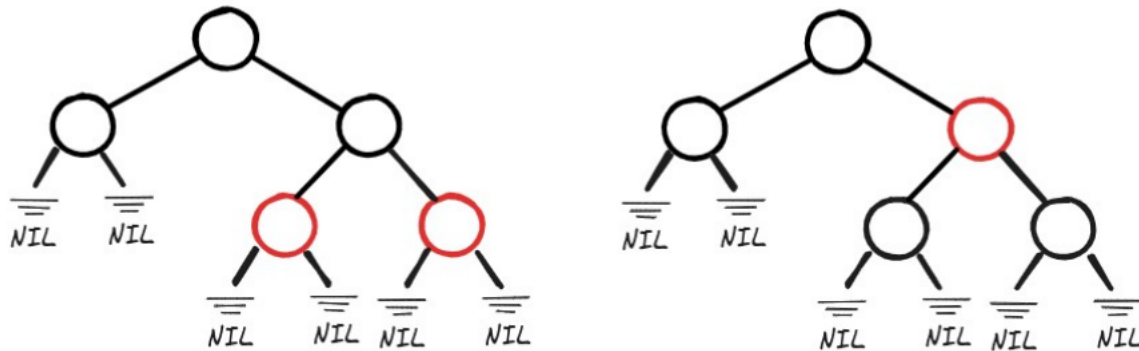


Figura 40: Exemplos de árvores de 5 nós que são árvores rubro-negras válidas.

Note que nas duas árvores anteriores, o número de nós pretos em qualquer caminho raiz-NIL é sempre igual a 2. Uma pergunta interessante é: qual é a altura máxima de uma árvore rubro-negra? A seguir iremos analisar como responder a essa questão.

5.1 Altura Máxima de Árvores Rubro-Negras

Lembre-se que a altura de uma árvore é definida como o número de arestas do maior caminho raiz-NIL. Assim, nosso objetivo é obter um limitante superior para o comprimento dos caminhos em uma árvore rubro-negra.

Teorema. Se todo caminho raiz-NIL de uma árvore rubro-negra T possui pelo menos k nós, então os primeiros k níveis da árvore T devem estar completos.

Esse resultado pode ser verificado facilmente por meio da contrapositiva. Recordando que

$$p \rightarrow q \equiv \neg q \rightarrow \neg p,$$

temos que:

Se os primeiros k níveis de T não estão completos, então existe um caminho raiz-NIL contendo menos de k nós.

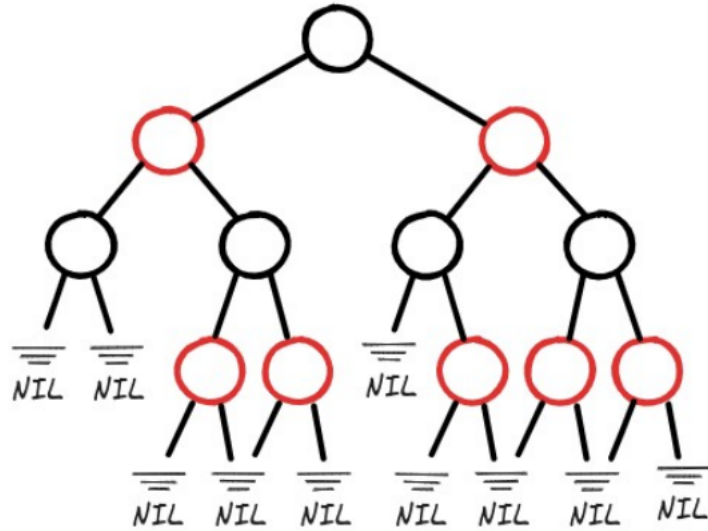


Figura 41: Se o k -ésimo nível de uma árvore não está completo, então existe um caminho raiz-NIL com menos de k nós.

Observe que, se o nível $k = 3$ de uma árvore não está completo, então necessariamente existirão caminhos raiz-NIL com apenas $k - 1 = 2$ nós. Lembre-se de que o comprimento de um caminho C é dado por $|C| - 1$, onde $|C|$ representa o número de nós no caminho C . Suponha agora que os k primeiros níveis da árvore estejam completos. Nesse caso, o número de nós da árvore T é pelo menos igual ao número de nós de uma árvore binária completa de altura $k - 1$. Assim,

$$n \geq 2^0 + 2^1 + 2^2 + \cdots + 2^{k-1} = \sum_{i=0}^{k-1} 2^i = 2^k - 1.$$

Logo,

$$2^k \leq n + 1,$$

o que implica

$$k \leq \log_2(n + 1).$$

Portanto, $k = O(\log n)$, mostrando que a altura da árvore cresce de forma logarítmica em relação ao número de nós. Agora utilizamos a propriedade 4 da definição de árvores rubro-negras, segundo a qual todo caminho raiz-NIL possui o mesmo número de nós pretos. Denotemos esse número por p . Claramente, temos um limitante superior para p , pois ele não pode exceder a altura da árvore. Assim,

$$p \leq \log_2(n + 1).$$

Entretanto, queremos limitar o número total de nós em qualquer caminho raiz-NIL, pois isso determina diretamente a altura da árvore. Observe que, pelas propriedades 2 e 3 da definição de árvores rubro-negras, não podem existir dois nós vermelhos consecutivos e a raiz é sempre preta.

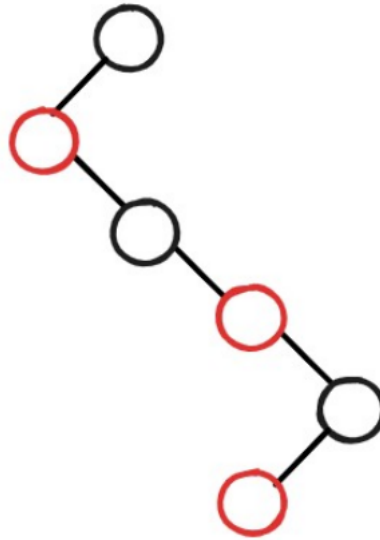


Figura 42: Em uma árvore rubro-negra, há no máximo um nó vermelho entre quaisquer dois nós pretos.

Portanto, ao longo de qualquer caminho, no máximo um nó vermelho pode aparecer entre dois nós pretos. Consequentemente, o número total de nós t em qualquer caminho raiz-NIL é no máximo o dobro do número de nós pretos:

$$t \leq 2p.$$

Substituindo o limite superior para p , obtemos

$$t \leq 2 \log_2(n + 1).$$

Esse resultado é fundamental, pois nos permite estabelecer a seguinte propriedade chave das árvores rubro-negras:

O comprimento do caminho da raiz até a folha mais distante é no máximo duas vezes o comprimento do caminho da raiz até a folha mais próxima.

Essa propriedade garante que árvores rubro-negras são *aproximadamente balanceadas*. Embora não sejam tão estritamente balanceadas quanto árvores AVL, elas ainda asseguram que as operações fundamentais de busca, inserção e remoção possuem complexidade

$$O(\log n).$$

Assim, árvores rubro-negras constituem uma estrutura de dados eficiente para manter conjuntos dinâmicos de elementos ordenados.

5.2 Inserção em Árvores Rubro-Negras

A ideia básica para inserir um novo elemento em uma árvore rubro-negra consiste em realizar inicialmente uma inserção semelhante à realizada em uma árvore binária de busca tradicional. Ou

seja, percorremos um caminho descendente na árvore, iniciando pela raiz, até encontrar a posição adequada para o novo nó. Esse novo nó é inserido como uma *folha vermelha*.

Entretanto, após essa inserção, algumas das propriedades que definem uma árvore rubro-negra podem ser violadas. Para restaurá-las, utilizamos uma combinação de *recolorações* e *rotações* na árvore, analisando a estrutura ao percorrer o caminho ascendente, isto é, retornando em direção à raiz. Nas análises a seguir, consideraremos que o nó corrente é o nó x .

Caso 1: Se y , o pai de x , não for vermelho, então nenhuma propriedade da árvore rubro-negra foi violada. Nesse caso, a árvore permanece válida e nenhuma ação adicional é necessária.



Figura 43: Se o pai do nó inserido x é preto, nada precisa ser feito.

Caso 2: Se y (pai de x) for vermelho, então a propriedade 3 da definição de árvores rubro-negras é violada, pois não podem existir dois nós vermelhos adjacentes. Sabemos ainda que, devido à propriedade 2, y não pode ser a raiz da árvore (pois a raiz é sempre preta). Além disso, o pai de y , denotado por w , deve ser necessariamente preto, também devido à propriedade 3.

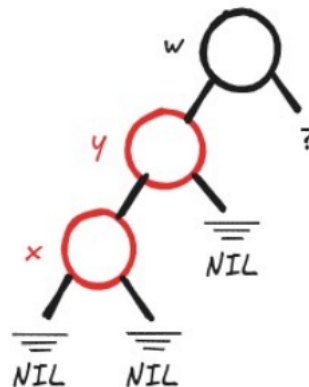


Figura 44: Se o pai do nó inserido x é vermelho, o avô deve ser preto.

A seguir analisamos dois subcasos possíveis.

Caso 2.1: w possui outro filho z de cor vermelha.

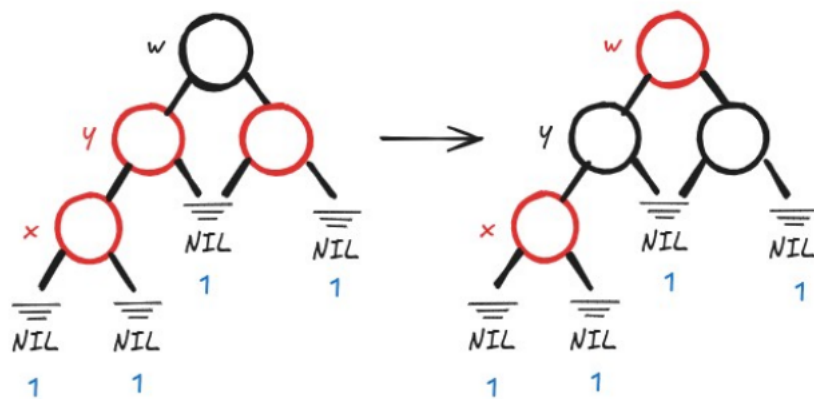


Figura 45: Se o avô do nó inserido tem outro filho vermelho, então ele fica vermelho e seus dois filhos ficam pretos.

Nesse caso, realizamos uma recoloração simples: os nós y e z passam a ser pretos, enquanto w passa a ser vermelho. Observe que a propriedade 4 permanece preservada, pois todos os caminhos raiz-NIL continuam passando pelo mesmo número de nós pretos. Além disso, a propriedade 3, que havia sido violada entre os nós x e y , é restaurada.

Entretanto, ao continuar subindo na árvore, será necessário verificar a situação do nó w , que agora é vermelho. Essa alteração pode fazê-lo violar a propriedade 3 em relação ao seu próprio pai. Caso w seja a raiz da árvore, basta recolori-lo para preto. Isso é permitido porque a raiz é o único nó presente em todos os caminhos raiz-NIL.

Caso 2.2: w não possui outro filho z de cor vermelha.

Nesse caso, w pode ter outro filho preto z ou mesmo não possuir outro filho.

Consideremos inicialmente a situação:

- a) w possui outro filho preto z .

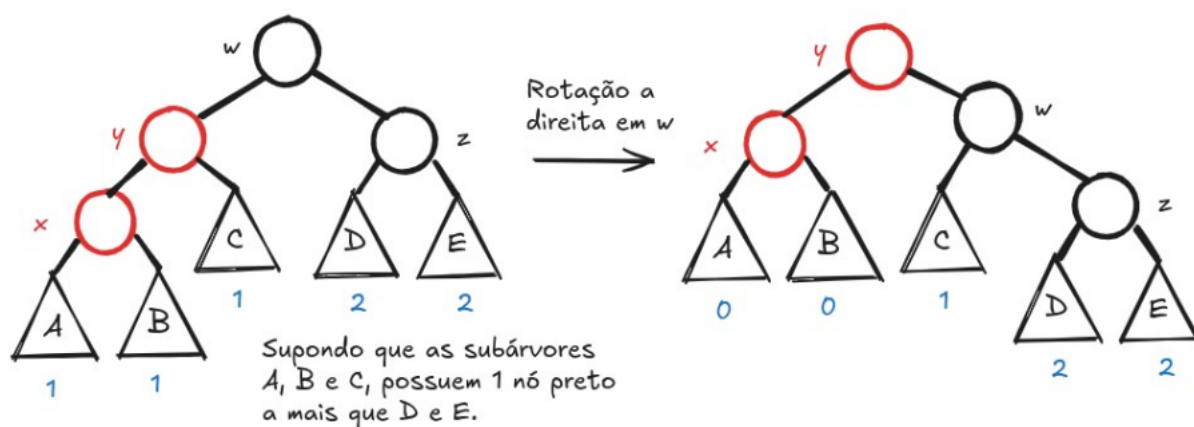


Figura 46: Se o avô w do nó inserido tem outro filho preto, então é preciso fazer uma rotação a direita em w e recolorir x de preto.

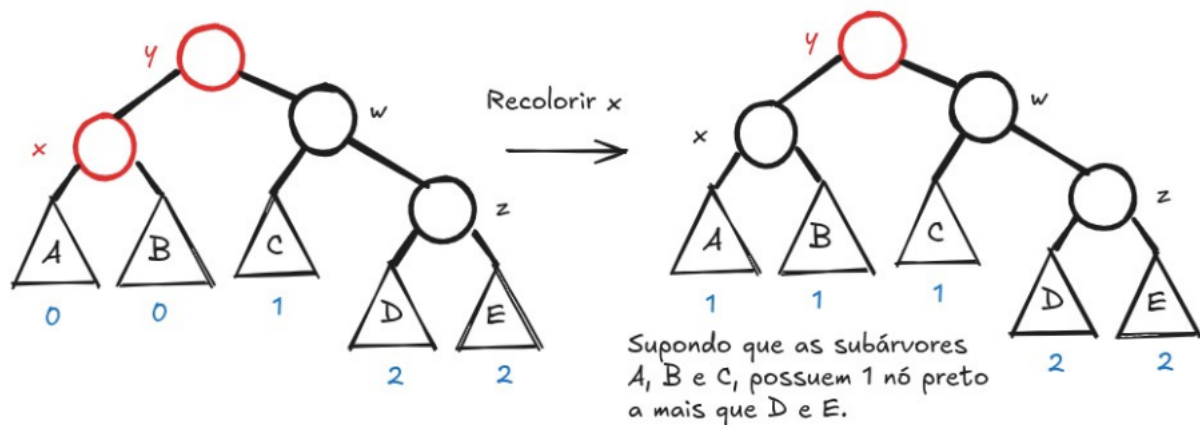


Figura 47: Se o avô w do nó inserido tem outro filho preto, então é preciso fazer uma rotação a direita em w e recolorir x de preto.

Nesse cenário, realizamos uma rotação à direita para inverter a relação entre w e y . Contudo, após a rotação, o número de nós pretos nos caminhos que vão da raiz até os filhos de x pode ser reduzido. Para corrigir esse problema, realizamos uma recoloração, tornando x preto. Isso restabelece tanto a propriedade referente ao número de nós pretos nos caminhos quanto a restrição de nós vermelhos adjacentes.

Ao continuar a subida na árvore (caminho ascendente), será necessário analisar a situação de y , que permanece vermelho e agora se tornou a raiz da subárvore. Ele pode eventualmente violar a propriedade 3 em relação ao seu pai. Caso y se torne a nova raiz da árvore, basta recolori-lo para preto, pois todo caminho raiz-NIL inclui a raiz.

- **b)** w não possui outro filho.

A mesma solução descrita anteriormente também funciona quando x é filho à esquerda de y .

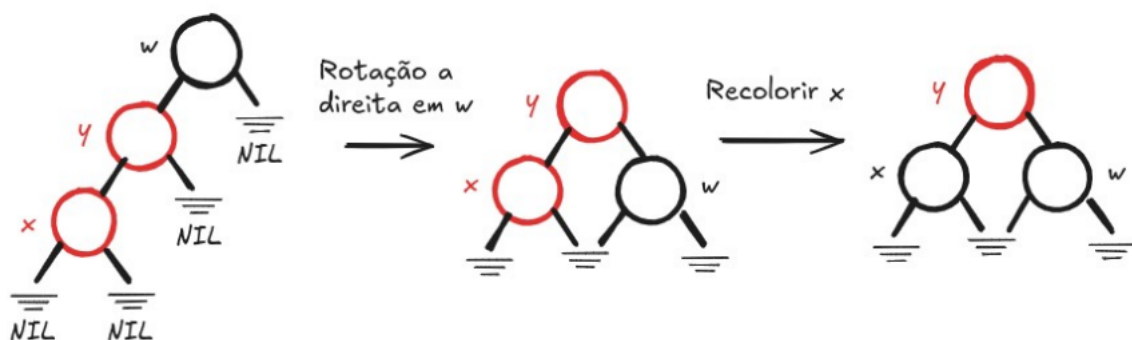


Figura 48: Se o avô w do nó inserido não tem outro filho, a solução anterior também funciona.

Entretanto, surge uma situação adicional quando x é filho à direita de y . Nesse caso, antes de aplicar a rotação principal, é necessário realizar uma rotação preliminar para transformar a configuração da árvore em um caso equivalente ao anterior, permitindo então aplicar o mesmo procedimento de correção.

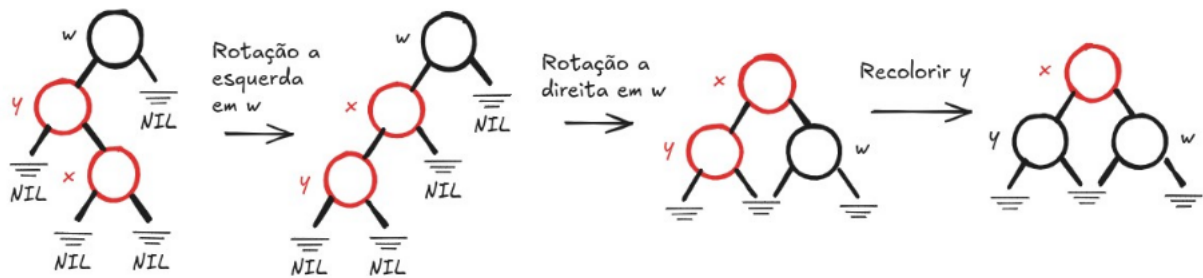


Figura 49: Se o avô w do nó inserido não tem outro filho e e o nó inserido x é filho a direita, é preciso aplicar uma rotação a esquerda em y (pai de x), seguida de uma rotação a direita em w e recolorir y de preto.

A seguir apresentamos os algoritmos necessários para a inserção em uma árvore rubro-negra. Inicialmente relembramos a operação de rotação à esquerda, que é uma das operações fundamentais utilizadas para manter as propriedades da estrutura.

```
Left_Rotate(T, x) {
    y = x.right
    x.right = y.left
    if y.left ≠ NIL
        y.left.p = x
    y.p = x.p
    if x.p == NIL
        T.root = y
    elif x == x.p.left
        x.p.left = y
    else
        x.p.right = y
    y.left = x
    x.p = y
}
```

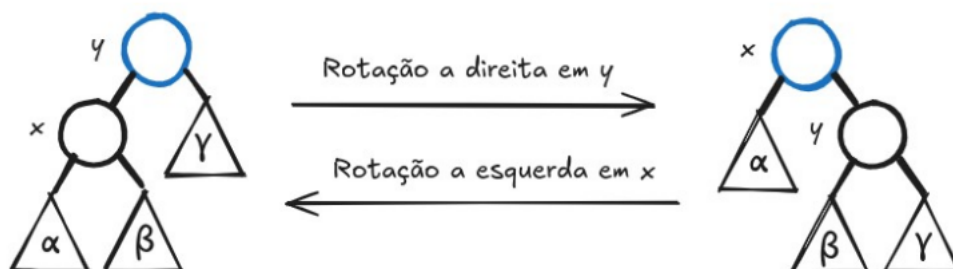


Figura 50: Ilustração gráfica do processo de rotação à esquerda e rotação à direita.

Em resumo, o processo envolve os seguinte passos:

- A subárvore esquerda de y torna-se a subárvore direita de x .

- Se a subárvore esquerda de y não é vazia, então x torna-se o pai dessa subárvore.
- O pai de x passa a ser o pai de y .
- Se x era a raiz da árvore, então y se torna a nova raiz.
- Caso contrário, se x era filho à esquerda, então y passa a ser o novo filho à esquerda.
- Caso contrário, x era filho à direita e y passa a ocupar essa posição.
- Finalmente, x torna-se o filho à esquerda de y .
- O nó y torna-se o novo pai de x .

Com base nesse procedimento, podemos escrever facilmente o algoritmo para realizar uma rotação à direita.

```
Right_Rotate(T, x) {
    y = x.left
    x.left = y.right
    if y.right ≠ NIL
        y.right.p = x
    y.p = x.p
    if x.p == NIL
        T.root = y
    elif x == x.p.right
        x.p.right = y
    else
        x.p.left = y
    y.right = x
    x.p = y
}
```

A seguir apresentamos o algoritmo para a inserção em uma árvore rubro-negra. Inicialmente o algoritmo realiza uma inserção semelhante àquela utilizada em árvores binárias de busca tradicionais, percorrendo um caminho descendente na árvore até encontrar a posição correta para o novo nó.

```
Red_Black_Insert(T, z) {
    x = T.root           // nó que será comparado com z (buscar)
    y = NIL              // será o pai de z
    while x ≠ NIL {
        y = x
        if z.key < x.key
            x = x.left
        else
```

```

        x = x.right
    }
    z.p = y          // encontrou posição correta
    if y == NIL
        T.root = z   // árvore estava vazia
    elif z.key < y.key
        y.left = z    // z é filho à esquerda
    else
        y.right = z   // z é filho à direita
    z.left = NIL
    z.right = NIL
    z.color = RED     // todo nó inserido inicia vermelho
    Red_Black_Insert_Fixup(T, z) // restaura propriedades da árvore
}

```

Observe que todo novo nó é inicialmente inserido com a cor vermelha. Após a inserção, a função `Red_Black_Insert_Fixup` é responsável por restaurar as propriedades da árvore rubro-negra por meio de recolorações e rotações apropriadas em um caminho bottom-up.

5.2.1 Ajustes após inserção

A função a seguir é responsável por restaurar as propriedades das árvores rubro-negras após a inserção de um novo nó. Esse procedimento é executado em um percurso ascendente (bottom-up), iniciando no nó recém inserido e caminhando em direção à raiz da árvore.

Durante esse processo, o algoritmo verifica se ocorre uma violação da propriedade que impede a existência de dois nós vermelhos consecutivos. Caso isso aconteça, o algoritmo realiza recolorações e rotações apropriadas até que todas as propriedades da árvore rubro-negra sejam novamente satisfeitas.

```

Red_Black_Insert_Fixup(T, z) {
    while z.p.color == RED {
        if z.p == z.p.p.left {           // pai de z é filho à esquerda?
            y = z.p.p.right               // y é o tio de z
            if y.color == RED {           // pai e tio são RED
                z.p.color = BLACK
                y.color = BLACK
                z.p.p.color = RED          // raiz da subárvore RED (avô)
                z = z.p.p
            }
        }
        else {
            if z == z.p.right {           // z é filho à direita
                z = z.p
                Left_Rotate(T, z)
            }
        }
    }
}

```

```

        }
        z.p.color = BLACK
        z.p.p.color = RED
        Right_Rotate(T, z.p.p)
    }
}
else {                                     // caso simétrico (espelhamento)
    y = z.p.p.left                         // y é o tio de z
    if y.color == RED {                   // pai e tio são RED
        z.p.color = BLACK
        y.color = BLACK
        z.p.p.color = RED
        z = z.p.p
    }
    else {
        if z == z.p.left {               // z é filho à esquerda
            z = z.p
            Right_Rotate(T, z)
        }
        z.p.color = BLACK
        z.p.p.color = RED
        Left_Rotate(T, z.p.p)
    }
}
}
T.root.color = BLACK    // raiz é sempre BLACK
}

```

Observe que o algoritmo trata dois casos principais, dependendo se o pai de z é filho à esquerda ou à direita de seu próprio pai. O segundo caso é simplesmente o espelhamento do primeiro. Em ambas as situações, as correções são realizadas por meio de uma combinação de recolorações dos nós e rotações na árvore. Por fim, após todos os ajustes, garante-se explicitamente que a raiz da árvore seja sempre preta, preservando assim uma das propriedades fundamentais das árvores rubro-negras.

As figuras a seguir mostram um exemplo ilustrativo, em que a chave 4 é inserida em uma árvore rubro-negra.

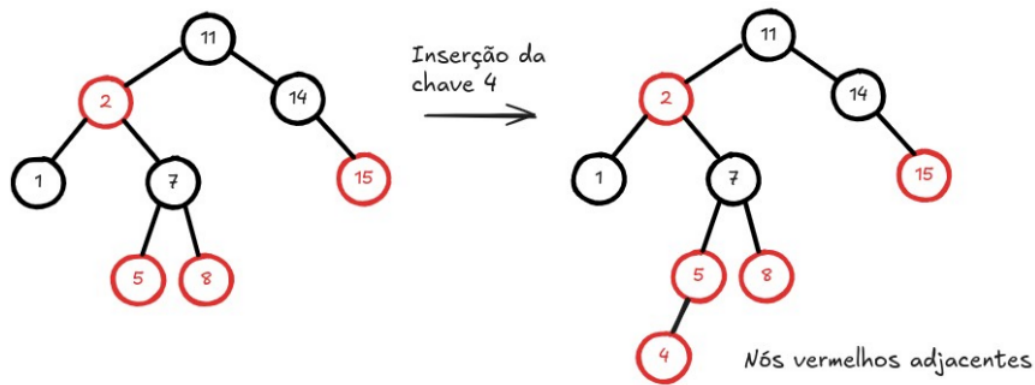


Figura 51: Ao inserir a chave 4, a árvore deixa de ser rubro-negra, pois existem 2 nós vermelhos adjacentes.

Note que após a inserção, as propriedades são violadas, mas aplicando as operações vistas anteriormente, as propriedades são restauradas. Neste ponto, devemos continuar subindo no caminho ascendente fazendo $z = z.p.p$, ou seja, z deve apontar para seu avô.

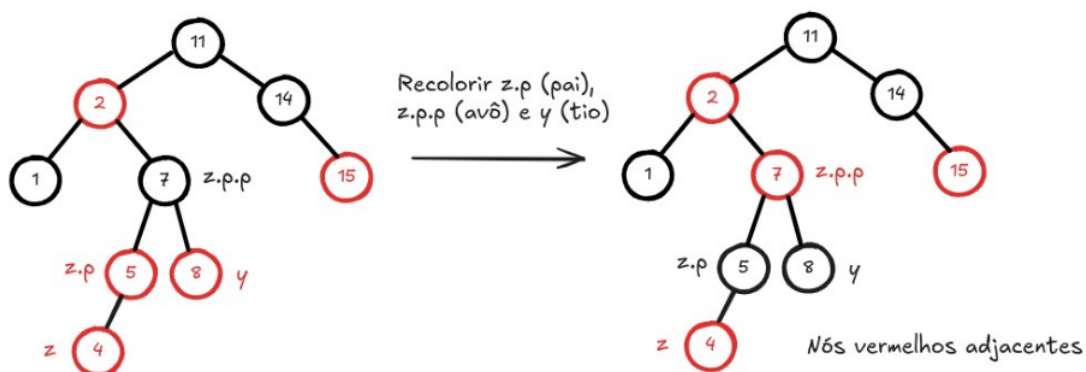


Figura 52: Para restaurar as propriedades da árvore rubro-negra, devemos recolorir o pai, o tio e o avô do nó folha inserido.

Assim, finalmente a árvore está com todas as quatro propriedades restauradas. Note que ela encontra-se balanceada, como era de se esperar.

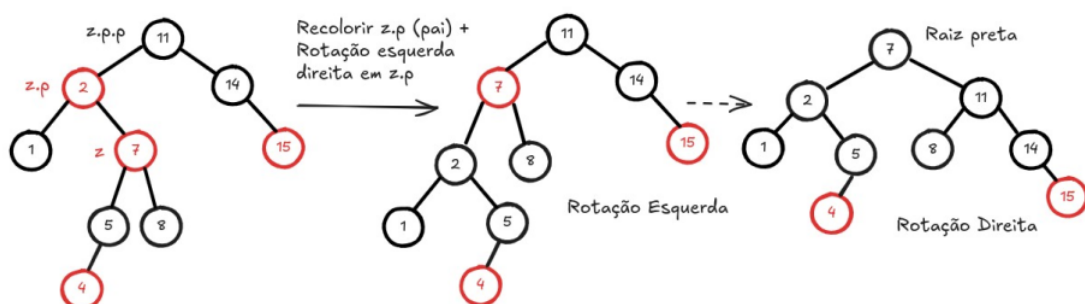


Figura 53: Ao fazer o avô ficar vermelho, precisamos continuar subindo na árvore para restaurar as propriedades.

5.3 Remoção em Árvores Rubro-Negras

A remoção de um nó em uma árvore rubro-negra é significativamente mais complexa do que a inserção. A principal dificuldade está no fato de que, além de preservar a propriedade de busca da árvore binária, também é necessário garantir que todas as propriedades que caracterizam uma árvore rubro-negra continuem válidas após a remoção.

Antes de apresentar o algoritmo, introduziremos uma pequena modificação na representação da árvore. Em vez de utilizar o ponteiro NIL como uma referência nula, todas as folhas externas da árvore são representadas por um único nó especial, denominado $T.nil$. Esse nó é compartilhado por toda a árvore e possui cor preta. Essa representação simplifica significativamente os algoritmos, pois elimina diversos testes para referências nulas e garante que toda folha da árvore possua um pai válido. A figura a seguir ilustra essa representação.

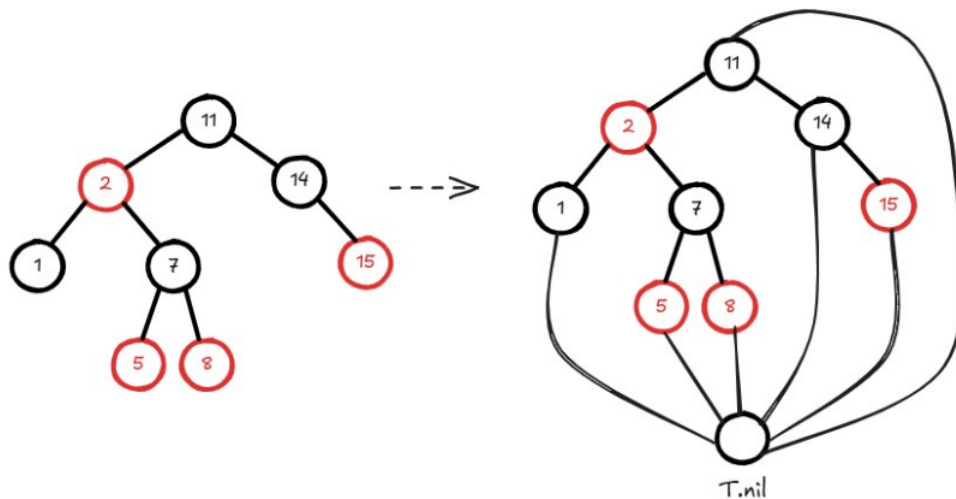


Figura 54: Em vez de usar a referência vazia NIL, criamos um único nó especial, denominado $T.nil$.

A estratégia utilizada para remover um nó é essencialmente a mesma empregada em árvores binárias de busca. Inicialmente, substituímos o nó a ser removido por outro nó que preserve a propriedade de busca da árvore. Entretanto, se o nó efetivamente removido for preto, a remoção poderá violar as propriedades das árvores rubro-negras. Nessa situação, será necessário executar um procedimento adicional de correção, denominado `RB-Delete-Fixup`, baseado em recolorações e rotações.

Para simplificar o algoritmo principal, definimos inicialmente uma primitiva auxiliar denominada `RB-Transplant`, cuja função é substituir uma subárvore enraizada em u pela subárvore enraizada em v . Essa função é equivalente ao algoritmo `Transplant` apresentado para árvores binárias de busca, com a diferença de que o nó v pode ser inclusive o nó sentinela $T.nil$.

```
RB-Transplant(T, u, v) {  
    if u.p == T.nil  
        T.root = v                // u é a raiz da árvore  
    elif u == u.p.left
```

```

    u.p.left = v                // u é filho à esquerda
else
    u.p.right = v              // u é filho à direita
v.p = u.p
}

```

A seguir apresentamos o algoritmo para remoção de um nó em uma árvore rubro-negra. Observe que ele é bastante semelhante ao algoritmo `Tree_Delete` utilizado em árvores binárias de busca. A principal diferença é que, ao final da remoção, pode ser necessário restaurar as propriedades da árvore rubro-negra.

```

RB-Delete(T, z) {
    y = z
    y_orig_color = y.color
    if z.left == T.nil {
        x = z.right
        // substitui z pelo filho à direita (ou T.nil)
        RB-Transplant(T, z, z.right)
    }
    elif z.right == T.nil {
        x = z.left
        // substitui z pelo filho à esquerda
        RB-Transplant(T, z, z.left)
    }
    else {
        // z possui dois filhos
        // encontra o sucessor de z
        y = Tree_Minimum(z.right)
        y_orig_color = y.color
        x = y.right
        if y.p == z
            // sucessor é filho direto de z
            x.p = y
        else {
            // remove y da posição original
            RB-Transplant(T, y, y.right)
            y.right = z.right
            y.right.p = y
        }
        // substitui z por y
        RB-Transplant(T, z, y)
        y.left = z.left
    }
}

```

```

    y.left.p = y
    // preserva a cor original de z
    y.color = z.color
}
if y_orig_color == BLACK
    // somente remover um nó preto pode
    // violar as propriedades da árvore
    RB-Delete-Fixup(T, x)
}

```

Observe que o algoritmo distingue os três casos clássicos da remoção em uma árvore binária de busca:

1. o nó possui apenas o filho direito;
2. o nó possui apenas o filho esquerdo;
3. o nó possui dois filhos.

Nos dois primeiros casos, basta substituir o nó removido por seu único filho. No terceiro caso, utiliza-se o sucessor em ordem, isto é, o menor elemento da subárvore direita, para preservar a propriedade de busca da árvore.

A variável `y_orig_color` armazena a cor do nó efetivamente removido da árvore. Essa informação é fundamental, pois apenas a remoção de um nó preto pode alterar a altura negra (*black-height*) dos caminhos `raiz-T.nil`. Quando isso ocorre, torna-se necessário executar o procedimento `RB-Delete-Fixup`, responsável por restaurar todas as propriedades das árvores rubro-negras por meio de uma sequência de recolorações e rotações.

Correção após a Remoção

Quando um nó é removido de uma árvore rubro-negra, nem sempre as propriedades que definem esse tipo de árvore permanecem válidas. Em particular, problemas podem surgir apenas quando o nó efetivamente removido da estrutura é **BLACK**. Nesse caso, a remoção pode alterar o número de nós pretos presente em alguns caminhos `raiz-NIL`, violando a Propriedade 4 da definição de árvores rubro-negras. Por outro lado, quando o nó removido é **vermelho**, nenhuma das propriedades é violada. Isso ocorre porque:

- o número de nós pretos em qualquer caminho `raiz-NIL` permanece inalterado;
- a remoção de um nó vermelho não pode criar dois nós vermelhos consecutivos.

Assim, a função `RB-Delete-Fixup` só precisa ser executada quando o nó removido originalmente era preto. Sua finalidade é restaurar as propriedades da árvore por meio de uma sequência de recolorações e rotações.

A seguir são apresentados os quatro casos fundamentais que podem ocorrer durante esse processo de correção. Inicialmente será considerado que o nó x , responsável por substituir o nó removido, é **filho à esquerda** de seu pai. Posteriormente, basta aplicar os casos simétricos quando x é filho à direita.

Caso 1: irmão vermelho

Neste primeiro caso, o nó x é preto, filho à esquerda de seu pai, e seu irmão w é vermelho.

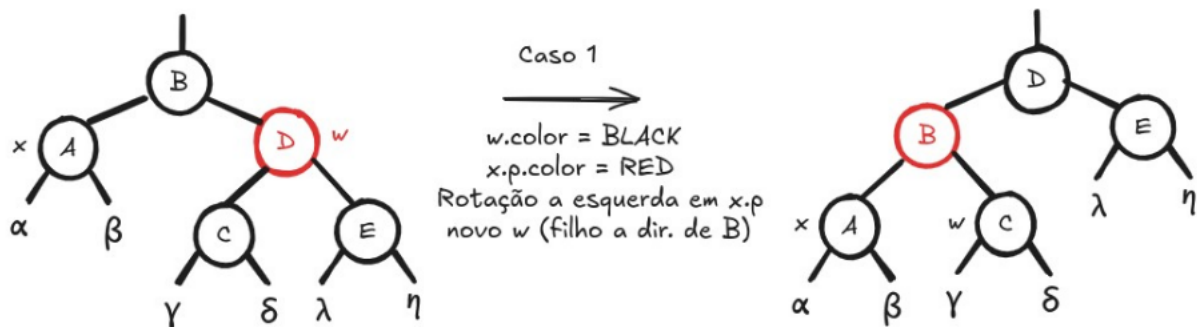


Figura 55: No caso 1, o nó preto é filho a esquerda possui um irmão vermelho.

A estratégia consiste em transformar essa configuração em uma das situações restantes. Para isso:

- recolore-se w para preto;
- recolore-se o pai de x para vermelho;
- realiza-se uma rotação à esquerda no pai de x .

Após essa transformação, o novo irmão de x será um nó preto, reduzindo o problema para um dos Casos 2, 3 ou 4.

Caso 2: irmão preto com dois filhos pretos

Neste caso, o irmão w é preto e ambos os seus filhos também são pretos.

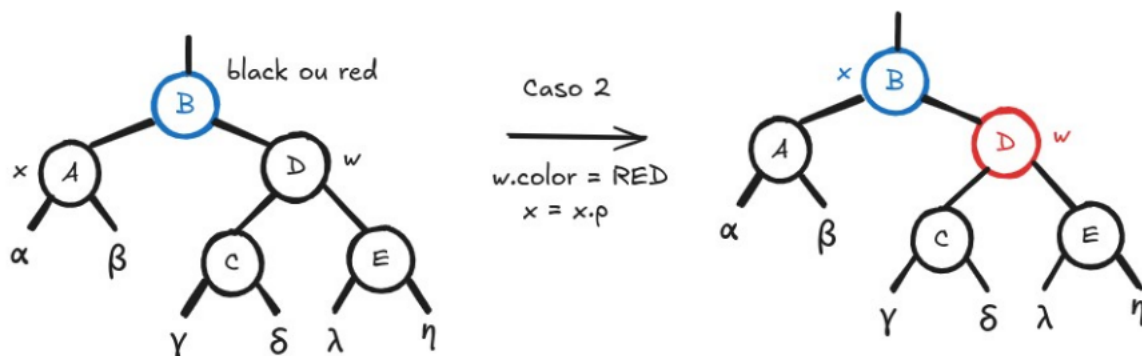


Figura 56: No caso 2, o nó preto é filho a esquerda possui um irmão preto com 2 filhos pretos.

A correção é bastante simples:

- recolor-se w para vermelho;
- redefine-se x como sendo seu pai.

Essa operação transfere o problema para um nível superior da árvore, onde o processo continua.

Caso 3: irmão preto com filho esquerdo vermelho

Agora o irmão w é preto, seu filho esquerdo é vermelho e seu filho direito é preto.

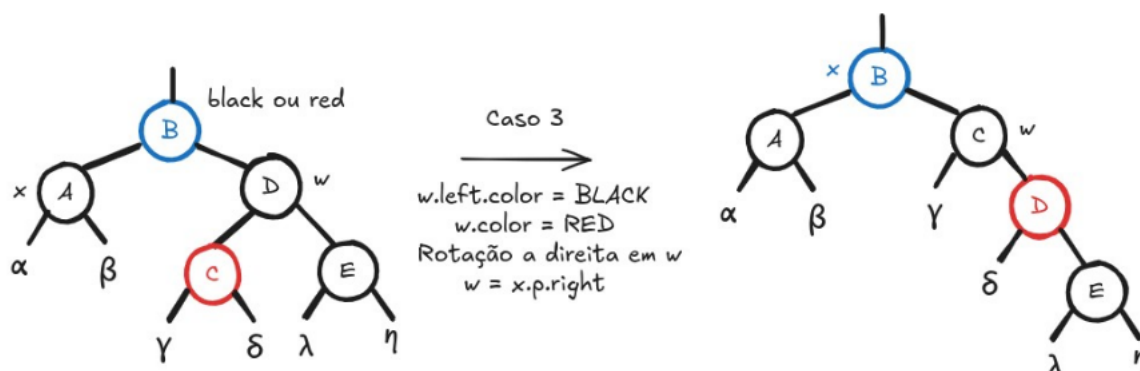


Figura 57: No caso 3, o nó preto é filho a esquerda possui um irmão preto com com filho a esquerda vermelho.

Esse caso é convertido no Caso 4 através das seguintes operações:

- recolor-se o filho esquerdo de w para preto;
- recolor-se w para vermelho;
- realiza-se uma rotação à direita em w .

Após essa rotação, o novo irmão de x satisfaz exatamente as condições do Caso 4.

Caso 4: irmão preto com filho direito vermelho

Este é o último caso e também o único que elimina definitivamente o problema. O irmão w é preto e possui um filho direito vermelho. O filho esquerdo pode ser vermelho ou preto.



Figura 58: No caso 4, o nó preto é filho a esquerda possui um irmão preto com filho a direita vermelho.

As operações realizadas são:

- atribuir a w a cor originalmente presente no pai de x ;
- recolorir o pai de x para preto;
- recolorir o filho direito de w para preto;
- realizar uma rotação à esquerda no pai de x .

Após essas operações, todas as propriedades da árvore rubro-negra são restauradas e o algoritmo pode ser encerrado.

Casos Simétricos Os quatro casos apresentados assumem que o nó x é filho à esquerda de seu pai. Quando x é filho à direita, aplicam-se exatamente as mesmas ideias de correção, bastando inverter as direções “esquerda” e “direita” em todas as rotações e testes de filhos. Consequentemente, o algoritmo completo precisa tratar um total de **oito casos**: quatro para quando x é filho à esquerda e quatro casos simétricos para quando x é filho à direita.

A seguir apresentamos o algoritmo RB-Delete-Fixup, responsável por restaurar as propriedades das árvores rubro-negras após a remoção de um nó preto. O algoritmo implementa os oito casos discutidos anteriormente: quatro quando o nó x é filho à esquerda de seu pai e quatro casos simétricos quando x é filho à direita.

```
RB-Delete-Fixup(T, x) {
    while x ≠ T.root and x.color == BLACK {
        if x == x.p.left {
            w = x.p.right           // irmão de x
            // Caso 1
            if w.color == RED {
                w.color = BLACK
                x.p.color = RED
                Left-Rotate(T, x.p)
                w = x.p.right
            }
        }
    }
}
```

```

    }
    // Caso 2
    if w.left.color == BLACK and w.right.color == BLACK {
        w.color = RED
        x = x.p
    }
    else {
        // Caso 3
        if w.right.color == BLACK {
            w.left.color = BLACK
            w.color = RED
            Right-Rotate(T, w)
            w = x.p.right
        }
        // Caso 4
        w.color = x.p.color
        x.p.color = BLACK
        w.right.color = BLACK
        Left-Rotate(T, x.p)
        x = T.root          // encerra o loop
    }

}

else {
    // Casos simétricos (x é filho à direita)
    w = x.p.left
    // Caso 5
    if w.color == RED {
        w.color = BLACK
        x.p.color = RED
        Right-Rotate(T, x.p)
        w = x.p.left
    }
    // Caso 6
    if w.right.color == BLACK and w.left.color == BLACK {
        w.color = RED
        x = x.p
    }
    else {
        // Caso 7
        if w.left.color == BLACK {

```

```

        w.right.color = BLACK
        w.color = RED
        Left-Rotate(T, w)
        w = x.p.left
    }
    // Caso 8
    w.color = x.p.color
    x.p.color = BLACK
    w.left.color = BLACK
    Right-Rotate(T, x.p)
    x = T.root           // encerra o loop
}
}
}
x.color = BLACK
}

```

5.4 Árvores AVL versus Árvores Rubro-Negras

Do ponto de vista assintótico, árvores AVL e árvores rubro-negras oferecem exatamente as mesmas garantias de desempenho. As operações de busca, inserção e remoção possuem complexidade

$$O(\log n),$$

uma vez que ambas mantêm a altura da árvore proporcional ao logaritmo do número de elementos armazenados.

Entretanto, apesar de apresentarem a mesma complexidade assintótica, o comportamento prático dessas duas estruturas é diferente devido à forma como o balanceamento é realizado.

As árvores AVL utilizam um critério de balanceamento bastante rígido: para todo nó, a diferença entre as alturas das subárvores esquerda e direita deve ser, no máximo, uma unidade. Como consequência, a altura da árvore tende a ser menor do que a altura de uma árvore rubro-negra com o mesmo número de elementos. Isso reduz o comprimento médio dos caminhos percorridos durante as buscas, tornando a operação de pesquisa ligeiramente mais rápida.

Por outro lado, manter esse balanceamento mais estrito exige um número maior de rotações durante inserções e, principalmente, durante remoções. Após remover um nó de uma árvore AVL, pode ser necessário propagar o rebalanceamento até a raiz da árvore, realizando diversas rotações ao longo do caminho.

As árvores rubro-negras adotam uma estratégia diferente. Em vez de impor um balanceamento rigoroso, elas apenas restringem a distribuição dos nós pretos ao longo dos caminhos raiz–NIL. Como consequência, a árvore pode ficar um pouco menos balanceada do que uma árvore AVL, mas continua

garantindo altura

$$h \leq 2 \log_2(n + 1),$$

o que preserva a complexidade $O(\log n)$ para todas as operações.

A principal vantagem dessa abordagem é que inserções e remoções exigem significativamente menos rotações. Na inserção, o algoritmo executa no máximo duas rotações, sendo que muitas vezes apenas recolorações são suficientes para restaurar as propriedades da árvore. Na remoção, embora existam diversos casos de correção, o número de rotações também permanece pequeno, sendo compensado principalmente por mudanças de cor.

Em resumo, pode-se dizer que as árvores AVL priorizam o desempenho das buscas, enquanto as árvores rubro-negras priorizam a eficiência das atualizações na estrutura. Dessa forma, a escolha entre ambas depende do perfil da aplicação:

- aplicações dominadas por consultas (buscas) tendem a se beneficiar de árvores AVL, devido ao seu balanceamento mais rigoroso;
- aplicações com grande quantidade de inserções e remoções normalmente apresentam melhor desempenho utilizando árvores rubro-negras, já que estas necessitam de menos rotações e menos operações de rebalanceamento.

Essa diferença explica por que diversas bibliotecas e sistemas operacionais modernos adotam árvores rubro-negras como estrutura de dados padrão para implementar conjuntos ordenados, mapas associativos e tabelas de símbolos. Embora as árvores AVL apresentem buscas ligeiramente mais rápidas, o menor custo de manutenção das árvores rubro-negras faz com que elas ofereçam melhor desempenho global em cenários com atualizações frequentes.

5.5 Considerações Finais

As árvores rubro-negras constituem uma das estruturas de dados balanceadas mais importantes da Ciência da Computação. Embora seu funcionamento seja significativamente mais complexo do que o de árvores binárias de busca convencionais, elas conseguem garantir que as operações de busca, inserção e remoção sejam executadas em tempo $O(\log n)$, independentemente da ordem de inserção das chaves.

O principal diferencial das árvores rubro-negras é o fato de seu balanceamento ser mantido por meio de um conjunto simples de propriedades envolvendo apenas duas cores. Diferentemente das árvores AVL, que controlam explicitamente o fator de balanço de cada nó, as árvores rubro-negras permitem pequenas diferenças entre as alturas das subárvores, desde que todos os caminhos da raiz até um nó `NIL` contenham o mesmo número de nós pretos. Essa estratégia produz árvores suficientemente balanceadas para garantir excelente desempenho, ao mesmo tempo em que reduz o número de rotações necessárias durante inserções e remoções.

Do ponto de vista prático, essa característica torna as árvores rubro-negras particularmente adequadas para aplicações nas quais a estrutura sofre atualizações frequentes. Em cenários com grande

quantidade de inserções e remoções, elas costumam apresentar desempenho superior ao das árvores AVL, uma vez que demandam menos operações de rebalanceamento.

As árvores rubro-negras são amplamente empregadas em bibliotecas e sistemas reais. A implementação das coleções ordenadas da linguagem Java (`TreeMap` e `TreeSet`), bem como os contêineres associativos `map`, `multimap`, `set` e `multiset` da Biblioteca Padrão da linguagem C++ (STL), utilizam árvores rubro-negras como estrutura de dados subjacente. Além disso, sistemas operacionais, compiladores, bancos de dados e diversos mecanismos de gerenciamento de memória empregam árvores rubro-negras para manter conjuntos ordenados de forma eficiente.

Embora sua implementação seja consideravelmente mais trabalhosa do que a de árvores AVL, especialmente devido aos diversos casos envolvidos na remoção de nós, as árvores rubro-negras representam um excelente compromisso entre simplicidade de manutenção, eficiência assintótica e desempenho prático. Por essa razão, são consideradas uma das estruturas fundamentais para o desenvolvimento de algoritmos e sistemas que exigem manipulação dinâmica de conjuntos ordenados.

6 Hash Tables

Nos capítulos anteriores estudamos diversas estruturas de dados para armazenar conjuntos ordenados de chaves, como árvores AVL, árvores rubro-negras, tries e skip lists. Todas essas estruturas oferecem operações eficientes de busca, inserção e remoção, tipicamente com complexidade $O(\log n)$. Entretanto, em muitas aplicações práticas, a ordem das chaves não é a principal preocupação. O objetivo mais importante é simplesmente responder rapidamente à pergunta: “o elemento x pertence ao conjunto?” Considere, por exemplo, um sistema de autenticação que precisa verificar se um nome de usuário já existe, um compilador que mantém uma tabela de símbolos, um navegador que armazena páginas em cache ou um banco de dados que deseja localizar um registro a partir de uma chave única. Nesses cenários, não é necessário percorrer os elementos em ordem crescente; o que realmente importa é obter acesso quase imediato ao elemento procurado. As tabelas hash (*hash tables*) foram criadas justamente para resolver esse problema. A ideia central consiste em utilizar uma função matemática, denominada *função hash*, para transformar uma chave arbitrária em um índice de um vetor. Em vez de percorrer uma árvore ou uma lista, o algoritmo calcula diretamente a posição onde a chave deve estar armazenada. Quando a função hash é bem projetada e os elementos estão distribuídos de maneira adequada, as operações de busca, inserção e remoção possuem tempo médio $O(1)$, isto é, tempo constante. Essa eficiência faz das tabelas hash uma das estruturas de dados mais utilizadas em sistemas reais. Linguagens modernas de programação oferecem implementações prontas, como `HashMap` em Java, `dict` em Python e `unordered_map` em C++. Além disso, caches, tabelas de símbolos, índices de bancos de dados e inúmeros sistemas distribuídos dependem de tabelas hash para fornecer acesso extremamente rápido às informações. Neste capítulo estudaremos os princípios fundamentais das tabelas hash, incluindo funções hash, tratamento de colisões e análise de desempenho, mostrando por que essa estrutura se tornou uma das ferramentas mais importantes da programação moderna.

6.1 Tabelas de Acesso Direto

As tabelas de espalhamento (*Hash Tables*) são estruturas de dados projetadas para realizar operações de busca, inserção e remoção de forma extremamente eficiente. Elas constituem uma das principais estruturas utilizadas para implementar dicionários, mapas associativos e conjuntos dinâmicos, estando presentes em praticamente todas as linguagens modernas de programação.

Sob hipóteses razoáveis sobre a distribuição das chaves e a qualidade da função de espalhamento, pode-se demonstrar que as operações de busca, inserção e remoção possuem complexidade média

$$O(1),$$

ou seja, tempo constante.

A ideia fundamental por trás de uma tabela hash deriva do conceito de **endereçamento direto**. Como visto anteriormente, um vetor permite acessar qualquer posição diretamente a partir de seu

índice, sem necessidade de percorrer os elementos armazenados. Essa propriedade faz com que o acesso a qualquer posição do vetor seja realizado em tempo constante.

Entretanto, para utilizar a própria chave como índice de um vetor, é necessário que o conjunto de possíveis chaves seja relativamente pequeno. Esse tipo de estrutura recebe o nome de **Tabela de Acesso Direto**. Por exemplo, considere um conjunto universo de chaves

$$U = \{0, 1, \dots, m - 1\},$$

onde m representa o número total de chaves possíveis. Suponha ainda que cada elemento possua uma chave única pertencente a esse universo.

Uma Tabela de Acesso Direto (TAD) consiste simplesmente em um vetor

$$T[0 \dots m - 1],$$

no qual cada posição corresponde exatamente a uma chave do universo U . Dessa forma, uma chave k é armazenada diretamente na posição $T[k]$. Caso nenhum elemento possua essa chave, a posição correspondente contém o valor NIL.

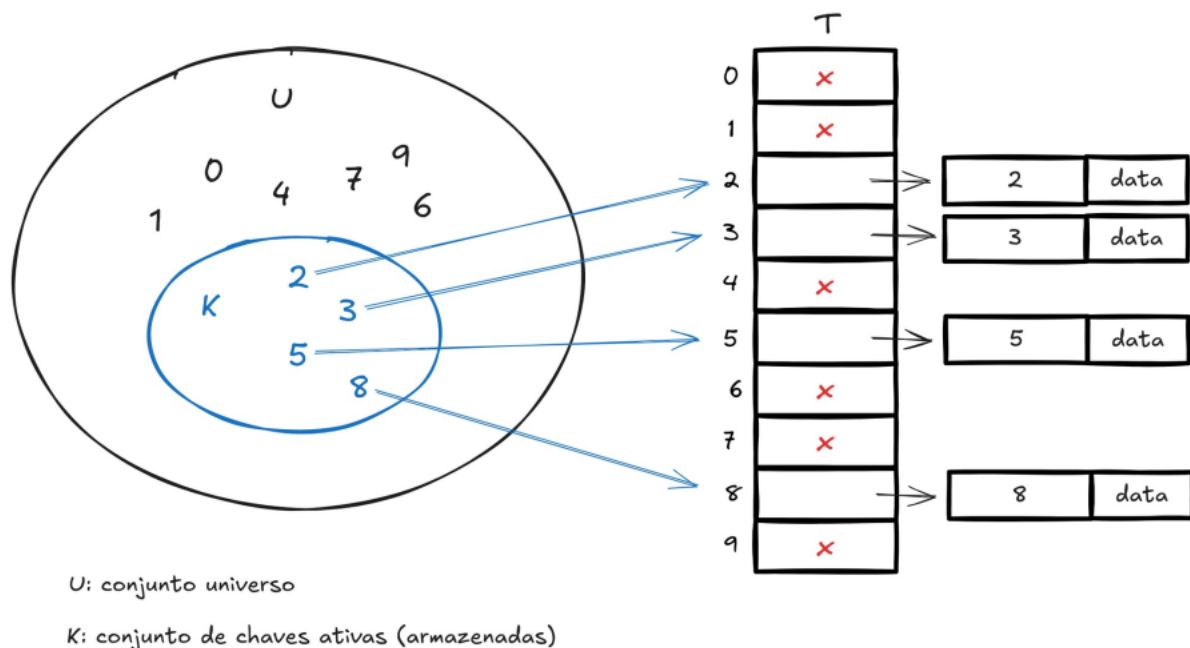


Figura 59: Em uma Tabela de Acesso Direto, uma chave k sempre deve ser associada ao slot k .

Como consequência, todas as operações básicas são extremamente simples e executadas em tempo constante.

```
TAD-Search(T, k) {
    return T[k]
}
```

```
TAD-Insert(T, x) {  
    T[x.key] = x  
}  
  
TAD-Delete(T, x) {  
    T[x.key] = NIL  
}
```

É imediato verificar que as três operações possuem complexidade

$$O(1),$$

uma vez que cada uma delas realiza apenas um único acesso ao vetor. Apesar dessa excelente eficiência, as Tabelas de Acesso Direto apresentam uma limitação importante. Seu consumo de memória é proporcional ao tamanho do universo de chaves, isto é,

$$O(|U|).$$

Quando o conjunto universo é muito grande, armazenar um vetor com uma posição para cada chave possível pode ser inviável, ou até mesmo impossível.

Na prática, esse problema é bastante comum. Em muitas aplicações, o universo de chaves pode conter milhões ou bilhões de valores possíveis, enquanto apenas uma pequena fração deles é efetivamente utilizada. Em outras palavras, se denotarmos por $K \subseteq U$ o conjunto de chaves realmente armazenadas, normalmente temos

$$|K| \ll |U|.$$

Nessa situação, a maior parte das posições da tabela permaneceria vazia, resultando em um enorme desperdício de memória.

As tabelas hash surgem justamente para contornar essa limitação. Em vez de utilizar a própria chave como índice do vetor, elas empregam uma *função de espalhamento* (*hash function*), responsável por transformar uma chave arbitrária em um índice pertencente a um vetor muito menor. Dessa forma, preserva-se o acesso praticamente imediato aos elementos, reduzindo drasticamente a quantidade de memória necessária.

6.2 Tabelas de Espalhamento

Como discutido na seção anterior, as Tabelas de Acesso Direto tornam-se inviáveis quando o universo de chaves U é muito grande e apenas uma pequena parcela dessas chaves é efetivamente utilizada. Nessas situações, as **Tabelas de Espalhamento** (*Hash Tables*) oferecem uma solução elegante e eficiente.

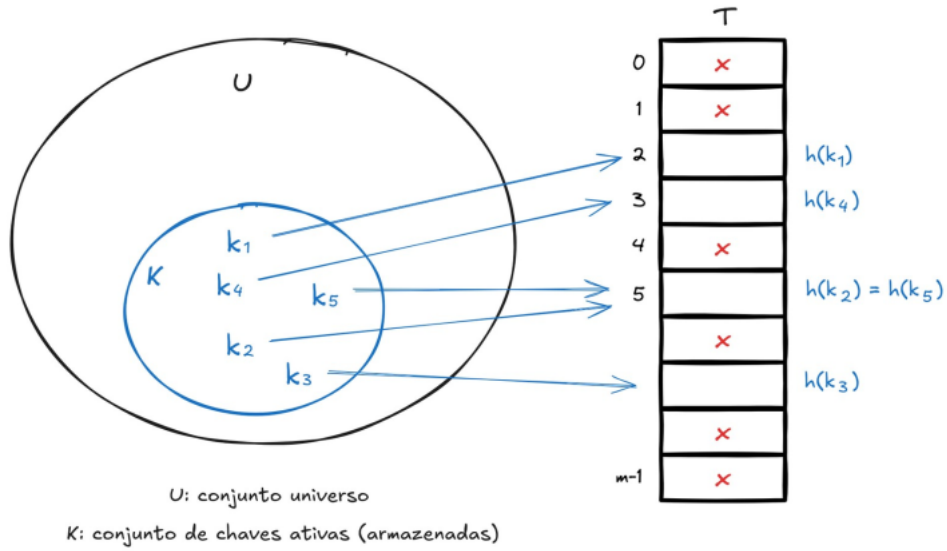


Figura 60: Em uma Tabela de Espalhamento, uma chave k deve ser associada ao slot $h(k)$, onde $h(\cdot)$ é uma função hash.

A ideia central consiste em armazenar apenas as chaves que realmente pertencem ao conjunto dinâmico. Se denotarmos por K o conjunto de chaves ativas, normalmente temos

$$|K| \ll |U|.$$

Assim, uma tabela hash reduz significativamente o consumo de memória, fazendo com que a complexidade de espaço passe de

$$O(|U|)$$

para

$$O(|K|),$$

enquanto preserva tempo médio constante para as operações de busca, inserção e remoção. A principal diferença entre uma Tabela de Acesso Direto e uma Tabela de Espalhamento está na forma como as chaves são associadas às posições do vetor. Na Tabela de Acesso Direto, uma chave k é armazenada diretamente na posição

$$T[k].$$

Já em uma Tabela de Espalhamento, a chave não é utilizada diretamente como índice. Em vez disso, aplica-se uma função denominada **função de espalhamento** ou **função hash**, representada por

$$h : U \longrightarrow \{0, 1, \dots, m - 1\},$$

onde m representa o número de posições (slots) da tabela hash e, em geral,

$$m \ll |U|.$$

Dessa forma, uma chave k é armazenada na posição

$$T[h(k)].$$

O valor $h(k)$ é denominado **valor hash** da chave k . A função hash é responsável por transformar uma chave pertencente a um universo potencialmente muito grande em um índice válido da tabela.

A grande vantagem dessa abordagem é que a tabela passa a ter apenas m posições, em vez de uma posição para cada elemento do universo de chaves. Isso torna possível armazenar conjuntos dinâmicos muito grandes utilizando uma quantidade de memória significativamente menor.

Entretanto, essa economia de espaço introduz um novo desafio. Como o número de chaves possíveis é muito maior que o número de posições disponíveis na tabela, é inevitável que duas ou mais chaves distintas sejam mapeadas para o mesmo índice. Esse fenômeno recebe o nome de **colisão**.

$$k_1 \neq k_2 \implies h(k_1) = h(k_2).$$

Do ponto de vista matemático, isso é consequência direta do fato de que a função hash mapeia um conjunto muito maior (U) em um conjunto muito menor ($\{0, \dots, m-1\}$). Pelo Princípio da Casa dos Pombos, colisões são inevitáveis sempre que

$$|U| > m.$$

Portanto, o objetivo de uma boa função hash não é eliminar colisões, isso é impossível na maioria das aplicações, mas sim distribuí-las da forma mais uniforme possível. Em outras palavras, deseja-se que as chaves sejam espalhadas quase aleatoriamente pelos slots da tabela, evitando concentrações excessivas em determinadas posições.

Nos próximos tópicos estudaremos duas questões fundamentais para o sucesso das tabelas hash: como projetar boas funções de espalhamento e como tratar eficientemente as colisões inevitavelmente produzidas por essas funções.

6.3 Tratamento de Colisões por Encadeamento Lógico

Como o número de possíveis chaves do conjunto universo é muito maior do que o número de posições disponíveis na tabela de espalhamento, colisões são inevitáveis. Uma das estratégias mais utilizadas para lidar com esse problema é o **encadeamento separado** (*separate chaining*), também denominado **encadeamento lógico**.

A ideia consiste em associar a cada posição da tabela uma lista encadeada contendo todos os elementos cujo valor de hash é igual àquela posição. Em outras palavras, se duas ou mais chaves produzem o mesmo valor de hash, todas elas passam a ser armazenadas na mesma lista.

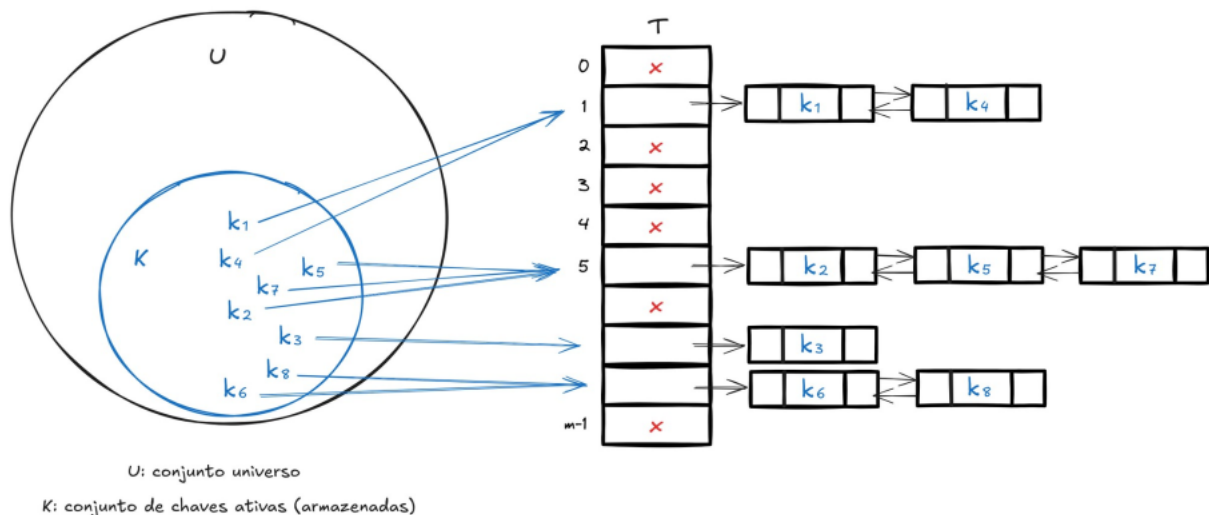


Figura 61: Em uma Tabela de Espalhamento com tratamento de colisões por encadeamento lógico, todas as chaves k_i que colidem em um mesmo slot são armazenadas em uma lista duplamente encadeada com cabeça em $T[h(k_i)]$.

Assim, cada posição da tabela contém uma referência para o primeiro elemento da lista correspondente. Caso nenhum elemento seja mapeado para determinado índice, o respectivo ponteiro contém o valor NIL.

A implementação das operações básicas torna-se bastante simples. A inserção consiste em adicionar um novo elemento na lista correspondente ao seu valor de hash. A busca percorre apenas a lista associada ao índice calculado pela função de espalhamento, enquanto a remoção elimina o elemento da lista após sua localização.

As primitivas básicas para inserção, busca e remoção são apresentadas a seguir.

```
// Insere x na cabeça da lista T[h(x.key)]
TH-Insert(T, x) {
    Insert_Head(T[h(x.key)], x)
}

// Busca a chave k na lista T[h(k)]
TH-Search(T, k) {
    Search(T[h(k)], k)
}

// Remove o elemento x da lista T[h(x.key)]
TH-Delete(T, x) {
    Delete(T[h(x.key)], x)
}
```

A análise de complexidade dessas operações é bastante intuitiva.

- **Inserção:** se o novo elemento for inserido na cabeça da lista encadeada, a operação possui

complexidade $\mathcal{O}(1)$.

- **Busca:** o tempo de execução depende do número de elementos presentes na lista correspondente ao valor de hash da chave pesquisada. Quanto menor o número de colisões, menor será o comprimento médio dessas listas. A análise detalhada dessa complexidade será apresentada na próxima subseção.
- **Remoção:** uma vez localizado o elemento, sua remoção pode ser realizada em tempo constante $\mathcal{O}(1)$, desde que as listas sejam implementadas como listas duplamente encadeadas. Nesse caso, não é necessário percorrer a lista para localizar o predecessor do nó a ser removido, bastando ajustar as referências de seus vizinhos. A primitiva `TH-Delete` assume que já existe uma referência direta para o elemento a ser removido.

Observe que, nessa estratégia, o desempenho da tabela de espalhamento depende diretamente da qualidade da função de hash. Quanto mais uniforme for a distribuição das chaves pelos diferentes slots da tabela, menores serão as listas encadeadas e, conseqüentemente, mais próximas de tempo constante serão as operações de busca, inserção e remoção no caso médio.

6.4 Análise da Complexidade da Busca

Nesta subseção analisaremos a complexidade da operação de busca em Tabelas de Espalhamento com tratamento de colisões por encadeamento lógico. Diferentemente das operações de inserção e remoção, cujo custo é relativamente simples de determinar, o desempenho da busca depende diretamente da distribuição das chaves produzida pela função de hash.

Considere uma Tabela de Espalhamento T composta por m slots que armazena um total de n chaves. Define-se o **fator de carga** (*load factor*) da tabela por

$$\alpha = \frac{n}{m},$$

que representa o número médio de elementos armazenados em cada lista encadeada. Observe que o fator de carga pode assumir valores menores, iguais ou maiores que 1. Quanto maior o valor de α , maior tende a ser o comprimento médio das listas encadeadas e, conseqüentemente, maior será o tempo necessário para realizar uma busca. O pior caso ocorre quando todas as n chaves são mapeadas para o mesmo slot da tabela. Nesse cenário, toda a estrutura reduz-se a uma única lista encadeada contendo n elementos, fazendo com que a busca possua complexidade

$$\mathcal{O}(n).$$

Felizmente, essa situação é extremamente improvável quando se utiliza uma boa função de hash. Para analisar o comportamento esperado da estrutura, assume-se a seguinte hipótese.

Hipótese de Hashing Simples Uniforme (HSU) Assumiremos que toda chave do conjunto universo possui a mesma probabilidade de ser mapeada para qualquer um dos m slots da tabela:

$$\forall k \in U, \quad P(h(k) = i) = \frac{1}{m}, \quad i = 0, 1, \dots, m-1.$$

Essa hipótese implica que a função de hash distribui as chaves de maneira aproximadamente uniforme entre os slots da tabela. Seja n_i o número de elementos armazenados na lista correspondente ao slot i . Naturalmente,

$$n = n_0 + n_1 + \dots + n_{m-1},$$

e, pela hipótese de uniformidade,

$$E[n_i] = \frac{n}{m} = \alpha.$$

Além disso, assumiremos que o cálculo da função de hash pode ser realizado em tempo constante, isto é,

$$T_h = \mathcal{O}(1).$$

Portanto, o custo da busca passa a depender apenas do número de elementos presentes na lista encadeada correspondente. Devemos analisar separadamente dois casos.

Busca sem sucesso

Nesse caso, a chave procurada não pertence à tabela, isto é,

$$k \notin T.$$

Teorema 1. *Sob a hipótese de hashing simples uniforme, o tempo esperado para uma busca sem sucesso é*

$$\mathcal{O}(1 + \alpha).$$

Demonstração.

Pela hipótese de hashing simples uniforme, qualquer chave inexistente possui probabilidade igual de ser mapeada para qualquer um dos m slots da tabela. Após o cálculo do valor de hash, a busca percorre toda a lista encadeada correspondente ao slot calculado. Como o comprimento esperado dessa lista é

$$E[n_i] = \alpha,$$

o tempo esperado da operação é dado por

$$T(n) = \mathcal{O}(1 + \alpha),$$

onde o termo constante corresponde ao cálculo da função de hash. Observe que o custo cresce linearmente com o fator de carga. Portanto, quanto menor for α , mais eficiente será a busca.

Busca com sucesso

Considere agora o caso em que a chave procurada pertence à tabela,

$$k \in T.$$

Nesse cenário, a análise é ligeiramente diferente, pois listas maiores possuem maior probabilidade de serem acessadas.

Teorema 2. *Sob a hipótese de hashing simples uniforme, o tempo esperado para uma busca com sucesso também é*

$$\mathcal{O}(1 + \alpha).$$

Demonstração.

Assuma inicialmente que cada uma das n chaves armazenadas possui probabilidade igual de ser pesquisada. Ao buscar uma chave k , o algoritmo realiza:

1. o cálculo da função de hash (tempo constante);
2. a inspeção dos elementos que aparecem antes de k na lista encadeada.

Como os novos elementos são inseridos na cabeça da lista, todos os elementos posicionados antes de k foram inseridos posteriormente. Cada nova chave possui probabilidade $1/m$ de ser inserida na mesma lista que k . Assim, o número esperado de elementos examinados pode ser escrito como

$$T(n) = \frac{1}{n} \sum_{i=1}^n \left(1 + \frac{n-i}{m} \right).$$

Desenvolvendo a expressão,

$$T(n) = 1 + \frac{1}{nm} \sum_{i=1}^n (n-i).$$

Como

$$\sum_{i=1}^n (n-i) = \frac{n(n-1)}{2},$$

segue que

$$T(n) = 1 + \frac{n-1}{2m}.$$

Substituindo

$$\alpha = \frac{n}{m},$$

obtemos

$$T(\alpha) = 1 + \frac{\alpha}{2} - \frac{\alpha}{2n}.$$

Como o último termo tende rapidamente a zero quando n cresce, concluímos que

$$T(\alpha) = \mathcal{O}(1 + \alpha).$$

Interpretação prática

O resultado anterior possui uma consequência extremamente importante. Se o número de elementos armazenados cresce proporcionalmente ao tamanho da tabela, isto é,

$$n = cm,$$

para alguma constante positiva c , então

$$\alpha = \frac{n}{m} = c = \mathcal{O}(1).$$

Consequentemente,

$$T(n) = \mathcal{O}(1 + \alpha) = \mathcal{O}(1).$$

Em outras palavras, desde que o fator de carga seja mantido aproximadamente constante, por exemplo, aumentando o tamanho da tabela quando necessário (processo conhecido como *rehashing*), as operações de busca em uma Tabela de Espalhamento possuem tempo esperado constante. Esse resultado explica por que Hash Tables são uma das estruturas de dados mais eficientes para implementar dicionários, tabelas de símbolos, caches, índices e diversas outras aplicações que exigem consultas rápidas.

6.5 Funções Hash

O desempenho de uma Tabela de Espalhamento depende fundamentalmente da qualidade da função de hash utilizada. Uma boa função de hash deve distribuir as chaves da forma mais uniforme possível pelos slots da tabela, minimizando a ocorrência de colisões. Em geral, assume-se que o conjunto universo de chaves é formado pelos números naturais,

$$\mathbb{N} = \{0, 1, 2, 3, \dots\}.$$

Quando as chaves não são naturalmente representadas por números inteiros, é necessário convertê-las para esse domínio antes da aplicação da função de hash. Por exemplo, caracteres podem ser convertidos por meio de seus códigos ASCII (ou Unicode), enquanto strings podem ser interpretadas como sequências de inteiros obtidas a partir desses códigos. Existem diversas estratégias para construir funções de hash. Nesta seção apresentaremos duas das mais utilizadas: o método da divisão e o método da multiplicação.

6.5.1 Método da Divisão

O método da divisão é provavelmente a técnica mais simples para construir uma função de hash. A ideia consiste em utilizar o resto da divisão inteira da chave k pelo número de slots da tabela, denotado por m . A função de hash é definida por

$$h(k) = k \bmod m.$$

Como o resto da divisão inteira sempre pertence ao intervalo

$$0 \leq h(k) \leq m - 1,$$

a função produz diretamente um índice válido para a Tabela de Espalhamento.

Exemplo Considere uma tabela com

$$m = 12$$

slots e uma chave

$$k = 100.$$

Como

$$100 \bmod 12 = 4,$$

o elemento será armazenado na posição 4 da tabela. Observe que essa função requer apenas uma operação de divisão inteira, podendo ser calculada em tempo constante,

$$\mathcal{O}(1).$$

Apesar de sua simplicidade, a escolha do valor de m exerce grande influência sobre a qualidade do espalhamento. Em particular, deve-se evitar escolher

$$m = 2^p,$$

para algum inteiro positivo p . Nesse caso,

$$h(k) = k \bmod 2^p,$$

o que equivale simplesmente a utilizar os p bits menos significativos da chave. Por exemplo,

$$2^1 = 10_2,$$

$$2^2 = 100_2,$$

$$2^3 = 1000_2,$$

$$2^4 = 10000_2,$$

$$2^5 = 100000_2.$$

Consequentemente, se os bits menos significativos das chaves apresentarem algum padrão (situação bastante comum em diversas aplicações), muitas colisões poderão ocorrer. Por essa razão, costuma-se escolher m como um número primo, preferencialmente próximo de uma potência de dois. Como números primos possuem poucos divisores, essa escolha tende a produzir uma distribuição mais uniforme das chaves pelos slots da tabela.

6.5.2 Método da Multiplicação

Outra estratégia bastante utilizada é o método da multiplicação. Diferentemente do método da divisão, sua eficiência é menos sensível à escolha do tamanho da tabela. O procedimento é composto por duas etapas. Primeiramente, multiplica-se a chave k por uma constante real

$$0 < A < 1.$$

Em seguida, extrai-se apenas a parte fracionária desse produto e multiplica-se o resultado pelo número de slots da tabela. A função de hash é dada por

$$h(k) = \lfloor m(kA \bmod 1) \rfloor,$$

onde

$$kA \bmod 1 = kA - \lfloor kA \rfloor$$

representa a parte fracionária de kA . Uma vantagem importante desse método é que o tamanho da tabela pode ser escolhido como

$$m = 2^p,$$

sem que isso comprometa significativamente a qualidade do espalhamento das chaves. Isso simplifica bastante a implementação, especialmente em arquiteturas digitais, nas quais operações envolvendo potências de dois são extremamente eficientes. Embora qualquer constante

$$0 < A < 1$$

possa ser utilizada, alguns valores produzem distribuições significativamente melhores que outros. Um valor clássico, sugerido por Knuth, é

$$A = \frac{\sqrt{5} - 1}{2} \approx 0.6180339887,$$

conhecido como o inverso da razão áurea. Essa constante apresenta excelentes propriedades de espalhamento para uma ampla variedade de conjuntos de chaves e, por esse motivo, tornou-se uma escolha bastante popular na literatura. Em aplicações práticas, tanto o método da divisão quanto o método da multiplicação apresentam custo computacional constante. Entretanto, a qualidade da distribuição das chaves depende fortemente das características dos dados de entrada e da escolha adequada dos parâmetros da função de hash.

6.6 Hashing Universal

Nas seções anteriores assumimos que a função de hash produz uma distribuição aproximadamente uniforme das chaves pelos slots da tabela. Entretanto, essa hipótese pode deixar de ser válida em aplicações reais, principalmente quando as chaves não são aleatórias ou quando um adversário conhece previamente a função de hash utilizada. Suponha, por exemplo, que um usuário malicioso conheça a função de hash empregada por uma Tabela de Espalhamento. Nesse caso, ele pode construir um conjunto de chaves que produzam exatamente o mesmo valor de hash. Como consequência, todos os elementos serão armazenados na mesma lista encadeada, fazendo com que a estrutura se comporte como uma lista linear. Nesse cenário, a complexidade média da busca deixa de ser constante e passa a ser

$$\mathcal{O}(n),$$

eliminando completamente a principal vantagem das Hash Tables. Para reduzir esse tipo de vulnerabilidade, foi proposta a técnica denominada **hashing universal**. A ideia central consiste em não utilizar uma única função de hash fixa, mas sim selecionar aleatoriamente uma função pertencente a uma família previamente construída de funções de hash. Dessa forma, mesmo que um adversário conheça a família de funções utilizada, ele não saberá qual função específica foi escolhida pela estrutura de dados. Essa estratégia torna as colisões previsíveis muito mais difíceis de serem exploradas e garante que o comportamento esperado da tabela permaneça próximo do ideal.

6.6.1 Família Universal de Funções Hash

Seja U o conjunto universo de chaves e seja

$$H = \{h_1, h_2, \dots, h_t\}$$

uma família finita de funções de hash que mapeiam elementos de U nos índices da tabela

$$\{0, 1, \dots, m-1\}.$$

Dizemos que H é uma **família universal** se, para quaisquer duas chaves distintas

$$k, l \in U, \quad k \neq l,$$

o número de funções da família que provocam colisão entre elas é no máximo

$$\frac{|H|}{m}.$$

Equivalentemente, escolhendo-se uma função de hash uniformemente ao acaso em H , a probabilidade de duas chaves distintas colidirem satisfaz

$$P(h(k) = h(l)) \leq \frac{1}{m}.$$

Observe que essa propriedade reproduz, na prática, a hipótese de hashing simples uniforme discutida anteriormente.

6.6.2 Construção de uma Família Universal

Uma forma clássica de construir uma família universal de funções de hash consiste em escolher inicialmente um número primo p suficientemente grande para que todas as possíveis chaves satisfaçam

$$0 \leq k < p.$$

Definem-se então os conjuntos

$$\mathbb{Z}_p = \{0, 1, \dots, p-1\}$$

e

$$\mathbb{Z}_p^* = \{1, 2, \dots, p-1\}.$$

Para cada escolha de parâmetros

$$a \in \mathbb{Z}_p^* \quad \text{e} \quad b \in \mathbb{Z}_p,$$

define-se a função de hash

$$h_{a,b}(k) = ((ak + b) \bmod p) \bmod m.$$

Observe que essa função realiza duas operações sucessivas:

1. uma transformação afim sobre a chave, seguida da redução módulo p ;
2. uma segunda redução módulo m , produzindo um índice válido para a tabela de espalhamento.

A família completa é definida por

$$H_{p,m} = \{h_{a,b} : a \in \mathbb{Z}_p^*, b \in \mathbb{Z}_p\}.$$

Como existem $(p - 1)$ escolhas possíveis para o parâmetro a e p escolhas para o parâmetro b , essa família contém

$$|H_{p,m}| = p(p - 1)$$

funções de hash distintas. Uma característica interessante dessa construção é que o tamanho da tabela m pode ser escolhido livremente, não sendo necessário que ele seja um número primo.

6.6.3 Como o Hashing Universal é Utilizado na Prática?

Uma dúvida natural é a seguinte: se a função de hash é escolhida aleatoriamente, devemos selecionar uma nova função a cada inserção? A resposta é não. Na prática, escolhe-se aleatoriamente uma única função

$$h_{a,b} \in H_{p,m}$$

durante a criação da Tabela de Espalhamento. Essa função permanece sendo utilizada em todas as operações de inserção, busca e remoção. Entretanto, ao longo da execução da aplicação, pode ocorrer uma concentração excessiva de elementos em alguns poucos slots, indicando que a distribuição deixou de ser satisfatória. Nessa situação, escolhe-se uma nova função da família universal e realiza-se o processo conhecido como **rehashing**.

O rehashing consiste em recalcular o valor de hash de todas as chaves já armazenadas utilizando a nova função selecionada e redistribuí-las na tabela. Embora essa operação possua custo linear, ela é realizada apenas ocasionalmente e permite restaurar uma distribuição aproximadamente uniforme das chaves. Essa estratégia torna as Tabelas de Espalhamento significativamente mais robustas, especialmente em aplicações sujeitas a entradas adversariais, como servidores Web, bancos de dados e sistemas distribuídos, nos quais ataques baseados em colisões de hash podem comprometer seriamente o desempenho da estrutura.

6.7 Endereçamento Aberto (*Open Addressing*)

No método de **endereçamento aberto**, todas as chaves são armazenadas diretamente nos slots da própria Tabela de Espalhamento. Diferentemente do tratamento de colisões por encadeamento, não existem listas auxiliares: cada posição da tabela pode armazenar, no máximo, uma única chave. Quando ocorre uma colisão, isto é, quando duas chaves são mapeadas para o mesmo slot, a estratégia consiste em procurar outra posição livre na tabela de acordo com uma sequência de sondagem (*probe sequence*) definida pela função hash. Como todas as chaves são armazenadas diretamente na tabela, existe a possibilidade de que todos os slots sejam ocupados, impossibilitando novas inserções. Por esse motivo, o fator de carga

$$\alpha = \frac{n}{m},$$

onde n representa o número de chaves armazenadas e m o número de slots da tabela, deve satisfazer

$$\alpha \leq 1.$$

Na prática, entretanto, procura-se manter α significativamente menor que 1, pois o desempenho degrada rapidamente quando a tabela se aproxima da capacidade máxima. A principal ideia do endereçamento aberto consiste em utilizar uma função hash parametrizada por uma variável inteira i , que representa o número de tentativas já realizadas para inserir ou localizar uma chave.

Assim, na primeira tentativa ($i = 0$), a função hash aponta para um determinado slot. Caso esse slot esteja ocupado, incrementa-se i e calcula-se uma nova posição. O processo continua até que seja encontrada uma posição livre (durante uma inserção), a chave desejada (durante uma busca) ou que toda a tabela tenha sido examinada. Os algoritmos de inserção e busca são apresentados a seguir.

```
Hash_Insert(T, k) {  
    i = 0  
    repeat {  
        q = h(k, i)  
        if T[q] == NIL {  
            T[q] = k  
            return q  
        }  
        else  
            i = i + 1  
    } until i == m  
    error "overflow"  
}
```

O algoritmo anterior realiza sucessivas tentativas de inserção até encontrar um slot vazio. Caso todos os m slots estejam ocupados, conclui-se que a tabela está cheia e nenhuma nova chave pode

ser inserida. O procedimento de busca utiliza exatamente a mesma sequência de sondagem utilizada durante a inserção. Isso garante que, se uma chave estiver armazenada na tabela, ela será encontrada.

```
Hash_Search(T, k) {  
    i = 0  
    repeat {  
        q = h(k, i)  
        if T[q] == k  
            return q  
        i = i + 1  
    } until T[q] == NIL or i == m  
    return NIL  
}
```

Observe que a busca é encerrada em duas situações distintas:

- quando a chave procurada é encontrada;
- quando é atingido um slot vazio (NIL), indicando que a chave nunca foi inserida na tabela.

Caso nenhuma dessas condições ocorra antes que todos os m slots sejam examinados, conclui-se que a chave não pertence à Tabela de Espalhamento. O aspecto mais importante do endereçamento aberto é a definição da função de sondagem

$$h(k, i),$$

que determina a sequência de posições visitadas durante as operações de busca e inserção. Uma boa função de sondagem deve minimizar o número de colisões secundárias e distribuir uniformemente as tentativas de acesso pelos diferentes slots da tabela. Nas próximas seções serão apresentados os três principais métodos de endereçamento aberto:

- **Linear Probing:** utiliza incrementos constantes entre posições consecutivas;
- **Quadratic Probing:** utiliza deslocamentos quadráticos para reduzir a formação de agrupamentos (*clustering*);
- **Double Hashing:** utiliza uma segunda função hash para definir os deslocamentos, produzindo uma sequência de sondagem muito mais uniforme e, em geral, oferecendo o melhor desempenho entre os métodos clássicos.

6.8 Linear Probing (Sondagem Linear)

A técnica de *Linear Probing* é o método mais simples de endereçamento aberto. Sua ideia consiste em utilizar uma função hash auxiliar

$$h' : U \rightarrow \{0, 1, \dots, m-1\},$$

e definir a função de sondagem como

$$h(k, i) = (h'(k) + i) \bmod m,$$

para

$$i = 0, 1, \dots, m-1.$$

Na primeira tentativa ($i = 0$), a chave é inserida na posição indicada por $h'(k)$. Caso esse slot esteja ocupado, examina-se a posição imediatamente seguinte; se ela também estiver ocupada, verifica-se a próxima, e assim sucessivamente, sempre avançando uma posição por vez (com retorno ao início da tabela quando necessário). Em outras palavras, a sequência de sondagem assume a forma

$$h'(k), h'(k) + 1, h'(k) + 2, h'(k) + 3, \dots$$

tomando sempre o resto da divisão por m . A principal desvantagem desse método é o surgimento do chamado *Primary Clustering* (agrupamento primário). À medida que colisões ocorrem, formam-se blocos consecutivos de posições ocupadas. Novas chaves que colidem com qualquer posição desse bloco tendem a ser inseridas imediatamente após ele, fazendo com que o agrupamento cresça continuamente. Como consequência, tanto as inserções quanto as buscas tornam-se progressivamente mais lentas.

6.9 Quadratic Probing (Sondagem Quadrática)

A sondagem quadrática procura reduzir o problema do agrupamento primário utilizando deslocamentos não lineares. Nesse método, a função de sondagem é definida por

$$h(k, i) = (h'(k) + c_1 i + c_2 i^2) \bmod m,$$

onde $h'(k)$ é uma função hash auxiliar, c_1 e c_2 são constantes previamente definidas e

$$i = 0, 1, \dots, m-1.$$

A primeira posição examinada continua sendo $h'(k)$. Entretanto, em caso de colisão, os próximos deslocamentos passam a crescer quadraticamente, produzindo uma sequência de sondagem mais espaçada do que na sondagem linear. Essa estratégia reduz significativamente o agrupamento primário, pois as posições visitadas deixam de ser consecutivas. Entretanto, ainda existe um problema importante: sempre que duas chaves possuem a mesma posição inicial,

$$h'(k_1) = h'(k_2),$$

elas percorrem exatamente a mesma sequência de sondagem, isto é,

$$h(k_1, i) = h(k_2, i), \quad \forall i.$$

Esse fenômeno é conhecido como *Secondary Clustering* (agrupamento secundário). Embora menos prejudicial que o agrupamento primário, ele ainda aumenta o número médio de colisões.

6.10 Double Hashing (Espalhamento Duplo)

O método de *Double Hashing* é considerado uma das estratégias mais eficientes para o tratamento de colisões em tabelas de espalhamento com endereçamento aberto. Sua principal vantagem é produzir sequências de sondagem muito próximas de uma permutação aleatória dos slots da tabela, reduzindo significativamente tanto o agrupamento primário quanto o secundário. Nesse método, utilizam-se duas funções hash independentes, definindo

$$h(k, i) = (h_1(k) + i h_2(k)) \bmod m.$$

A primeira tentativa de inserção ou busca ocorre na posição

$$h_1(k),$$

enquanto a segunda função hash determina o tamanho do deslocamento entre duas sondagens consecutivas. Observe que, diferentemente dos métodos anteriores, tanto a posição inicial quanto os deslocamentos dependem da própria chave. Assim, duas chaves que colidem inicialmente tendem a seguir caminhos completamente diferentes na tabela, reduzindo drasticamente a probabilidade de novas colisões. Para garantir que toda a tabela possa ser percorrida durante a sondagem, é desejável que os valores produzidos por $h_2(k)$ sejam relativamente primos com o tamanho da tabela m . Existem duas estratégias bastante utilizadas para satisfazer essa condição:

- escolher m como uma potência de 2 e definir $h_2(k)$ de forma que sempre produza valores ímpares;
- escolher m como um número primo e definir

$$h_2(k) = 1 + (k \bmod m'),$$

onde $m' < m$, normalmente adotando-se

$$m' = m - 1.$$

Uma escolha bastante comum é

$$h_1(k) = k \bmod m$$

e

$$h_2(k) = 1 + (k \bmod m').$$

A figura a seguir ilustra um exemplo de espalhamento duplo.

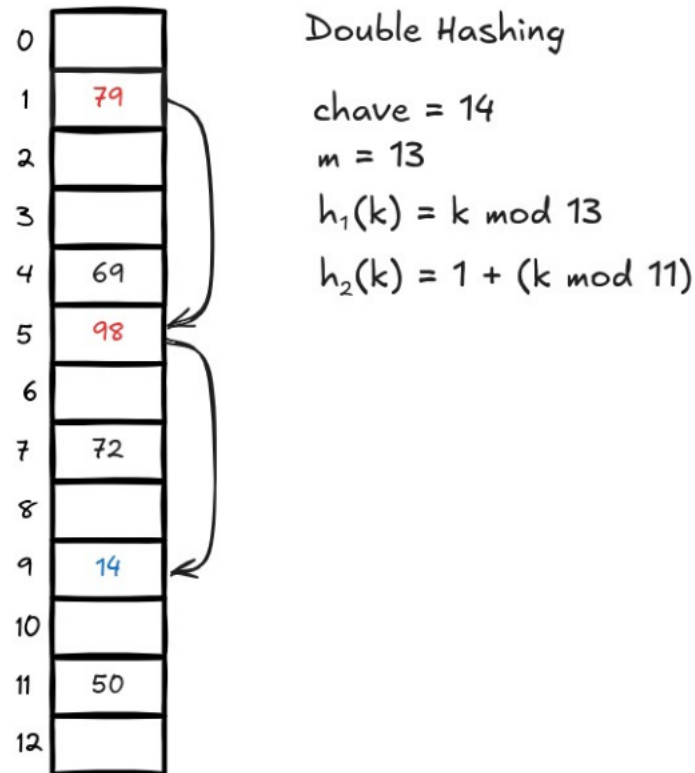


Figura 62: Sequências de colisões até a inserção da chave 14 usando uma Hash Table de endereçamento aberto com Hashing Duplo.

Considere uma Tabela de Espalhamento com

$$m = 13,$$

utilizando as funções

$$h_1(k) = k \bmod 13$$

e

$$h_2(k) = 1 + (k \bmod 11).$$

Suponha que desejamos inserir a chave $k = 14$. Na primeira tentativa ($i = 0$),

$$h_1(14) = 14 \bmod 13 = 1,$$

de modo que o algoritmo tenta inserir a chave no slot 1. Como essa posição já está ocupada, calcula-se

$$h_2(14) = 1 + (14 \bmod 11) = 1 + 3 = 4.$$

Na segunda tentativa ($i = 1$), obtém-se

$$(1 + 1 \cdot 4) \bmod 13 = 5.$$

Supondo que o slot 5 também esteja ocupado, realiza-se uma terceira tentativa ($i = 2$):

$$(1 + 2 \cdot 4) \bmod 13 = 9.$$

Como o slot 9 está livre, a chave é finalmente inserida nessa posição. Observe que o deslocamento permanece constante para uma mesma chave, mas varia de uma chave para outra. Essa característica faz com que diferentes elementos percorram sequências de sondagem distintas, tornando o espalhamento muito mais uniforme e reduzindo significativamente o número de colisões.

6.11 Análise da Complexidade no Endereçamento Aberto

Nesta seção analisaremos o desempenho das operações em Tabelas de Espalhamento com endereçamento aberto. Assumiremos, assim como na análise anterior, a hipótese de **hashing uniforme**, isto é, cada sequência de sondagem é igualmente provável entre todas as sequências possíveis. O principal resultado é apresentado a seguir.

Teorema 3. *Considere uma Tabela de Espalhamento com endereçamento aberto e fator de carga*

$$\alpha = \frac{n}{m} < 1,$$

onde n representa o número de chaves armazenadas e m o número de slots da tabela. Sob a hipótese de hashing uniforme, o número esperado de sondagens realizadas durante uma busca sem sucesso (isto é, para uma chave $k \notin T$) é, no máximo,

$$\frac{1}{1 - \alpha}.$$

Demonstração. Como $\alpha < 1$, existe pelo menos um slot vazio na tabela. Em uma busca sem sucesso, todas as sondagens, exceto a última, necessariamente visitam posições ocupadas. Seja X a variável aleatória que representa o número de sondagens realizadas em slots ocupados antes que um slot vazio seja encontrado. Defina, para $i = 1, 2, \dots$, o evento

$$A_i = \{\text{a } i\text{-ésima sondagem visita um slot ocupado}\}.$$

Assim,

$$\{X \geq i\} = A_1 \cap A_2 \cap \cdots \cap A_{i-1}.$$

Pela regra do produto,

$$P(X \geq i) = P(A_1)P(A_2|A_1) \cdots P(A_{i-1}|A_1 \cap \cdots \cap A_{i-2}).$$

Na primeira sondagem, a probabilidade de encontrar um slot ocupado é

$$\frac{n}{m}.$$

Na segunda, supondo que a primeira também tenha encontrado um slot ocupado, essa probabilidade torna-se

$$\frac{n-1}{m-1},$$

e, em geral,

$$P(A_j|A_1 \cap \cdots \cap A_{j-1}) = \frac{n-(j-1)}{m-(j-1)}.$$

Portanto,

$$P(X \geq i) = \frac{n}{m} \cdot \frac{n-1}{m-1} \cdots \frac{n-(i-2)}{m-(i-2)}.$$

Como $n < m$, vale

$$\frac{n-j}{m-j} \leq \frac{n}{m} = \alpha,$$

para todo $0 \leq j < n$. Logo,

$$P(X \geq i) \leq \alpha^{i-1}.$$

Resta agora calcular o valor esperado de X . Para qualquer variável aleatória discreta não negativa, vale a identidade (ver figura 63)

$$E[X] = \sum_{i=1}^{\infty} P(X \geq i).$$

Assim,

$$E[X] \leq \sum_{i=1}^{\infty} \alpha^{i-1}.$$

Efetuada a substituição de variável

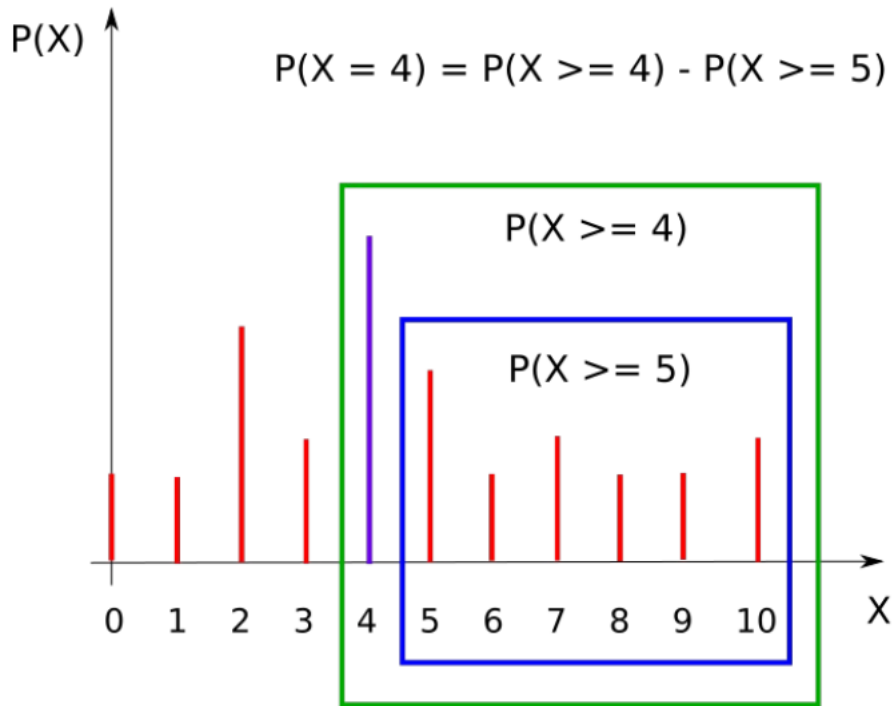


Figura 63: Para qualquer variável aleatória discreta não negativa X , vale a identidade $P(X = i) = P(X \geq i) - P(X \geq i + 1)$.

$$k = i - 1,$$

obtemos

$$E[X] \leq \sum_{k=0}^{\infty} \alpha^k.$$

O lado direito corresponde à soma de uma progressão geométrica infinita de razão α , cuja soma é

$$\sum_{k=0}^{\infty} \alpha^k = \frac{1}{1 - \alpha}, \quad 0 \leq \alpha < 1.$$

Concluimos, portanto,

$$E[X] \leq \frac{1}{1 - \alpha},$$

como desejado. □

Esse resultado mostra que o desempenho do endereçamento aberto depende diretamente do fator de carga da tabela. Enquanto α permanecer significativamente menor que 1, o número esperado de sondagens permanece pequeno. Por exemplo,

α	Número esperado de sondagens
0.50	2
0.75	4
0.90	10
0.95	20

Observe que o crescimento é bastante acentuado quando a tabela se aproxima da ocupação total. Isso explica por que implementações práticas costumam realizar *rehashing* quando o fator de carga ultrapassa um determinado limite (tipicamente entre 0.70 e 0.80). Além disso, se o tamanho da tabela é mantido proporcional ao número de elementos armazenados, isto é,

$$m = cn,$$

para alguma constante $c > 1$, então

$$\alpha = \frac{1}{c},$$

também é constante. Consequentemente,

$$\frac{1}{1 - \alpha} = O(1),$$

o que implica que a busca sem sucesso possui complexidade esperada constante.

Teorema 4. *A inserção de uma nova chave em uma Tabela de Espalhamento com endereçamento aberto e fator de carga $\alpha < 1$ requer, em média, no máximo*

$$\frac{1}{1 - \alpha}$$

sondagens.

A demonstração decorre diretamente do teorema anterior. De fato, para inserir uma nova chave, basta percorrer a sequência de sondagem até encontrar o primeiro slot vazio. Como esse procedimento é exatamente o mesmo realizado durante uma busca sem sucesso, ambos possuem o mesmo número esperado de sondagens.

Teorema 5. *Considere uma Tabela de Espalhamento com endereçamento aberto e fator de carga*

$$\alpha = \frac{n}{m} < 1.$$

Sob a hipótese de hashing uniforme, o número esperado de sondagens realizadas em uma busca com sucesso (isto é, para uma chave $k \in T$) é, no máximo,

$$\frac{1}{\alpha} \ln \left(\frac{1}{1 - \alpha} \right).$$

Demonstração. Uma busca com sucesso percorre exatamente a mesma sequência de sondagens utilizada quando a chave foi inserida na tabela. Suponha que a chave procurada tenha sido a $(i + 1)$ -ésima chave inserida na estrutura. No instante de sua inserção, já existiam i elementos armazenados, de modo que o fator de carga era

$$\alpha_i = \frac{i}{m}.$$

Pelo teorema anterior, o número esperado de sondagens necessárias para inserir essa chave é limitado por

$$E[X_i] \leq \frac{1}{1 - \alpha_i} = \frac{1}{1 - \frac{i}{m}} = \frac{m}{m - i}.$$

Como todas as n chaves armazenadas possuem a mesma probabilidade de serem consultadas, o número esperado de sondagens em uma busca com sucesso é a média desses valores:

$$\frac{1}{n} \sum_{i=0}^{n-1} \frac{m}{m - i}.$$

Como

$$\alpha = \frac{n}{m},$$

podemos escrever

$$\frac{m}{n} = \frac{1}{\alpha},$$

obtendo

$$\frac{1}{\alpha} \sum_{i=0}^{n-1} \frac{1}{m - i}.$$

Efetuada a mudança de variável

$$k = m - i,$$

temos

$$i = 0 \implies k = m,$$

e

$$i = n - 1 \implies k = m - n + 1.$$

Assim,

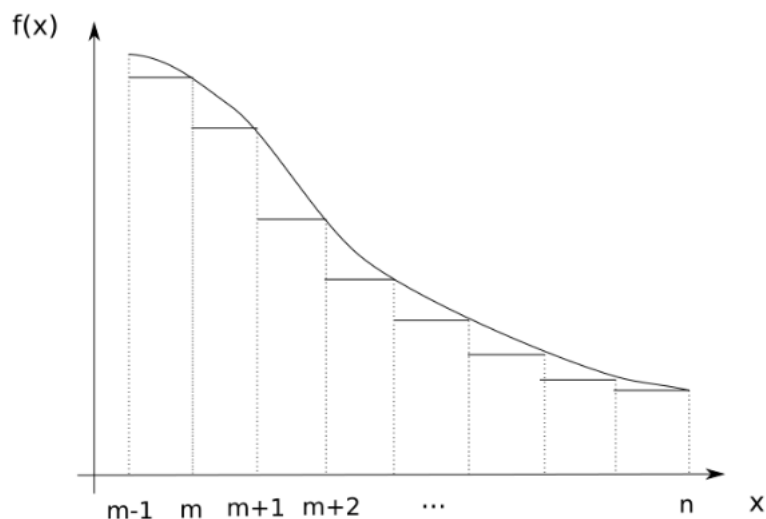


Figura 64: Para qualquer função monotonicamente decrescente, a soma das áreas dos retângulos é menor que a área sob a curva (integral).

$$\frac{1}{\alpha} \sum_{k=m-n+1}^m \frac{1}{k}.$$

Como a função

$$f(x) = \frac{1}{x}$$

é positiva e monotonicamente decrescente, vale a desigualdade (veja figura 64)

$$\sum_{k=a}^b f(k) \leq \int_{a-1}^b f(x) dx.$$

Aplicando esse resultado,

$$\begin{aligned} \frac{1}{\alpha} \sum_{k=m-n+1}^m \frac{1}{k} &\leq \frac{1}{\alpha} \int_{m-n}^m \frac{1}{x} dx \\ &= \frac{1}{\alpha} [\ln x]_{m-n}^m \\ &= \frac{1}{\alpha} (\ln m - \ln(m-n)) \\ &= \frac{1}{\alpha} \ln \left(\frac{m}{m-n} \right). \end{aligned}$$

Dividindo numerador e denominador da fração por m , obtemos

$$\frac{m}{m-n} = \frac{1}{1 - \frac{n}{m}} = \frac{1}{1 - \alpha}.$$

Portanto,

$$E[X] \leq \frac{1}{\alpha} \ln \left(\frac{1}{1 - \alpha} \right),$$

concluindo a demonstração. □

O resultado anterior mostra que a busca com sucesso é ainda mais eficiente do que a busca sem sucesso. Mesmo quando a tabela apresenta um fator de carga elevado, o número esperado de sondagens permanece bastante reduzido. Por exemplo,

α	Número esperado de sondagens
0.50	1.386
0.75	1.848
0.90	2.558
0.95	3.153

Esses valores evidenciam que o desempenho do endereçamento aberto permanece excelente mesmo para fatores de carga relativamente altos. Entretanto, à medida que α se aproxima de 1, o número de colisões cresce rapidamente, tornando recomendável aumentar o tamanho da tabela por meio de um processo de *rehashing* antes que ela fique excessivamente cheia.

Surge então uma questão natural: *é possível reduzir ainda mais a probabilidade de colisões?* Para responder a essa pergunta, estudaremos na próxima seção alguns resultados probabilísticos que ajudam a compreender o comportamento das funções hash e a ocorrência de colisões em Tabelas de Espalhamento.

6.12 Calculando Probabilidades de Colisões

Uma questão natural no estudo de Tabelas de Espalhamento é determinar a probabilidade de ocorrência de colisões. Considere uma tabela hash com m posições (slots) na qual serão inseridas n chaves. A pergunta que desejamos responder é:

Qual é a probabilidade de ocorrer pelo menos uma colisão durante a inserção dessas n chaves?

Antes de responder essa questão, é importante desfazer um equívoco bastante comum. Muitas pessoas imaginam que colisões somente se tornam frequentes quando a tabela está quase completamente ocupada. Por exemplo, poderia parecer razoável acreditar que uma tabela preenchida em apenas 25% apresentaria aproximadamente 25% de probabilidade de colisões. Entretanto, essa intuição está incorreta.

6.12.1 O paradoxo do aniversário

O fenômeno responsável por esse comportamento é conhecido como **Paradoxo do Aniversário**. Considere um grupo de pessoas e a seguinte pergunta:

Qual é o menor número de pessoas necessário para que a probabilidade de duas delas fazerem aniversário no mesmo dia seja superior a 50%?

Como existem 365 dias possíveis para aniversários (desconsiderando anos bissextos), é comum imaginar que seriam necessárias aproximadamente 183 pessoas para atingir essa probabilidade. No entanto, esse raciocínio é incorreto. O resultado surpreendente é que apenas **23 pessoas** são suficientes para que a probabilidade de duas delas compartilharem a mesma data de aniversário ultrapasse 50%.

A explicação está no fato de que não estamos interessados em comparar o aniversário de uma pessoa específica com as demais, mas sim em considerar *todos os pares possíveis* de pessoas. Em um grupo de 23 indivíduos existem

$$\binom{23}{2} = \frac{23 \times 22}{2} = 253$$

pares distintos de pessoas. Embora a probabilidade de um par específico compartilhar aniversário seja apenas

$$\frac{1}{365},$$

existem centenas de pares possíveis, fazendo com que a probabilidade total cresça rapidamente. Uma aproximação bastante intuitiva consiste em assumir independência entre os pares e escrever

$$P(\text{colisão}) \approx 1 - \left(\frac{364}{365}\right)^{253} \approx 50.05\%.$$

Esse resultado mostra que colisões podem ocorrer muito antes de o espaço disponível estar próximo da capacidade máxima.

6.12.2 Probabilidade de colisões em Tabelas Hash

O mesmo raciocínio pode ser aplicado às Tabelas de Espalhamento. Considere uma tabela com m posições e suponha que n chaves sejam distribuídas aleatoriamente entre esses slots. A probabilidade de ocorrer pelo menos uma colisão é

$$P_{m,n}(\text{colisão}) = 1 - P_{m,n}(\text{sem colisão}).$$

Para que não ocorra nenhuma colisão,

- a primeira chave pode ocupar qualquer posição;
- a segunda deve evitar exatamente uma posição;

- a terceira deve evitar duas posições;
- e assim sucessivamente.

Logo,

$$P_{m,n}(\text{sem colisão}) = 1 \cdot \frac{m-1}{m} \cdot \frac{m-2}{m} \dots \frac{m-n+1}{m},$$

ou, de forma compacta,

$$P_{m,n}(\text{sem colisão}) = \frac{m!}{(m-n)! m^n}.$$

Consequentemente,

$$P_{m,n}(\text{colisão}) = 1 - \frac{m!}{(m-n)! m^n}$$

representa a probabilidade de ocorrer pelo menos uma colisão durante a inserção de n chaves.

Exemplos

Para $m = 365$ e $n = 23$, obtém-se

$$P(\text{colisão}) \approx 50.07\%,$$

que corresponde exatamente ao Paradoxo do Aniversário. Se aumentarmos para

$$n = 57,$$

a probabilidade passa para aproximadamente

$$99.01\%,$$

ou seja, é praticamente certo que existirá pelo menos uma colisão. Agora considere uma Tabela Hash com

$$m = 1000$$

slots.

Mesmo utilizando apenas

$$n = 250$$

chaves (ocupação de apenas 25%), obtemos

$$P(\text{colisão}) \approx 99.99999999999984\%.$$

Esse resultado mostra que colisões são praticamente inevitáveis, mesmo quando a tabela está muito longe de sua capacidade máxima. A Tabela 4 apresenta alguns valores ilustrativos.

Tabela 4: Probabilidade de pelo menos uma colisão para $m = 1000$.

Número de chaves (n)	Probabilidade de colisão
38	50.93%
50	71.22%
75	94.19%
100	99.40%
125	99.97%

Observe que apenas 38 chaves já produzem uma probabilidade superior a 50% de ocorrência de colisão, apesar da tabela possuir 1000 posições disponíveis. Esses resultados demonstram que o objetivo do projeto de uma Tabela Hash não é eliminar completamente as colisões, o que é matematicamente impossível quando $|U| > m$, mas sim tratá-las de maneira eficiente.

6.13 Número esperado de colisões

Além da probabilidade de ocorrência, também é interessante estimar o número médio de colisões produzidas durante a inserção das chaves. Podemos interpretar a inserção como um experimento em que cada chave é lançada aleatoriamente em uma dentre m caixas. A primeira chave nunca produz colisão. A segunda possui probabilidade

$$\frac{1}{m}$$

de colidir com a primeira. A terceira possui probabilidade

$$\frac{2}{m}$$

de colidir com alguma das duas anteriores. De forma geral, a i -ésima chave possui probabilidade

$$\frac{i-1}{m}$$

de gerar uma colisão. Assim, o número esperado de colisões é

$$N_m = \sum_{i=1}^{n-1} \frac{i}{m} = \frac{1}{m} \sum_{i=1}^{n-1} i.$$

Utilizando a identidade

$$\sum_{i=1}^{n-1} i = \frac{n(n-1)}{2},$$

obtemos

$$N_m = \frac{n(n-1)}{2m}.$$

A Tabela 5 apresenta alguns exemplos para $m = 1000$.

Tabela 5: Número esperado de colisões para uma tabela com 1000 slots.

Número de chaves (n)	Colisões esperadas
100	≈ 5
250	≈ 31
500	≈ 125
750	≈ 281
1000	≈ 500

Observe que, quando a tabela está completamente preenchida ($n = m$), espera-se aproximadamente

$$\frac{m}{2}$$

colisões. Esse resultado reforça uma importante recomendação prática: sempre que possível, a Tabela Hash deve possuir um número de slots significativamente maior que o número esperado de elementos. Em aplicações reais, é comum dimensionar a tabela com um fator de carga inferior a 0,75, reduzindo o número médio de colisões e preservando o desempenho das operações de busca, inserção e remoção.

Embora métodos como encadeamento lógico e endereçamento aberto tratem colisões de forma bastante eficiente, existem aplicações em que se deseja eliminá-las completamente. Para esses casos, estudaremos na próxima seção a técnica conhecida como **hashing perfeito**, capaz de garantir buscas em tempo constante sem colisões para conjuntos estáticos de chaves.

6.14 Perfect Hashing (Espalhamento Perfeito)

As técnicas de hashing estudadas até o momento apresentam excelente desempenho no caso médio. Entretanto, no pior caso, colisões podem fazer com que a complexidade da busca aumente para $O(n)$. Existe, porém, uma situação particular em que é possível garantir buscas em tempo constante mesmo no pior caso: quando o conjunto de chaves é **estático**, isto é, nenhuma chave será inserida ou removida após a construção da estrutura. Esse cenário é bastante comum em diversas aplicações práticas. Alguns exemplos incluem:

- tabelas de palavras reservadas de compiladores;
- catálogos de arquivos gravados em mídias somente leitura (CD-ROM, DVD, firmware);
- dicionários estáticos utilizados por mecanismos de busca;
- tabelas de símbolos previamente conhecidas.

Nesses casos, todo o conjunto de chaves é conhecido antecipadamente. Como consequência, pode-se investir um tempo maior na construção da estrutura para obter buscas extremamente rápidas posteriormente. Essa estratégia é conhecida como **Perfect Hashing** (Espalhamento Perfeito).

Ideia geral

O Perfect Hashing utiliza uma estratégia de hashing em dois níveis baseada em hashing universal. No primeiro nível, uma função hash distribui todas as chaves em uma tabela principal. Em seguida, cada posição dessa tabela possui sua própria tabela hash secundária, construída de forma que não existam colisões. O funcionamento pode ser resumido da seguinte maneira:

1. Utiliza-se uma função hash primária $h(k)$ para distribuir as n chaves em uma tabela principal com m posições.
2. Todas as chaves que forem mapeadas para um mesmo slot j são armazenadas em uma tabela hash secundária S_j .
3. Cada tabela secundária possui sua própria função hash, escolhida especificamente para eliminar completamente as colisões.

A principal característica do método é que, ao invés de utilizar listas encadeadas para resolver colisões, cada slot da tabela principal armazena uma nova Tabela de Espalhamento.

Tabelas hash secundárias

Suponha que o slot j da tabela principal receba n_j chaves. A tabela secundária correspondente terá tamanho

$$m_j = n_j^2.$$

Pode-se demonstrar que esse tamanho é suficiente para que exista uma função hash universal capaz de distribuir todas essas n_j chaves sem qualquer colisão. Cada tabela secundária utiliza sua própria função hash

$$h_j(k) = ((a_j k + b_j) \bmod p_j) \bmod m_j,$$

em que:

- a_j e b_j são escolhidos aleatoriamente;
- normalmente utiliza-se $p_j = p$, sendo p um número primo suficientemente grande;
- $m_j = n_j^2$.

Como não existem colisões no segundo nível, cada busca realiza apenas:

1. um acesso à tabela principal;
2. um acesso à tabela secundária correspondente.

Logo, a busca possui complexidade

$$O(1)$$

inclusive no pior caso.

Exemplo

Considere o conjunto estático de chaves

$$K = \{10, 22, 37, 40, 52, 60, 70, 72, 75\}.$$

Suponha que a função hash primária seja

$$h(k) = ((ak + b) \bmod p) \bmod m,$$

com

$$a = 3, \quad b = 42, \quad p = 101, \quad m = 9.$$

Nesse caso,

$$h(75) = 2,$$

portanto, a chave 75 é inicialmente direcionada ao slot 2 da tabela principal. Suponha que o slot 2 possua uma tabela secundária S_2 de tamanho

$$m_2 = 9,$$

e que sua função hash seja

$$h_2(k) = ((a_2k + b_2) \bmod p) \bmod m_2,$$

com

$$a_2 = 10, \quad b_2 = 18.$$

Como

$$h_2(75) = 7,$$

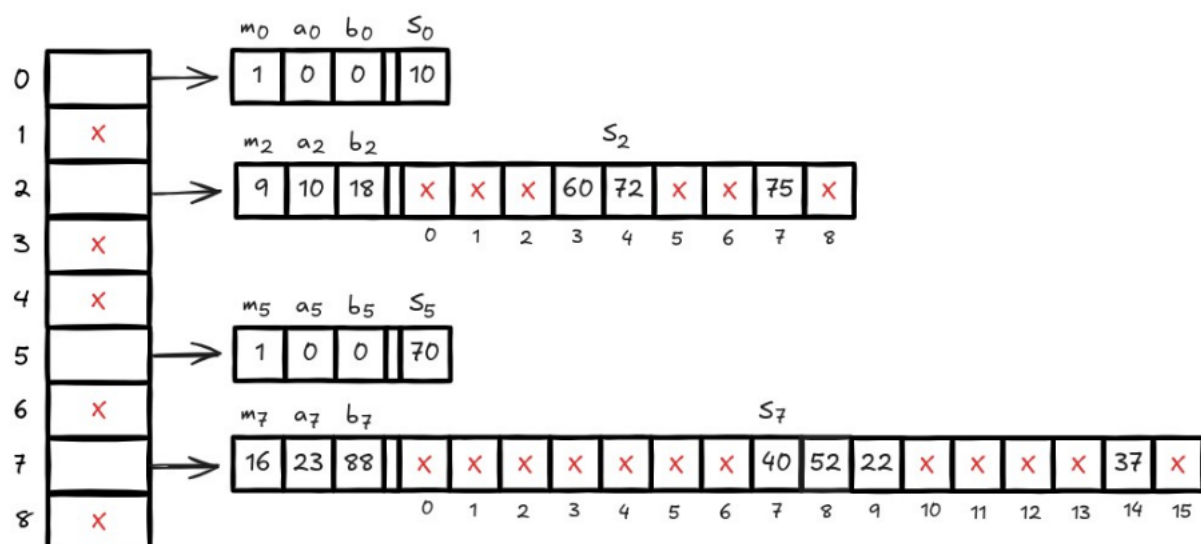


Figura 65: Hashing perfeito com estrutura de dois níveis: o primeiro nível composto por encadeamento lógico e o segundo nível composto por endereçamento aberto.

a chave 75 será armazenada na posição 7 da tabela secundária S_2 . Como a função hash secundária foi construída especificamente para eliminar colisões, nenhuma outra chave será armazenada nessa mesma posição.

Construção da estrutura

O Perfect Hashing exige uma etapa de pré-processamento antes que a estrutura possa ser utilizada. Primeiramente, é necessário determinar quantas chaves serão armazenadas em cada tabela secundária. Somente depois disso é possível definir o tamanho adequado de cada uma delas e construir suas respectivas funções hash. Esse custo adicional de construção é justamente o preço pago para obter buscas em tempo constante garantidas.

Algoritmo básico

Uma forma simples de construir uma estrutura de Perfect Hashing é apresentada a seguir. Inicialmente, define-se:

$$m = n,$$

isto é, a tabela principal possui exatamente o mesmo número de posições que o número de chaves. Escolhe-se também um número primo

$$p > \max(K),$$

e sorteiam-se aleatoriamente os parâmetros a e b para construir a função hash primária

$$h(k) = ((ak + b) \bmod p) \bmod m.$$

Em seguida, executa-se o seguinte procedimento.

1. Para cada chave k :

- calcule $h(k)$;
- incremente o contador n_j correspondente ao slot $j = h(k)$.

2. Calcule o somatório

$$S = n_0^2 + n_1^2 + \dots + n_{m-1}^2$$

3. Se $S > 2n$, então descarte a função hash primária e escolha novos valores para a e b .

4. Caso contrário, para cada slot j , faça $m_j = n_j^2$ e construa uma função hash secundária

$$h_j(k) = ((a_j k + b_j) \bmod p) \bmod m_j$$

escolhendo aleatoriamente a_j e b_j .

5. Teste cada função hash secundária.

6. Caso alguma delas produza colisões, escolha novos valores de a_j e b_j somente para aquela tabela secundária.

7. Repita os passos até que todas as tabelas secundárias estejam livres de colisões.

Complexidade

Durante a construção da estrutura, podem ser necessários diversos testes de funções hash até que todas as tabelas secundárias fiquem livres de colisões. Entretanto, esse custo é pago apenas uma única vez. Após concluída a construção, cada operação de busca realiza exatamente:

1. um cálculo da função hash primária;
2. um acesso à tabela principal;
3. um cálculo da função hash secundária;
4. um acesso à tabela secundária.

Todas essas operações possuem custo constante. Assim, a busca apresenta complexidade

$$O(1)$$

inclusive no pior caso.

Vantagens e limitações

O Perfect Hashing é uma técnica extremamente eficiente para conjuntos estáticos de chaves, oferecendo desempenho previsível e eliminando completamente as colisões durante as consultas. Por outro lado, essa abordagem apresenta algumas limitações importantes:

- o conjunto de chaves deve ser conhecido antecipadamente;
- inserções e remoções não são suportadas de forma eficiente;
- a construção da estrutura pode exigir diversas tentativas até encontrar funções hash adequadas;
- existe um custo adicional de memória devido às tabelas secundárias.

Apesar dessas restrições, o Perfect Hashing é amplamente utilizado em aplicações nas quais o conjunto de dados permanece inalterado por longos períodos, permitindo consultas extremamente rápidas e com desempenho garantido. Veremos um exemplo a seguir.

$$K = \{8, 22, 36, 75, 61, 13, 84, 58\} \quad h(k) = ((ak + b) \bmod p) \bmod m$$
$$p = 87, a = 64, b = 5 \quad h_j(k) = ((a_j k + b_j) \bmod p) \bmod m$$

Passo 1:

Selecione p como um número primo maior que a maior das chaves e gere a e b aleatoriamente

0	
1	
2	
3	
4	
5	
6	
7	

$K = \{8, 22, 36, 75, 61, 13, 84, 58\}$
 $p = 87, a = 64, b = 5$

$$h(k) = ((ak + b) \bmod p) \bmod m$$

$$h_j(k) = ((a_j k + b_j) \bmod p) \bmod m$$

Passo 2:

Para cada chave k , calcule $h(k)$ e conte quantas chaves caem em cada slot

	contagem (n_j)
0	0
1	1
2	2
3	0
4	1
5	1
6	1
7	2

$K = \{8, 22, 36, 75, 61, 13, 84, 58\}$
 $p = 87, a = 64, b = 5$

$$h(k) = ((ak + b) \bmod p) \bmod m$$

$$h_j(k) = ((a_j k + b_j) \bmod p) \bmod m$$

Passo 3:

Verifique se $\sum_{j=0}^{m-1} n_j < 2n$

	contagem (n_j)
0	0
1	1
2	2
3	0
4	1
5	1
6	1
7	2

$$\sum_{j=0}^{m-1} n_j = 1 + 4 + 1 + 1 + 1 + 4 = 12 < 16$$

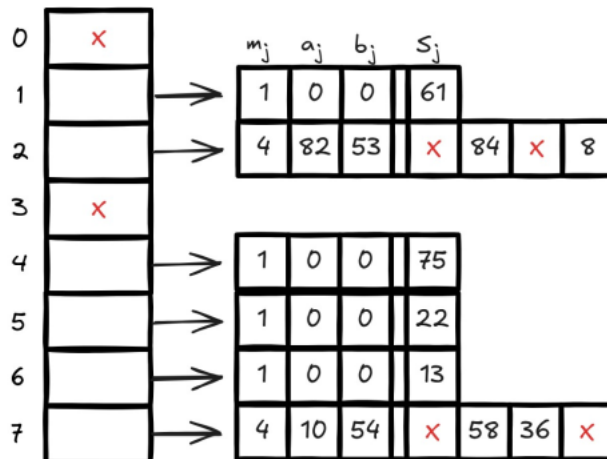
OK

$$K = \{8, 22, 36, 75, 61, 13, 84, 58\} \quad h(k) = ((ak + b) \bmod p) \bmod m$$

$$p = 87, a = 64, b = 5 \quad h_j(k) = ((a_j k + b_j) \bmod p) \bmod m$$

Passo 4:

Distribua as chaves nas tabelas hash do segundo nível



6.15 Considerações Finais

As Tabelas de Espalhamento constituem uma das estruturas de dados mais importantes da Ciência da Computação moderna. Sua principal característica é transformar operações de busca, inserção e remoção em operações de tempo constante no caso médio, tornando-as extremamente eficientes para aplicações que manipulam grandes volumes de dados.

Ao longo deste capítulo, observamos que o desempenho de uma Hash Table depende essencialmente de três fatores: a qualidade da função hash, o fator de carga da estrutura e a estratégia adotada para o tratamento de colisões. Uma função hash bem projetada deve distribuir as chaves de maneira aproximadamente uniforme entre os slots da tabela, minimizando a ocorrência de colisões e preservando o desempenho esperado das operações.

Foram estudadas duas estratégias clássicas para o tratamento de colisões. O *encadeamento lógico* destaca-se por sua simplicidade e por permitir que o fator de carga seja superior a 1, enquanto o *endereçoamento aberto* armazena todas as chaves diretamente na tabela, proporcionando excelente localidade de memória e desempenho prático, desde que o fator de carga permaneça suficientemente baixo. Também foram discutidas diferentes estratégias de sondagem, como sondagem linear, sondagem quadrática e espalhamento duplo (*double hashing*), cada uma apresentando vantagens e limitações específicas.

Outro aspecto importante abordado foi a escolha da função hash. Métodos clássicos, como divisão e multiplicação, oferecem soluções simples e eficientes para diversas aplicações. Entretanto, em cenários sujeitos a entradas adversárias ou maliciosas, o *hashing universal* torna-se uma alternativa bastante robusta, reduzindo significativamente a probabilidade de concentrações excessivas de chaves em poucos slots.

Também foi apresentado o conceito de *Perfect Hashing*, uma técnica destinada a conjuntos estáticos de chaves. Embora seu processo de construção seja mais elaborado, essa abordagem elimina completamente as colisões durante as consultas, garantindo tempo de busca $O(1)$ inclusive no pior caso.

Do ponto de vista probabilístico, um dos resultados mais importantes deste capítulo é que colisões são inevitáveis quando o número de possíveis chaves é muito maior que o tamanho da tabela. O chamado *Paradoxo do Aniversário* mostrou que a probabilidade de ocorrência de colisões cresce muito mais rapidamente do que nossa intuição normalmente sugere. Felizmente, as análises apresentadas demonstram que, mesmo na presença de colisões, uma Tabela de Espalhamento continua apresentando desempenho constante no caso médio, desde que o fator de carga seja mantido aproximadamente constante.

Na prática, Hash Tables estão presentes em praticamente todas as áreas da Computação. Elas são amplamente utilizadas na implementação de dicionários, conjuntos (*sets*), tabelas de símbolos de compiladores, caches, bancos de dados, sistemas de arquivos, roteadores de redes, mecanismos de indexação, sistemas distribuídos, tabelas de tradução de endereços e inúmeras bibliotecas das principais linguagens de programação. Estruturas como `unordered_map` e `unordered_set` da linguagem C++, `HashMap` da linguagem Java e os dicionários (`dict`) da linguagem Python são implementações altamente otimizadas de Hash Tables.

Em resumo, Tabelas de Espalhamento representam um excelente exemplo de como técnicas probabilísticas podem ser empregadas para obter estruturas de dados extremamente eficientes. Embora não garantam desempenho constante em todas as situações, sua simplicidade, rapidez e excelente desempenho no caso médio fazem delas uma das estruturas de dados mais utilizadas na prática, sendo frequentemente a primeira escolha sempre que operações de busca rápida são mais importantes do que a manutenção da ordenação dos elementos.

7 Grafos: Fundamentos Básicos

Os grafos constituem uma das estruturas matemáticas mais importantes da Ciência da Computação. De forma bastante simples, um grafo é um modelo utilizado para representar um conjunto de objetos e os relacionamentos existentes entre eles. Enquanto outras estruturas de dados, como listas, pilhas, filas e árvores, organizam informações de maneira essencialmente linear ou hierárquica, os grafos permitem representar relações arbitrárias entre elementos, tornando-os extremamente versáteis.

A importância dos grafos vai muito além do estudo teórico. Atualmente, uma parcela significativa dos problemas computacionais pode ser modelada naturalmente como um grafo. Em muitos casos, compreender o problema significa, antes de tudo, construir um grafo que represente corretamente seus elementos e suas conexões. Uma vez construída essa representação, diversos algoritmos clássicos podem ser empregados para responder perguntas complexas de forma eficiente.

A Internet é talvez o exemplo mais conhecido. Cada computador, roteador ou servidor pode ser representado como um vértice, enquanto as conexões físicas ou lógicas entre esses dispositivos correspondem às arestas do grafo. Sempre que uma página da Web é acessada, protocolos de roteamento utilizam algoritmos sobre grafos para determinar o caminho mais eficiente para transportar os pacotes de dados entre a origem e o destino.

As redes sociais também são naturalmente modeladas por grafos. Cada usuário corresponde a um vértice, enquanto amizades, seguidores ou conexões profissionais são representados pelas arestas. Essa representação permite identificar comunidades, sugerir novos contatos, detectar perfis influentes, recomendar conteúdos e estudar a propagação de informações (ou até mesmo de notícias falsas) pela rede.

Outro exemplo importante são os sistemas de navegação, como aplicativos de GPS. Nesse contexto, cruzamentos e cidades são representados por vértices, enquanto ruas e rodovias correspondem às arestas do grafo. Associando pesos às arestas, como distância, tempo de viagem ou custo de pedágio, é possível utilizar algoritmos para encontrar rotas ótimas entre dois pontos.

Grafos também aparecem em diversas outras áreas da Computação e da Engenharia, por exemplo:

- redes de computadores e protocolos de roteamento;
- circuitos elétricos e eletrônicos;
- redes de distribuição de energia elétrica;
- sistemas de transporte urbano, ferroviário e aéreo;
- dependências entre tarefas em sistemas operacionais;
- compiladores, por meio de grafos de fluxo de controle e dependência;
- bancos de dados relacionais e bancos de dados orientados a grafos;

- mecanismos de busca na Web;
- biologia computacional, para representar interações entre proteínas e genes;
- inteligência artificial, aprendizado de máquina e redes neurais baseadas em grafos (*Graph Neural Networks*).

Um exemplo particularmente interessante é o mecanismo de busca do Google. A Web pode ser interpretada como um enorme grafo direcionado, no qual cada página representa um vértice e cada hiperlink constitui uma aresta. Algoritmos como o *PageRank* exploram essa estrutura para estimar a relevância de cada página de acordo com o número e a qualidade dos links recebidos.

Outro exemplo atual são as redes neurais sobre grafos (*Graph Neural Networks* – GNNs), uma das áreas mais promissoras da Inteligência Artificial. Diferentemente das redes neurais tradicionais, que operam sobre dados estruturados em vetores ou matrizes, as GNNs são capazes de aprender diretamente sobre estruturas relacionais complexas, sendo aplicadas em problemas como recomendação de produtos, descoberta de novos medicamentos, previsão de interações moleculares e análise de redes sociais.

O estudo de grafos é particularmente interessante porque muitos problemas aparentemente distintos podem ser modelados utilizando a mesma estrutura matemática. Encontrar o menor caminho entre duas cidades, identificar comunidades em uma rede social, escalonar tarefas em um processador, detectar ciclos em um programa ou planejar a expansão de uma rede elétrica são exemplos de problemas que, embora pertençam a áreas completamente diferentes, compartilham a mesma modelagem baseada em grafos.

Neste capítulo serão apresentados os conceitos fundamentais dessa importante estrutura de dados. Inicialmente estudaremos a terminologia básica, diferentes formas de representação computacional e as principais propriedades dos grafos. Esses conceitos servirão de base para os capítulos seguintes, nos quais serão desenvolvidos algoritmos clássicos para busca, caminhos mínimos, ordenação topológica, dentre outros problemas fundamentais da Ciência da Computação.

Ao final deste estudo, o leitor perceberá que os grafos constituem uma linguagem universal para representar relacionamentos e conexões, estando presentes em praticamente todos os ramos da Computação moderna. Dominar seus conceitos e algoritmos é, portanto, uma habilidade indispensável para qualquer profissional ou pesquisador da área.

7.1 Definições Básicas

Os grafos constituem uma das estruturas matemáticas mais importantes da Ciência da Computação. Antes de estudarmos algoritmos sobre grafos, é necessário estabelecer alguns conceitos fundamentais.

Definição 1. Um *grafo* é um par ordenado

$$G = (V, E),$$

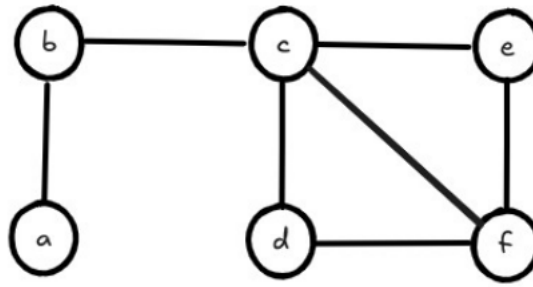


Figura 66: Exemplo de grafo básico simples com 6 vértices e 7 arestas.

em que:

- V é um conjunto não vazio de vértices;
- $E \subseteq V \times V$ é um conjunto de pares de vértices, denominado conjunto de arestas.

Denotamos por $N(v)$ o conjunto dos vértices adjacentes ao vértice v , denominado **vizinhança** de v . Formalmente,

$$N(v) = \{u \in V : (u, v) \in E\}.$$

Por exemplo, no grafo mostrado na Figura 66, temos

$$N(b) = \{a, c\}.$$

Definição 2. O **grau** de um vértice v , denotado por $d(v)$, corresponde ao número de arestas incidentes em v . Em grafos simples, isto é, grafos sem laços e sem arestas múltiplas, o grau de um vértice coincide com o número de seus vizinhos.

Por exemplo,

$$d(a) = 1, \quad d(b) = 2, \quad d(c) = 4.$$

Definição 3. A **lista de graus** de um grafo $G = (V, E)$ é a sequência formada pelos graus de todos os vértices, ordenados em ordem não decrescente. Por exemplo,

$$LG = (1, 2, 2, 2, 3, 4).$$

Um dos resultados mais importantes da teoria dos grafos é o chamado **Lema do Aperto de Mãos** (*Handshaking Lemma*).

Teorema 6 (Handshaking Lema). Se $G = (V, E)$ possui $n = |V|$ vértices e $m = |E|$ arestas, então

$$\sum_{i=1}^n d(v_i) = 2m.$$

Demonstração. Cada aresta possui exatamente duas extremidades. Ao calcularmos a soma dos graus dos vértices, cada aresta é contabilizada exatamente duas vezes: uma vez para cada extremidade. Como o grafo possui m arestas, concluímos que o número total de incidências entre vértices e arestas é igual a $2m$. Portanto,

$$\sum_{i=1}^n d(v_i) = 2m.$$

□

Esse resultado é extremamente útil, pois fornece uma condição necessária para a existência de um grafo com uma determinada sequência de graus. Além disso, ele será utilizado repetidamente na análise de diversos algoritmos sobre grafos. Uma consequência direta do Lema do Aperto de Mãos é um resultado bastante conhecido da Teoria dos Grafos.

Teorema 7. *Em qualquer grafo, o número de vértices de grau ímpar é sempre par.*

Demonstração. Particione o conjunto de vértices em dois subconjuntos:

$$P = \{v \in V : d(v) \text{ é par}\}$$

e

$$I = \{v \in V : d(v) \text{ é ímpar}\}.$$

Pelo Lema do Aperto de Mãos,

$$\sum_{v \in V} d(v) = 2m,$$

onde $2m$ é um número par. Assim,

$$\sum_{u \in I} d(u) = 2m - \sum_{v \in P} d(v).$$

Como a soma dos graus dos vértices pertencentes a P é composta apenas por números pares, ela também é um número par. Logo, a soma dos graus dos vértices pertencentes a I deve ser igualmente par. Entretanto, a soma de números ímpares é par somente quando a quantidade de parcelas é par. Consequentemente,

$$|I|$$

deve ser um número par. Portanto, todo grafo possui um número par de vértices com grau ímpar. □

Esse teorema possui diversas aplicações importantes. Um exemplo clássico aparece na caracterização de grafos que possuem caminhos e circuitos eulerianos, assunto que será estudado posteriormente.

7.2 Representações Computacionais de Grafos

Uma mesma estrutura de grafo pode ser armazenada na memória do computador de diferentes formas. A escolha da representação influencia diretamente a complexidade de diversas operações, bem como o consumo de memória. As três representações mais utilizadas são: matriz de adjacências, matriz de incidência e lista de adjacências.

7.3 Matriz de Adjacências

Considere um grafo $G = (V, E)$ com $n = |V|$ vértices. A **matriz de adjacências** é uma matriz quadrada

$$A \in \mathbb{R}^{n \times n},$$

definida por

$$A_{ij} = \begin{cases} 1, & \text{se } v_j \in N(v_i), \\ 0, & \text{caso contrário.} \end{cases}$$

Em grafos simples, essa matriz possui diversas propriedades importantes:

- a diagonal principal é nula,

$$A_{ii} = 0;$$

- seus elementos pertencem ao conjunto $\{0, 1\}$;
- em grafos não direcionados,

$$A = A^T,$$

isto é, a matriz é simétrica;

- a soma dos elementos da linha i corresponde ao grau do vértice v_i ,

$$d(v_i) = \sum_{j=1}^n A_{ij}.$$

A principal vantagem dessa representação é permitir verificar a existência de uma aresta entre dois vértices em tempo constante. Por outro lado, seu custo de armazenamento é elevado, pois são necessários

$$O(n^2)$$

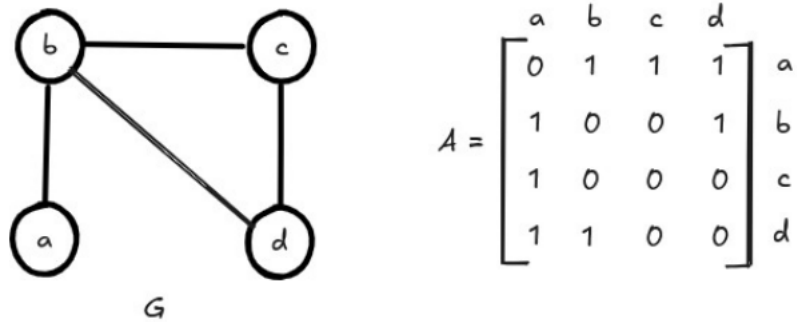


Figura 67: Exemplo de representação de um grafo G via matriz de adjacências

elementos de memória, independentemente do número de arestas do grafo. Considerando 1 byte para armazenar cada posição da matriz, são necessários aproximadamente

$$n^2 \text{ bytes}$$

em memória contígua.

7.4 Matriz de Incidência

Outra possibilidade consiste em utilizar uma **matriz de incidência**. Nesse caso, constrói-se uma matriz

$$M \in \mathbb{R}^{n \times m},$$

na qual cada linha representa um vértice e cada coluna representa uma aresta. Cada coluna indica quais vértices são incidentes à respectiva aresta. Embora essa representação seja útil em algumas aplicações matemáticas e em problemas de otimização, ela possui elevado custo de armazenamento, exigindo

$$O(nm)$$

posições de memória, ou aproximadamente

$$nm \text{ bytes.}$$

Por esse motivo, seu uso é relativamente raro em implementações práticas, sendo restrito a aplicações específicas, como por exemplo métodos espectrais.

7.5 Lista de Adjacências

A representação mais utilizada em aplicações computacionais é a **lista de adjacências**. Nessa estrutura, cada vértice mantém uma referência para uma lista encadeada contendo todos os seus vértices

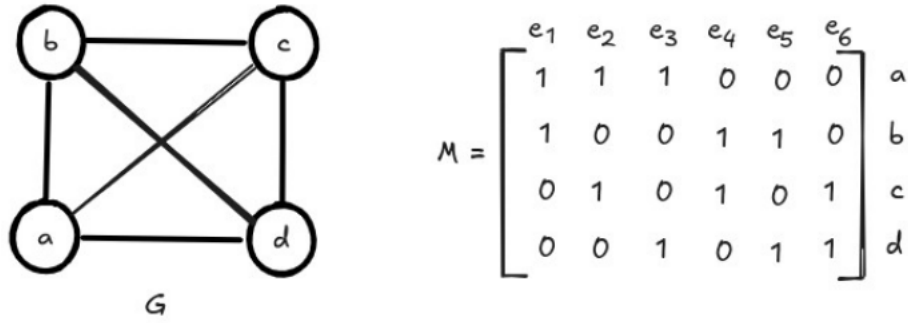


Figura 68: Exemplo de representação de um grafo G via matriz de incidência

vizinhos. O número total de elementos armazenados em todas as listas é igual à soma dos graus dos vértices. Pelo Lema do Aperto de Mãos,

$$\sum_{i=1}^n d(v_i) = 2m.$$

Portanto, em grafos não direcionados, cada aresta aparece exatamente duas vezes nas listas de adjacências: uma vez para cada extremidade. Assim, o espaço necessário para armazenar todas as listas é

$$O(n + m),$$

onde o termo $O(n)$ corresponde ao vetor de referências para as listas e o termo $O(m)$ corresponde aos nós das listas encadeadas.

Supondo que cada inteiro ocupe 32 bits (4 bytes), cada aresta é armazenada duas vezes, consumindo aproximadamente

$$2m \times 4 = 8m$$

bytes de memória, além do pequeno custo adicional das referências das listas. Diferentemente da matriz de adjacências, os elementos não precisam ocupar posições contíguas de memória.

7.6 Quando utilizar cada representação?

A escolha entre matriz e lista de adjacências depende principalmente da densidade do grafo. Definimos a densidade como

$$d = \frac{m}{n^2}.$$

Comparando apenas o espaço necessário para armazenar as arestas, temos:

$$8m < n^2.$$

Lista de adjacências

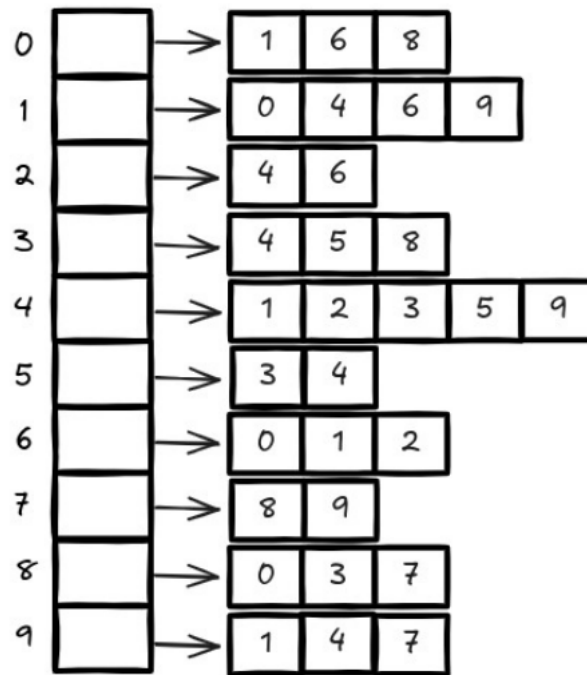


Figura 69: Exemplo de representação de um grafo G via lista de adjacências.

Dividindo ambos os lados por n^2 , obtemos

$$d = \frac{m}{n^2} < \frac{1}{8} = 0.125.$$

Assim, quando a densidade do grafo é inferior a aproximadamente 12,5%, a lista de adjacências tende a utilizar menos memória do que a matriz de adjacências. Como exemplo, considere

$$n = 100 \quad \text{e} \quad m = 1000.$$

Nesse caso,

$$d = \frac{1000}{100^2} = 0.1,$$

ou seja, $d = 10\% < 12.5\%$. Trata-se de um grafo esparsos, tornando a lista de adjacências a escolha mais adequada. Por fim, vale observar que um grafo simples não direcionado possui no máximo

$$\frac{n(n-1)}{2}$$

arestas. Logo, sua densidade máxima é

$$d_{\max} = \frac{\frac{n(n-1)}{2}}{n^2} = \frac{n-1}{2n} = \frac{1}{2} - \frac{1}{2n},$$

de modo que

$$\lim_{n \rightarrow \infty} d_{\max} = \frac{1}{2}.$$

Isso significa que, mesmo em um grafo completo, o máximo valor de densidade é igual a $1/2$.

7.7 Tipos de Grafos

Embora a definição geral de grafo seja bastante simples, diversas classes especiais de grafos possuem propriedades particulares e aparecem frequentemente em aplicações práticas.

7.7.1 Grafos direcionados

Um **grafo direcionado** (ou *digrafo*) é um grafo cujas arestas possuem orientação. Nesse caso,

$$E \subseteq V \times V,$$

de modo que o par ordenado (u, v) representa uma aresta que parte do vértice u e chega ao vértice v . Diferentemente dos grafos não direcionados, em geral,

$$(u, v) \neq (v, u),$$

e, conseqüentemente, a matriz de adjacências deixa de ser simétrica. Em grafos direcionados distinguimos dois conceitos importantes:

- grau de entrada (*in-degree*), denotado por $d^-(v)$;
- grau de saída (*out-degree*), denotado por $d^+(v)$.

Vale a relação

$$\sum_{v \in V} d^-(v) = \sum_{v \in V} d^+(v) = m.$$

7.7.2 Grafos ponderados

Em muitas aplicações, cada aresta possui um custo, distância, capacidade ou tempo associado. Nesses casos utiliza-se um **grafo ponderado**, no qual existe uma função

$$w : E \rightarrow \mathbb{R},$$

que associa um peso a cada aresta. Esse tipo de grafo aparece naturalmente em problemas de roteamento, navegação GPS, planejamento logístico e redes de comunicação. Em muitos cenários, essa função associa uma distância entre dois vértices, criando pesos não negativos.

7.7.3 Multigrafos

Em um **multigrafo** é permitido existir mais de uma aresta ligando o mesmo par de vértices. Quando também são permitidas arestas que ligam um vértice a ele mesmo, chamadas de **laços** (*self-loops*), obtém-se uma estrutura ainda mais geral. Ao longo desta apostila, salvo indicação em contrário, consideraremos grafos simples, isto é, sem laços e sem arestas múltiplas.

7.7.4 Caminhos, Ciclos e Conectividade

Grande parte dos algoritmos em grafos baseia-se nos conceitos de caminho e conectividade.

Definição 4. Um *caminho* é uma sequência de vértices

$$(v_0, v_1, \dots, v_k)$$

tal que

$$(v_i, v_{i+1}) \in E, \quad i = 0, \dots, k-1.$$

O comprimento de um caminho corresponde ao número de arestas que o compõem, isto é, k . Quando todos os vértices do caminho são distintos, ele recebe o nome de **caminho simples**.

Definição 5. Um *ciclo* é um caminho cujo primeiro e último vértices coincidem.

Quando nenhum outro vértice é repetido ao longo do percurso, dizemos que o ciclo é simples. A existência ou não de ciclos desempenha papel fundamental em diversos problemas computacionais, como ordenação topológica, detecção de dependências circulares e construção de árvores geradoras.

7.7.5 Conectividade

Um grafo é dito **conexo** quando existe um caminho entre qualquer par de vértices. Caso contrário, o grafo é denominado **desconexo**. Em grafos direcionados distinguimos dois conceitos:

- **fortemente conexo**: existe caminho orientado entre quaisquer dois vértices em ambas as direções;
- **fracamente conexo**: o grafo torna-se conexo quando ignoramos a orientação das arestas.

7.7.6 Subgrafos

Definição 6. Um *subgrafo* de um grafo

$$G = (V, E)$$

é outro grafo

$$G' = (V', E'),$$

tal que

$$V' \subseteq V \quad e \quad E' \subseteq E.$$

Subgrafos são amplamente utilizados na decomposição de problemas complexos e aparecem em diversos algoritmos de otimização. Quando um grafo não é conexo, ele pode ser decomposto em várias partes independentes.

Definição 7. Uma *componente conexa* é um subgrafo conexo maximal.

A palavra *maximal* significa que nenhum outro vértice pode ser acrescentado ao subgrafo sem perder a propriedade de conexidade. A identificação das componentes conexas é um dos problemas fundamentais em teoria dos grafos e pode ser resolvida eficientemente por algoritmos de busca em profundidade (DFS) ou busca em largura (BFS), que serão estudados nos próximos capítulos.

7.8 Considerações Finais

Neste capítulo foram apresentados os conceitos fundamentais da Teoria dos Grafos, estabelecendo a base necessária para o estudo dos algoritmos que serão desenvolvidos ao longo dos próximos capítulos.

Inicialmente, introduzimos a definição formal de um grafo, bem como conceitos essenciais, como vizinhança, grau de um vértice e lista de graus. Em seguida, demonstramos dois resultados clássicos da teoria: o Lema do Aperto de Mãos, que relaciona a soma dos graus ao número de arestas, e a importante propriedade de que todo grafo possui um número par de vértices de grau ímpar.

Também discutimos as principais representações computacionais de grafos, matriz de adjacências, matriz de incidência e lista de adjacências, destacando suas vantagens, limitações e aplicações. Em particular, vimos que listas de adjacências constituem a representação preferida para grafos esparsos, enquanto matrizes de adjacências tornam-se mais adequadas para grafos densos ou quando consultas de adjacência em tempo constante são prioritárias.

Por fim, apresentamos algumas das principais classes de grafos, bem como os conceitos de caminhos, ciclos, conectividade, subgrafos e componentes conexas. Esses conceitos constituem o vocabulário básico da Teoria dos Grafos e serão utilizados continuamente nos algoritmos estudados posteriormente.

Nos capítulos seguintes, utilizaremos esses fundamentos para desenvolver algoritmos clássicos de busca em grafos, árvores geradoras mínimas, caminhos mínimos, ordenação topológica, componentes fortemente conexas e diversos outros problemas fundamentais da Computação. O leitor perceberá que praticamente toda aplicação envolvendo redes, sistemas distribuídos, roteamento, planejamento, redes sociais ou inteligência artificial pode ser modelada naturalmente por meio de grafos, tornando essa uma das estruturas de dados mais importantes de toda a Ciência da Computação.

8 Busca em Grafos: BFS e DFS

A busca em grafos constitui um dos tópicos mais importantes da Ciência da Computação. Diversos problemas computacionais podem ser formulados como a exploração sistemática de um grafo, na qual o objetivo é visitar seus vértices e arestas de maneira organizada para extrair informações sobre sua estrutura ou encontrar soluções para problemas específicos.

Alguns exemplos clássicos incluem a verificação de conectividade entre dois vértices, a determinação de componentes conexas, a detecção de ciclos, a construção de árvores geradoras, a ordenação topológica de tarefas, a busca por caminhos mínimos e inúmeras aplicações em redes de computadores, sistemas de navegação, redes sociais, bioinformática e inteligência artificial.

De maneira geral, um algoritmo de busca parte de um vértice inicial $s \in V$ e visita progressivamente os vértices alcançáveis a partir dele. Durante esse processo, cada vértice é marcado para evitar visitas repetidas e suas arestas são exploradas segundo uma estratégia bem definida. Ao término da execução, obtém-se uma árvore (ou floresta) de busca, que registra como cada vértice foi descoberto.

Os dois algoritmos fundamentais para exploração de grafos são:

- **Busca em Largura** (*Breadth-First Search – BFS*), que visita inicialmente todos os vértices mais próximos da origem antes de avançar para níveis mais distantes;
- **Busca em Profundidade** (*Depth-First Search – DFS*), que procura explorar cada caminho o mais profundamente possível antes de realizar retrocessos (*backtracking*).

Embora ambos percorram todos os vértices alcançáveis do grafo, as estruturas produzidas e as aplicações de cada algoritmo são bastante diferentes. A BFS é particularmente adequada para problemas envolvendo menores caminhos em grafos não ponderados, enquanto a DFS serve de base para diversos algoritmos clássicos, como detecção de ciclos, ordenação topológica, identificação de componentes conexas e componentes fortemente conexas, pontes, vértices de articulação e muitas outras aplicações.

Assumindo que o grafo seja representado por listas de adjacências, tanto a Busca em Largura quanto a Busca em Profundidade executam em tempo

$$O(|V| + |E|),$$

pois cada vértice é visitado uma única vez e cada aresta é examinada no máximo duas vezes em grafos não direcionados (uma vez a partir de cada extremidade), ou uma única vez em grafos direcionados. Essa complexidade é ótima, uma vez que qualquer algoritmo precisa, no pior caso, examinar todos os vértices e arestas para explorar completamente um grafo.

Nas próximas seções estudaremos detalhadamente esses dois algoritmos, analisando seus princípios de funcionamento, implementações, propriedades e principais aplicações na resolução de problemas computacionais.

8.1 Árvores e suas Propriedades

As árvores constituem uma das estruturas mais importantes da Teoria dos Grafos e da Ciência da Computação. Diversos problemas computacionais podem ser formulados como a tarefa de extrair, a partir de um grafo arbitrário, uma árvore que satisfaça determinada propriedade de interesse.

Ao longo deste livro veremos, por exemplo, algoritmos capazes de construir árvores de busca, árvores geradoras, árvores geradoras mínimas e árvores de caminhos mínimos. Em todos esses casos, parte-se de um grafo qualquer e busca-se uma árvore que preserve alguma característica desejável, como profundidade mínima, custo mínimo ou menor distância entre vértices. Antes de estudarmos esses algoritmos, é importante compreender algumas propriedades fundamentais das árvores.

Definição 8. Um grafo $G = (V, E)$ é uma **árvore** se, e somente se, é conexo e acíclico.

Em outras palavras, uma árvore é um grafo no qual existe um caminho entre quaisquer dois vértices e que não contém ciclos.

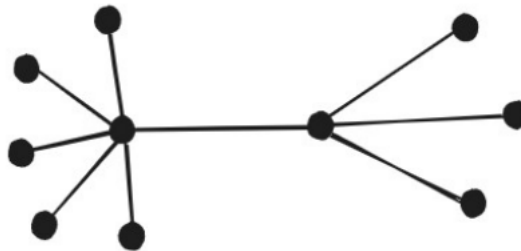


Figura 70: Exemplo de um grafo acíclico e conexo (árvore).

Um dos resultados mais importantes sobre árvores é o seguinte.

Teorema 8. Um grafo $G = (V, E)$ é uma árvore se, e somente se, existe um único caminho simples entre quaisquer dois vértices distintos $u, v \in V$.

Demonstração. ($\Rightarrow$)

Suponha que G seja uma árvore.

Como G é conexo, existe pelo menos um caminho entre quaisquer dois vértices u e v .

Suponha, por absurdo, que existam dois caminhos simples distintos ligando u a v . Percorrendo um dos caminhos e retornando pelo outro, obtemos necessariamente um ciclo, contradizendo o fato de que uma árvore é acíclica.

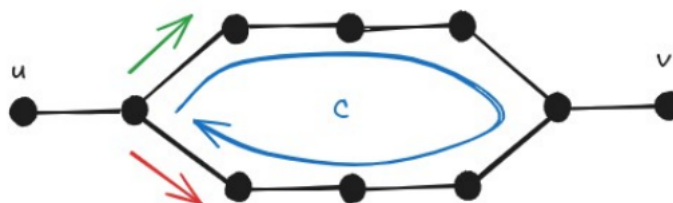


Figura 71: Se há dois caminhos entre um vértice u e outro v , então há um ciclo C no grafo.

Logo, entre quaisquer dois vértices existe exatamente um caminho simples.

($\Leftarrow$)

Agora suponha que exista exatamente um caminho simples entre quaisquer dois vértices distintos. Como existe um caminho entre todo par de vértices, o grafo é conexo.

Resta mostrar que ele é acíclico.

Suponha, novamente por absurdo, que exista um ciclo em G . Escolha dois vértices distintos pertencentes a esse ciclo. Percorrendo o ciclo em sentidos opostos, obtemos dois caminhos simples distintos ligando esses vértices, o que contradiz a hipótese de unicidade.

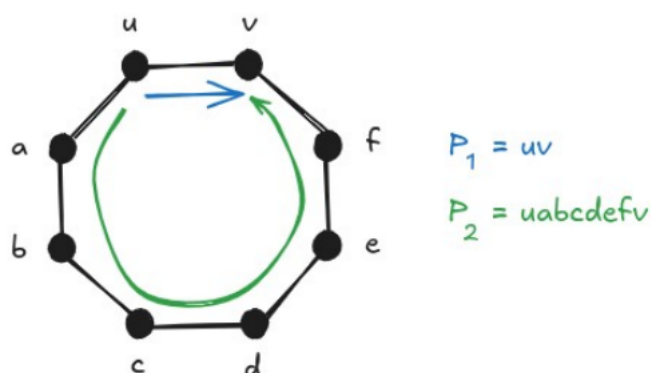


Figura 72: Se há um ciclo C no grafo, então existe 2 caminhos entre um vertice v e outro v .

Portanto, o grafo não possui ciclos e é acíclico.

Como G é simultaneamente conexo e acíclico, concluímos que G é uma árvore.

□

Esse teorema fornece uma caracterização extremamente importante das árvores. Na prática, ele mostra que uma árvore estabelece exatamente uma maneira de conectar quaisquer dois vértices, eliminando ambiguidades e ciclos. Essa propriedade torna as árvores estruturas ideais para representar hierarquias, sistemas de arquivos, redes de comunicação, árvores de decisão e diversas outras aplicações computacionais.

Uma das propriedades mais importantes das árvores diz respeito ao número de arestas em função da quantidade de vértices. Esse resultado será utilizado diversas vezes ao longo do estudo de algoritmos em grafos.

Teorema 9. *Toda árvore com n vértices possui exatamente $n - 1$ arestas.*

Demonstração. Faremos a demonstração por indução sobre o número de vértices.

Caso base:

Para $n = 1$, a árvore é composta por um único vértice e nenhuma aresta. Logo,

$$|E| = 0 = 1 - 1 = n - 1.$$

Portanto, a propriedade é válida para $n = 1$.

Hipótese de indução:

Assuma que toda árvore com k vértices possui exatamente $k - 1$ arestas.

Passo indutivo:

Considere uma árvore T com $k + 1$ vértices. Toda árvore com pelo menos dois vértices possui pelo menos uma folha, isto é, um vértice de grau igual a 1. Seja v uma folha de T . Removendo v e a única aresta incidente a ele, obtemos um novo grafo T' . Observe que:

- T' continua conexo, pois a remoção de uma folha não desconecta os demais vértices;
- T' continua acíclico, pois remover arestas jamais cria ciclos.

Logo, T' também é uma árvore. Além disso,

$$|V(T')| = k.$$

Pela hipótese de indução,

$$|E(T')| = k - 1.$$

Como removemos exatamente uma aresta ao eliminar a folha,

$$|E(T)| = |E(T')| + 1 = (k - 1) + 1 = k.$$

Como

$$|V(T)| = k + 1,$$

segue que

$$|E(T)| = (k + 1) - 1.$$

Portanto, a propriedade também vale para árvores com $k + 1$ vértices. Concluimos, pelo Princípio da Indução Matemática, que toda árvore com n vértices possui exatamente $n - 1$ arestas. $\square$

8.2 Relação entre busca em grafos e árvores

À primeira vista, pode não ser evidente a relação entre algoritmos de busca em grafos e árvores. No entanto, essa conexão é bastante profunda e constitui a base dos principais algoritmos de busca, como a Busca em Largura (BFS) e a Busca em Profundidade (DFS).

A ideia central é que realizar uma busca em um grafo consiste em visitar sistematicamente seus vértices, registrando o caminho utilizado para alcançar cada um deles. Ao final desse processo, obtém-se uma árvore, denominada **árvore de busca**, que representa exatamente como os vértices foram descobertos. Essa construção é possível graças a uma propriedade fundamental das árvores.

Teorema 10. Em uma árvore existe exatamente um único caminho simples entre quaisquer dois vértices.

Essa propriedade implica que cada vértice pode ser identificado de maneira única pelo caminho que parte da raiz da árvore até esse vértice. Em outras palavras, o caminho desempenha o papel de um identificador único. Em árvores binárias de busca, por exemplo, um elemento pode ser localizado seguindo uma sequência única de decisões (*esquerda* ou *direita*) a partir da raiz. Essa sequência funciona como um índice lógico para recuperar o elemento desejado.

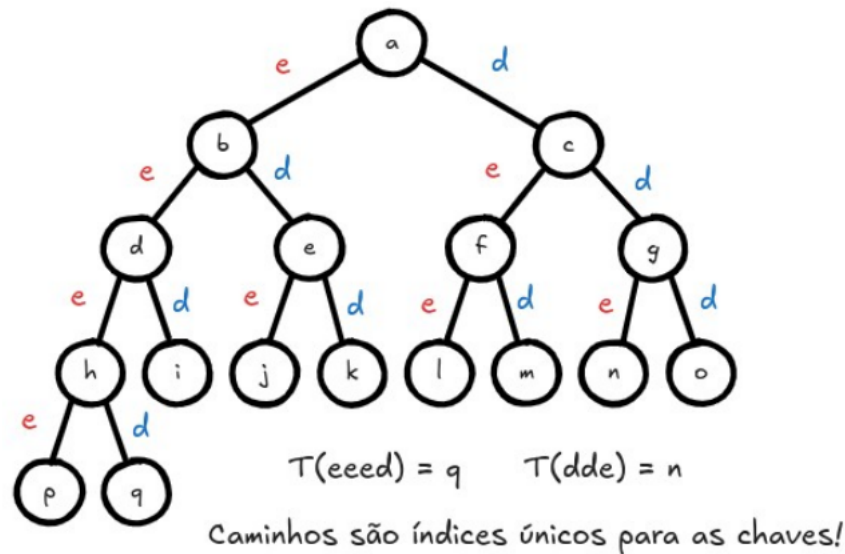


Figura 73: Em uma árvore binária, todo nó pode ser unicamente identificado por uma sequência de direções: esquerda e direita.

Em árvores mais gerais, nas quais um vértice pode possuir um número arbitrário de filhos, o princípio permanece o mesmo: o caminho da raiz até o vértice identifica unicamente sua posição na estrutura.

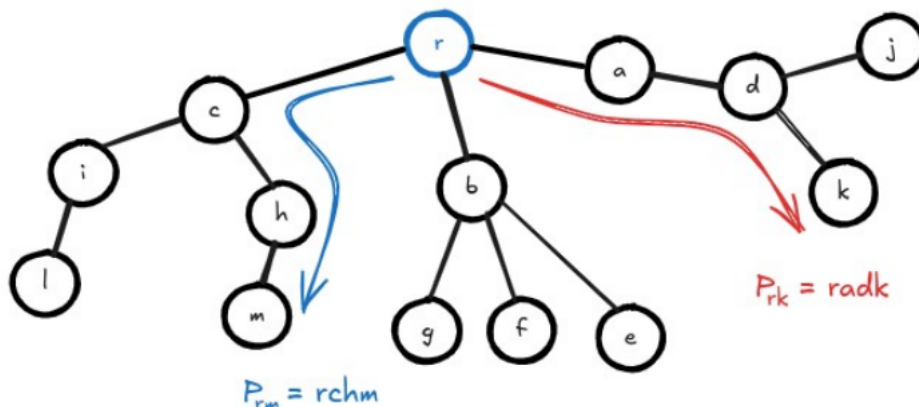


Figura 74: Caminhos únicos são como índices para os vértices de um grafo G.

Assim, quando executamos um algoritmo de busca sobre um grafo $G = (V, E)$, o resultado não é apenas a visita aos vértices, mas também a construção de uma árvore T , enraizada no vértice inicial

r , denominada **árvore de busca**. Nessa árvore, cada vértice (exceto a raiz) possui exatamente um pai, correspondente ao vértice a partir do qual ele foi descoberto pela primeira vez.

Portanto, pode-se interpretar um algoritmo de busca como um procedimento que transforma um grafo arbitrário em uma árvore de busca. Essa árvore preserva a conectividade do componente alcançado e fornece um caminho único da raiz até cada vértice visitado, permitindo reconstruir percursos, calcular distâncias, identificar ancestrais e servir de base para diversos outros algoritmos em teoria dos grafos.

8.3 Busca em Largura (Breadth-First Search – BFS)

A **Busca em Largura** (*Breadth-First Search* – BFS) é um dos algoritmos fundamentais para percorrer grafos. Sua principal característica é visitar os vértices em ordem crescente de distância em relação ao vértice inicial. A ideia central consiste em explorar o grafo por níveis. Todos os vértices localizados a uma mesma distância da raiz são visitados antes que qualquer vértice do nível seguinte seja processado. Essa estratégia garante que os primeiros caminhos encontrados até cada vértice sejam também os caminhos de menor comprimento em grafos não ponderados. Para controlar o estado da busca, o algoritmo associa algumas informações a cada vértice.

Variáveis utilizadas

- **color(v)**: indica o estado de exploração do vértice v , podendo assumir três valores:
 - **WHITE**: o vértice ainda não foi descoberto;
 - **GRAY**: o vértice foi descoberto, mas ainda possui vizinhos a serem explorados;
 - **BLACK**: todos os vizinhos do vértice já foram processados.
- $\lambda(v)$: distância (em número de arestas) entre o vértice inicial s e o vértice v .
- $\pi(v)$: predecessor de v na árvore de busca, isto é, o vértice a partir do qual v foi descoberto.
- Q : fila (*First-In, First-Out* – FIFO) utilizada para armazenar os vértices descobertos que ainda precisam ser processados.

As duas operações básicas realizadas sobre a fila são:

- **push**: insere um elemento no final da fila;
- **pop**: remove o elemento localizado no início da fila.

O algoritmo completo é apresentado a seguir.

```

BFS( $G, s$ ) {
  for each  $v \in V - s$  {
     $v.color = WHITE$ 
     $\lambda(v) = \infty$ 
     $\pi(v) = NIL$ 
  }
   $s.color = GRAY$ 
   $\lambda(s) = 0$ 
   $\pi(s) = NIL$ 
   $Q = \emptyset$ 
  push( $Q, s$ )
  while  $Q \neq \emptyset$  {
     $u = pop(Q)$ 
    for each  $v \in N(u)$  {
      if  $v.color == WHITE$  {
         $v.color = GRAY$ 
         $\lambda(v) = \lambda(u) + 1$ 
         $\pi(v) = u$ 
        push( $Q, v$ )
      }
    }
     $u.color = BLACK$ 
  }
}

```

O algoritmo recebe como entrada um grafo não ponderado $G = (V, E)$ e um vértice inicial s . Como saída, produz uma árvore denominada **árvore BFS** (*BFS tree*), formada pelas arestas utilizadas para descobrir cada vértice pela primeira vez.

8.3.1 Exemplo de execução da Busca em Largura

A seguir, apresentamos um exemplo completo da execução do algoritmo BFS. Considere que a busca é iniciada no vértice raiz s . Em cada iteração, um vértice é removido da fila, seus vizinhos ainda não descobertos são identificados e inseridos na fila, tendo suas distâncias e predecessores atualizados.

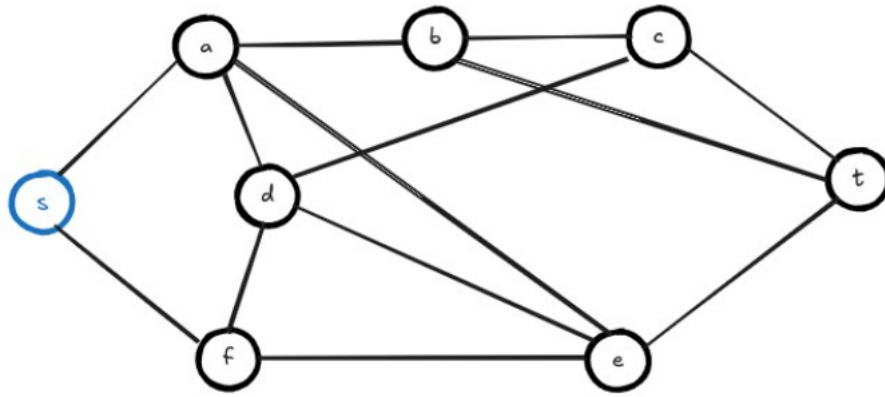


Figura 75: Desejamos encontrar a árvore de busca em largura com raiz no vértice s .

A Tabela 6 apresenta o *trace* completo da execução do algoritmo.

Tabela 6: Trace da execução do algoritmo BFS a partir do vértice s .

Iter.	$u = \text{pop}(Q)$	Novos vértices descobertos	Atualizações realizadas
0	s	$\{a, f\}$	$\lambda(a) = 1, \pi(a) = s$ $\lambda(f) = 1, \pi(f) = s$
1	a	$\{b, d, e\}$	$\lambda(b) = 2, \pi(b) = a$ $\lambda(d) = 2, \pi(d) = a$ $\lambda(e) = 2, \pi(e) = a$
2	f	$\emptyset$	Nenhuma atualização.
3	b	$\{c, t\}$	$\lambda(c) = 3, \pi(c) = b$ $\lambda(t) = 3, \pi(t) = b$
4	d	$\emptyset$	Nenhuma atualização.
5	e	$\emptyset$	Nenhuma atualização.
6	c	$\emptyset$	Nenhuma atualização.
7	t	$\emptyset$	Nenhuma atualização.

A Tabela 7 apresenta a configuração da fila Q ao final de cada iteração do algoritmo de Busca em Largura. Observe que, por se tratar de uma fila (*FIFO – First In, First Out*), os vértices são processados exatamente na ordem em que foram descobertos.

Tabela 7: Evolução da fila durante a execução da BFS.

Iteração	Conteúdo da fila Q
0	$[s]$
1	$[a, f]$
2	$[f, b, d, e]$
3	$[b, d, e]$
4	$[d, e, c, t]$
5	$[e, c, t]$
6	$[c, t]$
7	$[t]$
8	$\emptyset$

Note que a fila cresce à medida que novos vértices são descobertos e diminui conforme esses vértices são processados. Quando a fila torna-se vazia, significa que todos os vértices alcançáveis a partir da raiz já foram visitados, encerrando a execução do algoritmo.

Observe que os vértices são descobertos exatamente em ordem crescente de distância em relação à raiz s . Inicialmente, são visitados todos os vértices a uma aresta da raiz (a e f), em seguida todos os vértices a duas arestas (b , d e e) e, por fim, os vértices a três arestas (c e t). Essa característica é justamente o que garante que a árvore produzida pela BFS armazene os caminhos mínimos (em número de arestas) entre o vértice inicial e todos os demais vértices alcançáveis do grafo. A árvore de busca em largura é codificada pelo seguinte mapa de predecessores:

v	s	a	b	c	d	e	f	t
$\pi(v)$	---	s	a	b	a	a	s	b
$\lambda(v)$	0	1	2	3	2	2	1	3

Figura 76: Mapa de predecessores que codifica a árvore de busca em largura.

8.3.2 Complexidade da Busca em Largura

Uma das principais vantagens do algoritmo de Busca em Largura (Breadth-First Search – BFS) é sua elevada eficiência. Quando o grafo é representado por listas de adjacências, o algoritmo possui complexidade linear em relação ao tamanho da entrada, isto é,

$$T(n, m) = O(n + m),$$

onde:

- $n = |V|$ é o número de vértices do grafo;

- $m = |E|$ é o número de arestas.

Essa complexidade pode ser demonstrada analisando separadamente cada parte do algoritmo.

Inicialização Inicialmente, todos os vértices do grafo são percorridos para inicializar suas variáveis de controle (`color`, λ e π). Esse procedimento executa exatamente uma operação para cada vértice. Assim, o custo da etapa de inicialização é

$$O(n).$$

Processamento da fila Durante a execução do algoritmo, cada vértice é inserido na fila exatamente uma única vez. Isso ocorre porque um vértice só é inserido quando sua cor passa de `WHITE` para `GRAY`. A partir desse momento, ele nunca mais volta ao estado `WHITE`, impossibilitando uma segunda inserção. Da mesma forma, cada vértice também é removido da fila exatamente uma única vez. Como as operações de inserção (`push`) e remoção (`pop`) possuem custo constante, segue que o custo total associado à fila é

$$O(n).$$

Percorrendo as listas de adjacências A parte mais importante da análise consiste no laço

```
for each  $v \in N(u)$  {
    if  $v.\text{color} == \text{WHITE}$  {
         $v.\text{color} = \text{GRAY}$ 
         $\lambda(v) = \lambda(u) + 1$ 
         $\pi(v) = u$ 
        push( $Q$ ,  $v$ )
    }
}
```

À primeira vista, pode parecer que esse laço pode ser executado muitas vezes. Entretanto, basta observar que cada lista de adjacências é percorrida apenas quando seu respectivo vértice é removido da fila. Logo, cada lista é visitada exatamente uma única vez. Se denotarmos por $d(v)$ o grau do vértice v , o número total de iterações realizadas pelos laços internos é

$$\sum_{v \in V} d(v).$$

Pelo Lema do Aperto de Mãos (Handshaking Lemma), sabemos que

$$\sum_{v \in V} d(v) = 2m$$

para grafos não direcionados. Como o fator constante 2 é desprezado na notação assintótica, o custo dessa etapa é

$$O(m).$$

No caso de grafos direcionados, a soma dos graus de saída é exatamente igual ao número de arestas, isto é,

$$\sum_{v \in V} d^+(v) = m,$$

o que conduz ao mesmo resultado assintótico.

Complexidade total Somando os custos das três etapas analisadas, obtemos

$$T(n, m) = O(n) + O(n) + O(m).$$

Como a soma de funções assintóticas é dominada pelo maior termo, segue que

$$T(n, m) = O(n + m).$$

Complexidade espacial Além do tempo de execução, também é interessante analisar o consumo de memória do algoritmo. A BFS armazena, para cada vértice, três informações auxiliares:

- a cor do vértice;
- sua distância até a raiz;
- seu predecessor na árvore de busca.

Além disso, mantém uma fila contendo, no pior caso, até n vértices simultaneamente. Consequentemente, o espaço adicional utilizado pelo algoritmo é

$$O(n),$$

desconsiderando a memória necessária para armazenar o próprio grafo.

Discussão O resultado

$$O(n + m)$$

é considerado ótimo para algoritmos de busca em grafos representados por listas de adjacências, pois cada vértice é processado exatamente uma vez e cada aresta é examinada, no máximo, uma única vez (ou duas vezes em grafos não direcionados, uma por cada extremidade).

Na prática, isso significa que a BFS escala muito bem mesmo para grafos de grande porte. Por esse motivo, ela é amplamente empregada em aplicações como redes de computadores, sistemas de navegação, redes sociais, motores de busca, análise de conectividade e diversos outros problemas em que é necessário explorar sistematicamente a estrutura de um grafo.

8.4 Corretude da Busca em Largura

Uma das propriedades mais importantes da Busca em Largura (BFS) é que ela calcula corretamente a distância mínima da raiz para todos os vértices alcançáveis do grafo.

Teorema 11. *Ao término da execução da BFS iniciada no vértice s , vale que*

$$\lambda(v) = d(s, v), \quad \forall v \in V,$$

onde $d(s, v)$ denota a distância geodésica entre s e v , isto é, o número de arestas do menor caminho entre esses dois vértices.

Demonstração. A demonstração será feita por contradição. Inicialmente, observe que a BFS nunca atribui a um vértice uma distância menor que a distância real. Portanto,

$$\lambda(v) \geq d(s, v), \quad \forall v \in V.$$

Suponha, por absurdo, que exista algum vértice para o qual

$$\lambda(v) > d(s, v).$$

Entre todos os vértices que satisfazem essa propriedade, considere aquele que é o primeiro a ser removido da fila Q .

Como

$$\lambda(v) > d(s, v),$$

segue que existe um caminho entre s e v , caso contrário teríamos

$$d(s, v) = \infty,$$

o que contradiz a hipótese.

Seja

$$P_{sv}^*$$

um caminho mínimo entre s e v , e seja u o predecessor imediato de v nesse caminho. Pela definição de caminho mínimo,

$$d(s, v) = d(s, u) + 1.$$

Além disso, como v foi escolhido como o primeiro vértice cujo valor de λ é incorreto, todos os vértices removidos anteriormente da fila possuem distância correta. Em particular,

$$\lambda(u) = d(s, u).$$

Logo,

$$\lambda(v) > d(s, v) = d(s, u) + 1 = \lambda(u) + 1,$$

ou seja,

$$\lambda(v) > \lambda(u) + 1.$$

Entretanto, quando o vértice u é removido da fila, exatamente uma das seguintes situações ocorre para seu vizinho v .

1. v é branco (WHITE).

Nesse caso, a BFS descobre v pela primeira vez e executa

$$\lambda(v) = \lambda(u) + 1,$$

contradizendo a desigualdade anterior.

2. v é preto (BLACK).

Se v já foi finalizado, então ele necessariamente saiu da fila antes de u . Como a BFS visita os vértices em ordem crescente de distância, segue que

$$\lambda(v) \leq \lambda(u),$$

o que novamente contradiz

$$\lambda(v) > \lambda(u) + 1.$$

3. v é cinza (GRAY).

Nesse caso, v já havia sido descoberto anteriormente por algum vértice w que deixou a fila antes de u . Assim,

$$\lambda(w) \leq \lambda(u),$$

e, pela própria BFS,

$$\lambda(v) = \lambda(w) + 1.$$

Portanto,

$$\lambda(v) = \lambda(w) + 1 \leq \lambda(u) + 1,$$

novamente contradizendo

$$\lambda(v) > \lambda(u) + 1.$$

Em todos os casos chegamos a uma contradição. Logo, não existe vértice para o qual

$$\lambda(v) > d(s, v).$$

Como já sabemos que

$$\lambda(v) \geq d(s, v),$$

concluimos necessariamente que

$$\boxed{\lambda(v) = d(s, v), \quad \forall v \in V.}$$

Assim, a BFS calcula corretamente a distância mínima da raiz para todos os vértices alcançáveis do grafo. $\square$

8.5 Busca em Profundidade (Depth-First Search – DFS)

A Busca em Profundidade (*Depth-First Search* – DFS) é um dos algoritmos fundamentais para exploração de grafos. Sua estratégia consiste em explorar o grafo da forma mais profunda possível antes de retroceder. Em outras palavras, sempre que um vértice é descoberto, o algoritmo tenta visitar imediatamente um de seus vizinhos ainda não explorados, repetindo esse processo recursivamente até atingir um vértice sem vizinhos não visitados. Somente então ocorre o retrocesso (*backtracking*) para continuar a exploração. Essa estratégia é análoga à exploração de um labirinto: ao encontrar uma bifurcação, escolhe-se um caminho e segue-se por ele até não haver mais possibilidades, retornando apenas quando necessário.

8.5.1 Variáveis utilizadas

Durante a execução do algoritmo, cada vértice v mantém as seguintes informações:

- $v.d$ (*discovery time*): instante em que o vértice é descoberto pela primeira vez;

- $v.f$ (*finishing time*): instante em que o processamento do vértice é concluído, isto é, após todos os seus vizinhos terem sido explorados;
- $v.color$: estado atual do vértice, podendo assumir três valores:
 - **WHITE**: vértice ainda não descoberto;
 - **GRAY**: vértice descoberto, porém ainda em processamento;
 - **BLACK**: vértice completamente processado.
- $\pi(v)$: predecessor de v na árvore de busca, ou seja, o vértice a partir do qual v foi descoberto.

Além dessas variáveis, utiliza-se um contador global denominado `time`, responsável por registrar os tempos de descoberta e finalização dos vértices.

8.5.2 Implementação recursiva

Conceitualmente, a DFS utiliza uma pilha (*Last-In First-Out* – LIFO) para controlar a exploração dos vértices. Entretanto, na prática, essa pilha pode ser substituída pela própria pilha de chamadas da linguagem de programação por meio da recursão. Essa abordagem torna a implementação mais simples e elegante, dispensando a criação explícita de uma estrutura de pilha. Como desvantagem, deve-se observar que, em grafos muito grandes ou muito profundos, a implementação recursiva pode provocar estouro da pilha de execução (*stack overflow*). Nesses casos, uma implementação iterativa utilizando uma pilha explícita é mais indicada.

```
DFS(G) {
    for each  $u \in V$  {
         $u.color = WHITE$ 
         $\pi(u) = NIL$ 
    }
    time = 0
    for each  $u \in V$  {
        if  $u.color == WHITE$ 
            DFS_visit(G, u)
    }
}

DFS_visit(G, u) {
    time = time + 1
     $u.d = time$ 
     $u.color = GRAY$ 
    for each  $v \in N(u)$  {
        if  $v.color == WHITE$  {
```

```

         $\pi(v) = u$ 
        DFS_visit( $G, v$ )
    }
}
u.color = BLACK
time = time + 1
u.f = time
}

```

Também é possível implementar a Busca em Profundidade de forma iterativa. Nesse caso, basta substituir a recursão por uma pilha explícita. Observe que essa implementação é bastante semelhante ao algoritmo BFS. A principal diferença está na estrutura de dados utilizada para armazenar os vértices descobertos:

- a BFS utiliza uma **fila (FIFO)**, explorando os vértices por níveis;
- a DFS utiliza uma **pilha (LIFO)**, aprofundando a exploração sempre que possível.

Essa pequena modificação altera completamente a ordem de visita dos vértices e, consequentemente, as propriedades da árvore gerada.

Assim como a Busca em Largura, a DFS recebe como entrada um grafo não ponderado G e produz como saída uma árvore de busca, denominada **árvore DFS (DFS tree)**. Entretanto, diferentemente da BFS, a árvore produzida pela DFS **não preserva os menores caminhos** entre a raiz e os demais vértices. Sua principal característica é refletir a ordem em que os vértices são descobertos durante a exploração em profundidade.

Os tempos de descoberta (d) e de finalização (f) associados a cada vértice desempenham um papel fundamental em diversos algoritmos sobre grafos, como detecção de ciclos, ordenação topológica, identificação de componentes fortemente conexas, cálculo de pontos de articulação e pontes, entre várias outras aplicações que serão estudadas a seguir. O exemplo abaixo ilustra o trace completo do algoritmo DFS.

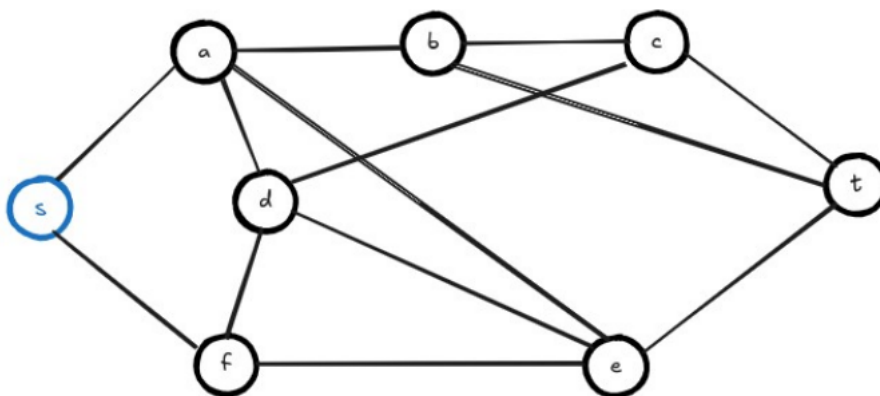


Figura 77: Desejamos encontrar a árvore de busca em profundidade com raiz no vértice s .

Tabela 8: Trace da execução do algoritmo DFS a partir do vértice s .

u	$u.color$	$u.d$	$V' = \{v \in N(u) \mid v.color = \text{WHITE}\}$	Atualização de $\pi(v)$	$u.f$
s	G, B	1	$\{a, f\}$	$\pi(a) = s, \pi(f) = s$	16
a	G, B	2	$\{b, d, e\}$	$\pi(b) = a, \pi(d) = a, \pi(e) = a$	15
b	G, B	3	$\{c, t\}$	$\pi(c) = b, \pi(t) = b$	14
c	G, B	4	$\{d, t\}$	$\pi(d) = c, \pi(t) = c$	13
d	G, B	5	$\{e, f\}$	$\pi(e) = d, \pi(f) = d$	12
e	G, B	6	$\{f, t\}$	$\pi(f) = e, \pi(t) = e$	11
f	G, B	7	$\emptyset$	—	8
t	G, B	9	$\emptyset$	—	10

8.5.3 Complexidade da Busca em Profundidade

Assim como a Busca em Largura (BFS), o algoritmo de Busca em Profundidade (DFS) possui complexidade linear em relação ao tamanho do grafo quando este é representado por listas de adjacências.

Teorema 12. *Seja $G = (V, E)$ um grafo representado por listas de adjacências. A versão recursiva do algoritmo DFS possui complexidade de tempo*

$$O(n + m),$$

onde $n = |V|$ é o número de vértices e $m = |E|$ é o número de arestas.

Demonstração. A análise pode ser realizada examinando separadamente os principais componentes do algoritmo. Inicialmente, o algoritmo percorre todos os vértices para inicializar seus atributos (`color`, `parent`, etc.). Esse procedimento requer

$$O(n)$$

operações. Em seguida, o laço principal percorre novamente todos os vértices para verificar quais ainda permanecem não descobertos (`WHITE`), iniciando uma nova chamada de `DFS_Visit` sempre que necessário. Esse segundo percurso também possui custo

$$O(n).$$

Resta analisar o custo das chamadas recursivas da função `DFS_Visit`. Observe que cada vértice é descoberto exatamente uma única vez. Após sua descoberta, sua cor muda de `WHITE` para `GRAY`, impedindo novas chamadas recursivas sobre esse mesmo vértice. Portanto, existem exatamente n chamadas da função `DFS_Visit`. Durante a execução de `DFS_Visit`, o algoritmo percorre integralmente a lista de adjacências do vértice corrente. Como cada lista é percorrida apenas uma única

vez, o número total de vizinhos examinados é

$$\sum_{v \in V} d(v),$$

onde $d(v)$ denota o grau do vértice v . Pelo Lema do Aperto de Mãos (*Handshaking Lemma*), sabemos que

$$\sum_{v \in V} d(v) = 2m$$

para grafos não direcionados. Assim, o custo total gasto na exploração das listas de adjacências é

$$O(2m) = O(m).$$

Somando todos os custos, obtemos

$$O(n) + O(n) + O(m) = O(2n + m).$$

Como fatores constantes são desprezados na notação assintótica, concluímos que

$$O(2n + m) = O(n + m).$$

Logo, a complexidade de tempo da Busca em Profundidade é linear no tamanho do grafo. $\square$

8.5.4 Propriedades da DFS-tree e Vértices de Corte

Uma das principais vantagens da Busca em Profundidade (DFS) é que ela produz muito mais informações sobre a estrutura do grafo do que a Busca em Largura (BFS). Além da árvore de busca propriamente dita, a DFS registra os instantes de descoberta e finalização de cada vértice, permitindo identificar ciclos, pontes, componentes biconexas e vértices de corte. A árvore produzida pela DFS, denominada *DFS-tree*, possui diversas propriedades importantes que serão exploradas ao longo deste capítulo.

Durante a execução da DFS, cada aresta do grafo pode ser classificada conforme sua função na exploração. Em grafos não direcionados existem apenas dois tipos de arestas:

- **Arestas da árvore (tree edges):** são as arestas utilizadas para descobrir um novo vértice. Essas arestas compõem a DFS-tree.
- **Arestas de retorno (back edges):** conectam um vértice a algum de seus ancestrais na DFS-tree. A existência de uma aresta de retorno indica a presença de um ciclo no grafo.

A ideia de ancestral mais alto

Considere um vértice u da DFS-tree.

Durante a exploração da subárvore enraizada em u , pode ocorrer que algum descendente possua uma aresta de retorno para um ancestral de u . Quando isso acontece, existe um caminho alternativo ligando essa subárvore ao restante do grafo. Por outro lado, se nenhum vértice da subárvore consegue alcançar um ancestral de u , então toda a comunicação dessa subárvore com o restante do grafo depende exclusivamente do vértice u . Essa observação leva ao conceito de *valor low*. Para cada vértice u , define-se

$$low(u),$$

como o menor tempo de descoberta que pode ser alcançado a partir de u , utilizando:

- zero ou mais arestas da DFS-tree; e
- no máximo uma aresta de retorno.

Equivalentemente,

$$low(u) = \min \begin{cases} d(u), \\ d(w), \text{ para cada aresta de retorno } (x, w), \\ low(v), \text{ para cada filho } v \text{ de } u. \end{cases}$$

O valor $low(u)$ indica o ancestral mais antigo da DFS-tree que pode ser alcançado pela subárvore enraizada em u sem utilizar o pai de u . Quanto menor for $low(u)$, maior é a conectividade dessa subárvore com o restante do grafo.

Caracterização dos vértices de corte

Um vértice é denominado **vértice de corte** (ou *articulation point*) quando sua remoção aumenta o número de componentes conexas do grafo. A DFS fornece uma maneira extremamente eficiente de identificar esses vértices.

Teorema 13. *Seja u um vértice da DFS-tree. Então:*

1. *A raiz da DFS-tree é um vértice de corte se, e somente se, possui dois ou mais filhos.*
2. *Um vértice interno u é vértice de corte se existe algum filho v tal que*

$$low(v) \geq d(u).$$

Demonstração. Considere inicialmente um vértice interno u . Se existe um filho v tal que

$$low(v) \geq d(u),$$

isso significa que nenhum vértice da subárvore de v consegue alcançar um ancestral de u através de uma aresta de retorno. Consequentemente, toda comunicação entre essa subárvore e o restante do

grafo ocorre obrigatoriamente por meio de u . Logo, a remoção de u desconecta a subárvore de v , tornando u um vértice de corte. Reciprocamente, suponha que

$$low(v) < d(u)$$

para todo filho v de u . Nesse caso, cada subárvore possui pelo menos uma aresta de retorno que alcança algum ancestral de u . Assim, mesmo removendo u , continua existindo um caminho alternativo ligando cada subárvore ao restante do grafo. Portanto, u não é vértice de corte.

Finalmente, considere a raiz da DFS-tree. Como ela não possui ancestral, o único modo de sua remoção desconectar o grafo é possuir duas ou mais subárvores independentes. Logo, a raiz é vértice de corte exatamente quando possui pelo menos dois filhos. $\square$

Por exemplo, considere o grafo a seguir.

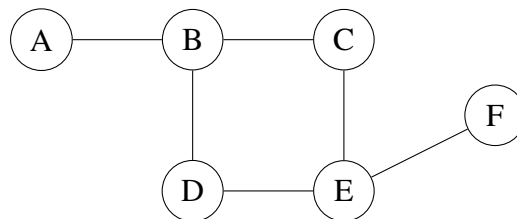


Figura 78: Grafo utilizado no exemplo de vértices de corte.

Após a aplicação da DFS, temos a seguinte árvore:

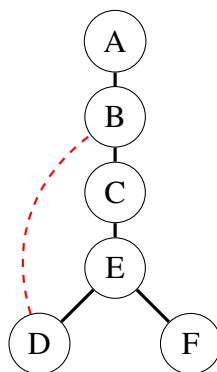


Figura 79: DFS-tree (arestas sólidas) e aresta de retorno (tracejada).

A rotulação com tempos de entrada e os valores de $low(v)$ para cada vértice é mostrada na tabela a seguir. Os valores da coluna $low(v)$ representam o menor tempo de descoberta (*discovery time*) que pode ser alcançado a partir do vértice v , permitindo percorrer zero ou mais arestas da DFS-tree seguidas de, no máximo, uma aresta de retorno (*back edge*). No exemplo da Tabela 9, o vértice A possui $low(A) = 1$, pois é a raiz da árvore e não existe caminho que alcance um ancestral anterior a ele. O vértice B também possui $low(B) = 2$, uma vez que não há arestas de retorno que permitam atingir um ancestral de B . Já os vértices C , D e E possuem valor $low = 2$ porque existe uma aresta de retorno ligando D a B . Essa aresta permite que, a partir de D , seja alcançado o vértice B , cujo

tempo de descoberta é $d(B) = 2$. Como o valor de low é propagado durante o retorno da recursão, tanto E quanto C herdam esse mesmo menor valor. Finalmente, o vértice F possui $low(F) = 6$, pois é uma folha da DFS-tree e não possui nenhuma aresta de retorno para um ancestral. Dessa forma, o menor vértice que pode ser alcançado a partir de F é ele próprio, resultando em $low(F) = d(F) = 6$.

Tabela 9: Tempos de descoberta e valores low dos vértices da DFS-tree.

Vértice	$d(v)$	$low(v)$
A	1	1
B	2	2
C	3	2
D	5	2
E	4	2
F	6	6

Aplicando o teorema dos vértices de corte, temos a seguinte conclusão:

- O vértice A é a raiz e possui apenas um único filho: não é vértice de corte.
- O vértice B possui $d(B) = 2$ e note existe um filho na DFS-tree (C) que possui valor de low igual a 2. Isso define B como vértice de corte.
- O vértice C possui $d(C) = 3$ e note seu único filho na DFS-tree E possui valor de low igual a 2. Isso não define C como vértice de corte.
- O vértice D possui $d(D) = 5$ e não possui filho na DFS-tree. Portanto, D não é vértice de corte.
- O vértice E possui $d(E) = 4$ e e note que seu único filho na DFS-tree F possui valor de low igual a 6. Portanto, E é vértice de corte.
- O vértice F possui $d(F) = 6$ e não possui filho na DFS-tree. Portanto, F não é vértice de corte.

Sendo assim, no grafo G existem apenas dois vértices de corte: B e E.

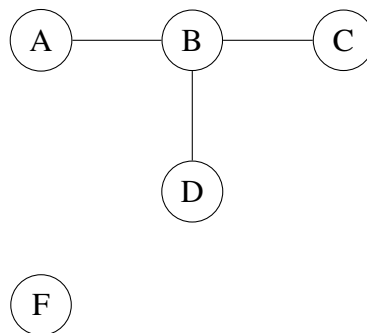


Figura 80: Grafo após a remoção do vértice de corte E.

8.5.5 Detecção de Ciclos com Busca em Profundidade (DFS)

A detecção de ciclos é um dos problemas fundamentais em Teoria dos Grafos e possui inúmeras aplicações práticas. Em compiladores, por exemplo, é utilizada para verificar dependências circulares entre módulos. Em sistemas operacionais, permite detectar situações de *deadlock*. Em bancos de dados, auxilia na identificação de dependências cíclicas entre tabelas. Em gerenciamento de projetos, é empregada para verificar a consistência de grafos de precedência. Além disso, diversos algoritmos clássicos, como ordenação topológica e algoritmos para componentes fortemente conexas, dependem diretamente da detecção de ciclos.

A Busca em Profundidade (Depth-First Search – DFS) é a técnica mais utilizada para esse problema. Sua principal vantagem é que, durante a exploração recursiva dos vértices, ela mantém implicitamente o caminho atualmente percorrido. Assim, qualquer aresta que retorne para um vértice já pertencente a esse caminho indica imediatamente a existência de um ciclo.

Ideia básica Considere inicialmente um grafo não direcionado. Durante a execução da DFS, cada vértice recebe um predecessor $\pi(v)$, formando uma árvore DFS. Ao explorar um vértice u , três situações podem ocorrer para um vizinho v :

- v ainda não foi visitado (WHITE): a busca continua normalmente a partir de v ;
- v é o predecessor imediato de u : essa aresta corresponde ao retorno natural da árvore DFS e deve ser ignorada;
- v já foi visitado e não é o predecessor de u : existe um caminho da árvore ligando v até u , juntamente com a aresta (u, v) , formando um ciclo.

Portanto, em grafos não direcionados, basta encontrar uma aresta ligando o vértice atual a um vértice previamente visitado que não seja seu pai na árvore DFS.

Adaptação do algoritmo A adaptação do algoritmo DFS é extremamente simples. Basta verificar, durante a exploração dos vizinhos, se um vértice já descoberto não corresponde ao predecessor do vértice atual.

```
CycleDFS(G) {  
    for each vertex  $u \in V$  {  
        u.color = WHITE  
         $\pi(u) = NIL$   
    }  
    for each vertex  $u \in V$  {  
        if u.color == WHITE {  
            if DFS_Cycle(u)  
                return TRUE  
        }  
    }  
}
```

```

    }
}
return FALSE
}

DFS_Cycle(u) {
    u.color = GRAY
    for each  $v \in N(u)$  {
        if v.color == WHITE
             $\pi(v) = u$ 
            if DFS_Cycle(v)
                return TRUE
        }
    else {
        if  $v \neq \pi(u)$  then
            return TRUE
        }
    }
    u.color = BLACK
    return FALSE
}

```

O algoritmo retorna imediatamente quando encontra o primeiro ciclo. Caso toda a busca termine sem detectar nenhuma dessas situações, o grafo é acíclico.

Por que o algoritmo funciona? Suponha que durante a exploração do vértice u seja encontrada uma aresta (u, v) em que v já foi descoberto e $v \neq \pi(u)$. Como v já pertence à árvore DFS, existe um caminho único entre v e u formado pelas arestas da própria árvore. Ao acrescentarmos a aresta adicional (u, v) , obtemos um caminho fechado, caracterizando um ciclo.

Por outro lado, se o único vizinho já visitado for exatamente o predecessor de u , então essa aresta apenas representa o retorno natural utilizado pela DFS e não caracteriza um ciclo. Assim, o algoritmo detecta corretamente a existência de ciclos em grafos não direcionados.

8.6 Ordenação Topológica

A ordenação topológica é um dos problemas mais importantes envolvendo grafos direcionados. Seu objetivo consiste em encontrar uma ordenação linear dos vértices de um grafo direcionado acíclico (*Directed Acyclic Graph* – DAG), de modo que toda aresta direcionada (u, v) satisfaça a seguinte propriedade:

u aparece antes de v .

Em outras palavras, se existe uma dependência de u para v , então u deve necessariamente aparecer antes de v na ordenação. Esse problema possui inúmeras aplicações práticas. Alguns exemplos incluem:

- planejamento de disciplinas em um curso, respeitando os pré-requisitos;
- escalonamento de tarefas em sistemas operacionais;
- gerenciamento de projetos (PERT/CPM);
- compilação de programas e resolução de dependências entre módulos;
- construção de pacotes em gerenciadores como `apt`, `yum` e `pip`.

Todos esses problemas possuem uma característica em comum: algumas tarefas somente podem ser executadas após a conclusão de outras.

8.6.1 Quando a ordenação topológica existe?

Nem todo grafo direcionado admite uma ordenação topológica. Considere dois vértices A e B tais que existam as arestas

$$A \rightarrow B \quad \text{e} \quad B \rightarrow A.$$

Nesse caso, A deveria aparecer antes de B , mas simultaneamente B deveria aparecer antes de A , o que é impossível. Generalizando essa ideia, temos o seguinte resultado.

Teorema 14. *Um grafo direcionado admite uma ordenação topológica se, e somente se, ele é acíclico.*

Assim, antes de realizar a ordenação topológica, é conveniente verificar se o grafo possui ciclos, conforme o algoritmo de detecção de ciclos apresentado anteriormente.

8.6.2 A ideia da solução usando DFS

A Busca em Profundidade fornece naturalmente uma solução elegante para esse problema. Durante a execução da DFS, cada vértice recebe dois tempos:

$$v.d$$

(tempo de descoberta)

e

$$v.f$$

(tempo de finalização).

O tempo de finalização corresponde ao instante em que toda a subárvore enraizada em v já foi completamente explorada. Considere uma aresta direcionada

$$u \rightarrow v.$$

Como a DFS visita todos os descendentes antes de finalizar um vértice, necessariamente temos

$$f(u) > f(v).$$

Ou seja, o predecessor termina sua exploração somente após todos os vértices alcançáveis por ele terem sido completamente processados. Essa propriedade sugere imediatamente uma solução: ordenar os vértices em ordem decrescente dos tempos de finalização.

Durante a DFS, sempre que um vértice é finalizado (BLACK), ele é inserido no início de uma lista. Ao término da busca, essa lista corresponde exatamente à ordenação topológica. Uma implementação simples é apresentada a seguir.

```
TopologicalSort(G) {
  for each vertex u in V {
    u.color = WHITE
    u.parent = NIL
  }
  L = empty list
  for each vertex u in V {
    if u.color == WHITE
      DFS_Topo(u)
  }
  return L
}

DFS_Topo(u) {
  u.color = GRAY
  for each v in N(u) {
    if v.color == WHITE then {
      v.parent = u
      DFS_Topo(v)
    }
  }
  u.color = BLACK
  insert_front(L, u)
}
```

Observe que a única modificação em relação ao algoritmo DFS tradicional consiste em inserir o vértice no início da lista quando sua exploração termina. Alternativamente, pode-se armazenar todos os tempos de finalização e, ao final da busca, ordenar os vértices em ordem decrescente de $f(v)$.

Exemplo

Considere o seguinte conjunto de dependências:

$$A \rightarrow C, \quad B \rightarrow C, \quad C \rightarrow D, \quad C \rightarrow E, \quad D \rightarrow F, \quad E \rightarrow F.$$

Uma possível ordenação topológica é

$$A, B, C, D, E, F.$$

Outra ordenação igualmente válida é

$$B, A, C, E, D, F.$$

Observe que ambas respeitam todas as dependências do grafo. Esse exemplo mostra que uma ordenação topológica, em geral, não é única.

Uma aplicação da ordenação topológica seria a seguinte: imagine que você queira “ensinar” um robô humanoide a se vestir. Claramente, existe uma ordem natural que precisa ser seguida durante o processo. Por exemplo, não podemos calçar o sapato e depois colocar as meias. Suponha que o grafo acíclico direcionado que representa as dependências seja dado conforme a figura a seguir.

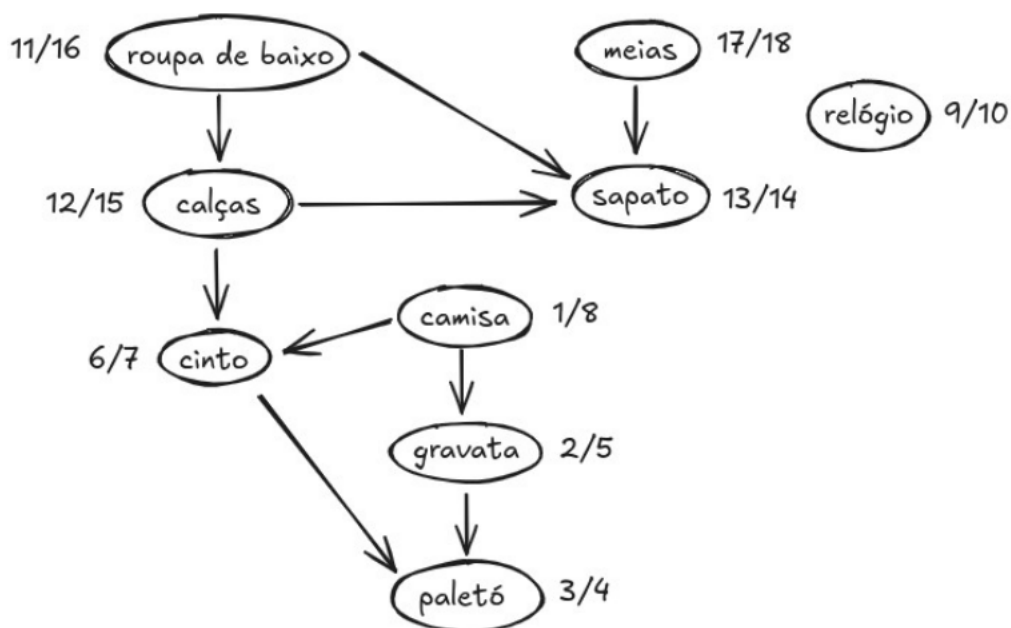


Figura 81: Diagrama de dependência entre as peças de roupa para o robô humanoide se vestir com os tempos de entrada e saída de cada vértice.

Ordenando os vértices pelos tempos de saída, temos:

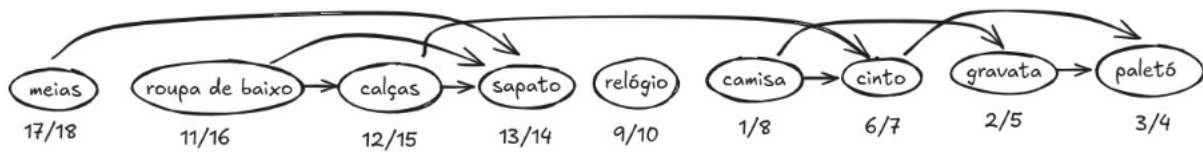


Figura 82: Resultado da ordenação topológica do problema do robô humanóide.

Note que ao desenhar o grafo com os vértices em ordem topológica, todas as arestas apontam para a direita. A seguir, demonstramos um resultado fundamental para a corretude do algoritmo `Topological_Sort`.

Teorema 15. *Um grafo direcionado $G = (V, E)$ é acíclico se, e somente se, uma execução da Busca em Profundidade (DFS) em G não gera nenhuma back edge (aresta de retorno).*

Demonstração. A demonstração será feita mostrando as duas implicações.

($\Rightarrow$) Se G é acíclico, então a DFS não gera back edges.

Faremos a prova por contraposição:

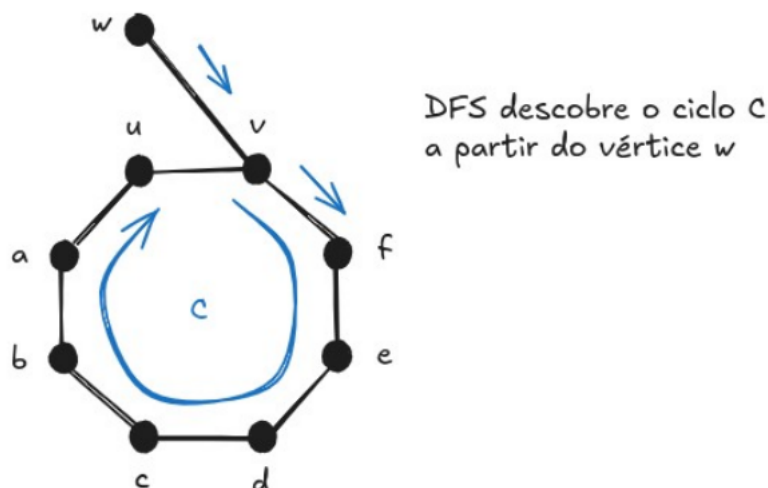
DFS gera uma back edge $\Rightarrow G$ contém um ciclo.

Suponha que, durante a execução da DFS, seja encontrada uma back edge (u, v) . Por definição, uma back edge conecta um vértice u a um de seus ancestrais v na árvore DFS. Como v é ancestral de u , existe necessariamente um caminho na árvore DFS de v até u , formado apenas por arestas da árvore:

$$v \rightsquigarrow u.$$

Além disso, a aresta (u, v) fecha esse caminho, produzindo o ciclo

$$v \rightsquigarrow u \rightarrow v.$$



Logo, G contém um ciclo, o que conclui a contraposição. Portanto, se G é acíclico, nenhuma *back edge* pode ser produzida pela DFS.

($\Leftarrow$) Se a DFS não gera *back edges*, então G é acíclico.

Também utilizaremos a contraposição:

$$G \text{ contém um ciclo} \implies \text{a DFS gera uma } \textit{back edge}.$$

Suponha que G contenha um ciclo

$$C = (v_1, v_2, \dots, v_k, v_1).$$

Considere v como o primeiro vértice desse ciclo descoberto pela DFS. Seja u o vértice imediatamente anterior a v no ciclo, isto é, existe a aresta direcionada

$$(u, v) \in E.$$

No instante em que v é descoberto, todos os demais vértices do ciclo ainda possuem cor `WHITE`, pois v foi o primeiro vértice do ciclo a ser visitado. Consequentemente, existe um caminho composto apenas por vértices brancos de v até u ao longo do ciclo. Pelas propriedades da DFS, todos esses vértices serão descobertos recursivamente antes que v seja finalizado. Em particular, u tornar-se-á um descendente de v na árvore DFS.

Como a aresta (u, v) não pertence à árvore (ela fecha o ciclo), ela conecta um descendente ao seu ancestral, caracterizando exatamente uma *back edge*. Logo, toda vez que um grafo direcionado contém um ciclo, a DFS necessariamente produz uma *back edge*.

Como ambas as implicações foram demonstradas, concluímos que um grafo direcionado é acíclico se, e somente se, a Busca em Profundidade não produz nenhuma *back edge*. $\square$

8.6.3 Por que o algoritmo funciona?

O resultado apresentado a seguir estabelece a corretude do algoritmo de ordenação topológica baseado em Busca em Profundidade (DFS). Sua importância decorre do fato de mostrar que basta ordenar os vértices em ordem decrescente dos tempos de finalização para obter uma ordenação topológica válida.

Esse teorema está diretamente relacionado ao resultado anterior, que afirma que um grafo direcionado é acíclico se, e somente se, a DFS não produz *back edges*. A ausência de *back edges* garante que toda aresta do grafo conecta um vértice que será finalizado *depois* de seu sucessor, propriedade fundamental para a corretude da ordenação topológica.

Teorema 16. *O algoritmo `Topological_Sort` produz uma ordenação topológica correta para qualquer grafo direcionado acíclico (DAG).*

Demonstração. Considere a execução do algoritmo DFS sobre um DAG $G = (V, E)$, obtendo os tempos de finalização $f(v)$ para todo vértice $v \in V$. O algoritmo `Topological_Sort` consiste

simplesmente em ordenar os vértices em ordem decrescente desses tempos. Portanto, basta demonstrar que, para toda aresta direcionada

$$(u, v) \in E,$$

vale

$$f(u) > f(v).$$

Assim, o vértice u sempre aparecerá antes de v na ordenação produzida. Considere uma aresta arbitrária (u, v) examinada durante a execução da DFS. No momento em que essa aresta é explorada, existem apenas três possibilidades para a cor do vértice v .

Caso 1: v é WHITE.

Nesse caso, v ainda não foi descoberto. O algoritmo realiza uma chamada recursiva para visitar v , fazendo com que ele se torne descendente de u na árvore DFS. Como um descendente sempre é finalizado antes de seu ancestral, temos

$$f(v) < f(u).$$

Caso 2: v é GRAY.

Essa situação é impossível. De fato, se v está cinza, então ele é um ancestral de u na pilha de recursão. Logo, a aresta (u, v) seria uma *back edge*. Entretanto, pelo teorema anterior, um grafo direcionado acíclico não possui *back edges*. Surge, portanto, uma contradição. Assim, esse caso nunca ocorre em um DAG.

Caso 3: v é BLACK.

Nesse caso, v já foi completamente explorado e seu tempo de finalização já foi determinado. Como a busca ainda está processando o vértice u , seu tempo de finalização ainda não foi atribuído. Consequentemente,

$$f(v) < f(u).$$

Em todos os casos possíveis conclui-se que, para qualquer aresta $(u, v) \in E$,

$$f(u) > f(v).$$

Logo, ao ordenar os vértices em ordem decrescente dos tempos de finalização, todo predecessor aparece antes de seus sucessores, satisfazendo exatamente a definição de ordenação topológica. Portanto, o algoritmo `Topological_Sort` produz uma ordenação topológica correta para qualquer grafo direcionado acíclico. $\square$

Complexidade

A ordenação topológica herda praticamente toda a complexidade da Busca em Profundidade. A DFS visita cada vértice exatamente uma vez e examina cada aresta exatamente uma vez. Além disso, a inserção de um vértice no início da lista pode ser realizada em tempo constante. Assim, o custo total é

$$T(n, m) = O(n + m),$$

onde

$$n = |V| \quad \text{e} \quad m = |E|.$$

Portanto, a ordenação topológica pode ser obtida em tempo linear no tamanho do grafo, sendo um dos algoritmos mais eficientes para problemas de planejamento e escalonamento de tarefas.

Observações

É importante destacar que:

- a ordenação topológica existe apenas para grafos direcionados acíclicos (DAGs);
- um DAG pode possuir diversas ordenações topológicas válidas;
- se durante a DFS for detectada uma *back edge*, então o grafo contém um ciclo e nenhuma ordenação topológica pode existir.

A estreita relação entre detecção de ciclos e ordenação topológica faz da Busca em Profundidade uma das ferramentas mais versáteis para o processamento de grafos direcionados ou não.

8.7 Considerações Finais

Neste capítulo foram apresentados os principais fundamentos da busca em grafos, uma das técnicas mais importantes da Ciência da Computação. Embora os algoritmos estudados sejam conceitualmente simples, eles constituem a base para uma grande variedade de algoritmos clássicos utilizados em áreas como otimização, inteligência artificial, visão computacional, mineração de dados, bioinformática, redes de computadores e engenharia de software.

Inicialmente, estudamos a Busca em Largura (*Breadth-First Search* – BFS), cuja principal característica consiste em explorar o grafo nível a nível. Demonstramos que a árvore produzida pelo algoritmo preserva as menores distâncias entre o vértice de origem e todos os demais vértices alcançáveis, tornando a BFS a ferramenta natural para problemas de caminhos mínimos em grafos não ponderados.

Em seguida, apresentamos a Busca em Profundidade (*Depth-First Search* – DFS), que adota uma estratégia de exploração completamente distinta, aprofundando-se o máximo possível antes de retroceder. Apesar de também possuir complexidade linear $O(n + m)$, a DFS revela uma quantidade muito maior de informações estruturais sobre o grafo por meio de sua árvore de busca e dos tempos de descoberta e finalização dos vértices.

A partir dessas propriedades, mostramos como a DFS pode ser adaptada para resolver diversos problemas clássicos. Entre eles destacam-se a identificação de vértices de corte, a detecção de ciclos em grafos direcionados e não direcionados e a ordenação topológica de grafos direcionados acíclicos. Em todos esses casos, observa-se que pequenas adaptações sobre um mesmo algoritmo permitem resolver problemas aparentemente bastante distintos.

Outro aspecto importante discutido neste capítulo foi a análise de complexidade. Tanto a BFS quanto a DFS executam em tempo linear em relação ao tamanho do grafo, isto é,

$$O(n + m),$$

onde $n = |V|$ representa o número de vértices e $m = |E|$ o número de arestas. Essa eficiência explica por que esses algoritmos são amplamente empregados como blocos fundamentais em sistemas computacionais que processam grafos de grande porte.

Os conceitos apresentados neste capítulo servirão de base para diversos tópicos mais avançados que serão estudados posteriormente. Algoritmos para componentes fortemente conexas, árvores geradoras mínimas, caminhos mínimos, fluxos em redes, emparelhamentos, análise de redes complexas e diversos métodos modernos de mineração de grafos utilizam diretamente as ideias desenvolvidas pela BFS e pela DFS.

Em síntese, mais do que algoritmos específicos, a busca em grafos fornece uma maneira sistemática de explorar estruturas relacionais complexas. Compreender profundamente suas propriedades permite resolver uma enorme variedade de problemas computacionais de forma elegante e eficiente, tornando a BFS e a DFS ferramentas indispensáveis no repertório de qualquer cientista ou engenheiro da computação.

Tabela 10: Comparação entre Busca em Largura (BFS) e Busca em Profundidade (DFS).

Característica	BFS (Breadth-First Search)	DFS (Depth-First Search)
Estratégia de exploração	Visita os vértices por níveis, explorando primeiro todos os vizinhos de mesma distância da raiz.	Aprofunda a exploração o máximo possível antes de retroceder (<i>backtracking</i>).
Estrutura de dados	Fila (FIFO).	Pilha (LIFO) ou recursão.
Árvore produzida	Árvore BFS.	Árvore DFS.
Menores caminhos	Sempre encontra os menores caminhos em grafos não ponderados.	Não garante menores caminhos.
Distância da raiz	Calcula corretamente a distância geodésica $\lambda(v) = d(s, v)$.	Não calcula distâncias mínimas.
Ordem de visita	Exploração em largura.	Exploração em profundidade.
Retrocesso	Não realiza retrocesso explícito.	Retrocede sempre que não existem vizinhos não visitados.
Tempos de descoberta e finalização	Não utiliza tempos de descoberta/finalização.	Armazena tempos de descoberta (d) e finalização (f).
Complexidade	$O(n + m)$	$O(n + m)$
Consumo de memória	Pode armazenar muitos vértices simultaneamente na fila (largura do grafo).	Armazena apenas o caminho atual de exploração (profundidade do grafo).
Aplicações típicas	Menores caminhos em grafos não ponderados, níveis de distância, redes sociais, difusão de informação, navegação.	Deteção de ciclos, ordenação topológica, componentes fortemente conexas, pontes, pontos de articulação e exploração exaustiva.
Ideia intuitiva	"Explore todos os vizinhos antes de avançar."	"Avance o máximo possível antes de voltar."

9 Caminhos mínimos em grafos ponderados

Em diversos problemas computacionais, não basta apenas determinar se existe um caminho entre dois vértices de um grafo. Frequentemente, deseja-se encontrar o *melhor* caminho segundo algum critério de otimização. Quando as arestas do grafo possuem pesos associados, o problema mais comum consiste em encontrar um caminho cuja soma dos pesos seja mínima. Esse problema é conhecido como **problema dos caminhos mínimos** (*Shortest Path Problem*).

Os caminhos mínimos aparecem naturalmente em inúmeras aplicações do mundo real. Sistemas de navegação por GPS utilizam algoritmos de caminhos mínimos para determinar rotas entre cidades. Redes de computadores empregam técnicas semelhantes para encaminhar pacotes de dados de forma eficiente. Em logística e transporte, esses algoritmos são utilizados para otimizar rotas de entrega. Em robótica, permitem planejar trajetórias entre posições distintas minimizando distância, tempo ou consumo de energia.

Formalmente, seja $G = (V, E)$ um grafo ponderado, em que cada aresta $(u, v) \in E$ possui um peso associado $w(u, v)$. O peso de um caminho é definido como a soma dos pesos de todas as arestas que o compõem. O objetivo é determinar, para um vértice de origem s , um caminho de custo mínimo até cada vértice alcançável do grafo.

Neste capítulo estudaremos inicialmente o algoritmo de **Dijkstra**, um dos algoritmos mais importantes da Ciência da Computação. Sua principal ideia consiste em expandir progressivamente um conjunto de vértices cujas distâncias mínimas já foram determinadas, escolhendo sempre o vértice mais promissor para ser processado em seguida. Essa estratégia gulosa (*greedy*) permite obter algoritmos extremamente eficientes, amplamente utilizados em aplicações práticas.

Além da descrição detalhada do algoritmo, analisaremos sua complexidade computacional e apresentaremos uma demonstração formal de sua corretude. Veremos que a eficiência do método depende diretamente da estrutura de dados utilizada para implementar a fila de prioridades responsável pela seleção dos vértices.

Entretanto, apesar de sua importância e eficiência, o algoritmo de Dijkstra possui uma limitação fundamental: sua corretude depende da hipótese de que todos os pesos das arestas sejam não negativos. Quando existem arestas com pesos negativos, a estratégia gulosa utilizada pelo algoritmo deixa de ser válida e as garantias teóricas sobre a optimalidade das soluções encontradas não podem mais ser asseguradas.

Para lidar com essa situação mais geral, estudaremos posteriormente o algoritmo de **Bellman-Ford**. Diferentemente de Dijkstra, esse algoritmo é capaz de encontrar caminhos mínimos mesmo na presença de pesos negativos. Além disso, ele possui uma propriedade extremamente importante: consegue detectar ciclos de peso negativo, situação na qual o problema dos caminhos mínimos deixa de possuir solução bem definida.

Ao final deste capítulo, o leitor compreenderá não apenas como os algoritmos de Dijkstra e Bellman-Ford funcionam, mas também quais hipóteses garantem sua corretude, quais são suas limitações e em quais cenários cada abordagem deve ser utilizada.

9.1 Algoritmo de Dijkstra

Considere um grafo ponderado $G = (V, E, w)$, em que a função

$$w : E \rightarrow \mathbb{R}_{\geq 0}$$

associa um peso não negativo a cada aresta do grafo. O problema consiste em determinar, para um vértice de origem s , os caminhos de menor custo até todos os demais vértices alcançáveis. Antes de apresentar o algoritmo de Dijkstra, é conveniente formalizar o conceito de caminho mínimo e introduzir a operação de *relaxamento*, utilizada por praticamente todos os algoritmos clássicos para caminhos mínimos.

9.1.1 Caminhos mínimos

Definição 9. *Seja*

$$P = (v_0, v_1, \dots, v_n)$$

um caminho em um grafo ponderado $G = (V, E, w)$. O custo (ou peso) de P é definido como a soma dos pesos de suas arestas:

$$w(P) = \sum_{i=0}^{n-1} w(v_i, v_{i+1}).$$

Um caminho P^ entre os vértices v_0 e v_n é denominado **caminho mínimo** (ou caminho ótimo) se*

$$w(P^*) \leq w(P)$$

para qualquer outro caminho P ligando v_0 a v_n .

Em outras palavras, um caminho mínimo é aquele cuja soma dos pesos das arestas é a menor possível.

9.1.2 A operação de relaxamento

A operação fundamental utilizada pelos algoritmos de caminhos mínimos é denominada **relaxamento** (*edge relaxation*). Para cada vértice v , mantemos duas informações:

- $\lambda(v)$: melhor estimativa conhecida, até o momento, para o custo do caminho entre a origem s e o vértice v ;
- $\pi(v)$: predecessor de v na árvore de caminhos mínimos construída pelo algoritmo.

Inicialmente, os valores de $\lambda(v)$ representam apenas estimativas. À medida que novas arestas são examinadas, essas estimativas tornam-se progressivamente menores, aproximando-se dos verdadeiros

custos mínimos. A ideia do relaxamento é extremamente simples. Suponha que já conhecemos um caminho de custo $\lambda(u)$ da origem até o vértice u . Se existe uma aresta (u, v) de peso $w(u, v)$, então podemos alcançar v passando por u , com custo

$$\lambda(u) + w(u, v).$$

Se esse novo custo for menor que a estimativa atual de v , então encontramos um caminho melhor e devemos atualizar as informações armazenadas. O procedimento é apresentado a seguir.

```
relax(u, v, w) {  
    if  $\lambda(v) > \lambda(u) + w(u, v)$  {  
         $\lambda(v) = \lambda(u) + w(u, v)$   
         $\pi(v) = u$   
    }  
}
```

Uma forma equivalente consiste em escrever

```
relax(u, v, w) {  
     $\lambda(v) = \min\{\lambda(v), \lambda(u) + w(u, v)\}$   
    if  $\lambda(v)$  was updated  
         $\pi(v) = u$   
}
```

Observe que a operação de relaxamento possui uma propriedade importante:

$$\lambda(v)$$

nunca aumenta. Ela permanece inalterada ou diminui, indicando que um caminho de menor custo foi encontrado.

A diferença entre os diversos algoritmos de caminhos mínimos reside justamente na estratégia utilizada para aplicar essa operação. Em particular, cada algoritmo responde de maneira diferente às seguintes perguntas:

- Quais arestas devem ser relaxadas?
- Quantas vezes cada aresta deve ser relaxada?
- Em que ordem os relaxamentos devem ocorrer?

O algoritmo de Dijkstra responde a essas perguntas utilizando uma estratégia gulosa (*greedy*). A cada iteração, o algoritmo seleciona o vértice cuja estimativa de distância é a menor entre todos aqueles que ainda não foram processados. Em seguida, relaxa todas as arestas incidentes nesse vértice. A principal hipótese por trás dessa estratégia é que, quando um vértice possui a menor estimativa

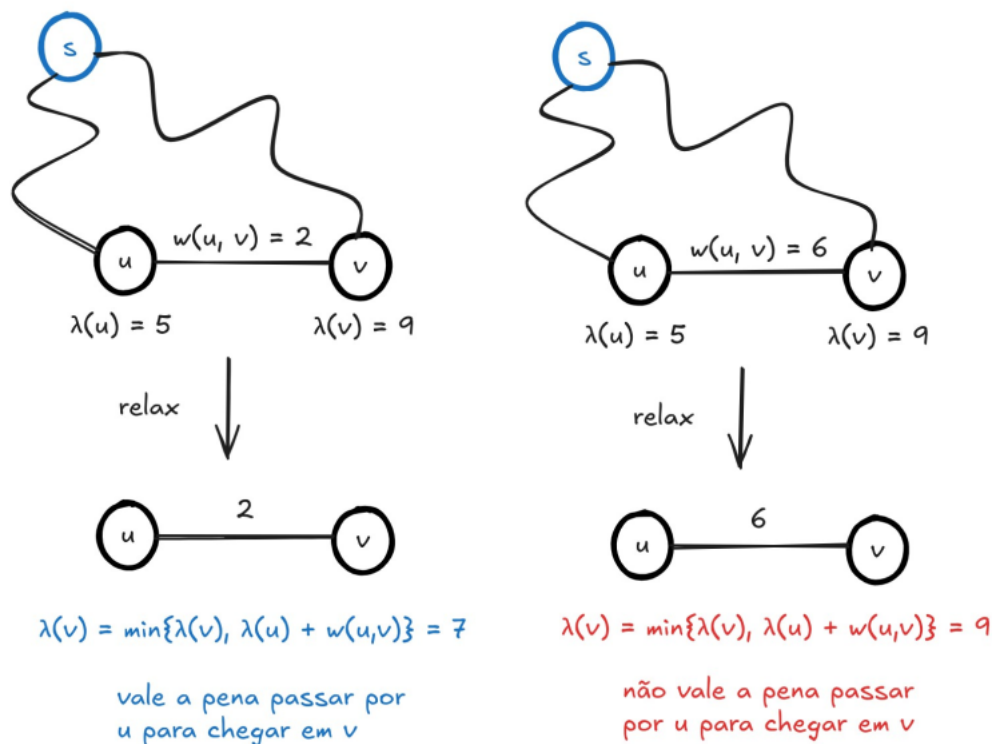


Figura 83: A operação relaxamento nunca aumenta o valor de $\lambda(v)$.

disponível, seu caminho mínimo já foi definitivamente determinado. Essa propriedade somente é válida quando todas as arestas possuem pesos não negativos.

Para implementar essa política de seleção eficiente, utiliza-se uma **fila de prioridades**. Nela, quanto menor o valor de $\lambda(v)$, maior é a prioridade do vértice. Em termos intuitivos, o algoritmo expande primeiro os ramos mais baratos da árvore de caminhos mínimos. Como todo subcaminho de um caminho mínimo também é um caminho mínimo, os caminhos curtos descobertos inicialmente servem de base para construir caminhos ótimos mais longos. Esse princípio é conhecido como **princípio da otimalidade**, um dos conceitos fundamentais da Programação Dinâmica. O algoritmo utiliza as seguintes variáveis.

- $\lambda(v)$: menor custo conhecido da origem s até v ;
- $\pi(v)$: predecessor de v na árvore de caminhos mínimos;
- Q : fila de prioridades contendo os vértices ainda não processados;
- S : conjunto dos vértices cujos caminhos mínimos já foram definitivamente determinados.

A fila de prioridades oferece três primitivas fundamentais.

- $\text{Insert}(Q, v)$: insere o vértice v em Q ;
- $\text{ExtractMin}(Q)$: remove o vértice de menor valor de $\lambda(v)$;

- `DecreaseKey(Q, v,  $\lambda(v)$ )` : atualiza a prioridade do vértice após uma redução em seu custo.

```

Dijkstra(G, w, s) {
    for each vertex  $v \in V$  {
         $\lambda(v) = \infty$ 
         $\pi(v) = NIL$ 
    }
     $\lambda(s) = 0$ 
     $S = \emptyset$ 
     $Q = \emptyset$ 
    for each vertex  $v \in V$ 
        Insert( $Q, v$ )
    while  $Q \neq \emptyset$  {
         $u = \text{ExtractMin}(Q)$ 
         $S = S \cup u$ 
        for each vertex  $v \in N(u)$  {
            if  $\lambda(v) > \lambda(u) + w(u, v)$  {
                 $\lambda(v) = \lambda(u) + w(u, v)$ 
                 $\pi(v) = u$ 
                DecreaseKey( $Q, v, \lambda(v)$ )
            }
        }
    }
}

```

Ao término da execução, os ponteiros $\pi(v)$ definem a árvore de caminhos mínimos enraizada em s , enquanto os valores $\lambda(v)$ correspondem aos custos mínimos da origem até cada vértice alcançável.

9.1.3 Dijkstra como uma generalização da BFS

Existe uma relação bastante interessante entre o algoritmo de Dijkstra e a Busca em Largura (BFS). Na BFS, todas as arestas possuem implicitamente o mesmo peso (tipicamente igual a 1). Dessa forma, basta utilizar uma fila FIFO para garantir que os vértices sejam explorados em ordem crescente da distância até a origem.

No algoritmo de Dijkstra, entretanto, as arestas podem possuir pesos distintos. Consequentemente, a simples ordem de descoberta dos vértices deixa de refletir a ordem de seus custos. Por esse motivo, a fila FIFO é substituída por uma fila de prioridades, responsável por selecionar sempre o vértice cujo caminho conhecido possui o menor custo.

Assim, pode-se interpretar o algoritmo de Dijkstra como uma generalização da Busca em Largura para grafos ponderados com pesos não negativos. Ambos os algoritmos expandem a árvore de busca a partir dos menores caminhos conhecidos; a diferença é que, enquanto a BFS mede distância em número de arestas, Dijkstra mede distância pela soma dos pesos das arestas.

Exemplo de execução do algoritmo de Dijkstra

A seguir apresentamos uma simulação completa da execução do algoritmo de Dijkstra. Em cada iteração, o vértice de menor custo é removido da fila de prioridades, suas arestas são relaxadas e as estimativas de custo dos vértices adjacentes são atualizadas quando um caminho mais curto é encontrado. Considere o grafo G a seguir, em que desejamos encontrar os caminhos mínimos de s os demais vértices de G .

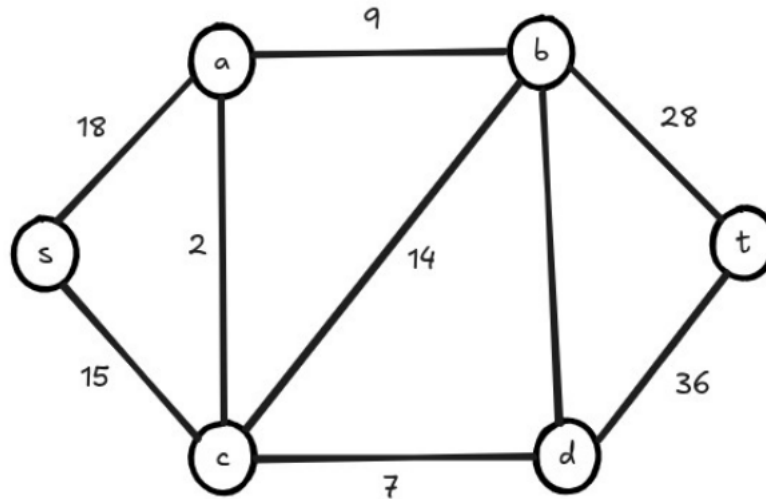


Figura 84: Grafo de entrada para o algoritmo de Dijkstra com raiz no vértice s .

A Tabela 11 resume toda a execução do algoritmo.

Tabela 11: Trace completo do algoritmo de Dijkstra.

Iteração	u	$V' = \{v \in N(u) \cap Q\}$	Relaxamentos realizados
0	s	$\{a, c\}$	$\lambda(a) = \min\{\infty, 0 + 18\} = 18, \quad \pi(a) = s$ $\lambda(c) = \min\{\infty, 0 + 15\} = 15, \quad \pi(c) = s$
1	c	$\{a, b, d\}$	$\lambda(a) = \min\{18, 15 + 2\} = 17, \quad \pi(a) = c$ $\lambda(b) = \min\{\infty, 15 + 14\} = 29, \quad \pi(b) = c$ $\lambda(d) = \min\{\infty, 15 + 7\} = 22, \quad \pi(d) = c$
2	a	$\{b\}$	$\lambda(b) = \min\{29, 17 + 9\} = 26, \quad \pi(b) = a$
3	d	$\{b, t\}$	$\lambda(b) = \min\{26, 22 + 10\} = 26$ (não altera) $\lambda(t) = \min\{\infty, 22 + 36\} = 58, \quad \pi(t) = d$
4	b	$\{t\}$	$\lambda(t) = \min\{58, 26 + 28\} = 54, \quad \pi(t) = b$
5	t	$\emptyset$	Nenhum relaxamento.

A seguir, são mostradas as configurações da fila de prioridades a cada iteração do algoritmo.

Tabela 12: Evolução dos valores de $\lambda(v)$ ao longo da execução.

Iteração	Extraído	$\lambda(s)$	$\lambda(a)$	$\lambda(b)$	$\lambda(c)$	$\lambda(d)$	$\lambda(t)$
Inicial	—	0	∞	∞	∞	∞	∞
0	s	0	18	∞	15	∞	∞
1	c	0	17	29	15	22	∞
2	a	0	17	26	15	22	∞
3	d	0	17	26	15	22	58
4	b	0	17	26	15	22	54
5	t	0	17	26	15	22	54

A árvore de caminhos mínimos resultante é mostrada a seguir:

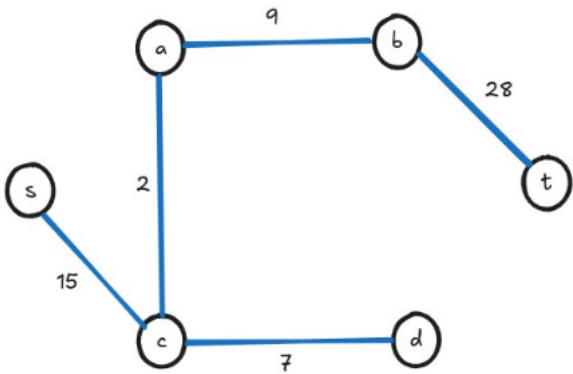


Figura 85: Árvore de caminhos mínimos obtida pelo algoritmo de Dijkstra.

O mapa de predecessores que codifica a árvore em questão é dado por:

v	s	a	b	c	d	t
$\pi(v)$	---	s	a	b	a	b
$\lambda(v)$	0	17	26	15	22	54

Figura 86: Mapa de predecessores que codifica a árvore de caminhos mínimos.

9.1.4 Análise da Complexidade

A complexidade do algoritmo de Dijkstra depende diretamente da estrutura de dados utilizada para implementar a fila de prioridades. As duas implementações mais comuns utilizam um vetor (array) simples ou um *heap* binário (*min-heap*). A seguir analisaremos ambos os casos.

Caso 1: Fila de prioridades implementada com um vetor

Nesta implementação, a fila de prioridades é representada por um vetor contendo todos os vértices ainda não processados. A cada iteração é necessário percorrer esse vetor para localizar o vértice com menor valor de $\lambda(v)$. Os custos das principais operações são:

- Inicialização dos vetores $\lambda(v)$ e $\pi(v)$: $O(n)$;
- Inserção inicial dos vértices em Q : $O(n)$;
- O laço principal (`while`) é executado exatamente n vezes;
- Cada operação `ExtractMin` percorre todo o vetor, exigindo $O(n)$;
- O processamento dos vizinhos de um vértice u percorre todos os seus vizinhos, isto é, $d(u)$ vértices;
- A operação `DecreaseKey` consiste apenas na atualização do valor de $\lambda(v)$ no vetor, portanto possui custo $O(1)$.

Assim, o custo total do algoritmo pode ser escrito como

$$T(n) = O(n) + O(n) + n \cdot O(n) + \sum_{u \in V} O(d(u)).$$

Pelo Lema do Aperto de Mãos,

$$\sum_{u \in V} d(u) = 2m,$$

onde m é o número de arestas do grafo. Logo,

$$T(n) = O(n) + O(n^2) + O(m).$$

Como em qualquer grafo simples vale

$$m \leq n(n-1)/2 = O(n^2),$$

segue que

$$\boxed{T(n) = O(n^2)}.$$

Essa implementação é simples e possui excelente desempenho em grafos densos, nos quais o número de arestas é próximo de n^2 .

Caso 2: Fila de prioridades implementada com um heap binário

Uma implementação mais eficiente consiste em representar a fila de prioridades por um *min-heap*. Nesse caso, a remoção do vértice de menor prioridade torna-se significativamente mais rápida. Os custos das operações passam a ser:

- Inicialização dos vetores: $O(n)$;
- Construção inicial do heap: $O(n)$;
- O laço principal continua sendo executado n vezes;
- Cada operação `ExtractMin`: $O(\log n)$;
- O processamento das listas de adjacência continua visitando cada aresta exatamente uma vez;
- Cada operação `DecreaseKey`: $O(\log n)$.

O custo total torna-se

$$T(n) = O(n) + n O(\log n) + \sum_{u \in V} (O(d(u)) + O(d(u) \log n)).$$

Aplicando novamente o Lema do Aperto de Mãos,

$$\sum_{u \in V} d(u) = 2m,$$

obtemos

$$T(n) = O(n) + O(n \log n) + O(m) + O(m \log n).$$

Como, em grafos conexos,

$$m \geq n - 1,$$

o termo dominante é $m \log n$, resultando em

$$\boxed{T(n) = O(m \log n)}.$$

Essa é a implementação normalmente utilizada na prática, principalmente quando o grafo é esparso.

Qual implementação deve ser utilizada?

A escolha da melhor implementação depende da densidade do grafo. A implementação baseada em vetor possui custo

$$O(n^2),$$

enquanto a implementação com heap binário possui custo

$$O(m \log n).$$

A implementação com vetor será mais eficiente quando

$$n^2 < m \log n,$$

ou, equivalentemente,

$$\frac{m}{n^2} > \frac{1}{\log n}.$$

Como a densidade de um grafo é definida por

$$d = \frac{m}{n^2},$$

podemos escrever

$$d > \frac{1}{\log n}.$$

Portanto,

- se o grafo é **denso**, isto é,

$$d > \frac{1}{\log n},$$

a implementação com vetor tende a ser mais eficiente;

- se o grafo é **esparso**, isto é,

$$d < \frac{1}{\log n},$$

a implementação com heap binário apresenta melhor desempenho.

Exemplo. Considere um grafo com

$$n = 1024 = 2^{10}.$$

Como

$$\log_2(1024) = 10,$$

o ponto de equilíbrio ocorre quando

$$d = \frac{1}{10} = 0.1.$$

Assim,

- se $d > 0.1$, a implementação baseada em vetor tende a ser mais rápida;
- se $d < 0.1$, a implementação baseada em heap binário é preferível.

Como

$$m = d n^2,$$

o valor correspondente é

$$m = 0.1 \times 1024^2 \approx 104\,858$$

arestas. Portanto, apenas grafos bastante densos justificam a implementação mais simples baseada em vetor, enquanto a implementação com *heap* binário é a escolha natural para a maioria dos problemas encontrados na prática.

9.1.5 Prova de Corretude do Algoritmo de Dijkstra

A seguir demonstraremos que o algoritmo de Dijkstra calcula corretamente os caminhos mínimos quando todos os pesos das arestas são não negativos.

Teorema 17. *Seja $G = (V, E, w)$ um grafo ponderado em que*

$$w : E \rightarrow \mathbb{R}_{\geq 0}.$$

Ao término do algoritmo de Dijkstra, vale que

$$\lambda(v) = d(s, v), \quad \forall v \in V,$$

onde $d(s, v)$ representa o custo do caminho mínimo entre a origem s e o vértice v .

Demonstração. Inicialmente, observe que o algoritmo mantém sempre a seguinte propriedade:

$$\lambda(v) \geq d(s, v), \quad \forall v \in V,$$

isto é, o valor armazenado em $\lambda(v)$ nunca subestima o verdadeiro custo do caminho mínimo. Isso ocorre porque a operação de relaxamento apenas atualiza $\lambda(v)$ utilizando o custo de algum caminho existente entre s e v . A demonstração será feita por contradição.

1. Suponha que u esteja saindo da fila e que ele é o primeiro vértice para o qual

$$\lambda(u) > d(s, u)$$

exatamente no instante em que u é removido da fila de prioridades e inserido no conjunto S .

2. Claramente, $u \neq s$, pois

$$\lambda(s) = d(s, s) = 0.$$

3. Note que u é alcançável a partir de s , pois caso contrário $\lambda(u) = d(s, u) = \infty$. Logo, existe um caminho mínimo

$$P_{su}^*.$$

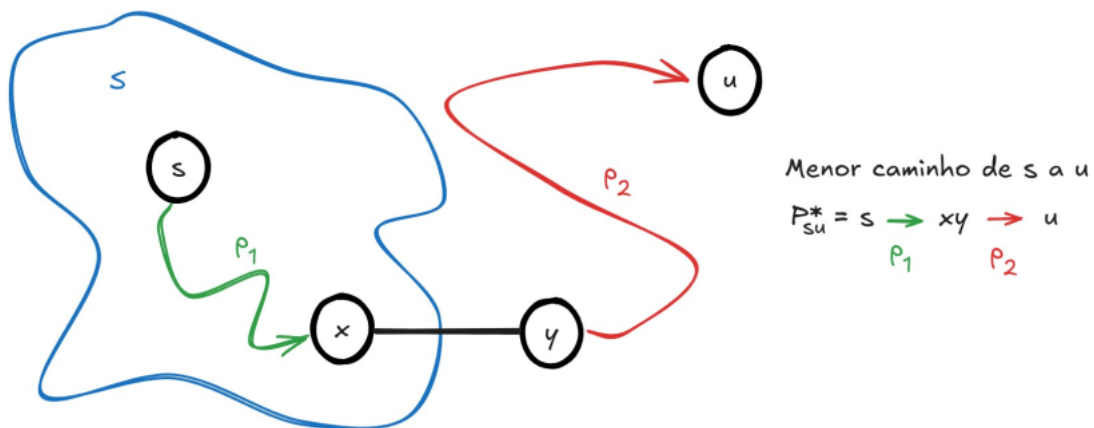
4. No instante imediatamente anterior à inserção de u em S , sabemos que

$$s \in S \quad \text{e} \quad u \notin S.$$

Portanto, ao percorrer o caminho mínimo P_{su}^* , existe um primeiro vértice que ainda não pertence a S .

5. Seja y esse primeiro vértice e seja x seu predecessor imediato em P_{su}^* . Assim,

$$x \in S \quad \text{e} \quad y \notin S.$$



6. Como x já pertence ao conjunto S , pela hipótese de indução do algoritmo,

$$\lambda(x) = d(s, x).$$

Além disso, quando x foi processado, a aresta (x, y) foi necessariamente relaxada. Logo,

$$\lambda(y) \leq \lambda(x) + w(x, y) = d(s, x) + w(x, y) = d(s, y).$$

Como $\lambda(y)$ nunca pode ser menor que o verdadeiro caminho mínimo, conclui-se que

$$\boxed{\lambda(y) = d(s, y).}$$

7. Como y aparece antes de u no caminho mínimo e todos os pesos são não negativos,

$$d(s, y) \leq d(s, u).$$

Utilizando a propriedade inicial do algoritmo,

$$d(s, u) \leq \lambda(u),$$

obtemos

$$\lambda(y) = d(s, y) \leq d(s, u) \leq \lambda(u).$$

8. Entretanto, tanto y quanto u ainda pertencem ao conjunto $V - S$. Como o algoritmo sempre escolhe o vértice de menor valor de λ para entrar em S , segue que

$$\lambda(u) \leq \lambda(y).$$

9. Das duas desigualdades anteriores concluímos que

$$\lambda(y) = d(s, y) = d(s, u) = \lambda(u).$$

o que contradiz a hipótese inicial $\lambda(u) > d(s, u)$.

Portanto, nossa hipótese inicial é falsa. Consequentemente, não existe vértice que seja inserido em S com um valor incorreto de λ , isto é,

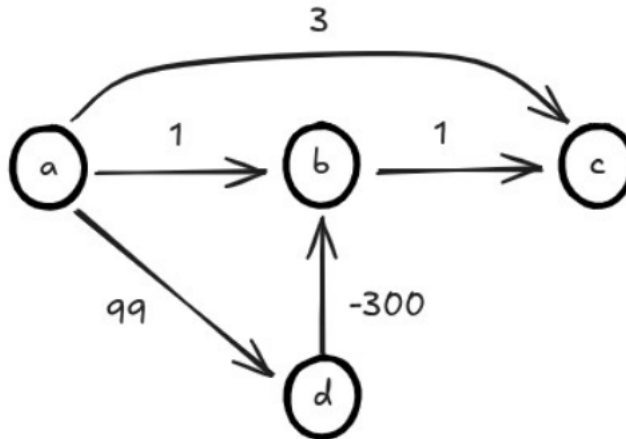
$$\boxed{\lambda(v) = d(s, v), \quad \forall v \in V.}$$

Assim, o algoritmo de Dijkstra sempre produz corretamente a árvore de caminhos mínimos. $\square$

9.2 Limitações do Algoritmo de Dijkstra: Arestas com Pesos Negativos

A corretude do algoritmo de Dijkstra depende de uma hipótese fundamental: **todas as arestas do grafo devem possuir pesos não negativos**. Quando essa condição é violada, o algoritmo pode produzir resultados incorretos. A razão é que o algoritmo adota uma estratégia *gulosa* (*greedy*). Em cada iteração, ele remove da fila de prioridades o vértice com menor valor de $\lambda(v)$ e assume que esse valor já corresponde ao custo definitivo do caminho mínimo. Em outras palavras, uma vez que um vértice é removido da fila e inserido no conjunto S , seu valor nunca mais será alterado.

Essa estratégia é correta apenas quando todos os pesos são não negativos. Nesse caso, qualquer caminho alternativo descoberto posteriormente necessariamente terá custo maior ou igual ao caminho já encontrado, impossibilitando melhorias futuras. Entretanto, na presença de arestas com pesos negativos, essa propriedade deixa de ser válida. Um vértice cujo caminho mínimo aparentemente já foi determinado pode, posteriormente, ser alcançado por outro caminho que utilize uma aresta de peso negativo, produzindo um custo ainda menor. Como o algoritmo de Dijkstra nunca reconsidera vértices já removidos da fila de prioridades, essa melhoria não será propagada, comprometendo a corretude da solução. O exemplo a seguir apresenta um exemplo desse comportamento.



Considere o grafo ponderado da figura e suponha que desejamos calcular os caminhos mínimos a partir do vértice A . A Tabela 13 apresenta a execução passo a passo do algoritmo.

Tabela 13: Execução do algoritmo de Dijkstra na presença de pesos negativos.

Iteração	u	$V' = \{v \in N(u) \cap Q\}$	Atualizações realizadas
0	A	$\{B, C, D\}$	$\lambda(B) = 1, \pi(B) = A$ $\lambda(C) = 3, \pi(C) = A$ $\lambda(D) = 99, \pi(D) = A$
1	B	$\{C\}$	$\lambda(C) = \min(3, 1 + 1) = 2$ $\pi(C) = B$
2	C	$\emptyset$	Nenhuma atualização.
3	D	$\emptyset$	Nenhuma atualização.

A evolução dos valores de $\lambda(v)$ é apresentada na Tabela 14.

Tabela 14: Valores de $\lambda(v)$ durante a execução do algoritmo.

Iteração	A	B	C	D
0	0	∞	∞	∞
1	0	1	3	99
2	0	1	2	99
3	0	1	2	99

À primeira vista, o algoritmo parece produzir uma solução consistente. Entretanto, suponha agora que exista uma aresta de peso negativo de D para B , por exemplo,

$$w(D, B) = -200.$$

Nesse caso, o caminho

$$A \rightarrow D \rightarrow B$$

possui custo

$$99 + (-200) = -101,$$

que é muito menor do que o custo anteriormente atribuído a B , igual a 1. O problema é que, quando o algoritmo finalmente processa o vértice D , o vértice B já foi removido da fila de prioridades e inserido no conjunto S . Portanto, ele é considerado definitivamente resolvido e não poderá mais ser atualizado. Consequentemente, embora o algoritmo pudesse descobrir que

$$\lambda(B) = -101,$$

essa informação jamais seria propagada para seus sucessores, produzindo uma árvore de caminhos mínimos incorreta. Esse exemplo evidencia exatamente por que a prova de corretude do algoritmo de Dijkstra exige pesos não negativos. Durante a demonstração, utiliza-se a propriedade

$$d(s, y) \leq d(s, u),$$

válida apenas quando todas as arestas possuem peso maior ou igual a zero. Com pesos negativos, um vértice localizado mais adiante em um caminho pode possuir custo menor que seus predecessores, invalidando o argumento utilizado na prova. Em resumo, o algoritmo de Dijkstra **não possui garantias teóricas de corretude na presença de arestas com pesos negativos**. Para resolver esse problema é necessário utilizar algoritmos específicos, como o algoritmo de Bellman–Ford, que permite múltiplas relaxações das arestas e é capaz de corrigir sucessivamente os valores de $\lambda(v)$ até que todos os caminhos mínimos sejam encontrados.

9.3 O Algoritmo de Bellman–Ford

A principal limitação do algoritmo de Dijkstra decorre do fato de que ele considera definitivo o custo de um vértice assim que este é removido da fila de prioridades. Caso exista uma aresta de peso negativo, um caminho melhor pode ser descoberto posteriormente, mas o algoritmo não revisita vértices já finalizados. O algoritmo de Bellman–Ford adota uma estratégia completamente diferente. Em vez de tornar permanentemente definitivo o valor de um vértice, ele permite que todos os caminhos sejam continuamente refinados por meio de sucessivas operações de relaxamento.

A ideia central é extremamente simples: em vez de selecionar apenas um subconjunto de arestas para relaxar, o algoritmo percorre **todas as arestas do grafo**, repetindo esse processo diversas vezes. Assim, sempre que um caminho melhor é descoberto, essa melhoria é propagada gradualmente para os demais vértices nas iterações seguintes. Essa característica torna o algoritmo capaz de lidar naturalmente com arestas de peso negativo. Mesmo que uma melhoria seja encontrada tardiamente, ela ainda poderá ser propagada nas próximas passagens pelas arestas do grafo. Mais precisamente, o algoritmo baseia-se na seguinte propriedade:

Todo caminho simples em um grafo com $|V|$ vértices possui, no máximo, $|V| - 1$ arestas.

Logo, após realizar exatamente $|V| - 1$ passagens sobre todas as arestas, qualquer caminho mínimo possível já teve oportunidade suficiente para propagar seu custo até o vértice de destino. O algoritmo é descrito a seguir.

```
Bellman_Ford(G, w, s) {
  for each  $v \in V$  {
     $\lambda(v) = \infty$ 
     $\pi(v) = NIL$ 
  }
   $\lambda(s) = 0$ 
  for  $i = 1$  to  $|V| - 1$  {
    for each edge  $(u, v)$  in  $E$ 
       $\lambda(v) = \min\{\lambda(v), \lambda(u) + w(u, v)\}$ 
      if  $\lambda(v)$  was updated
         $\pi(v) = u$ 
  }
}
```

Observe que, diferentemente do algoritmo de Dijkstra, a ordem em que as arestas são relaxadas é irrelevante para a corretude do método. Qualquer ordem pode ser utilizada, pois eventuais melhorias serão naturalmente propagadas nas próximas iterações. Como exemplo, considere o mesmo grafo apresentado na seção anterior, utilizando a seguinte ordem de relaxamento:

$$(A, B), (A, C), (A, D), (B, C), (D, B).$$

Como o grafo possui quatro vértices, o algoritmo executará exatamente três iterações completas sobre o conjunto de arestas.

Primeira iteração

$$(A, B) : \quad \lambda(B) = \min(\infty, 1) = 1$$

$$(A, C) : \quad \lambda(C) = \min(\infty, 3) = 3$$

$$(A, D) : \lambda(D) = \min(\infty, 99) = 99$$

$$(B, C) : \lambda(C) = \min(3, 1 + 1) = 2$$

$$(D, B) : \lambda(B) = \min(1, 99 - 300) = -201.$$

Observe que, ao contrário do algoritmo de Dijkstra, o vértice B pode ser atualizado mesmo após já ter sido utilizado anteriormente na mesma iteração.

Segunda iteração

Agora, como $\lambda(B)$ foi reduzido para -201 , essa melhoria pode ser propagada para seus vizinhos.

$$(A, B) : \lambda(B) = \min(-201, 1) = -201$$

$$(A, C) : \lambda(C) = \min(2, 3) = 2$$

$$(A, D) : \lambda(D) = 99$$

$$(B, C) : \lambda(C) = \min(2, -201 + 1) = -200$$

$$(D, B) : \lambda(B) = \min(-201, 99 - 300) = -201.$$

Perceba que somente na segunda passagem o novo valor de B consegue influenciar o custo do vértice C .

Terceira iteração

Executando novamente o relaxamento de todas as arestas, verifica-se que nenhuma atualização adicional ocorre:

$$(A, B) : \lambda(B) = -201$$

$$(A, C) : \lambda(C) = -200$$

$$(A, D) : \lambda(D) = 99$$

$$(B, C) : \lambda(C) = -200$$

$$(D, B) : \lambda(B) = -201.$$

Como nenhuma distância foi modificada, conclui-se que todos os caminhos mínimos já foram encontrados. A principal diferença entre Bellman–Ford e Dijkstra pode ser resumida da seguinte forma:

- O algoritmo de **Dijkstra** considera que um vértice nunca mais precisará ser atualizado após ser removido da fila de prioridades. Essa hipótese é válida apenas quando todos os pesos são não negativos.

- O algoritmo de **Bellman–Ford** nunca assume que uma estimativa é definitiva. Em vez disso, ele continua relaxando todas as arestas repetidamente, permitindo que melhorias descobertas em uma iteração sejam propagadas nas iterações seguintes.

É justamente essa propagação sucessiva das melhorias que torna o algoritmo capaz de encontrar corretamente os caminhos mínimos mesmo na presença de arestas com pesos negativos (desde que o grafo não contenha ciclos de peso negativo, situação que será discutida na próxima seção).

9.4 Dijkstra com Múltiplas Fontes (Multi-Source Dijkstra)

Em diversos problemas práticos, não estamos interessados em encontrar os menores caminhos a partir de um único vértice de origem, mas sim a partir de um *conjunto* de vértices. Nesses casos, utiliza-se uma generalização natural do algoritmo de Dijkstra, conhecida como **Multi-Source Dijkstra**. A ideia consiste em determinar, para cada vértice do grafo, a menor distância até *qualquer* vértice pertencente a um conjunto de fontes previamente definido. Mais formalmente, seja

$$S = \{s_1, s_2, \dots, s_k\} \subseteq V$$

um conjunto de vértices de origem. O objetivo passa a ser calcular

$$d(S, v) = \min_{s_i \in S} d(s_i, v),$$

ou seja, a distância do vértice v até a fonte mais próxima.

Ideia básica

A adaptação do algoritmo é surpreendentemente simples. Em vez de inicializar apenas um vértice com distância zero, inicializamos simultaneamente todas as fontes:

$$\lambda(s) = 0, \quad \forall s \in S.$$

Todos esses vértices são inseridos inicialmente na fila de prioridades. Os demais continuam com distância infinita:

$$\lambda(v) = \infty, \quad \forall v \notin S.$$

A partir desse ponto, o algoritmo prossegue exatamente da mesma forma que o algoritmo de Dijkstra tradicional. Na prática, é como se todas as fontes expandissem simultaneamente suas árvores de caminhos mínimos. À medida que essas árvores crescem, cada vértice passa a pertencer naturalmente à fonte que consegue alcançá-lo com menor custo. A única diferença em relação ao algoritmo tradicional encontra-se na etapa de inicialização.

```

MultiSource_Dijkstra(G, w, S) {
  for each  $v \in V$  {
     $\lambda(v) = \infty$ 
     $\pi(v) = NIL$ 
  }
   $Q = \emptyset$ 
  for each  $s \in S$  {
     $\lambda(s) = 0$ 
    Insert( $Q, s$ )
  }
  while  $Q \neq \emptyset$  {
     $u = \text{ExtractMin}(Q)$ 
    for each  $v \in N(u)$  {
      if  $\lambda(v) > \lambda(u) + w(u, v)$  {
         $\lambda(v) = \lambda(u) + w(u, v)$ 
         $\pi(v) = u$ 
        DecreaseKey( $Q, v, \lambda(v)$ )
      }
    }
  }
}

```

Observe que o restante do algoritmo permanece exatamente igual ao Dijkstra convencional.

Superfonte (Super Source)

Existe outra forma bastante elegante de interpretar o algoritmo. Pode-se adicionar ao grafo um novo vértice artificial s^* , denominado *superfonte*, conectado a todas as fontes reais por arestas de peso zero.

$$w(s^*, s_i) = 0, \quad \forall s_i \in S.$$

Em seguida, basta executar o algoritmo de Dijkstra tradicional tomando s^* como origem. Como todas as arestas incidentes na superfonte possuem custo zero, o algoritmo considera automaticamente todas as fontes simultaneamente. Essa interpretação é bastante utilizada em demonstrações teóricas e simplifica a implementação de diversos algoritmos em grafos.

Complexidade

A complexidade assintótica permanece exatamente a mesma do algoritmo de Dijkstra. Utilizando uma fila de prioridades baseada em um heap binário, obtém-se

$$O(m \log n),$$

onde

$$n = |V| \quad \text{e} \quad m = |E|.$$

A única diferença é a inserção inicial de múltiplos vértices na fila de prioridades, cujo custo é desprezível em relação ao restante da execução.

Aplicações

O algoritmo Multi-Source Dijkstra aparece naturalmente em diversos problemas reais.

Localização de serviços Considere um mapa urbano contendo vários hospitais. Deseja-se determinar qual hospital está mais próximo de cada residência da cidade. Nesse caso:

- as fontes correspondem aos hospitais;
- cada cruzamento é um vértice;
- as ruas representam as arestas;
- o peso das arestas corresponde ao tempo de deslocamento.

Após executar o algoritmo, cada residência conhece automaticamente a distância até o hospital mais próximo.

Redes de distribuição Empresas de energia elétrica, água, gás ou telecomunicações frequentemente precisam determinar qual estação de distribuição atende cada consumidor com o menor custo. Cada subestação é utilizada como uma fonte, produzindo uma partição natural da rede.

Planejamento logístico Em sistemas de transporte, diversas centrais de distribuição podem abastecer diferentes cidades. O algoritmo permite determinar automaticamente qual centro de distribuição é responsável por cada destino, minimizando o custo de transporte.

Robótica Em navegação autônoma, um robô pode precisar encontrar rapidamente o ponto seguro mais próximo entre vários pontos de recarga ou estações de manutenção. Todos esses pontos são utilizados como fontes simultâneas.

Processamento de imagens Uma aplicação bastante importante ocorre em Visão Computacional. Ao representar uma imagem como um grafo, determinados pixels podem ser marcados como sementes (*seeds*) pertencentes ao objeto ou ao fundo. Executando o algoritmo Multi-Source Dijkstra, cada pixel é associado automaticamente à semente que pode alcançá-lo com menor custo. Essa ideia constitui a base de diversos algoritmos de segmentação de imagens, como o *Image Foresting Transform (IFT)* e métodos de *watershed* baseados em grafos.

Redes de computadores Em protocolos de roteamento, pode ser necessário determinar o roteador mais próximo dentre vários servidores ou gateways disponíveis. O algoritmo calcula simultaneamente as menores distâncias até todos eles, reduzindo significativamente o custo computacional em relação à execução repetida do algoritmo para cada origem.

Observação

Uma alternativa para resolver esse problema seria executar o algoritmo de Dijkstra individualmente para cada uma das k fontes. Entretanto, essa estratégia possui custo

$$O(km \log n),$$

enquanto o algoritmo Multi-Source realiza exatamente uma única execução de Dijkstra,

$$O(m \log n),$$

independentemente do número de fontes. Quando o conjunto de fontes é grande, essa diferença representa um ganho computacional significativo.

Exemplo ilustrativo

O exemplo a seguir ilustra a execução passo a passo do algoritmo de *Multi-Source Dijkstra*. Considere o grafo ponderado apresentado na figura correspondente e suponha que desejamos particioná-lo em dois agrupamentos. Para isso, escolhem-se dois vértices sementes: A e E .

Assim, ambos são considerados origens simultâneas do algoritmo, sendo inicializados com distância zero:

$$\lambda(A) = \lambda(E) = 0.$$

Todos os demais vértices recebem distância infinita. Durante a execução, cada vértice passa a pertencer naturalmente à semente que consegue alcançá-lo com menor custo acumulado.

Ao término da execução, cada vértice encontra-se associado à semente que o alcança por meio do caminho de menor custo. Dessa forma, o algoritmo produz simultaneamente duas árvores de caminhos mínimos, uma enraizada em A e outra em E , induzindo naturalmente uma partição do grafo em

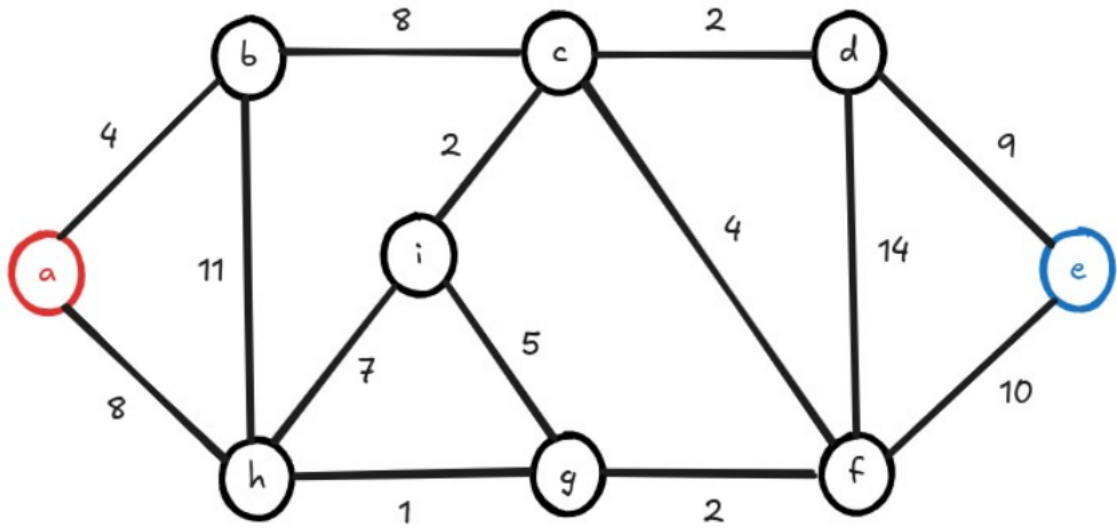


Tabela 15: Execução passo a passo do algoritmo Multi-Source Dijkstra.

Iteração	u	$V' = \{v \in N(u) \mid v \in Q\}$	Atualizações realizadas
1	A	$\{B, H\}$	$\lambda(B) = \min(\infty, 4) = 4, \pi(B) = A$ $\lambda(H) = \min(\infty, 8) = 8, \pi(H) = A$
2	E	$\{D, F\}$	$\lambda(D) = \min(\infty, 9) = 9, \pi(D) = E$ $\lambda(F) = \min(\infty, 10) = 10, \pi(F) = E$
3	B	$\{C, H\}$	$\lambda(C) = \min(\infty, 4 + 8) = 12, \pi(C) = B$ $\lambda(H) = \min(8, 4 + 11) = 8$ (não altera)
4	H	$\{G, I\}$	$\lambda(G) = \min(\infty, 8 + 1) = 9, \pi(G) = H$ $\lambda(I) = \min(\infty, 8 + 7) = 15, \pi(I) = H$
5	D	$\{C, F\}$	$\lambda(C) = \min(12, 9 + 2) = 11, \pi(C) = D$ $\lambda(F) = \min(10, 9 + 14) = 10$ (não altera)
6	G	$\{F, I\}$	$\lambda(F) = \min(10, 9 + 14) = 10$ (não altera) $\lambda(I) = \min(15, 9 + 5) = 14, \pi(I) = G$
7	F	$\{C\}$	$\lambda(C) = \min(11, 10 + 4) = 11$ (não altera)
8	C	$\{I\}$	$\lambda(I) = \min(14, 11 + 2) = 13, \pi(I) = C$
9	I	$\emptyset$	Nenhuma atualização.

Tabela 16: Evolução dos custos mínimos durante a execução do algoritmo.

Iteração	A	B	C	D	E	F	G	H	I
0	0	∞	∞	∞	0	∞	∞	∞	∞
1	0	4	∞	∞	0	∞	∞	8	∞
2	0	4	∞	9	0	10	∞	8	∞
3	0	4	12	9	0	10	∞	8	∞
4	0	4	12	9	0	10	9	8	15
5	0	4	11	9	0	10	9	8	15
6	0	4	11	9	0	10	9	8	14
7	0	4	11	9	0	10	9	8	14
8	0	4	11	9	0	10	9	8	13

dois agrupamentos. Esse mecanismo constitui a base de diversos algoritmos de crescimento competitivo em grafos, sendo amplamente utilizado em problemas de segmentação de imagens, agrupamento de dados e particionamento de redes.

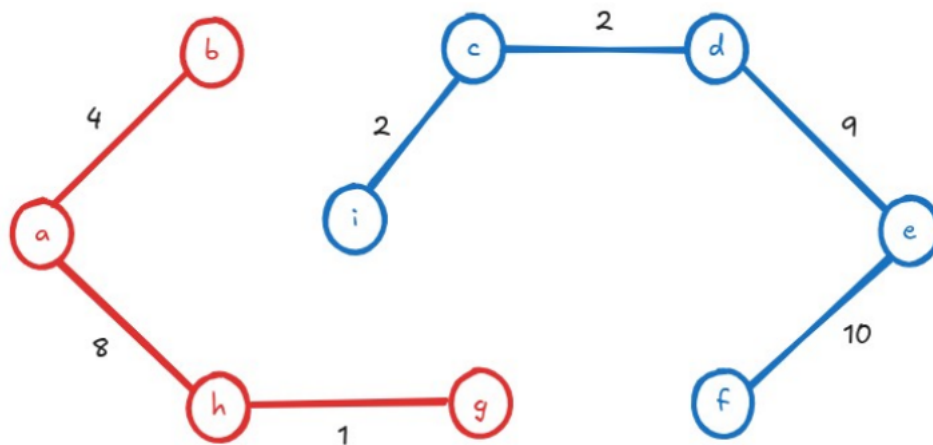


Figura 87: Floresta de caminhos ótimos obtidas a partir do exemplo anterior com o algoritmo de Dijkstra multisource.

9.5 A heurística A*

Embora o algoritmo de Dijkstra seja um dos métodos mais eficientes para encontrar caminhos mínimos em grafos com pesos não negativos, ele apresenta uma limitação importante: durante sua execução, expande todos os vértices cuja distância até a origem é menor que a distância do destino. Em muitos problemas práticos, isso significa explorar uma região muito maior do que a realmente necessária.

Quando o objetivo é encontrar o caminho mínimo entre uma origem específica s e um único destino t , é possível reduzir significativamente o número de vértices visitados utilizando informações adicionais sobre o problema. Essa é a ideia central do algoritmo A* (A-Star), proposto por *Peter Hart*, *Nils Nilsson* e *Bertram Raphael* em 1968.

A principal diferença em relação ao algoritmo de Dijkstra está no critério utilizado para selecionar o próximo vértice da fila de prioridades. Em vez de considerar apenas o custo acumulado desde a origem, o algoritmo A* também estima o custo restante até o destino.

Função de avaliação

Considere:

- $g(v)$: custo conhecido do caminho da origem s até o vértice v ;
- $h(v)$: estimativa do custo restante de v até o destino t ;
- $f(v)$: custo estimado total do caminho passando por v .

A função utilizada para ordenar a fila de prioridades é

$$f(v) = g(v) + h(v).$$

No contexto desta apostila,

$$g(v) = \lambda(v),$$

de modo que

$$f(v) = \lambda(v) + h(v).$$

Assim, um vértice terá maior prioridade quanto menor for o valor de $f(v)$. Observe que, enquanto o algoritmo de Dijkstra utiliza apenas a informação já conhecida ($\lambda(v)$), o algoritmo A* incorpora também uma estimativa do futuro por meio da função heurística $h(v)$.

Interpretação intuitiva

Considere o problema de encontrar o menor caminho entre duas cidades. O algoritmo de Dijkstra expande todos os caminhos em ordem crescente de custo acumulado, sem considerar a direção do destino. Como consequência, diversos vértices localizados em direções completamente opostas podem ser explorados. Já o algoritmo A* privilegia vértices que, além de apresentarem baixo custo acumulado, também parecem estar mais próximos do destino. Em outras palavras,

“Dijkstra olha apenas para o caminho já percorrido.”

“A* olha simultaneamente para o caminho percorrido e para o caminho que ainda falta percorrer.”

Essa simples modificação costuma reduzir drasticamente o número de vértices expandidos.

A função heurística

O desempenho do algoritmo depende diretamente da qualidade da heurística $h(v)$. Idealmente,

$$h(v) \approx d(v, t),$$

onde $d(v, t)$ representa a distância real entre v e o destino. Naturalmente, se conhecêssemos exatamente esse valor, o problema já estaria resolvido. Portanto, a função heurística procura apenas fornecer uma boa aproximação. Alguns exemplos bastante utilizados são:

- distância Euclidiana;
- distância Manhattan;
- distância em linha reta (grande círculo) em sistemas de navegação;
- funções heurísticas obtidas por aprendizado de máquina.

Quanto mais próxima a heurística estiver da distância real, menor tende a ser o número de vértices explorados.

Heurísticas admissíveis

Para que o algoritmo A* preserve a garantia de encontrar o caminho mínimo, a heurística deve satisfazer uma propriedade fundamental.

Definição 10. Uma heurística é dita **admissível** quando nunca superestima a distância até o destino:

$$h(v) \leq d(v, t), \quad \forall v \in V.$$

Ou seja, a estimativa deve ser sempre otimista.

Em outras palavras, estimativa $\leq$ distância real. Caso essa condição seja satisfeita, o algoritmo A* continua produzindo caminhos mínimos. Por outro lado, se a heurística superestimar o custo restante, o algoritmo poderá priorizar caminhos incorretos e perder sua garantia de otimalidade.

Heurísticas consistentes

Existe uma condição ainda mais forte, conhecida como *consistência* (ou *monotonicidade*). Uma heurística é consistente quando

$$h(u) \leq w(u, v) + h(v),$$

para toda aresta $(u, v) \in E$. Essa desigualdade lembra diretamente a desigualdade triangular da Geometria. Toda heurística consistente é automaticamente admissível. Além disso, quando a heurística é consistente, cada vértice é removido da fila de prioridades apenas uma única vez, exatamente como ocorre no algoritmo de Dijkstra.

Algoritmo

A implementação do algoritmo A* é praticamente idêntica à do algoritmo de Dijkstra. A única diferença encontra-se no critério utilizado pela fila de prioridades. Em vez de ordenar os vértices pelo valor de $\lambda(v)$, utiliza-se

$$f(v) = \lambda(v) + h(v).$$

Assim, basta modificar a operação de atualização da fila.

```
A_Star(G, w, s, t) {  
  Inicializar  $\lambda(v) = \infty$  para todo  $v$   
   $\lambda(s) = 0$   
  Inserir  $s$  na fila de prioridades  
  Enquanto  $Q$  não estiver vazia {  
     $u = \text{ExtractMin}(Q)$  // menor  $f(u) = \lambda(u) + h(u)$   
    Se  $u == t$   
      terminar  
    Para cada vizinho  $v$  de  $u$   
      relax( $u, v$ )  
    Atualizar prioridade de  $v$  usando  $\lambda(v) + h(v)$   
  }  
}
```

Observe que o algoritmo pode ser interrompido imediatamente quando o destino t é removido da fila de prioridades. Nesse instante, seu caminho mínimo já foi determinado.

Complexidade

O algoritmo A* possui exatamente a mesma complexidade assintótica do algoritmo de Dijkstra. Utilizando um heap binário, obtém-se

$$O((m) \log n).$$

A diferença prática está no número de vértices efetivamente explorados. Quando a heurística é informativa, a busca concentra-se em uma pequena região do grafo próxima ao caminho ótimo, tornando o algoritmo muito mais rápido na prática. No pior caso, quando

$$h(v) = 0,$$

para todo vértice, temos

$$f(v) = \lambda(v),$$

fazendo com que o algoritmo A* se torne exatamente o algoritmo de Dijkstra. Por outro lado, quanto mais precisa for a heurística, menor será o número de vértices expandidos.

Aplicações

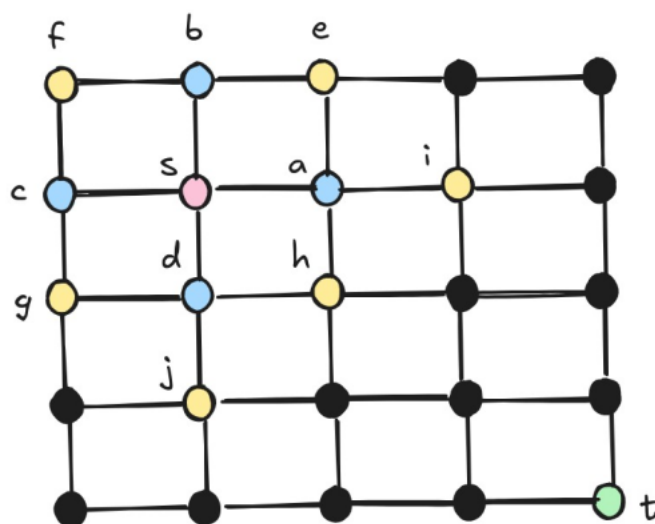
O algoritmo A* é atualmente um dos métodos mais utilizados para planejamento de trajetórias e navegação. Entre suas principais aplicações destacam-se:

- sistemas de navegação por GPS;
- robótica móvel;
- movimentação de personagens em jogos digitais;
- planejamento de trajetórias para veículos autônomos;
- navegação de drones;
- inteligência artificial para resolução de labirintos;
- roteamento em mapas e ambientes urbanos;
- planejamento de movimentos em manipuladores robóticos.

Sua popularidade deve-se ao excelente compromisso entre eficiência computacional e garantia de encontrar caminhos mínimos quando a heurística utilizada é admissível.

Exemplos

Considere o grafo ilustrado na figura a seguir, em que s representa o vértice de origem e t o vértice de destino.



Suponha que, após as primeiras iterações do algoritmo, os vértices a , b , c e d tenham sido descobertos com o mesmo custo acumulado:

$$\lambda(a) = \lambda(b) = \lambda(c) = \lambda(d) = 1.$$

No algoritmo de Dijkstra, todos esses vértices possuem exatamente a mesma prioridade na fila, pois o critério de ordenação depende exclusivamente do custo acumulado $\lambda(v)$. Consequentemente, qualquer um deles pode ser expandido primeiro. O algoritmo A*, por outro lado, utiliza também uma estimativa da distância restante até o destino. Seja $\gamma(v)$ a distância Euclidiana entre o vértice v e o destino t . Nesse exemplo, temos

$$\gamma(a) = \gamma(d) = \sqrt{2^2 + 3^2} = \sqrt{13},$$

e

$$\gamma(b) = \gamma(c) = \sqrt{3^2 + 4^2} = \sqrt{25} = 5.$$

Como a prioridade é determinada pela função

$$f(v) = \lambda(v) + \gamma(v),$$

obtém-se

$$f(a) = f(d) = 1 + \sqrt{13},$$

enquanto

$$f(b) = f(c) = 1 + 5 = 6.$$

Como

$$1 + \sqrt{13} < 6,$$

os vértices a e d serão removidos da fila de prioridades antes de b e c . Em outras palavras, embora todos estejam à mesma distância da origem, o algoritmo A* reconhece que a e d parecem estar geometricamente mais próximos do destino e, portanto, direciona a busca para essas regiões do grafo. O mesmo comportamento ocorre nas camadas seguintes da busca. Por exemplo, suponha que os vértices

$$e, f, g, h, i, j$$

tenham todos custo acumulado

$$\lambda(e) = \lambda(f) = \lambda(g) = \lambda(h) = \lambda(i) = \lambda(j) = 2.$$

Suas estimativas heurísticas, entretanto, são distintas. Por exemplo,

$$\gamma(e) = \sqrt{18}, \quad \gamma(h) = \sqrt{8}, \quad \gamma(j) = \sqrt{10}.$$

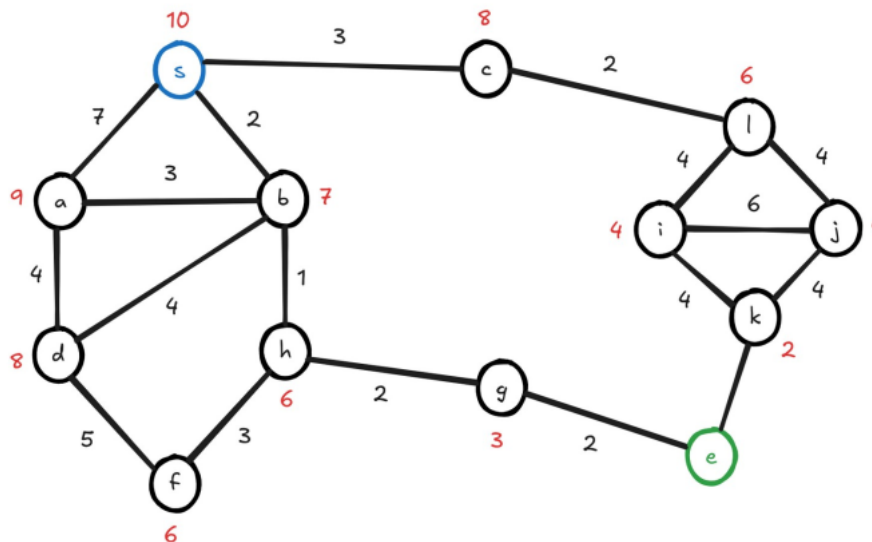
Como consequência,

$$f(h) = 2 + \sqrt{8} < f(j) = 2 + \sqrt{10} < f(e) = 2 + \sqrt{18},$$

fazendo com que o vértice h seja explorado antes de j , e ambos antes de e . Esse exemplo evidencia a principal característica do algoritmo A*: a função heurística faz com que o vértice de destino exerça uma espécie de “atração” sobre a busca. Em vez de expandir uniformemente todas as direções, como ocorre no algoritmo de Dijkstra, o A* concentra a exploração nas regiões que aparentam conduzir mais rapidamente ao objetivo.

Quanto mais precisa for a heurística utilizada, menor tende a ser o número de vértices visitados até que o destino seja alcançado. Por esse motivo, em problemas como navegação em mapas, robótica e jogos digitais, o algoritmo A* costuma ser significativamente mais eficiente que o algoritmo de Dijkstra, principalmente quando o destino pode mudar dinamicamente e um novo caminho precisa ser recalculado rapidamente.

Exemplo: Considere o grafo ponderado ilustrado na a seguir.



Os vértices representam cidades e o peso de cada aresta corresponde à distância entre elas. Suponha que estamos inicialmente na cidade s e desejamos encontrar um caminho mínimo até a cidade e . Diferentemente do algoritmo de Dijkstra, o algoritmo A* utiliza uma função heurística para direcionar a busca. Nesse exemplo, os números em vermelho associados aos vértices representam os valores da heurística

$$\gamma(v),$$

isto é, uma estimativa da distância restante entre o vértice v e o destino e . O objetivo é executar o algoritmo A* para determinar o caminho mínimo entre s e e . Na tabela a seguir, cada entrada apresenta o valor

$$f(v) = \lambda(v) + \gamma(v),$$

utilizado para ordenar a fila de prioridades.

Tabela 17: Evolução dos valores de prioridade $f(v) = \lambda(v) + \gamma(v)$.

Iteração	s	a	b	c	d	e	f	g	h	i	j	k	l
0	0	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞
1	0	$7 + 9$	$2 + 7$	$3 + 8$	∞	∞	∞	∞	∞	∞	∞	∞	∞
2	0	$5 + 9$	$2 + 7$	$3 + 8$	$6 + 8$	∞	∞	∞	$3 + 6$	∞	∞	∞	∞
3	0	$5 + 9$	$2 + 7$	$3 + 8$	$6 + 8$	∞	$6 + 6$	$5 + 3$	$3 + 6$	∞	∞	∞	∞
4	0	$5 + 9$	$2 + 7$	$3 + 8$	$6 + 8$	$7 + 0$	$6 + 6$	$5 + 3$	$3 + 6$	∞	∞	∞	∞

Tabela 18: Trace completo do algoritmo A*.

Iteração	u	$V' = \{v \in N(u) \mid v \in Q\}$	Atualizações realizadas
1	s	$\{a, b, c\}$	$\lambda(a) = \min(\infty, 7) = 7, \quad \pi(a) = s$ $\lambda(b) = \min(\infty, 2) = 2, \quad \pi(b) = s$ $\lambda(c) = \min(\infty, 3) = 3, \quad \pi(c) = s$
2	b	$\{a, d, h\}$	$\lambda(a) = \min(7, 2 + 3) = 5, \quad \pi(a) = b$ $\lambda(d) = \min(\infty, 2 + 4) = 6, \quad \pi(d) = b$ $\lambda(h) = \min(\infty, 2 + 1) = 3, \quad \pi(h) = b$
3	h	$\{f, g\}$	$\lambda(f) = \min(\infty, 3 + 3) = 6, \quad \pi(f) = h$ $\lambda(g) = \min(\infty, 3 + 2) = 5, \quad \pi(g) = h$
4	g	$\{e\}$	$\lambda(e) = \min(\infty, 5 + 2) = 7, \quad \pi(e) = g$

Observe que o algoritmo não visita todos os vértices do grafo. Graças à função heurística, a busca é direcionada para os vértices que parecem estar mais próximos do destino, reduzindo significativamente o número de expansões realizadas. Ao remover o vértice e da fila de prioridades, o algoritmo é encerrado, pois o menor caminho já foi encontrado. Reconstruindo-o a partir dos predecessores, obtemos

$$s \rightarrow b \rightarrow h \rightarrow g \rightarrow e,$$

cujos custos totais são

$$2 + 1 + 2 + 2 = 7.$$

Esse exemplo ilustra claramente a principal vantagem do algoritmo A*: em vez de explorar uniformemente todas as direções, como ocorre no algoritmo de Dijkstra, a heurística orienta a busca em direção ao objetivo, fazendo com que apenas uma pequena fração dos vértices do grafo seja efetivamente visitada.

9.6 Considerações Finais

Neste capítulo estudamos um dos problemas mais importantes da Teoria dos Grafos e da Ciência da Computação: o problema de determinação de caminhos mínimos em grafos ponderados. Diversas aplicações práticas podem ser modeladas por esse problema, desde sistemas de navegação por GPS até protocolos de roteamento na Internet, planejamento de trajetórias em robótica, logística, transporte urbano, redes de comunicação e inteligência artificial.

Inicialmente, formalizamos o conceito de caminho mínimo e introduzimos a operação de *relaxamento de arestas*, que constitui o princípio fundamental por trás da maioria dos algoritmos de caminhos mínimos. Embora extremamente simples, essa operação permite atualizar iterativamente estimativas de distância até que os menores caminhos sejam obtidos.

Em seguida, estudamos o algoritmo de Dijkstra, provavelmente o método mais conhecido para resolver o problema quando todos os pesos das arestas são não negativos. Vimos que sua eficiência depende diretamente da estrutura de dados utilizada para implementar a fila de prioridades, podendo variar entre $O(n^2)$ e $O((n + m) \log n)$. Além disso, demonstramos formalmente sua correteza, evidenciando que a estratégia gulosa adotada pelo algoritmo garante que cada vértice retirado da fila já possui sua distância mínima definitivamente determinada.

Também discutimos uma importante limitação do algoritmo de Dijkstra: sua incapacidade de lidar corretamente com arestas de peso negativo. Esse resultado não decorre de uma falha de implementação, mas sim da própria estratégia gulosa empregada pelo algoritmo, que pressupõe que nenhuma atualização futura poderá reduzir a distância de um vértice já finalizado.

Para contornar essa limitação, apresentamos o algoritmo de Bellman–Ford. Embora possua maior custo computacional, esse método relaxa repetidamente todas as arestas do grafo, permitindo que melhorias de custo sejam propagadas mesmo quando existem pesos negativos. Essa característica torna o algoritmo adequado para uma classe mais ampla de problemas e, além disso, permite detectar ciclos de peso negativo, situação na qual o problema de caminhos mínimos deixa de possuir solução bem definida.

Na sequência, exploramos o algoritmo de Dijkstra com múltiplas fontes (*Multisource Dijkstra*), uma generalização simples e extremamente útil do algoritmo original. Em vez de partir de um único vértice, a busca é iniciada simultaneamente em várias origens, permitindo resolver eficientemente problemas de particionamento do espaço, construção de diagramas de Voronoi em grafos, planejamento logístico, cobertura de redes e diversas aplicações em aprendizado de máquina e visão computacional.

Por fim, introduzimos a heurística A^* , que incorpora conhecimento adicional sobre o problema por meio de uma função heurística capaz de estimar a distância restante até o destino. Mostramos que, quando essa heurística é admissível, o algoritmo preserva a garantia de encontrar caminhos mínimos, mas normalmente visita um número muito menor de vértices do que o algoritmo de Dijkstra. Essa característica explica por que o A^* é amplamente utilizado em sistemas de navegação, robótica móvel, jogos digitais e planejamento de trajetórias em ambientes complexos.

Os algoritmos apresentados neste capítulo ilustram diferentes estratégias de projeto de algoritmos. O algoritmo de Dijkstra é um exemplo clássico da técnica gulosa (*Greedy Algorithms*); o algoritmo de Bellman–Ford baseia-se em relaxamentos sucessivos até a convergência; enquanto o algoritmo A^* combina otimização exata com conhecimento heurístico para reduzir o espaço de busca. Apesar de suas diferenças, todos compartilham o mesmo objetivo: explorar a estrutura do grafo de maneira eficiente para determinar caminhos de custo mínimo.

Epílogo

Ao longo desta apostila percorremos uma parte significativa dos fundamentos de Algoritmos e Estruturas de Dados, uma área que constitui a base de praticamente toda a Computação moderna. Iniciamos nossa jornada estudando a construção de algoritmos, técnicas de análise de complexidade e estruturas fundamentais para organização eficiente de informações. Em seguida, exploramos árvores, tabelas de espalhamento, grafos, algoritmos de busca e problemas clássicos de otimização, mostrando que, por trás de aplicações aparentemente distintas, existem princípios matemáticos e computacionais comuns.

Embora os tópicos apresentados sejam frequentemente ensinados como conteúdos independentes, um dos principais objetivos deste material foi evidenciar que todos eles fazem parte de uma mesma linguagem para modelar e resolver problemas. Uma árvore de busca, uma tabela hash, um grafo ou uma fila de prioridades não são apenas estruturas de dados diferentes: cada uma representa uma forma distinta de organizar informação para explorar propriedades específicas e obter algoritmos mais eficientes.

Mais do que memorizar algoritmos, espera-se que o leitor tenha desenvolvido uma maneira de pensar. A verdadeira habilidade de um cientista da computação consiste em abstrair um problema do mundo real, identificar a estrutura matemática que melhor o representa e escolher, ou até mesmo projetar, o algoritmo mais adequado para resolvê-lo. Em outras palavras, aprender algoritmos é aprender a raciocinar sobre eficiência, corretude e escalabilidade.

Outro aspecto importante é compreender que praticamente todo algoritmo representa um compromisso entre diferentes recursos computacionais. Em diversas situações, é possível reduzir o tempo de execução à custa de maior consumo de memória; em outras, economizar memória implica realizar mais processamento. Entender esses compromissos é essencial para o desenvolvimento de sistemas eficientes e robustos.

Naturalmente, os assuntos abordados nesta apostila representam apenas uma introdução a um universo muito mais amplo. Existem inúmeros tópicos avançados que ampliam e aprofundam os conceitos aqui estudados, como algoritmos aproximativos, algoritmos probabilísticos, algoritmos paralelos e distribuídos, estruturas de dados persistentes, algoritmos para processamento de grandes volumes de dados, teoria da complexidade computacional, otimização combinatória, geometria computacional, teoria dos grafos avançada, aprendizado de máquina e diversas outras áreas de pesquisa que continuam evoluindo rapidamente.

Entretanto, independentemente da área escolhida pelo leitor, os conhecimentos apresentados neste material continuarão sendo fundamentais. Sempre que um novo problema surgir, será necessário organizar informações, modelar relações entre objetos, analisar custos computacionais e projetar algoritmos eficientes. Os princípios estudados aqui permanecerão presentes, muitas vezes de forma invisível, sustentando tecnologias utilizadas diariamente por bilhões de pessoas.

Por fim, vale lembrar que aprender algoritmos é um processo essencialmente prático. A leitura fornece a base teórica, mas a verdadeira compreensão surge ao implementar soluções, testar diferentes

abordagens, analisar casos particulares, identificar limitações e, principalmente, resolver muitos problemas. Cada algoritmo implementado amplia a intuição do programador e fortalece sua capacidade de projetar soluções para desafios inéditos.

Esperamos que esta apostila sirva não apenas como material de estudo para a disciplina, mas também como uma referência que possa ser consultada ao longo da formação acadêmica e profissional do leitor. Mais importante ainda, esperamos que ela desperte a curiosidade para explorar tópicos mais avançados e incentive o desenvolvimento do pensamento algorítmico, uma das competências mais valiosas para qualquer profissional da Computação.

“Algoritmos são muito mais do que sequências de instruções: são uma forma de transformar ideias em soluções eficientes. Aprender a projetá-los é aprender a pensar de maneira sistemática, criativa e rigorosa sobre problemas.”

Informações sobre o autor

Alexandre L. M. Levada é Bacharel em Ciência da Computação pela Universidade Estadual Paulista (UNESP) em 2002. Concluiu seu Mestrado em Ciência da Computação pela Universidade Federal de São Carlos (UFSCar) em 2006 e seu Doutorado em Física Computacional pela Universidade de São Paulo (USP) em 2010. No mesmo ano, ingressou no Departamento de Computação da Universidade Federal de São Carlos. É professor associado no Departamento de Computação, onde lecionou diversas disciplinas de graduação e pós-graduação como lógica matemática, teoria dos grafos, matemática discreta, processamento digital de imagens, análise de sinais e sistemas, reconhecimento de padrões, visão computacional, algoritmos e estruturas de dados 1 e 2, projeto e análise de algoritmos, programação científica e otimização matemática. Seus principais interesses de pesquisa incluem reconhecimento de padrões, aprendizado de máquina e filtragem de ruído em sinais e imagens.

“Se você tiver um pão e eu tiver um euro, e eu uso o meu euro para comprar o seu pão, no final da troca eu terei o pão e você o euro. Parece um equilíbrio perfeito, não? A tem um euro, B tem um pão; depois, A tem o pão e B o euro. É uma transação justa, mas meramente material. Agora, imagine que você tem um soneto de Verlaine ou conhece o teorema de Pitágoras, e eu não tenho nada. Se me ensinar, no final dessa troca, eu terei aprendido o soneto e o teorema, mas você ainda os terá também. Nesse caso, não há apenas equilíbrio, mas crescimento. No primeiro, trocamos mercadorias. No segundo, compartilhamos conhecimento. E enquanto a mercadoria se consome, a cultura se expande infinitamente.”

Michel Serres, filósofo francês.

Referências

- W. CELES, R. CERQUEIRA, J. RANGEL. Introdução a Estrutura de Dados, 2a edição, Elsevier, 2016.
- CORMEN, T. H., Leiserson, C. E., Rivest, R. L., Stein, C. Introduction to Algorithms. 4 a edição, The MIT Press, 2022.
- ROUGHGARDEN, T. Algorithms Illuminated: The Omnibus Edition. SoundLikeYourself, 2022.
- KARUMANCHI, N. Data Structures and Algorithms Made Easy, CareerMonk Publications, 2010.
- LAMBERT, K. A. Fundamentals of Python: Data Structures, 2a edição, Cengage Learning, 2019.
- ALTHOFF, C. The Self-Taught Computer Scientist: The Beginners Guide to Data Structures & Algorithms, Wiley, 2022.
- BHARGAVA, A. Y. Grokking Algorithms: A Illustrated Guide for Programmers and Other Curious People, Manning Publications, 2016.
- KLEINBERG, J.; TARDOS, E. Algorithmic Design, Addison Wesley, 2005.
- U. AGARWAL, Algorithms Design and Analysis, Dhanpat Rai Publications, 2012.
- R. SEDGEWICK, K. WAYNE. Algorithms, 4th. ed., Addison-Wesley, 2011.
- LEVITIN, A. Introduction to the Design and Analysis of Algorithms, 3a edição, Pearson, 2012.
- SKIENA, S. S. The Algorithm Design Manual, 2a edição, Springer, 2008.
- S. DASGUPTA, C.H. PAPADIMITRIOU, U.V. VAZIRANI. Algorithms, McGraw-Hill, 2007.
- N. ZIVIANI. Projetos de algoritmos: com implementações em Pascal e C. 3. ed. rev. e ampl. São Paulo: Cengage Learning, 2012.
- R. SEDGEWICK. Algorithms in C, Parts 1-4: fundamentals, data structures, sorting, searching. 3rd. ed., Addison-Wesley, 1998.
- FELICE, M. C. S. Material e aulas das disciplinas Algoritmos e Estruturas de Dados I e II. Disponíveis em: <http://www.aloc.ufscar.br/felice/>
- Geeks for Geeks – A computer science portal for geeks. <https://www.geeksforgeeks.org>